PYTHON NÂNG CAO · PE-V4

Sổ tay nội dung giáo trình

48 bài · 8 module · bản dựng 13/10/2026

Tài liệu này là brief nội dung cho việc thiết kế visualization / motion-graphic. Mỗi bài có một khối "KỊCH BẢN MINH HOẠ" nói rõ animation cần thể hiện gì; phụ lục cuối sách là bảng tra toàn bộ mã bài kèm một dòng ý tưởng.

Toàn bộ nội dung được render tự động từ JSON bài học đã build (`scripts/pe-v4/out/lessons/*.json`, schema `studio-lesson/v1`). Không có câu nào được viết thêm trong lúc dựng PDF: thiếu dữ liệu thì tài liệu ghi là thiếu.

Mã bài (P01, T01, N01, PD01, M01, DA01…) là định danh duy nhất — dùng đúng mã khi đặt tên asset và khi trao đổi.

Chú giải nguồn lý thuyết. `SL n` = trang thứ n trong tệp slide gốc đã gộp của giảng viên (1–56 Multi Processing · 57–107 Multithreading · 108–135 GUI Toolkit). Số in trên slide có thể khác vì một slide có nhiều lớp hiện dần. `DOC …` = tài liệu chính thức của Python.

Bản dựng này có đủ 48/48 mã bài.

Phạm vi và yêu cầu đầu vào

YÊU CẦU ĐẦU VÀO

- Đã viết được hàm, vòng lặp, đọc/ghi file và class cơ bản bằng Python
- Trình duyệt có mạng để chạy Python (Pyodide) ngay trong bài; máy cài Python 3.12 để chạy thật subprocess, thread, Tkinter
- Bài numpy/pandas/matplotlib tải thêm thư viện ở lần mở đầu tiên (vài MB)

KỸ NĂNG ĐẦU RA

- Chạy chương trình con bằng subprocess: returncode, stdout/stderr, timeout, check=True
- Đa luồng với threading: start/join, Lock, Queue và đo thời gian chờ I/O
- Dựng GUI Tkinter: widget, biến, pack/grid, command và tách logic khỏi giao diện
- Nhập dữ liệu, truy vấn SQL với sqlite3 và tra cứu
- NumPy: mảng, vector hoá, lọc bằng mask
- pandas: đọc CSV/Excel, làm sạch, groupby, merge
- Matplotlib: line, bar, scatter, histogram, subplot có nhãn đầy đủ
- Dự án DA: từ dữ liệu bẩn tới báo cáo và biểu đồ

CÁCH ĐỌC MỖI BÀI

Mỗi bài theo đúng thứ tự khối: mục tiêu → cần nhớ trước → bối cảnh → điểm kiến thức → mẫu code và kết quả dự kiến → ngộ nhận → câu dự đoán và tự kiểm tra → bảng trace → template lắp mảnh → bài tập và bộ chấm → môi trường chạy → kịch bản minh hoạ.

Trong khối code, dấu » ở cuối dòng nghĩa là dòng gốc dài hơn khổ giấy nên bị bọc xuống dòng dưới — không phải ký tự của chương trình.

Mục lục

Số trang là trang PDF. Mỗi bài bắt đầu ở một trang mới.

M1 · Tiến trình và subprocess 5

P01 · Program, process và tài nguyên . . . 6
P02 · Vòng đời, fork, exec và wait . . . 11
P03 · Scheduling và context switch . . . 16
P04 · stdin, stdout, stderr và pipe . . . 21
P05 · subprocess.run và kết quả . . . 26
P06 · Popen, communicate và timeout . . . 31
P07 · multiprocessing: Process và Queue . . . 35

M2 · Đa luồng 40

T01 · Thread, tài nguyên và concurrency . . . 41
T02 · Thread target, start và join . . . 46
T03 · Subclass Thread . . . 50
T04 · GIL, CPU-bound và I/O-bound . . . 54
T05 · Lock, race condition và deadlock . . . 59
T06 · Queue và tác vụ lưu nền . . . 64

M3 · GUI Tkinter và dự án 69

G01 · GUI, CLI và cửa sổ . . . 70
G02 · Widget và callback . . . 74
G03 · Dữ liệu nhập và lựa chọn . . . 79
G04 · Layout: pack, place, grid . . . 84
G05 · OOP và tách logic khỏi giao diện . . . 89
G06 · Event loop và worker nền . . . 94
R02 · Dự án PW5–PW9: lưu, shell, thread và GUI . . . 99

M4 · Nhập dữ liệu, SQL và tra cứu 104

X01 · Nhập dữ liệu bằng NumPy và pandas . . . 105
X02 · Excel, Stata, SAS, HDF5 và MATLAB . . . 108
X03 · SQL với sqlite3 và pandas . . . 112
X04 · Matplotlib: dữ liệu và loại biểu đồ . . . 116
X05 · Matplotlib: tùy chỉnh và xuất . . . 120

M5 · NumPy 124

N01 · Array, dtype, shape và axis . . . 125
N02 · Indexing, mask và vectorization . . . 128
N03 · Broadcasting và chuẩn hoá dữ liệu . . . 131
N04 · Thống kê, NaN và trọng số . . . 134
N05 · View, copy và sai số số thực . . . 137

N06 · Đọc dữ liệu và mô phỏng tái lập 140

M6 · pandas **143**

PD01 · Series, DataFrame và index 144
PD02 · CSV, Excel và schema 147
PD03 · Làm sạch thiếu, trùng và sai kiểu 150
PD04 · Lọc, tính cột và cập nhật an toàn 154
PD05 · GroupBy và KPI có mẫu số rõ ràng 158
PD06 · Merge, concat và kiểm tra cardinality 162
PD07 · Long, wide, pivot và melt 166
PD08 · Ngày giờ, resample và rolling 170
PD09 · EDA, chất lượng dữ liệu và export 174

M7 · Matplotlib **178**

M01 · Figure, Axes và chọn biểu đồ 179
M02 · Phân bố, scatter và outlier 183
M03 · Subplot và heatmap đọc được 187
M04 · Màu, annotation và kể chuyện dữ liệu 191
M05 · Ticks, scale và ngày trên trục 195
M06 · Layout, xuất hình và kiểm tra chất lượng 199

M8 · Dự án DA **203**

DA01 · Mini-project: doanh thu từ raw tới chart 204
DA02 · Mini-project: điểm số và GPA 209

Phụ lục · Bảng tra ứng viên animation **213**

M1 · Tiến trình và subprocess

7 bài: P01, P02, P03, P04, P05, P06, P07

Ứng viên animation — ưu tiên cao (★★): không có · ưu tiên (★): P01, P02, P03, P04, P05, P06, P07

P01 · Program, process và tài nguyên

Mã bài: P01 · Trung cấp · M1 · Tiến trình và subprocess · 25 phút · 15 XP · thứ tự 1

MỤC TIÊU

Phân biệt program với process; đọc PID và đi ngược cây process.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

TABLE mô phỏng kết quả lệnh ps: mỗi dòng (pid, ppid, program). Viết count_processes(table, program) đếm số PROCESS đang chạy program đó (mỗi dòng là một process); ancestors(table, pid) trả list PID đi ngược theo ppid: [pid, cha, ông, ...] tới khi ppid không còn trong bảng (pid không có trong bảng → ValueError); current_identity() trả (os.getpid(), os.getppid()) của chính process đang chạy.

VÍ DỤ MINH HOẠ

Với TABLE mẫu: count_processes(TABLE, "python") → 3 (một program, ba process); ancestors(TABLE, 410) → [410, 305, 120, 1].

LÝ THUYẾT

Định nghĩa

Slide nói gọn: "process is a program in execution state (active)". Program là mã THỤ ĐỘNG nằm trên đĩa — không có gì đang chạy. Process là trạng thái thực thi đang hoạt động của mã đó, và một program có thể đang ứng với nhiều process cùng lúc.

- PID: số hệ điều hành cấp cho từng process, không cấp cho file .py
- PPID: PID của process cha đã tạo ra nó
- Trạng thái thực thi gồm context (program counter, thanh ghi), file descriptor và vùng nhớ
- Vùng nhớ chia ba phần: process stack, data section, heap

Ví dụ: cây process trên máy thật

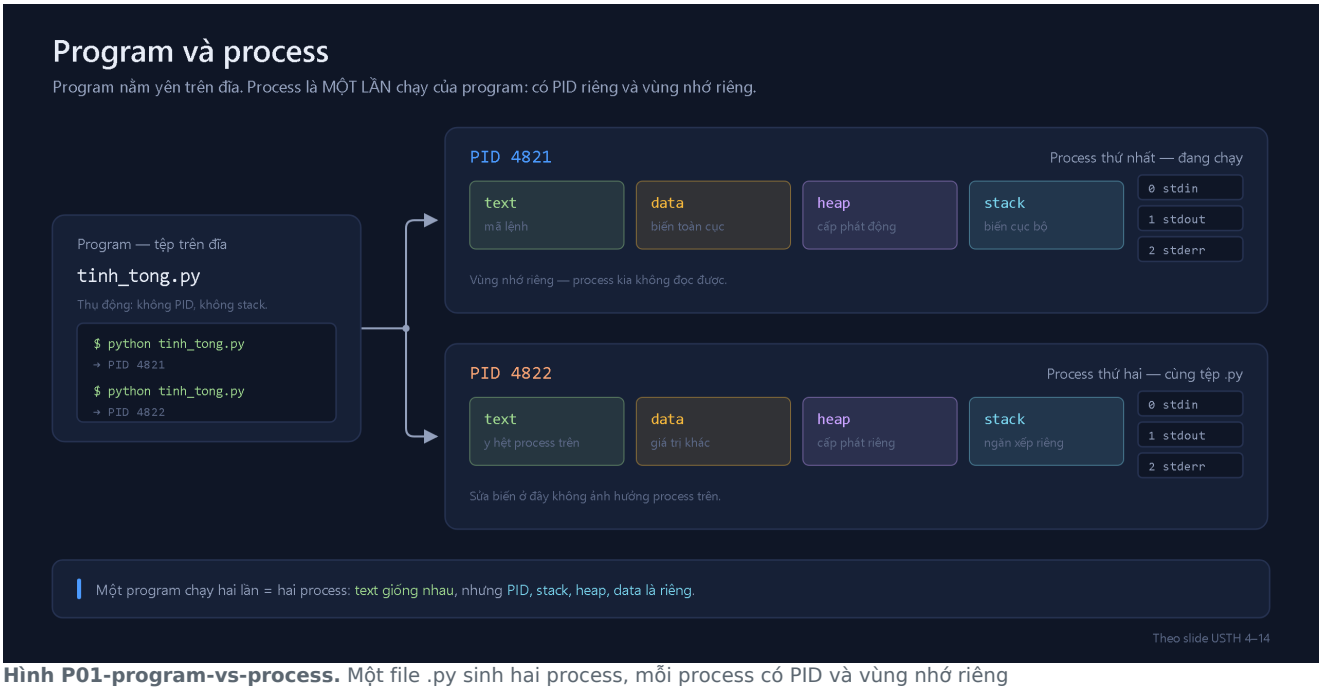
Lệnh pstree -A ở slide in ra init-+-cron, |-sshd---bash---pstree, |-screen---bash---python2... Mỗi nhánh là một process có PID riêng; bash tạo python, python tạo tiếp con thì thành cháu. TABLE của bài là một lát cắt của cây đó: ba dòng mang tên "python" là ba process của cùng một program.

Dùng ở đâu

- ps / pstree / Task Manager: xem process nào đang chạy và là con của ai
- Ghi log kèm os.getpid() để biết dòng log đến từ lần chạy nào khi chạy nhiều bản song song
- Giám sát: thấy process treo thì lần theo PPID để tìm cả nhánh cần dừng
- Studio chạy trong trình duyệt nên PID là số GIẢ (ổn định, con khác cha); máy thật mỗi lần một số

Đọc sơ đồ

Bên trái là một file .py nằm yên trên đĩa — đó là program. Hai khối bên phải là hai process sinh ra từ chính file ấy: mỗi khối có PID riêng và một vùng nhớ riêng (text, data, heap, stack), nên sửa biến trong khối này không đụng tới khối kia. Ba mũi tên 0/1/2 ở đáy mỗi khối là file descriptor stdin, stdout, stderr.



Hình P01-program-vs-process. Một file .py sinh hai process, mỗi process có PID và vùng nhớ riêng

Mô tả hình: Bên trái file .py trên đĩa; bên phải hai hộp process PID khác nhau, mỗi hộp có text/data/heap/stack và ba file descriptor 0, 1, 2

Nguồn lý thuyết: SL 4-18 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Program là mã nằm yên trên đĩa (file .py). Process là MỘT LẦN chạy program đó — có PID, bộ nhớ và trạng thái riêng.
2. Chạy cùng một file hai lần → hai process, hai PID. Một program có thể có nhiều process cùng lúc.
3. os.getpid() trả PID của process đang chạy dòng code đó; os.getppid() trả PID của process CHA. Nối ppid → pid ta được cây process.
4. Tài nguyên riêng của process: context (program counter, thanh ghi), stack, data, heap và file descriptor — 0, 1, 2 là stdin, stdout, stderr.
5. Process thường KHÔNG chung không gian địa chỉ: con sửa list của nó, list của cha không đổi. Trao dữ liệu phải qua pipe, file, queue (P04, P07).

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import os
import subprocess
import sys

print("Tôi là process", os.getpid(), "- cha của tôi là", os.getppid())

cmd = [sys.executable, "-c", "import os; print(os.getpid(), os.getppid())"]
for run in range(2):
    child_pid, child_ppid = subprocess.check_output(cmd, text=True).split()
    print("Lần", run + 1, "- con", child_pid, "có cha", child_ppid)
```

Kết quả dự kiến (theo JSON): count_processes(TABLE, "python") → 3; ancestors(TABLE, 410) → [410, 305, 120, 1]

NGỘ NHẬN THƯỜNG GẶP: "Program và process là một"

Program là mã thụ động; process là một lần thực thi đang hoạt động. Một program có thể có nhiều process cùng lúc (mở ba cửa sổ python là ba process), mỗi process có PID, stack, heap, file descriptor riêng. Vì vậy đếm process phải đếm từng DÒNG trong bảng ps, không đếm tên program.

CÂU DỰ ĐOÁN

Chạy cùng một script hai lần liên tiếp. Hai lần đó có nhất thiết cùng PID không?

sai	Có — cùng file thì cùng PID	PID gắn với process đang chạy, không gắn với program.
sai	Có, nếu chạy trên cùng một máy	Cùng máy vẫn là hai lần thực thi khác nhau.
ĐÚNG	Không — mỗi lần chạy là một process mới, hệ điều hành cấp PID mới	PID định danh process (một lần thực thi), không định danh file .py.

TỰ KIỂM TRA

1. os.getppid() trả về gì?

• PID của chính process hiện tại — Đó là os.getpid().
✔ PID của process cha đã tạo ra process hiện tại — Nối ppid → pid ta đi ngược được cây process.
• Số process con đang chạy — Không hàm os nào đếm con theo kiểu này.

Đáp án: Nối ppid → pid ta đi ngược được cây process.

2. File descriptor 0, 1, 2 của một process thường là gì?

• Ba vùng nhớ stack, data, heap — File descriptor tham chiếu tài nguyên I/O, không phải vùng nhớ.
• Ba process đầu tiên của hệ thống — Đó là PID, không phải file descriptor.
✔ stdin, stdout, stderr — Ba luồng chuẩn mỗi process có sẵn khi khởi động.

Đáp án: Ba luồng chuẩn mỗi process có sẵn khi khởi động.

3. Object tạo trong lúc chạy (vd. list(range(5))) nằm ở vùng nhớ nào của process?

• Program counter — PC chỉ là vị trí lệnh đang chạy.
✔ Heap — vùng cấp phát động — Stack chỉ giữ khung lời gọi hàm và tham chiếu tới object.
• File descriptor — File descriptor tham chiếu file, pipe, terminal.

Đáp án: Stack chỉ giữ khung lời gọi hàm và tham chiếu tới object.

TỰ GIẢI THÍCH

Vì sao count_processes(TABLE, "python") là 3 chứ không phải 1?

Mẫu trả lời: Ba dòng có program "python" là ba lần thực thi khác nhau với ba PID 305, 306, 410 — tức ba process, dù chỉ có một program "python". Đếm tên program khác nhau sẽ ra 1 và bỏ mất hai process đang chạy. Trường hợp đặc biệt: program không có dòng nào thì trả 0.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	ancestors(TABLE, 410) → chain = [410]
Bước 2	parent_of[410] là 305 → thêm 305
Bước 3	parent_of[305] là 120, parent_of[120] là 1 → thêm 120 rồi 1
Bước 4	parent_of[1] là 0, không có trong bảng → dừng, trả [410, 305, 120, 1]

CÂU PHẢN VÍ DỤ

Bảng [(1, 0, "init"), (7, 1, "code"), (8, 1, "code")]: count_processes(bảng, "code") là?

Lựa chọn: 2 | 3 | 1 · Đáp án: 2

TEMPLATE LẮP MẢNH

```
def count_processes(table, program):
    count = 0
    for pid, ppid, name in table:
        if {{slot1}}:
            count += 1
    return count


def ancestors(table, pid):
    parent_of = {}
    for row_pid, row_ppid, program in table:
        {{slot2}}
    if pid not in parent_of:
        raise ValueError(f"không có process {pid} trong bảng")
    chain = [pid]
    while parent_of[pid] in parent_of:
        pid = {{slot3}}
        chain.append(pid)
    return chain
```

Slot	Mảnh đúng
[Slot 1]	`name == program`
[Slot 2]	`parent_of[row_pid] = row_ppid`
[Slot 3]	`parent_of[pid]`

Mảnh bẫy	Vì sao sai
`parent_of[row_ppid] = row_pid`	Ngược chiều: một cha nhiều con thì con sau ghi đè con trước.
`row_ppid`	Biến của vòng for đã xong — chỉ là ppid của dòng cuối bảng.
`name in program`	"py" in "python" là True — đếm nhầm program khác tên.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp chỗ so sánh khi đếm, chỗ dựng bảng cha và bước đi lên cây.

Yêu cầu thêm: Đổi sang bảng của máy khác (PID không tăng dần, dòng con đứng trước dòng cha) và một program không có process nào; thử cả PID không có trong bảng.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

TABLE mô phỏng kết quả lệnh ps: mỗi dòng (pid, ppid, program). Viết count_processes(table, program) đếm số PROCESS đang chạy program đó (mỗi dòng là một process); ancestors(table, pid) trả list PID đi ngược theo ppid: [pid, cha, ông, ...] tới khi ppid không còn trong bảng (pid không có trong bảng → ValueError); current_identity() trả (os.getpid(), os.getppid()) của chính process đang chạy.

Thực hành: Đếm theo DÒNG (mỗi dòng một process). Dựng dict pid → ppid một lần, rồi đi ngược theo ppid tới khi ppid không còn trong bảng.

Gọi	Kết quả mong đợi
`count_processes(TABLE, "python")`	`3`
`ancestors(TABLE, 410)`	`[410, 305, 120, 1]`
`ancestors(TABLE, 999)`	`ValueError`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`count_processes(table, program)`	Đếm số process đang chạy một program. · table: list (pid, ppid, program); program: str · int (0 nếu không có) · Không in, không sửa table.
`ancestors(table, pid)`	Đi ngược cây process theo ppid. · table như trên; pid: int · list[int] bắt đầu bằng pid; pid lạ → ValueError · Không phụ thuộc thứ tự dòng trong table.
`current_identity()`	Cho biết process hiện tại là ai, cha là ai. · không · tuple (os.getpid(), os.getppid()) · Không in.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
TABLE: "python" có 3 process	`count_processes(TABLE, "python")`	`3`
Bảng khác: "sshd" có 1, "vim" có 0	`(count_processes([(1, 0, "systemd"), (900, 1, "sshd"), (77, 900, "bash"), (5000, 77, "python")], "sshd"), count_processes([(1, 0, "systemd"), (900, 1, "sshd"), (77, 900, "bash"), (5000, 77, "python")], "vim"))`	`(1, 0)`
Một program, hai process	`count_processes([(1, 0, "init"), (7, 1, "code"), (8, 1, "code")], "code")`	`2`
ancestors(TABLE, 410)	`ancestors(TABLE, 410)`	`[410, 305, 120, 1]`
Gốc cây: ancestors(TABLE, 1) → [1]	`ancestors(TABLE, 1)`	`[1]`
PID không tăng dần	`ancestors([(1, 0, "systemd"), (900, 1, "sshd"), (77, 900, "bash"), (5000, 77, "python")], 5000)`	`[5000, 77, 900, 1]`
Dòng con đứng trước dòng cha	`ancestors([(50, 7, "python"), (7, 1, "bash"), (1, 0, "init")], 50)`	`[50, 7, 1]`
PID không có trong bảng → ValueError	`_studio.call(ancestors, TABLE, 999).error`	`'ValueError'`
current_identity() = (os.getpid(), os.getppid())	`current_identity()`	`(__import__("os").getpid(), __import__("os").getppid())`

MÔI TRƯỜNG CHẠY

PID trong Studio là số GIẢ nhưng ổn định (process con luôn có PID khác cha); trên máy thật mỗi lần chạy là một số khác. Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng). Lệnh shell khác (ls, dir, ping…) không chạy được ở đây. Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. ancestors(TABLE, 410) → chain = [410]
Khung 2 — Bước 2. parent_of[410] là 305 → thêm 305
Khung 3 — Bước 3. parent_of[305] là 120, parent_of[120] là 1 → thêm 120 rồi 1
Khung 4 — Bước 4. parent_of[1] là 0, không có trong bảng → dừng, trả [410, 305, 120, 1]
Khung đối chiếu — ngộ nhận "Program và process là một". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Program là mã thụ động; process là một lần thực thi đang hoạt động. Một program có thể có nhiều process cùng lúc (mở ba cửa sổ python là ba process), mỗi process có PID, stack, heap, file descriptor riêng. Vì vậy đếm process phải đếm từng DÒNG trong bảng ps, không đếm tên program.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: parent_of[row_ppid] = row_pid, row_ppid, name in program
Khung kết quả — 3 cặp vào/ra. count_processes(TABLE, "python") → 3; ancestors(TABLE, 410) → [410, 305, 120, 1]; ancestors(TABLE, 999) → ValueError

Nguồn: pe-v4 · python-learning-data.json#P01 · parser pe-v4/build · file build 001-P01.json

P02 · Vòng đời, fork, exec và wait

Mã bài: P02 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 2

MỤC TIÊU

Theo dõi vòng đời process; chờ con để lấy exit code.

CẦN NHỚ TRƯỚC

P01

BỐI CẢNH

Viết next_state(state, event) theo sơ đồ 5 trạng thái: new –admit→ ready, ready –dispatch→ running, running –interrupt→ ready, running –io_wait→ waiting, waiting –io_done→ ready, running –exit→ terminated; cặp (state, event) khác → ValueError. lifecycle(events) đã cấp, dùng next_state. Rồi viết run_child(code): tạo process con bằng subprocess.Popen([sys.executable, "-c", code]), chờ con kết thúc bằng wait() và trả exit code.

VÍ DỤ MINH HOẠ

next_state("waiting", "io_done") → "ready"; run_child("raise SystemExit(3)") → 3; run_child("pass") → 0.

LÝ THUYẾT

Định nghĩa: năm trạng thái

Slide "Process States" định nghĩa đúng năm trạng thái và các sự kiện chuyển. Trạng thái là chỗ process đang đứng; sự kiện là cái làm nó đi tiếp. Sơ đồ chỉ có sáu đường — mọi cặp khác không có nghĩa.

- new: process vừa được tạo
- ready: chờ được scheduler giao cho một processor
- running: đang thực thi lệnh
- waiting: đang chờ một sự kiện (thường là I/O) xảy ra
- terminated: đã chạy xong

Ví dụ: fork rồi exec trên UNIX

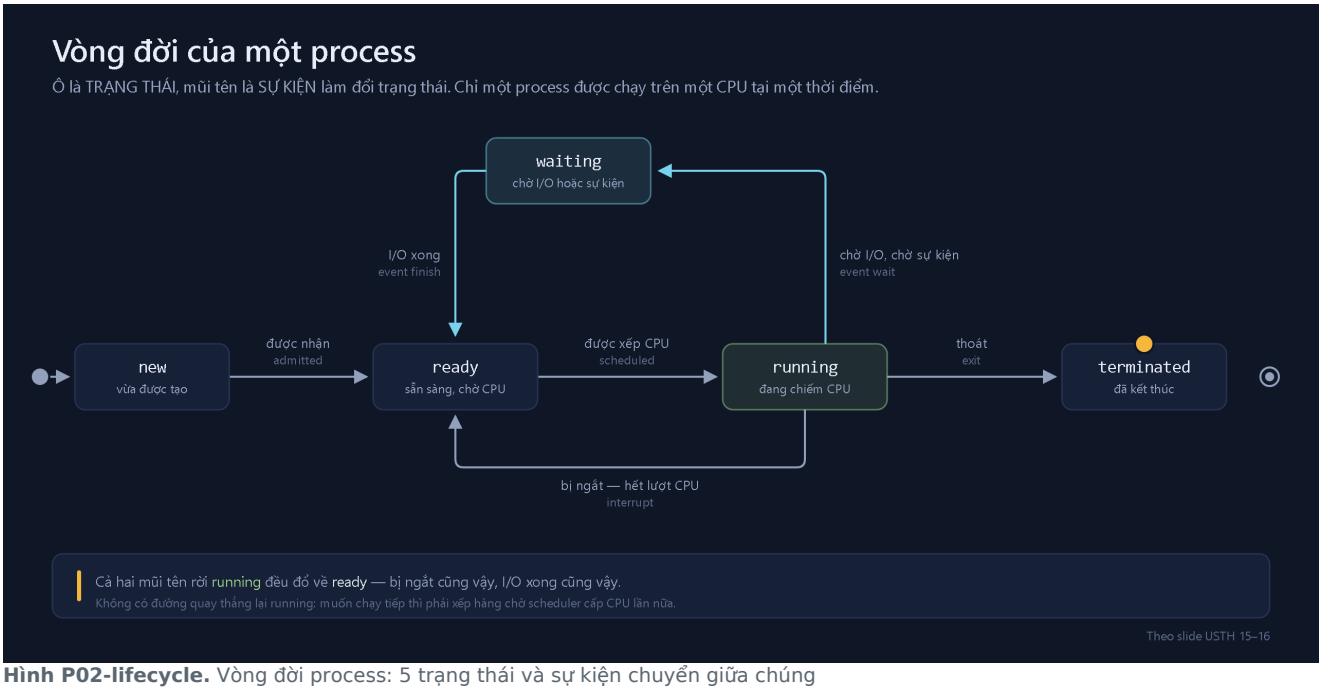
Slide nhấn: process mới KHÔNG được tạo từ con số không, mà qua hai bước. fork() nhân bản process đang chạy — open file, thanh ghi, vùng nhớ đều giống, TRỪ process id. Cha nhận về PID của con, con nhận 0, nhờ vậy if (pid == 0) tách được hai nhánh. Con gọi exec() để thay chương trình (PID giữ nguyên), còn cha gọi wait() để thu kết quả.

Dùng ở đâu

- subprocess trên Linux làm đúng việc đó: fork rồi exec trong process con
- Windows không có fork; nó tạo process bằng CreateProcess — WinExec là API cũ, đã deprecated
- wait() để lấy exit code, và để con đã xong không nằm lại thành zombie
- os.fork KHÔNG tồn tại trên Windows; trong Studio subprocess cũng chỉ là bản mô phỏng
- Mọi thứ chạy lệnh hộ bạn — shell, CI runner, máy chấm — đều dựng trên chuỗi tạo–chờ–thu mã này

Đọc sơ đồ

Ô là TRẠNG THÁI, chữ trên mũi tên là SỰ KIỆN. Chấm đen là lúc process ra đời, vòng tròn viền là lúc nó biến mất. Chú ý hai mũi tên rời khỏi running: interrupt (hết time slice) và io_wait (phải chờ). Cả hai nhánh đều quay về ready chứ không về thẳng running — ai chạy tiếp là quyền của scheduler.



Hình P02-lifecycle. Vòng đời process: 5 trạng thái và sự kiện chuyển giữa chúng

Mô tả hình: Sơ đồ new → ready → running; running → ready khi interrupt; running → waiting khi chờ I/O; waiting → ready khi I/O xong; running → terminated khi exit

Nguồn lý thuyết: SL 15-16, 22-29 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Vòng đời: new → ready (chờ tới lượt CPU) → running → terminated. Đang chạy mà phải chờ I/O thì sang waiting.
2. I/O xong → về ready, KHÔNG nhảy thẳng vào running: CPU có thể đang chạy process khác, scheduler mới quyết định ai chạy.
3. UNIX fork() nhân bản process: cha nhận PID của con, con nhận 0. exec() THAY chương trình trong chính process đó — PID giữ nguyên.
4. wait() cho cha chờ con kết thúc và thu exit code (con xong mà chưa được thu là zombie). Windows không có fork; nó tạo process bằng CreateProcess.
5. subprocess.Popen(...) tạo con rồi trả object NGAY, returncode còn None. wait() chặn tới khi con xong và trả exit code: 0, n của SystemExit(n), 1 nếu có ngoại lệ.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import subprocess
import sys

child = subprocess.Popen([sys.executable, "-c", "import time; time.sleep(0.5)"])
print("Vừa tạo con; returncode =", child.returncode)
code = child.wait()
print("wait() trả", code, "- returncode =", child.returncode, "- poll() =", child.poll())
```

Kết quả dự kiến (theo JSON): next_state("waiting", "io_done") → "ready"; run_child("raise SystemExit(3)") → 3

NGỘ NHẬN THƯỜNG GẶP: "exec() tạo ra một process mới"

exec() thay image chương trình của CHÍNH process đang chạy; PID không đổi. Tạo process mới là việc của fork() (UNIX) hay CreateProcess (Windows); subprocess trên Linux fork rồi exec trong con. Ngộ nhận đi kèm: đọc child.returncode ngay sau Popen để lấy exit code — lúc đó nó còn là None, phải wait() trước.

CÂU DỰ ĐOÁN

Ngay sau child = subprocess.Popen([...]) (chưa gọi wait/poll), child.returncode là gì?

sai	PID của con	PID nằm ở child.pid.
ĐÚNG	None — Popen trả object ngay, chưa ai thu trạng thái kết thúc của con	returncode chỉ được điền khi poll()/wait()/communicate() thấy con đã kết thúc.
sai	0	0 chỉ có sau khi con đã kết thúc thành công VÀ đã được thu.

TỰ KIỂM TRA

1. Sau os.fork() thành công, giá trị trả về trong process CON là?

✔ 0 — Cha nhận PID của con, con nhận 0 — nhờ vậy if pid == 0 tách được hai nhánh.

- PID của con — Đó là giá trị cha nhận.
- PID của cha — Con muốn biết cha thì gọi os.getppid().

Đáp án: Cha nhận PID của con, con nhận 0 — nhờ vậy if pid == 0 tách được hai nhánh.

2. Process đang waiting (chờ đọc đĩa) và I/O vừa xong. Nó chuyển sang...

✔ ready — xếp hàng chờ scheduler cho chạy tiếp — CPU có thể đang chạy process khác; I/O xong chỉ có nghĩa là SẴN SÀNG.

- terminated — Chờ I/O xong không có nghĩa là kết thúc.
- running ngay lập tức — Phải qua ready và được scheduler chọn (dispatch).

Đáp án: CPU có thể đang chạy process khác; I/O xong chỉ có nghĩa là SẴN SÀNG.

3. exec() có luôn tạo PID mới không?

- Có — exec không tạo thêm process.
- Chỉ trên Windows — Windows không có exec theo nghĩa UNIX; tạo process bằng CreateProcess.

✔ Không — nó thay chương trình của process hiện tại, PID giữ nguyên — Tạo process mới là fork()/CreateProcess.

Đáp án: Tạo process mới là fork()/CreateProcess.

TỰ GIẢI THÍCH

Vì sao I/O xong thì process về ready chứ không về running?

Mẫu trả lời: Trong lúc process chờ I/O, scheduler đã giao CPU cho process khác. Khi I/O xong, process chỉ SẴN SÀNG chạy tiếp nên phải xếp vào hàng ready chờ scheduler chọn lại (dispatch). Nhảy thẳng sang running là cướp CPU của process đang chạy — chỉ scheduler quyết định việc đó. Trường hợp đặc biệt: cặp không có trong sơ đồ (vd. waiting + dispatch) thì báo ValueError.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Popen([sys.executable, "-c", code]) → con được tạo, cha chạy tiếp ngay
Bước 2	child.returncode lúc này → None: chưa ai thu trạng thái
Bước 3	child.wait() → cha chặn tới khi con terminated
Bước 4	wait() trả về → exit code: 0, n của SystemExit(n), 1 nếu lỗi

CÂU PHẢN VÍ DỤ

run_child("import sys; sys.exit(7)") trả gì?

Lựa chọn: None | 0 | 7 · Đáp án: 7

TEMPLATE LẮP MẢNH

```
TRANSITIONS = {
    ("new", "admit"): "ready",
    ("ready", "dispatch"): "running",
    ("running", "interrupt"): "ready",
    ("running", "io_wait"): "waiting",
    ("waiting", "io_done"): {{slot1}},
    ("running", "exit"): "terminated",
}


def next_state(state, event):
    if (state, event) not in TRANSITIONS:
        {{slot2}}
    return TRANSITIONS[(state, event)]


def run_child(code):
    child = subprocess.Popen([sys.executable, "-c", code])
    return {{slot3}}
```

Slot	Mảnh đúng
[Slot 1]	`"ready"`
[Slot 2]	`raise ValueError("invalid transition")`
[Slot 3]	`child.wait()`

Mảnh bẫy	Vì sao sai
`return None`	Âm thầm trả None — lỗi chuyển trạng thái bị che mất.
`"running"`	I/O xong chưa chắc có CPU: phải về ready chờ dispatch.
`child.returncode`	Đọc ngay sau Popen là None: chưa ai chờ con.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp chuyển trạng thái hay nhầm, cách báo lỗi và lệnh chờ con.

Yêu cầu thêm: Thử chuỗi sự kiện có hai lần bị ngắt và một lần chờ I/O; thử con thoát bằng SystemExit(7) và con ném ZeroDivisionError.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết next_state(state, event) theo sơ đồ 5 trạng thái: new –admit→ ready, ready –dispatch→ running, running –interrupt→ ready, running –io_wait→ waiting, waiting –io_done→ ready, running –exit→ terminated; cặp (state, event) khác → ValueError. lifecycle(events) đã cấp, dùng next_state. Rồi viết run_child(code): tạo process con bằng subprocess.Popen([sys.executable, "-c", code]), chờ con kết thúc bằng wait() và trả exit code.

Thực hành: Viết sơ đồ thành dict {(state, event): trạng thái mới}; cặp không có trong dict thì raise ValueError. Với process con: Popen → wait() → trả giá trị của wait().

Gọi	Kết quả mong đợi
`next_state("waiting", "io_done")`	`"ready"`
`next_state("new", "dispatch")`	`ValueError`
`run_child("raise SystemExit(3)")`	`3`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`next_state(state, event)`	Một bước trong sơ đồ 5 trạng thái. · state, event: str · str trạng thái mới; cặp không hợp lệ → ValueError · Không in.
`run_child(code)`	Tạo process con chạy code bằng Python hiện tại và chờ nó. · code: str (mã Python) · int exit code của con · Con in gì thì in thẳng ra màn hình (không thu).

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
new --admit--> ready	`next_state("new", "admit")`	`'ready'`
I/O xong → ready, không phải running	`next_state("waiting", "io_done")`	`'ready'`
Bị ngắt (hết time slice) → ready	`next_state("running", "interrupt")`	`'ready'`
Vòng đời có chờ I/O và bị ngắt	`lifecycle(["admit", "dispatch", "interrupt", "dispatch", "io_wait", "io_done", "dispatch", "exit"])`	`['new', 'ready', 'running', 'ready', 'running', 'waiting', 'ready', 'running', 'terminated']`
Chuyển không hợp lệ → ValueError	`[_studio.call(next_state, s, e).error for s, e in [("new", "dispatch"), ("waiting", "dispatch"), ("terminated", "admit"), ("ready", "exit")]]`	`['ValueError'] * 4`
run_child("pass") → 0	`run_child("pass")`	`0`
SystemExit(3) → 3, sys.exit(7) → 7	`(run_child("raise SystemExit(3)"), run_child("import sys; sys.exit(7)"))`	`(3, 7)`
Ngoại lệ không bắt trong con → 1	`run_child("1/0")`	`1`
Dùng Popen rồi tự chờ (không gọi run)	`[c["kind"] for c in _studio.call(run_child, "pass").subprocess_calls]`	`['Popen']`

MÔI TRƯỜNG CHẠY

Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng). Lệnh shell khác (ls, dir, ping…) không chạy được ở đây. Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. Popen([sys.executable, "-c", code]) → con được tạo, cha chạy tiếp ngay
Khung 2 — Bước 2. child.returncode lúc này → None: chưa ai thu trạng thái
Khung 3 — Bước 3. child.wait() → cha chặn tới khi con terminated
Khung 4 — Bước 4. wait() trả về → exit code: 0, n của SystemExit(n), 1 nếu lỗi
Khung đối chiếu — ngộ nhận "exec() tạo ra một process mới". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. exec() thay image chương trình của CHÍNH process đang chạy; PID không đổi. Tạo process mới là việc của fork() (UNIX) hay CreateProcess (Windows); subprocess trên Linux fork rồi exec trong con. Ngộ nhận đi kèm: đọc child.returncode ngay sau Popen để lấy exit code — lúc đó nó còn là None, phải wait() trước.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: return None, "running", child.returncode
Khung kết quả — 3 cặp vào/ra. next_state("waiting", "io_done") → "ready"; next_state("new", "dispatch") → ValueError; run_child("raise SystemExit(3)") → 3

Nguồn: pe-v4 · python-learning-data.json#P02 · parser pe-v4/build · file build 002-P02.json

P03 · Scheduling và context switch

Mã bài: P03 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 3

MỤC TIÊU

Mô phỏng Round Robin và tính thời gian chờ.

CẦN NHỚ TRƯỚC

P02

BỐI CẢNH

Viết round_robin(jobs, quantum, switch=0): jobs là list (tên, burst) — burst nguyên dương, mọi job đến lúc 0, hàng ready theo đúng thứ tự list. Mỗi lượt lấy process đầu hàng, cho chạy min(quantum, phần còn lại) đơn vị và ghi (tên, bắt đầu, kết thúc) vào timeline; chưa xong thì về CUỐI hàng. Mỗi khi CPU chuyển sang một process KHÁC process vừa chạy, đồng hồ cộng thêm switch đơn vị (context switch, không ai làm việc). quantum <= 0 → ValueError; không sửa list jobs. Rồi viết waiting_times(jobs, timeline): dict tên → lúc hoàn thành − burst.

VÍ DỤ MINH HOẠ

jobs = [("P1", 5), ("P2", 3), ("P3", 1)], quantum = 2 → P1 0–2, P2 2–4, P3 4–5, P1 5–7, P2 7–8, P1 8–9; thời gian chờ {"P1": 4, "P2": 5, "P3": 4}.

LÝ THUYẾT

Định nghĩa: scheduler làm gì

Scheduler trả lời hai câu hỏi slide đặt ra: chạy process nào tiếp theo, và cho nó chạy bao lâu. Nó biết danh sách process, trạng thái và thông tin thống kê của từng process, rồi chọn theo độ ưu tiên.

- Preemption: hệ điều hành CƯỠNG BỨC dừng process đang chạy — Round Robin thuộc nhóm này
- Non-preemption (cooperation): process tự nguyện nhường CPU
- Short-term scheduler đổi process sau vài mili-giây; long-term tính bằng giây/phút cho batch job
- CPU-bound chủ yếu tính toán; I/O-bound chủ yếu chờ đĩa, mạng, máy in

Ví dụ: ba hàng đợi và bảng thuật toán

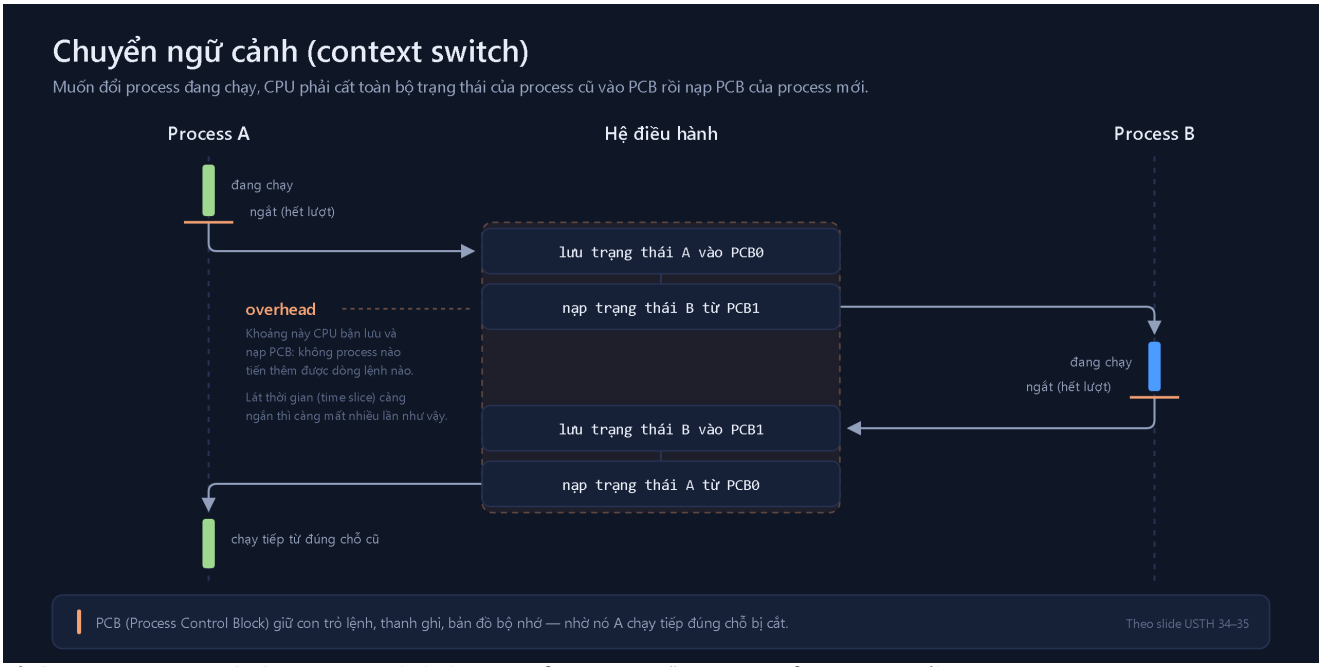
Slide chia process theo hàng đợi: job queue (vừa vào hệ thống, còn trên đĩa), ready queue (đã ở bộ nhớ chính, chờ CPU), device queue (chờ một thiết bị I/O) — đổi trạng thái là chuyển hàng. Bảng so sánh: FCFS và SJF không preempt; SRTF và Round Robin có; MLQ và MLFQ chia nhiều hàng con, khác nhau ở chỗ process có được chuyển hàng hay không.

Dùng ở đâu

- Giải thích vì sao máy chạy hàng trăm process trên vài lõi mà giao diện vẫn mượt
- Chọn quantum: nhỏ thì phản hồi nhanh nhưng tốn nhiều lần switch; lớn thì ngược lại
- Đọc số đo: bảng thuật toán cân nhắc thời gian chờ ω và thời gian phản hồi ρ
- round_robin của bài là MÔ PHỎNG bằng Python thuần, không đụng tới scheduler thật của máy

Đọc sơ đồ

Hai cột ngoài là hai process, cột giữa là hệ điều hành. A đang chạy thì có interrupt: CPU rời A, lưu ngữ cảnh của A vào PCB0, nạp ngữ cảnh của B từ PCB1, rồi B mới chạy. Khoảng tô đậm giữa hai đoạn mang nhãn "overhead" — trong khoảng đó không process nào làm được việc của mình, nên đổi càng nhiều lần càng tốn.



Hình P03-context-switch. Context switch: lưu PCB của process cũ, nạp PCB của process mới

Mô tả hình: Process A chạy, có interrupt, hệ điều hành lưu PCB0 rồi nạp PCB1, sau đó process B chạy; khoảng giữa ghi nhãn overhead

Nguồn lý thuyết: SL 30-43 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Một lõi CPU chỉ chạy một process tại một thời điểm. Xen kẽ đủ nhanh thì nhiều tác vụ cùng tiến triển (concurrency); nhiều lõi mới chạy song song thật.
2. Round Robin là preemptive: mỗi process được tối đa quantum (time slice); hết lượt mà chưa xong thì bị NGẮT và về CUỐI hàng ready.
3. FCFS là non-preemptive: process đầu hàng chạy tới hết rồi mới tới process sau.
4. Context switch lưu ngữ cảnh (program counter, thanh ghi — trong PCB) của process cũ, nạp của process mới. Lúc đó không ai làm việc: đó là overhead.
5. Thời gian chờ = lúc hoàn thành − burst (mọi job đến lúc 0). deque.popleft() lấy đầu hàng nhanh; list.pop(0) như ví dụ cũng đúng.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
ready = ["A", "B", "C"]
clock = 0
quantum = 3
for turn in range(4):
    current = ready.pop(0)
    clock = clock + quantum
    print(current, "chạy tới thời điểm", clock)
    ready.append(current)
```

Kết quả dự kiến (theo JSON): P1 5, P2 3, P3 1, quantum 2 → P1 0–2, P2 2–4, P3 4–5, …; switch = 1 → P1 0–2, P2 3–5, P1 6–7

NGỘ NHẬN THƯỜNG GẶP: "Hết quantum thì process vẫn chạy tiếp tới xong"

Đó là FCFS (non-preemptive). Round Robin là preemptive: hết time slice, scheduler NGẮT process, lưu ngữ cảnh và đưa nó về cuối hàng ready. Ngộ nhận thứ hai: context switch "miễn phí" — mỗi lần đổi process tốn thời gian lưu/nạp PCB mà không làm việc của ứng dụng, nên quantum quá nhỏ làm mọi job xong muộn hơn.

CÂU DỰ ĐOÁN

jobs = [("P1", 5), ("P2", 3), ("P3", 1)], quantum = 2. Process nào XONG ĐẦU TIÊN?

sai	P1 — nó đứng đầu hàng	Đứng đầu chỉ được chạy trước 2 đơn vị rồi bị ngắt.
sai	P2	P2 cần 3 > quantum nên lượt đầu chưa xong.
ĐÚNG	P3, lúc 5 — nó chỉ cần 1 đơn vị, tới lượt ở lần thứ ba	P1 0–2, P2 2–4, P3 4–5: P3 xong ngay trong lượt đầu của nó.

TỰ KIỂM TRA

1. Concurrency có bắt buộc CPU nhiều lõi không?
   - Có, ít nhất bằng số process — Hệ điều hành chạy hàng trăm process trên vài lõi.

• Có — Một lõi + time slice vẫn cho nhiều tác vụ cùng tiến triển.

✔ Không — một lõi xen kẽ các tác vụ cũng tạo concurrency; nhiều lõi mới chạy song song thật — Concurrency là cùng tiến triển; parallelism là cùng chạy một lúc.

Đáp án: Concurrency là cùng tiến triển; parallelism là cùng chạy một lúc.

2. Trong lúc context switch, CPU đang làm gì?

• Chạy cả hai process cùng lúc — Một lõi chỉ chạy một luồng lệnh tại một thời điểm.

✔ Lưu ngữ cảnh process cũ, nạp ngữ cảnh process mới (PCB) — không chạy code ứng dụng — Vì thế switch là overhead.

• Chạy tiếp process cũ — Process cũ vừa bị ngắt.

Đáp án: Vì thế switch là overhead.

3. Process phần lớn thời gian chờ đĩa/mạng gọi là gì?

• Preemptive — Preemptive là kiểu scheduler, không phải kiểu process.

✔ I/O-bound — Nó nằm trong waiting nhiều hơn running.

• CPU-bound — CPU-bound chủ yếu tính toán.

Đáp án: Nó nằm trong waiting nhiều hơn running.

TỰ GIẢI THÍCH

Khi switch > 0, vì sao giảm quantum làm các job xong MUỘN hơn?

Mẫu trả lời: Quantum nhỏ cắt mỗi job thành nhiều lượt hơn nên CPU đổi process nhiều lần hơn; mỗi lần đổi tốn switch đơn vị không làm việc của ai. Tổng burst giữ nguyên nhưng cộng thêm nhiều overhead, nên lúc hoàn thành và thời gian chờ đều tăng. Trường hợp đặc biệt: chỉ còn một process thì nó chạy các lượt liền nhau, không tốn switch.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	ready.popleft() → lấy process đầu hàng ready
Bước 2	đổi sang process khác → clock += switch (overhead)
Bước 3	run = min(quantum, left) → chạy tối đa một time slice
Bước 4	left > run → về cuối hàng với phần còn lại left - run

CÂU PHẢN VÍ DỤ

[("A", 4), ("B", 4)], quantum = 4: timeline là?

Lựa chọn: A 0–4, B 4–8 | B 0–4, A 4–8 | A 0–2, B 2–4, A 4–6, B 6–8 · Đáp án: A 0–4, B 4–8

TEMPLATE LẮP MẢNH

```
from collections import deque


def round_robin(jobs, quantum, switch=0):
    if quantum <= 0:
        raise ValueError("quantum must be positive")
    ready = deque(jobs)
    clock = 0
    last = None
    timeline = []
    while ready:
        name, left = ready.popleft()
        if last is not None and name != last:
            {{slot1}}
        run = {{slot2}}
        timeline.append((name, clock, clock + run))
        clock += run
        last = name
        if left > run:
            {{slot3}}
    return timeline
```

Slot	Mảnh đúng
[Slot 1]	`clock += switch`
[Slot 2]	`min(quantum, left)`
[Slot 3]	`ready.append((name, left - run))`

Mảnh bẫy	Vì sao sai
`ready.appendleft((name, left - run))`	Về ĐẦU hàng: process vừa bị ngắt lại chạy tiếp, process khác phải chờ mãi.
`clock += 1`	Chi phí switch là tham số, không cố định 1.

`left`	Chạy tới hết — thành FCFS, không còn preemptive.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ của vòng Round Robin.

Yêu cầu thêm: Thử quantum lớn hơn mọi burst (thành FCFS), quantum = 1, và switch = 1 khi một process phải chạy hai lượt liền nhau ở cuối.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết round_robin(jobs, quantum, switch=0): jobs là list (tên, burst) — burst nguyên dương, mọi job đến lúc 0, hàng ready theo đúng thứ tự list. Mỗi lượt lấy process đầu hàng, cho chạy min(quantum, phần còn lại) đơn vị và ghi (tên, bắt đầu, kết thúc) vào timeline; chưa xong thì về CUỐI hàng. Mỗi khi CPU chuyển sang một process KHÁC process vừa chạy, đồng hồ cộng thêm switch đơn vị (context switch, không ai làm việc). quantum <= 0 → ValueError; không sửa list jobs. Rồi viết waiting_times(jobs, timeline): dict tên → lúc hoàn thành − burst.

Thực hành: Hàng ready là deque các cặp (tên, phần còn lại). Mỗi lượt: popleft, cộng switch nếu đổi process, chạy min(quantum, phần còn lại), chưa xong thì append về cuối.

Gọi	Kết quả mong đợi
`round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2)`	`P1 0–2, P2 2–4, P3 4–5, P1 5–7, P2 7–8, P1 8–9`
`round_robin([("P1", 3), ("P2", 2)], 2, switch=1)`	`P1 0–2, P2 3–5, P1 6–7`
`round_robin([("A", 1)], 0)`	`ValueError`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`round_robin(jobs, quantum, switch=0)`	Mô phỏng Round Robin trên một lõi. · jobs: list (tên, burst > 0); quantum: int > 0; switch: int >= 0 · list (tên, bắt đầu, kết thúc) theo thứ tự chạy; quantum <= 0 → ValueError · Không sửa jobs, không in.
`waiting_times(jobs, timeline)`	Thời gian chờ của từng job. · jobs như trên; timeline do round_robin trả · dict tên → lúc hoàn thành − burst · Không in.

BỘ CHẤM (10 ca)

Ca	Gọi	Mong đợi
Ví dụ đề: quantum = 2	`round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2)`	`[('P1', 0, 2), ('P2', 2, 4), ('P3', 4, 5), ('P1', 5, 7), ('P2', 7, 8), ('P1', 8, 9)]`
quantum ≥ burst → như FCFS	`round_robin([("A", 4), ("B", 4)], 4)`	`[('A', 0, 4), ('B', 4, 8)]`
quantum = 1, ba job	`round_robin([("X", 2), ("Y", 1), ("Z", 2)], 1)`	`[('X', 0, 1), ('Y', 1, 2), ('Z', 2, 3), ('X', 3, 4), ('Z', 4, 5)]`
switch = 1 mỗi lần đổi process	`round_robin([("P1", 3), ("P2", 2)], 2, switch=1)`	`[('P1', 0, 2), ('P2', 3, 5), ('P1', 6, 7)]`
Cùng một process chạy liền hai lượt: không tốn switch	`round_robin([("P1", 5), ("P2", 1)], 2, switch=1)`	`[('P1', 0, 2), ('P2', 3, 4), ('P1', 5, 7), ('P1', 7, 8)]`
Thời gian chờ (ví dụ đề)	`waiting_times([("P1", 5), ("P2", 3), ("P3", 1)], round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2))`	`{'P1': 4, 'P2': 5, 'P3': 4}`
Thời gian chờ tính cả overhead switch	`waiting_times([("P1", 3), ("P2", 2)], round_robin([("P1", 3), ("P2", 2)], 2, switch=1))`	`{'P1': 4, 'P2': 3}`
Không có job → []	`round_robin([], 3)`	`[]`
quantum <= 0 → ValueError	`[_studio.call(round_robin, [("A", 1)], q).error for q in (0, -2)]`	`['ValueError', 'ValueError']`
Không sửa list jobs	`_studio.check('j = [("A", 3), ("B", 1)]\nround_robin(j, 1)\nassert j == [("A", 3), ("B", 1)], "round_robin không được sửa jobs"')`	`True`

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. ready.popleft() → lấy process đầu hàng ready
Khung 2 — Bước 2. đổi sang process khác → clock += switch (overhead)
Khung 3 — Bước 3. run = min(quantum, left) → chạy tối đa một time slice
Khung 4 — Bước 4. left > run → về cuối hàng với phần còn lại left - run
Khung đối chiếu — ngộ nhận "Hết quantum thì process vẫn chạy tiếp tới xong". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Đó là FCFS (non-preemptive). Round Robin là preemptive: hết time slice, scheduler NGẮT process, lưu ngữ cảnh và đưa nó về cuối hàng ready. Ngộ nhận thứ hai: context switch "miễn phí" — mỗi lần đổi process tốn thời gian lưu/nạp PCB mà không làm việc của ứng dụng, nên quantum quá nhỏ làm mọi job xong muộn hơn.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: ready.appendleft((name, left - run)), clock += 1, left
Khung kết quả — 3 cặp vào/ra. round_robin([("P1", 5), ("P2", 3), ("P3", 1)], 2) → P1 0–2, P2 2–4, P3 4–5, P1 5–7, P2 7–8, P1 8–9; round_robin([("P1", 3), ("P2", 2)], 2, switch=1) → P1 0–2, P2 3–5, P1 6–7; round_robin([("A", 1)], 0) → ValueError

Nguồn: pe-v4 · python-learning-data.json#P03 · parser pe-v4/build · file build 003-P03.json

P04 · stdin, stdout, stderr và pipe

Mã bài: P04 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 4

MỤC TIÊU

Phân biệt stdin, stdout, stderr và nối chúng với process con.

CẦN NHỚ TRƯỚC

P03

BỐI CẢNH

Hai phía của một process. (1) read_scores(): đọc HẾT các dòng từ stdin (sys.stdin); dòng rỗng bỏ qua, dòng là số nguyên thì giữ (int), dòng khác thì in "skip: <dòng đã strip>" ra STDERR — không ra stdout; trả list số. (2) run_sum(lines): chạy sum_numbers.py (đã cấp) làm process con bằng [sys.executable, "sum_numbers.py"], đưa các dòng vào stdin của con qua input= (mỗi dòng kết thúc bằng "\n"), thu riêng stdout và stderr dạng chuỗi; trả (tổng: int, list các dòng cảnh báo).

VÍ DỤ MINH HOẠ

stdin 5 / abc / -2 → read_scores() trả [5, -2] và stderr nhận "skip: abc". run_sum(["10", "x", "20"]) → (30, ["skip: x"]).

LÝ THUYẾT

Định nghĩa: ba luồng chuẩn

Mỗi process khi khởi động đã có sẵn ba luồng, mang đúng ba file descriptor đầu tiên. Mặc định cả ba nối với terminal — bàn phím vào, màn hình ra — nên trên màn hình chúng trông như một, dù thực ra là ba đường riêng, đổi đích được độc lập.

- stdin (0): đầu vào — input() và sys.stdin đọc từ đây
- stdout (1): KẾT QUẢ — print(x) ghi vào đây
- stderr (2): báo lỗi và cảnh báo — print(x, file=sys.stderr)
- Mỗi đầu có thể trỏ tới terminal, một file, hoặc một pipe sang process khác

Ví dụ: bốn cách nối của slide

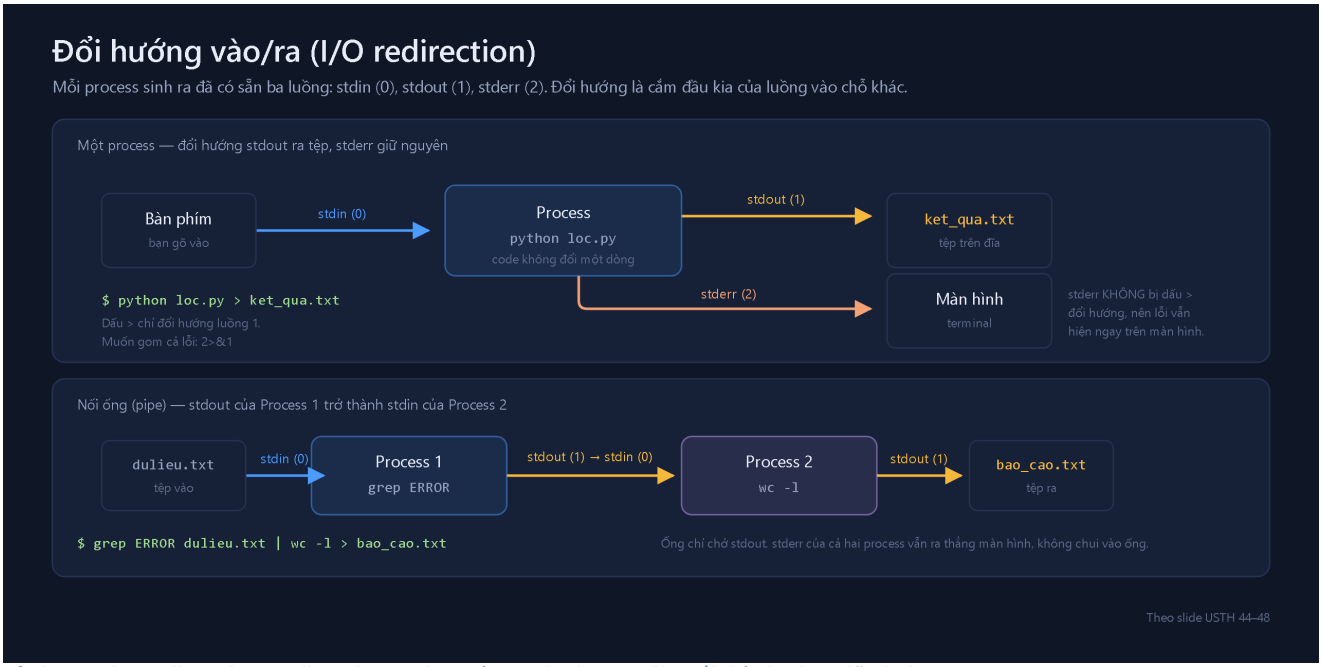
Slide vẽ cùng một hộp Process với bốn cấu hình. (1) Bàn phím → process → màn hình. (2) bc < input.txt: stdin lấy từ file. (3) ls -la > out.txt: stdout ghi ra file, còn stderr vẫn ra màn hình. (4) ps aux | grep term: stdout của process 1 thành stdin của process 2 qua một pipe, còn stderr của cả hai vẫn về màn hình.

Dùng ở đâu

- Pipeline shell và CI: mỗi công cụ đọc stdin, ghi kết quả ra stdout nên ghép được với nhau
- Giữ stderr riêng để log lỗi không trộn vào dữ liệu đang chuyển tiếp cho bước sau
- Trong Python: subprocess thay terminal bằng PIPE để cha đọc được đầu ra của con
- Studio không có shell nên không viết được | hay >; nó mô phỏng ba luồng riêng cho mỗi con

Đọc sơ đồ

Hộp giữa là process. Mũi tên vào bên trái là stdin (0); hai mũi tên ra là stdout (1) và stderr (2) — để ý stderr luôn tách thành một đường riêng, kể cả khi stdout đã đổi sang file. Phần dưới hình là biến thể pipe: stdout của process 1 cắm thẳng vào stdin của process 2, dữ liệu chảy qua bộ nhớ chứ không qua đĩa.



Hình P04-io-redirection. stdin/stdout/stderr: từ terminal, sang file, rồi thành pipe giữa hai process

Mô tả hình: Hộp process với stdin 0 vào, stdout 1 và stderr 2 ra; các biến thể nối tới bàn phím, màn hình, file và một pipe nối hai process

Nguồn lý thuyết: SL 44-48 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Mỗi process có ba luồng chuẩn: stdin (fd 0) — đầu vào, stdout (fd 1) — kết quả, stderr (fd 2) — báo lỗi. Trên terminal chúng trông lẫn vào nhau nhưng là ba luồng riêng.
2. input() và sys.stdin đọc fd 0; print(x) ghi stdout; print(x, file=sys.stderr) ghi stderr.
3. for line in sys.stdin đọc tới hết dữ liệu (EOF); mỗi dòng còn "\n" ở cuối nên phải strip().
4. Shell: < file làm stdin, > file nhận stdout, a | b nối stdout của a vào stdin của b bằng một pipe.
5. subprocess: input= đưa chuỗi vào stdin của con; capture_output=True tạo hai PIPE riêng; stderr=subprocess.STDOUT mới gộp. Thiếu text=True thì nhận bytes.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import subprocess
import sys

program = "import sys; print(5, flush=True); print('warn', file=sys.stderr)"
r = subprocess.run([sys.executable, "-c", program],
                   capture_output=True, text=True)
print("stdout:", repr(r.stdout))
print("stderr:", repr(r.stderr))

merged = subprocess.run([sys.executable, "-c", program],
                        stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
print("gộp:", repr(merged.stdout))
```

Kết quả dự kiến (theo JSON): stdin 5 / abc / -2 → [5, -2]; stderr: skip: abc; run_sum(["10", "x", "20"]) → (30, ["skip: x"])

NGỘ NHẬN THƯỜNG GẶP: "stderr cũng là stdout, in đâu cũng được"

stderr là luồng RIÊNG (fd 2). In cảnh báo ra stdout làm hỏng kết quả: chương trình đọc stdout của bạn (qua pipe, > out.txt, hay r.stdout) sẽ nhận cả "skip: abc" lẫn con số. Chỉ khi cấu hình gộp — 2>&1 trong shell, stderr=subprocess.STDOUT trong Python — hai luồng mới đi chung một đường.

CÂU DỰ ĐOÁN

Con chạy print(5) rồi print("warn", file=sys.stderr). Với capture_output=True, text=True, r.stdout là gì?

ĐÚNG	"5\n" — "warn" nằm riêng trong r.stderr	Hai luồng được thu bằng hai pipe khác nhau.
sai	"5\nwarn\n"	Chỉ gộp khi đặt stderr=subprocess.STDOUT.
sai	b"5\n"	text=True đã đổi bytes thành str.

TỰ KIỂM TRA

1. Lệnh shell ps aux | grep term làm gì với stdout của ps?
   - Ghi vào file tên grep — Ghi ra file là >.

• Gộp vào stderr — Pipe không đụng tới stderr.

✔ Nối nó vào stdin của grep qua một pipe — | là pipe giữa hai process.

Đáp án: | là pipe giữa hai process.

2. subprocess.run(..., capture_output=True) KHÔNG có text=True. r.stdout có kiểu gì?

• str — Cần text=True.

✔ bytes — Mặc định dữ liệu qua pipe là bytes; text=True (hoặc encoding=) mới cho str.

• list các dòng — Muốn list thì tự splitlines().

Đáp án: Mặc định dữ liệu qua pipe là bytes; text=True (hoặc encoding=) mới cho str.

3. for line in sys.stdin dừng khi nào?

✔ Khi hết dữ liệu (EOF): input= đã đọc hết, hoặc Ctrl+D (Linux) / Ctrl+Z rồi Enter (Windows) — Dòng rỗng giữa chừng vẫn là một dòng "\n", không làm dừng vòng lặp.

• Ở dòng rỗng đầu tiên — Dòng rỗng vẫn là một dòng.

• Sau đúng một dòng — Vòng for đọc tới EOF.

Đáp án: Dòng rỗng giữa chừng vẫn là một dòng "\n", không làm dừng vòng lặp.

TỰ GIẢI THÍCH

Vì sao "skip: abc" phải ra stderr chứ không ra stdout?

Mẫu trả lời: stdout là KẾT QUẢ mà bên khác đọc tiếp (qua pipe, > file, hay r.stdout trong subprocess). Nếu cảnh báo lẫn vào stdout, bên đọc gặp "skip: abc" thay vì con số và int() báo lỗi. Đặt cảnh báo ở stderr giữ stdout sạch mà người dùng vẫn thấy cảnh báo. Trường hợp đặc biệt: không có dòng hỏng thì stderr rỗng.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	input=data → các dòng đi vào stdin (fd 0) của con
Bước 2	con print(total) → con số đi vào pipe stdout
Bước 3	con print("skip: x", file=sys.stderr) → đi vào pipe stderr riêng
Bước 4	text=True → r.stdout và r.stderr là str, không phải bytes

CÂU PHẢN VÍ DỤ

run_sum(["4", "", "y", "6"]) trả gì?

Lựa chọn: (10, ["skip: y"]) | (10, []) | (4, ["skip: y"]) · Đáp án: (10, ["skip: y"])

TEMPLATE LẮP MẢNH

```
def read_scores():
    scores = []
    for line in sys.stdin:
        text = line.strip()
        if text == "":
            continue
        try:
            scores.append(int(text))
        except ValueError:
            {{slot1}}
    return scores


def run_sum(lines):
    data = "".join(line + "\n" for line in lines)
    result = subprocess.run([sys.executable, "sum_numbers.py"],
                            {{slot2}}, capture_output=True, {{slot3}})
    return int(result.stdout), result.stderr.splitlines()
```

Slot	Mảnh đúng
[Slot 1]	`print("skip:", text, file=sys.stderr)`
[Slot 2]	`input=data`
[Slot 3]	`text=True`

Mảnh bẫy	Vì sao sai
`stdin=data`	stdin= nhận PIPE hoặc file object; chuỗi phải đi qua input=.
`print("skip:", text)`	Ra stdout — lẫn vào kết quả.
`stderr=subprocess.STDOUT`	Không dùng chung với capture_output — và gộp thì int() gặp chữ.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp chỗ in cảnh báo và hai tham số của subprocess.run.

Yêu cầu thêm: Thử list rỗng, toàn dòng hỏng, số âm và dòng có khoảng trắng hai đầu; kiểm stdout chỉ chứa con số.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hai phía của một process. (1) read_scores(): đọc HẾT các dòng từ stdin (sys.stdin); dòng rỗng bỏ qua, dòng là số nguyên thì giữ (int), dòng khác thì in "skip: <dòng đã strip>" ra STDERR — không ra stdout; trả list số. (2) run_sum(lines): chạy sum_numbers.py (đã cấp) làm process con bằng [sys.executable, "sum_numbers.py"], đưa các dòng vào stdin của con qua input= (mỗi dòng kết thúc bằng "\n"), thu riêng stdout và stderr dạng chuỗi; trả (tổng: int, list các dòng cảnh báo).

Thực hành: Phía con: đọc sys.stdin, cảnh báo ra sys.stderr. Phía cha: input= đưa dữ liệu vào, capture_output=True tách hai pipe, text=True để có str.

Gọi	Kết quả mong đợi
`stdin: 5 / abc / -2`	`read_scores() → [5, -2]; stderr: skip: abc`
`run_sum(["10", "x", "20"])`	`(30, ["skip: x"])`
`run_sum([])`	`(0, [])`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`read_scores()`	Đọc số nguyên từ stdin của chính process. · không (đọc sys.stdin tới EOF) · list[int] · Dòng không phải số → "skip: <dòng>" ra stderr; stdout không in gì.
`run_sum(lines)`	Chạy sum_numbers.py làm process con và đọc kết quả. · lines: list[str] — mỗi phần tử một dòng · tuple (int tổng, list[str] cảnh báo) · Lệnh con: [sys.executable, "sum_numbers.py"]; dữ liệu qua input=.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
stdin 7, 8 → [7, 8]	`_studio.run(stdin=['7', '8'], entry='read_scores()').value`	`[7, 8]`
Bỏ dòng rỗng, giữ số âm, strip khoảng trắng	`_studio.run(stdin=['5', '', ' -2 ', '40'], entry='read_scores()').value`	`[5, -2, 40]`
Cảnh báo ra stderr, stdout sạch	`(lambda r: (r.ns['_got'], r.ns['_err'].getvalue(), r.stdout))(_studio.run(stdin=['5', 'abc', '3.5'], entry='import io, contextlib\n_err = io.StringIO()\nwith contextlib.redirect_stderr(_err):\n _got = read_scores()'))`	`([5], 'skip: abc\nskip: 3.5\n', '')`
stdin rỗng → []	`_studio.run(stdin=[], entry='read_scores()').value`	`[]`
run_sum: 10, x, 20	`run_sum(["10", "x", "20"])`	`(30, ['skip: x'])`
run_sum: số âm, nhiều dòng hỏng, dòng rỗng	`run_sum(["-4", "a", "b", "9", ""])`	`(5, ['skip: a', 'skip: b'])`
run_sum([]) → (0, [])	`run_sum([])`	`(0, [])`
Lệnh con là list [sys.executable, "sum_numbers.py"]	`[list(c["args"]) for c in _studio.call(run_sum, ["1"]).subprocess_calls]`	`[[__import__("sys").executable, "sum_numbers.py"]]`

MÔI TRƯỜNG CHẠY

Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng). Lệnh shell khác (ls, dir, ping…) không chạy được ở đây. Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật. File kèm bài: sum_numbers.py (module).

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. input=data → các dòng đi vào stdin (fd 0) của con
Khung 2 — Bước 2. con print(total) → con số đi vào pipe stdout
Khung 3 — Bước 3. con print("skip: x", file=sys.stderr) → đi vào pipe stderr riêng
Khung 4 — Bước 4. text=True → r.stdout và r.stderr là str, không phải bytes
Khung đối chiếu — ngộ nhận "stderr cũng là stdout, in đâu cũng được". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. stderr là luồng RIÊNG (fd 2). In cảnh báo ra stdout làm hỏng kết quả: chương trình đọc stdout của bạn (qua pipe, > out.txt, hay r.stdout) sẽ nhận cả "skip: abc" lẫn con số. Chỉ khi cấu hình gộp — 2>&1 trong shell, stderr=subprocess.STDOUT trong Python — hai luồng mới đi chung một đường.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: stdin=data, print("skip:", text), stderr=subprocess.STDOUT
Khung kết quả — 3 cặp vào/ra. stdin: 5 / abc / -2 → read_scores() → [5, -2]; stderr: skip: abc; run_sum(["10", "x", "20"]) → (30, ["skip: x"]); run_sum([]) → (0, [])

Nguồn: pe-v4 · python-learning-data.json#P04 · parser pe-v4/build · file build 004-P04.json

P05 · subprocess.run và kết quả

Mã bài: P05 · Trung cấp · M1 · Tiến trình và subprocess · 40 phút · 15 XP · thứ tự 5

MỤC TIÊU

Chạy process con bằng subprocess.run và đọc đúng kết quả.

CẦN NHỚ TRƯỚC

P04

BỐI CẢNH

Viết ba hàm, mỗi hàm chạy code Python trong một process con bằng [sys.executable, "-c", code]. exit_code(code) trả exit code (int) của con. output_of(code) trả stdout của con dạng str bằng subprocess.check_output (con thoát mã khác 0 thì để CalledProcessError bay ra). grade(code, expected, limit=2) là máy chấm mini: chạy con với capture_output=True, text=True, timeout=limit — quá giờ trả "TIMEOUT"; exit code khác 0 trả "ERROR <mã>"; stdout bỏ khoảng trắng hai đầu bằng expected trả "PASS"; còn lại "FAIL".

VÍ DỤ MINH HOẠ

grade("print(2+3)", "5") → "PASS"; grade("raise SystemExit(2)", "") → "ERROR 2"; output_of("print(7)") → "7\n"; exit_code("pass") → 0.

LÝ THUYẾT

Định nghĩa: run() là "chạy và chờ"

Bảng subprocess của slide xếp run() vào đúng ô "Run and wait": gọi run(args) là cha dừng lại tới khi con kết thúc, rồi nhận về một CompletedProcess. Đây là cách đơn giản nhất để chạy một chương trình khác từ trong Python.

- args là LIST: phần tử đầu là chương trình, các phần tử sau là từng đối số
- CompletedProcess mang returncode, stdout, stderr
- capture_output=True mới thu đầu ra; text=True mới cho str thay vì bytes
- check=True biến exit code khác 0 thành CalledProcessError; timeout= đặt hạn chờ

Ví dụ: exit code là lời khai của con

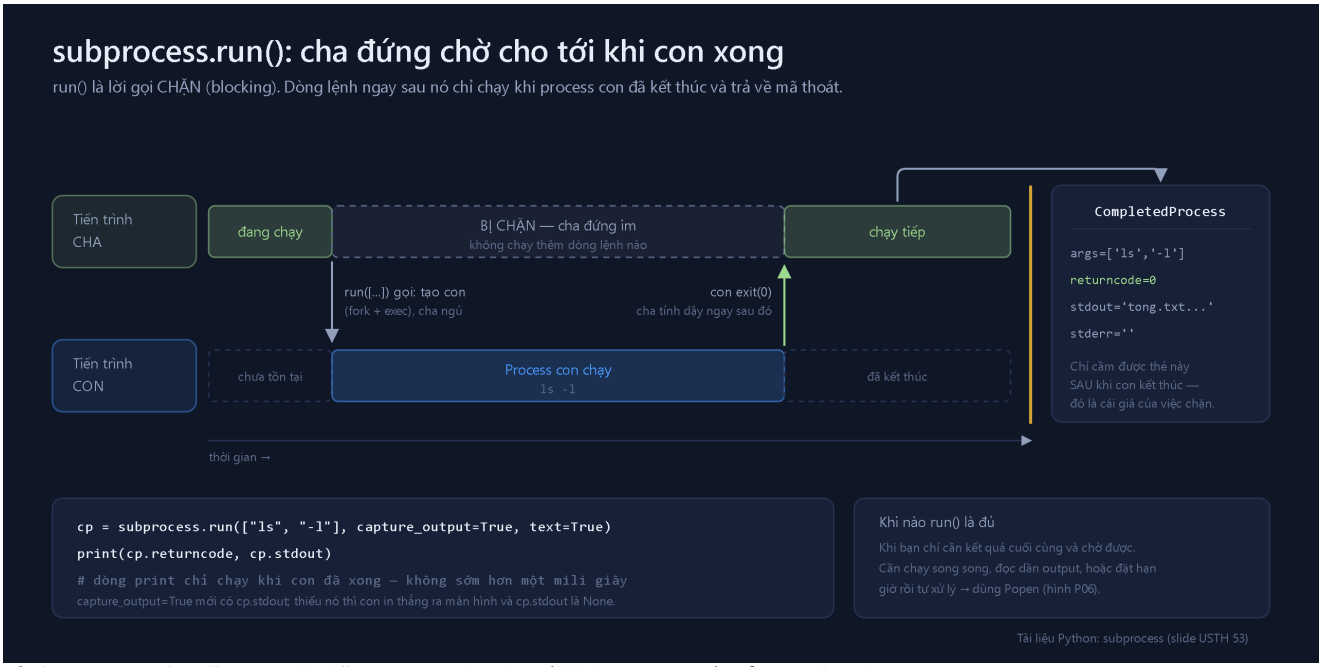
Mỗi process khi kết thúc trả về hệ điều hành một số nguyên. Quy ước: 0 là thành công, khác 0 là có chuyện. Python thoát mã n khi gặp SystemExit(n) và mã 1 khi có ngoại lệ không bắt. Cha KHÔNG nhận được object exception của con — chỉ nhận con số đó, còn traceback nằm trong stderr của con.

Dùng ở đâu

- Gọi công cụ ngoài từ Python: git, ffmpeg, pdflatex, hay một script Python khác
- Máy chấm bài và CI: chạy bài nộp, so stdout, lấy returncode làm tín hiệu đạt/trượt
- timeout= chặn chương trình lặp vô hạn — run() tự kill con rồi mới ném TimeoutExpired
- Dùng sys.executable thay chữ "python": trên Windows "python" có thể là stub của Microsoft Store
- Studio mô phỏng subprocess; time.sleep của con chỉ đẩy đồng hồ ảo nên vẫn thử timeout được

Đọc sơ đồ

Hai đường thời gian: cha ở trên, con ở dưới. Lúc gọi run(), đường của cha chuyển sang nét đứt — cha bị CHẶN, không làm gì khác. Đường của con chạy liền mạch tới exit. Khi con kết thúc, mũi tên quay lên mang theo hộp CompletedProcess với ba ô returncode, stdout, stderr; từ đó cha mới chạy tiếp.



Hình P05-run-timeline. run() chặn process cha cho tới khi con xong và trả CompletedProcess
Mô tả hình: Đường thời gian: cha bị chặn trong khi con chạy; con exit thì cha nhận CompletedProcess gồm returncode, stdout, stderr
Nguồn lý thuyết: SL 51-53; DOC subprocess (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. subprocess.run(args, ...) chạy con và CHỜ nó xong, trả CompletedProcess có returncode, stdout, stderr.
2. args là LIST: phần tử đầu là chương trình — sys.executable là đúng Python đang chạy, không phụ thuộc "python" trên PATH — rồi tới từng đối số.
3. Mặc định con in thẳng ra màn hình và result.stdout là None. capture_output=True mới thu; text=True để nhận str thay vì bytes.
4. Exit code khác 0 KHÔNG tự thành exception: tự kiểm returncode, hoặc check=True để nhận CalledProcessError (có .returncode). timeout= quá giờ → TimeoutExpired.
5. check_output(args, text=True) trả STDOUT của con, không phải exit code, và tự check: mã khác 0 → CalledProcessError.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import subprocess
import sys

result = subprocess.run([sys.executable, "-c", "print(2+3)"],
                        capture_output=True, text=True, check=True)
print(repr(result.stdout), result.returncode)

try:
    subprocess.run([sys.executable, "-c", "raise SystemExit(2)"], check=True)
except subprocess.CalledProcessError as error:
    print("exit code:", error.returncode)

print(repr(subprocess.check_output([sys.executable, "-c", "print('hi')"], text=True)))
```

Kết quả dự kiến (theo JSON): grade("print(2+3)", "5") → PASS; grade("raise SystemExit(2)", "") → ERROR 2

NGỘ NHẬN THƯỜNG GẶP: "check_output trả exit code"

check_output trả STDOUT của con. Viết if subprocess.check_output(...) == 0 là so chuỗi/bytes với 0 — luôn False; còn khi con thoát mã khác 0, check_output ném CalledProcessError chứ không trả mã. Lấy exit code bằng run(...).returncode, hoặc error.returncode trong except. Ngộ nhận anh em: tưởng exit code khác 0 tự thành lỗi — run() không ném gì nếu thiếu check=True.

CÂU DỰ ĐOÁN

subprocess.check_output([sys.executable, "-c", "print(1)"], text=True) trả về gì?

sai	0 — exit code	Đó là ngộ nhận phổ biến nhất về check_output.
ĐÚNG	"1\n" — stdout của con	check_output trả đầu ra; exit code nằm ở run().returncode hoặc trong exception.
sai	Một CompletedProcess	Đó là kiểu trả về của run().

TỰ KIỂM TRA

1. subprocess.run([sys.executable, "-c", "raise SystemExit(3)"]) (không có check) thì...

• Ném SystemExit trong process cha — SystemExit xảy ra trong process CON; cha chỉ thấy exit code.

✔ Trả CompletedProcess với returncode = 3, không ném lỗi — Chỉ check=True mới biến mã khác 0 thành CalledProcessError.

• Ném CalledProcessError — Cần check=True.

Đáp án: Chỉ check=True mới biến mã khác 0 thành CalledProcessError.

2. Vì sao viết [sys.executable, "-c", code] thay vì chuỗi "python -c " + code?

• Hai cách như nhau — Code có dấu cách hay dấu ngoặc kép sẽ bị tách sai.

✔ List giao từng đối số nguyên vẹn, không cần shell tách chuỗi; sys.executable chắc chắn là Python đang chạy — Chuỗi không có shell=True bị coi là TÊN chương trình trên Linux; "python" có thể là bản khác, hoặc stub của Microsoft Store trên Windows.

• Chuỗi chạy chậm hơn — Không phải chuyện tốc độ.

Đáp án: Chuỗi không có shell=True bị coi là TÊN chương trình trên Linux; "python" có thể là bản khác, hoặc stub của Microsoft Store trên Windows.

3. Con chạy 1/0 (không bắt lỗi). returncode là?

• 0 — Có lỗi thì không phải 0.

• ZeroDivisionError — Cha không nhận object exception của con, chỉ nhận mã thoát.

✔ 1 — ngoại lệ không bắt làm Python thoát mã 1, traceback ra stderr — Traceback nằm trong result.stderr nếu có capture.

Đáp án: Traceback nằm trong result.stderr nếu có capture.

TỰ GIẢI THÍCH

grade kiểm returncode TRƯỚC khi so stdout. Vì sao?

Mẫu trả lời: Con có thể in đúng "5" rồi mới gặp lỗi và thoát mã khác 0 — chỉ so stdout sẽ chấm PASS một chương trình hỏng. Exit code là tín hiệu thành công/thất bại chính thức của process; chỉ khi nó bằng 0 mới so đầu ra. Trường hợp đặc biệt: chạy quá limit thì không có exit code bình thường, trả TIMEOUT.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	run([sys.executable, "-c", code]) → cha chờ con kết thúc
Bước 2	quá limit giây → TimeoutExpired → "TIMEOUT"
Bước 3	returncode != 0 → "ERROR <mã>"
Bước 4	stdout.strip() == expected → "PASS", khác thì "FAIL"

CÂU PHẢN VÍ DỤ

grade("print(5); raise SystemExit(3)", "5") trả gì?

Lựa chọn: PASS | FAIL | ERROR 3 · Đáp án: ERROR 3

TEMPLATE LẮP MẢNH

```
def grade(code, expected, limit=2):
    try:
        result = subprocess.run([sys.executable, "-c", code],
                                capture_output=True, text=True, {{slot1}})
    except {{slot2}}:
        return "TIMEOUT"
    if {{slot3}}:
        return f"ERROR {result.returncode}"
    if result.stdout.strip() == expected:
        return "PASS"
    return "FAIL"
```

Slot	Mảnh đúng
[Slot 1]	`timeout=limit`
[Slot 2]	`subprocess.TimeoutExpired`
[Slot 3]	`result.returncode != 0`

Mảnh bẫy	Vì sao sai
`result.stderr`	Có chữ trong stderr chưa chắc là lỗi; tín hiệu chính thức là returncode.
`check=True`	Mã khác 0 thành CalledProcessError — không ai bắt, và mất timeout.
`subprocess.CalledProcessError`	Không có check=True thì run() không ném lỗi này; quá giờ là TimeoutExpired.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp máy chấm mini.

Yêu cầu thêm: Thử code in nhiều dòng, code in đúng rồi mới thoát mã 3, và code ngủ time.sleep(5) với limit = 1.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết ba hàm, mỗi hàm chạy code Python trong một process con bằng [sys.executable, "-c", code]. exit_code(code) trả exit code (int) của con. output_of(code) trả stdout của con dạng str bằng subprocess.check_output (con thoát mã khác 0 thì để CalledProcessError bay ra). grade(code, expected, limit=2) là máy chấm mini: chạy con với capture_output=True, text=True, timeout=limit — quá giờ trả "TIMEOUT"; exit code khác 0 trả "ERROR <mã>"; stdout bỏ khoảng trắng hai đầu bằng expected trả "PASS"; còn lại "FAIL".

Thực hành: Một lần run() là đủ cho grade: capture_output=True, text=True, timeout=limit. Bắt TimeoutExpired, rồi kiểm returncode, cuối cùng mới so stdout.

Gọi	Kết quả mong đợi
`grade("print(2+3)", "5")`	`"PASS"`
`grade("raise SystemExit(2)", "")`	`"ERROR 2"`
`grade("import time; time.sleep(5)", "", 1)`	`"TIMEOUT"`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`exit_code(code)`	Exit code của process con chạy code. · code: str · int · Con in thẳng ra màn hình (không thu).
`output_of(code)`	stdout của process con. · code: str · str; con thoát mã khác 0 → CalledProcessError · Dùng subprocess.check_output(..., text=True).
`grade(code, expected, limit=2)`	Máy chấm mini: chạy con, so đầu ra. · code, expected: str; limit: giây · "TIMEOUT" | "ERROR <mã>" | "PASS" | "FAIL" · Một lần subprocess.run(capture_output=True, text=True, timeout=limit).

BỘ CHẤM (12 ca)

Ca	Gọi	Mong đợi
exit_code("pass") → 0	`exit_code("pass")`	`0`
exit_code: SystemExit(4) → 4, 1/0 → 1	`(exit_code("raise SystemExit(4)"), exit_code("1/0"))`	`(4, 1)`
output_of trả stdout (str)	`output_of("print(6 * 7)")`	`'42\n'`
output_of: nhiều dòng	`output_of("for i in range(3): print(i)")`	`'0\n1\n2\n'`
output_of: con thoát mã 5 → CalledProcessError	`_studio.call(output_of, "raise SystemExit(5)").error`	`'CalledProcessError'`
grade: PASS và FAIL	`(grade("print(2+3)", "5"), grade("print(6)", "5"))`	`('PASS', 'FAIL')`
grade: in đúng rồi thoát mã 3 → ERROR 3	`grade("print(5); raise SystemExit(3)", "5")`	`'ERROR 3'`
grade: ngoại lệ trong con → ERROR 1	`grade("1/0", "x")`	`'ERROR 1'`
grade: quá giờ → TIMEOUT	`grade("import time; time.sleep(5)", "", 1)`	`'TIMEOUT'`
grade: ngủ ít hơn limit mặc định vẫn chấm	`grade("import time; time.sleep(1); print('ok')", "ok")`	`'PASS'`
grade: nhiều dòng, chỉ bỏ khoảng trắng hai đầu	`grade("print(1); print(2)", "1\n2")`	`'PASS'`
Lệnh con là list [sys.executable, "-c", code], không phải chuỗi	`(lambda a: (isinstance(a, (list, tuple)), list(a)))(_studio.call(exit_code, "raise SystemExit(4)").subprocess_calls[0]["args"])`	`(True, [__import__("sys").executable, "-c", "raise SystemExit(4)"])`

MÔI TRƯỜNG CHẠY

Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng). Lệnh shell khác (ls, dir, ping...) không chạy được ở đây. Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. run([sys.executable, "-c", code]) → cha chờ con kết thúc
Khung 2 — Bước 2. quá limit giây → TimeoutExpired → "TIMEOUT"
Khung 3 — Bước 3. returncode != 0 → "ERROR <mã>"
Khung 4 — Bước 4. stdout.strip() == expected → "PASS", khác thì "FAIL"
Khung đối chiếu — ngộ nhận "check_output trả exit code". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. check_output trả STDOUT của con. Viết if subprocess.check_output(...) == 0 là so chuỗi/bytes với 0 — luôn False; còn khi con thoát mã khác 0, check_output ném CalledProcessError chứ không trả mã. Lấy exit code bằng run(...).returncode, hoặc error.returncode trong except. Ngộ nhận anh em: tưởng exit code khác 0 tự thành lỗi — run() không ném gì nếu thiếu check=True.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: result.stderr, check=True, subprocess.CalledProcessError
Khung kết quả — 3 cặp vào/ra. grade("print(2+3)", "5") → "PASS"; grade("raise SystemExit(2)", "") → "ERROR 2"; grade("import time; time.sleep(5)", "", 1) → "TIMEOUT"

Nguồn: pe-v4 · python-learning-data.json#P05 · parser pe-v4/build · file build 005-P05.json

P06 · Popen, communicate và timeout

Mã bài: P06 · Trung cấp · M1 · Tiến trình và subprocess · 40 phút · 15 XP · thứ tự 6

MỤC TIÊU

Điều khiển process con bằng Popen: gửi input, chờ có giới hạn, dọn dẹp khi quá giờ.

CẦN NHỚ TRƯỚC

P05

BỐI CẢNH

Viết talk(code, data, limit=2): mở process con bằng subprocess.Popen([sys.executable, "-c", code]) với stdin, stdout, stderr đều là subprocess.PIPE và text=True; gửi data vào stdin và chờ bằng communicate(data, timeout=limit). Xong kịp thì trả ("ok", stdout, returncode). Quá giờ (TimeoutExpired) thì KHÔNG bỏ mặc con: kill() rồi communicate() thêm một lần để thu nốt phần đã in và thu hồi con; trả ("timeout", phần stdout thu được, returncode).

VÍ DỤ MINH HOẠ

talk("print(input().upper())", "hello\n") → ("ok", "HELLO\n", 0). Con in "start" rồi ngủ 10 giây, limit = 1 → ("timeout", "start\n", mã khác 0).

LÝ THUYẾT

Định nghĩa: Popen là "chạy nền"

Trong bảng subprocess của slide, Popen đứng ở ô "Run in background": nó khởi động con rồi trả object NGAY, cha không chờ. run() thực ra được xây trên Popen — gói sẵn phần giao tiếp và chờ, nên tiện nhưng không cho bạn xen vào giữa.

- stdin/stdout/stderr=subprocess.PIPE mở đường nói chuyện; luồng không khai thì nhận None
- communicate(input, timeout): gửi input, đóng stdin, đọc hết hai luồng, chờ con xong
- poll() hỏi "xong chưa" mà không chặn; wait() chặn tới khi xong
- terminate() xin con dừng tử tế; kill() ép dừng ngay

Ví dụ: timeout không giết ai

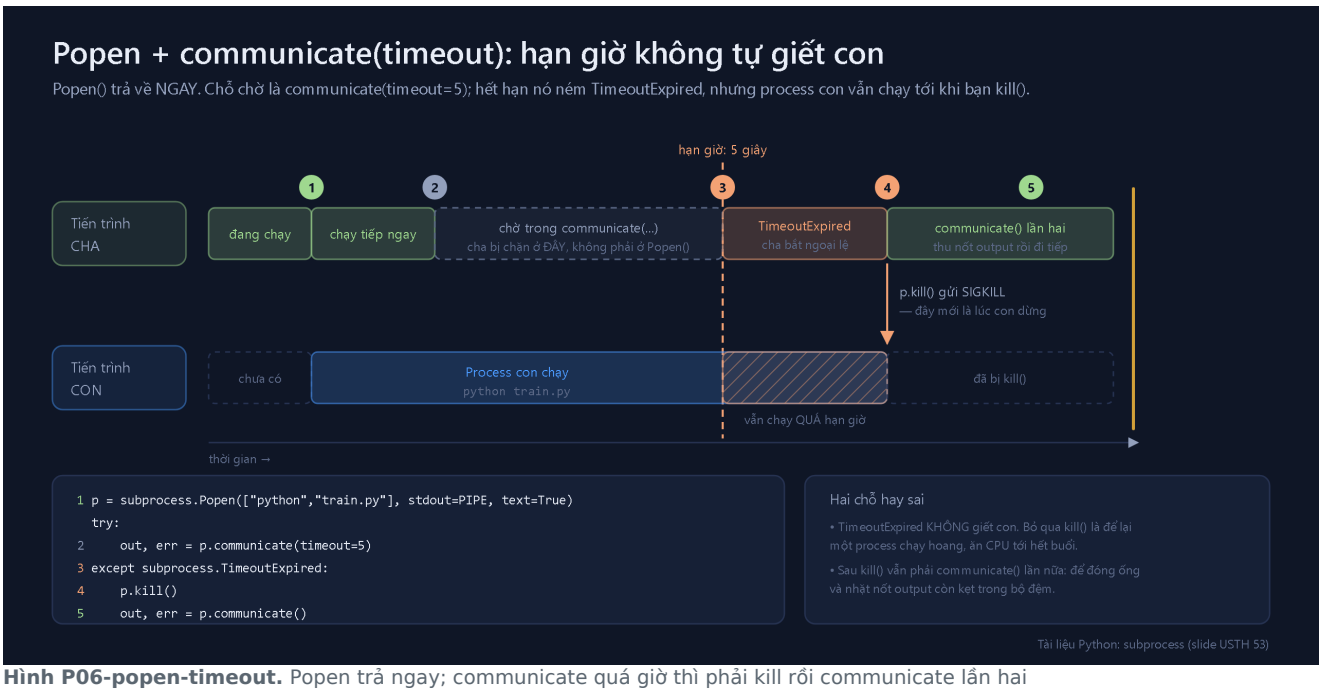
Tài liệu Python nói thẳng: communicate(timeout=...) ném TimeoutExpired nhưng KHÔNG đụng tới process con — con vẫn chạy, vẫn giữ pipe. Mẫu chuẩn trong tài liệu là bắt lỗi, gọi kill(), rồi gọi communicate() lần thứ hai để đọc nốt phần con đã in và thu hồi con. Khác hẳn run(timeout=...), vốn tự dọn con trước khi ném lỗi.

Dùng ở đâu

- Chạy việc dài ở nền trong khi cha vẫn làm việc khác, thỉnh thoảng poll() xem xong chưa
- Đối thoại nhiều lượt với một công cụ qua pipe (bc, một interpreter, một server con)
- Đặt hạn giờ cho mã lạ: quá hạn thì kill để không rò process và không kẹt pipe
- Mã thoát khi bị kill tuỳ hệ điều hành: -9 trên Linux (và trong Studio), 1 trên Windows

Đọc sơ đồ

Đọc theo trục thời gian ngang. Popen trả về ngay ở mốc đầu nên hai đường cha và con chạy song song. Hộp communicate(timeout) có một vạch hạn giờ: chạm vạch là TimeoutExpired, nhưng đường của con VẪN kéo dài qua vạch đó — đúng chỗ dễ nhầm. Chỉ sau kill() đường con mới dứt, và communicate() lần hai mới kéo nốt stdout còn trong pipe về cho cha.



Hình P06-popen-timeout. Popen trả ngay; communicate quá giờ thì phải kill rồi communicate lần hai

Mô tả hình: Trục thời gian: Popen trả ngay, communicate(timeout) ném TimeoutExpired nhưng con vẫn chạy, kill() dừng con, communicate() lần hai thu nốt stdout

Nguồn lý thuyết: SL 51-53; DOC subprocess (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Popen(args, ...) khởi động con rồi trả object NGAY — con chạy song song với cha. run() = Popen + communicate + chờ, gói sẵn.
2. communicate(input, timeout) gửi input vào stdin, đóng stdin, đọc hết stdout/stderr tới khi con xong, trả (stdout, stderr). Luồng không đặt PIPE → None.
3. poll() hỏi "xong chưa?" mà không chặn (None = còn chạy); wait() chặn tới khi xong; terminate()/kill() yêu cầu hệ điều hành dừng con.
4. Bị kill thì returncode khác 0 nhưng tuỳ hệ điều hành: -9 trên Linux (Studio cũng vậy), 1 trên Windows.
5. communicate(timeout=...) ném TimeoutExpired nhưng KHÔNG giết con. Mẫu chuẩn trong tài liệu Python: kill() rồi communicate() lần nữa để thu nốt dữ liệu.
6. run(timeout=...) thì tự kill và chờ con trước khi ném TimeoutExpired — Popen để bạn tự làm việc đó.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import subprocess
import sys

p = subprocess.Popen([sys.executable, "-c", "print(input()[::-1])"],
                     stdin=subprocess.PIPE, stdout=subprocess.PIPE, text=True)
print("Popen trả ngay, poll() =", p.poll())
out, err = p.communicate("hello\n", timeout=5)
print(repr(out), err, p.returncode)

slow = subprocess.Popen([sys.executable, "-c", "import time; time.sleep(10)"])
try:
    slow.communicate(timeout=0.5)
except subprocess.TimeoutExpired:
    slow.kill()
    slow.communicate()
    print("Quá giờ, đã kill; returncode =", slow.returncode)
```

Kết quả dự kiến (theo JSON): print(input().upper()) với "hello" → ("ok", "HELLO↵", 0); In "start" rồi ngủ 10 s, limit 1 → ("timeout", "start↵", mã ≠ 0)

NGỘ NHẬN THƯỜNG GẶP: "Bắt TimeoutExpired là xong việc"

Với Popen.communicate(timeout=...), TimeoutExpired không tự kill process: con vẫn chạy, vẫn giữ pipe và CPU; bỏ mặc nó là rò process. Phải kill() (hoặc terminate()) rồi gọi communicate() thêm một lần để thu phần output còn lại và thu hồi con — lúc đó returncode mới có giá trị.

CÂU DỰ ĐOÁN

p.communicate(timeout=1) ném TimeoutExpired. Lúc đó process con thế nào?

sai	Đã bị kill tự động	Đó là hành vi của run(timeout=...), không phải Popen.communicate.

sai	Đã kết thúc bình thường	Nếu đã kết thúc thì communicate trả kết quả chứ không ném.
ĐÚNG	Vẫn đang chạy — phải tự kill() rồi communicate() lại	TimeoutExpired chỉ báo cha đã chờ hết giờ; nó không đụng vào con.

TỰ KIỂM TRA

1. Popen() và run() khác nhau chính ở đâu?

• Popen không chạy được Python — Cả hai nhận cùng kiểu args.

• run chạy song song, Popen chạy tuần tự — Ngược lại: Popen không chờ.

✔ run chờ xong rồi trả kết quả; Popen trả object ngay để bạn tự điều khiển (gửi input, poll, kill…) — run được xây trên Popen.

Đáp án: run được xây trên Popen.

2. Ngay sau proc = Popen(...) (con đang chờ input), proc.poll() trả?

• Chặn tới khi con xong — Đó là wait().

✔ None — con còn chạy — poll không chặn; None nghĩa là chưa kết thúc.

• 0 — 0 chỉ có khi con đã kết thúc thành công.

Đáp án: poll không chặn; None nghĩa là chưa kết thúc.

3. Muốn communicate trả stdout dạng str (không phải bytes) cần gì?

• Không cần gì — Mặc định stdout không bị thu và pipe là bytes.

✔ stdout=subprocess.PIPE và text=True khi tạo Popen — Thiếu PIPE thì phần stdout là None; thiếu text=True thì là bytes.

• Gọi communicate(text=True) — communicate không có tham số text.

Đáp án: Thiếu PIPE thì phần stdout là None; thiếu text=True thì là bytes.

TỰ GIẢI THÍCH

Sau TimeoutExpired, vì sao phải gọi communicate() THÊM một lần sau kill()?

Mẫu trả lời: kill() chỉ yêu cầu hệ điều hành dừng con. communicate() lần hai đọc nốt dữ liệu còn trong pipe (vd. "start" con đã in trước khi bị giết) và chờ con thật sự kết thúc để thu hồi, nhờ đó returncode có giá trị và không còn process mồ côi. Trường hợp đặc biệt: con xong kịp thì không vào nhánh này.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Popen(...) → con chạy, cha nhận proc ngay
Bước 2	communicate(data, timeout=limit) → gửi data, đọc tới khi con xong
Bước 3	quá limit → TimeoutExpired, con VẪN chạy
Bước 4	kill() rồi communicate() → thu nốt output, returncode có giá trị

CÂU PHẢN VÍ DỤ

talk("import sys; print(len(sys.stdin.read().split()))", "a b c\n") trả gì?

Lựa chọn: ("ok", "1\n", 0) | ("ok", "3\n", 0) | ("timeout", "", 1) · Đáp án: ("ok", "3\n", 0)

TEMPLATE LẮP MẢNH

```
def talk(code, data, limit=2):
    proc = subprocess.Popen([sys.executable, "-c", code],
                            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE, text=True)
    try:
        out, err = {{slot1}}
    except subprocess.TimeoutExpired:
        {{slot2}}
        out, err = {{slot3}}
        return "timeout", out, proc.returncode
    return "ok", out, proc.returncode
```

Slot	Mảnh đúng
[Slot 1]	`proc.communicate(data, timeout=limit)`
[Slot 2]	`proc.kill()`
[Slot 3]	`proc.communicate()`

Mảnh bẫy	Vì sao sai
`proc.communicate(data)`	Không được gửi input lần hai sau khi đã bắt đầu giao tiếp (ValueError).
`proc.wait(timeout=limit)`	wait() không gửi data, không đọc pipe; nó trả exit code chứ không phải (out, err).

`pass`	Con vẫn chạy — TimeoutExpired không tự kill.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp mẫu chuẩn "timeout → kill → communicate".

Yêu cầu thêm: Thử con đọc nhiều dòng từ stdin, con in một dòng rồi ngủ lâu, và con ngủ ngắn hơn limit.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết talk(code, data, limit=2): mở process con bằng subprocess.Popen([sys.executable, "-c", code]) với stdin, stdout, stderr đều là subprocess.PIPE và text=True; gửi data vào stdin và chờ bằng communicate(data, timeout=limit). Xong kịp thì trả ("ok", stdout, returncode). Quá giờ (TimeoutExpired) thì KHÔNG bỏ mặc con: kill() rồi communicate() thêm một lần để thu nốt phần đã in và thu hồi con; trả ("timeout", phần stdout thu được, returncode).

Thực hành: Popen với ba PIPE + text=True; communicate(data, timeout=limit) trong try; except TimeoutExpired: kill(), communicate() lần hai, trả "timeout".

Gọi	Kết quả mong đợi
`talk("print(input().upper())", "hello\n")`	`("ok", "HELLO\n", 0)`
`Con in "start" rồi ngủ 10 giây, limit = 1`	`("timeout", "start\n", mã khác 0)`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`talk(code, data, limit=2)`	Gửi data cho process con, chờ có giới hạn, dọn dẹp khi quá giờ. · code, data: str; limit: giây · ("ok" | "timeout", stdout: str, returncode: int) · Quá giờ: kill() rồi communicate() lần hai — không để con chạy tiếp.

BỘ CHẤM (7 ca)

Ca	Gọi	Mong đợi
Gửi stdin, nhận stdout	`talk("print(input().upper())", "hello\n")`	`('ok', 'HELLO\n', 0)`
Con đọc nhiều dòng stdin	`talk("import sys; print(sum(int(x) for x in sys.stdin))", "1\n2\n39\n")`	`('ok', '42\n', 0)`
Trả đúng exit code của con	`talk("import sys; print('bye'); sys.exit(3)", "")`	`('ok', 'bye\n', 3)`
Quá giờ → "timeout"	`talk('import time; time.sleep(10)', "", 1)[0]`	`'timeout'`
Thu nốt phần con đã in trước khi bị kill	`talk("import time; print('start', flush=True); time.sleep(10)", "", 1)[:2]`	`('timeout', 'start\n')`
Sau kill, con đã kết thúc (returncode khác None và khác 0)	`(lambda rc: rc is not None and rc != 0)(talk('import time; time.sleep(10)', "", 1)[2])`	`True`
Ngủ ngắn hơn limit thì vẫn "ok"	`talk("import time; time.sleep(1); print('done')", "", 3)`	`('ok', 'done\n', 0)`

MÔI TRƯỜNG CHẠY

Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG subprocess: lệnh [sys.executable, "-c", code] hoặc [sys.executable, "ten_file.py", ...] chạy chương trình con trong một vùng nhớ riêng, có stdin/stdout/stderr riêng, returncode lấy từ sys.exit()/ngoại lệ, và time.sleep() của con chỉ đẩy một đồng hồ ảo (timeout vẫn được tính đúng). Lệnh shell khác (ls, dir, ping...) không chạy được ở đây. Hãy chạy lại file trên máy (Windows/Linux) để thấy process thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. Popen(...) → con chạy, cha nhận proc ngay
Khung 2 — Bước 2. communicate(data, timeout=limit) → gửi data, đọc tới khi con xong
Khung 3 — Bước 3. quá limit → TimeoutExpired, con VẪN chạy
Khung 4 — Bước 4. kill() rồi communicate() → thu nốt output, returncode có giá trị
Khung đối chiếu — ngộ nhận "Bắt TimeoutExpired là xong việc". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Với Popen.communicate(timeout=...), TimeoutExpired không tự kill process: con vẫn chạy, vẫn giữ pipe và CPU; bỏ mặc nó là rò process. Phải kill() (hoặc terminate()) rồi gọi communicate() thêm một lần để thu phần output còn lại và thu hồi con — lúc đó returncode mới có giá trị.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: proc.communicate(data), proc.wait(timeout=limit), pass
Khung kết quả — 2 cặp vào/ra. talk("print(input().upper())", "hello\n") → ("ok", "HELLO\n", 0); Con in "start" rồi ngủ 10 giây, limit = 1 → ("timeout", "start\n", mã khác 0)

Nguồn: pe-v4 · python-learning-data.json#P06 · parser pe-v4/build · file build 006-P06.json

P07 · multiprocessing: Process và Queue

Mã bài: P07 · Trung cấp · M1 · Tiến trình và subprocess · 35 phút · 15 XP · thứ tự 7

MỤC TIÊU

Chạy hàm ở process riêng và gửi kết quả về qua Queue.

CẦN NHỚ TRƯỚC

P06

BỐI CẢNH

Viết square(n, results): tính n * n và put cặp (n, n * n) vào Queue results. Viết run_workers(numbers): tạo một Queue, tạo một Process cho MỖI số (target=square, args=(n, results)), start hết, lấy đủ len(numbers) kết quả bằng results.get(), join hết, trả dict {n: n * n}. Lời gọi tạo process ở cấp ngoài cùng của file phải nằm trong if __name__ == "__main__":.

VÍ DỤ MINH HOẠ

run_workers([2, 3, 4]) → {2: 4, 3: 9, 4: 16} — ba process, mỗi process gửi một cặp qua Queue.

LÝ THUYẾT

Định nghĩa: hai cách tạo process

Chương Multi Processing của slide dạy subprocess — chạy một CHƯƠNG TRÌNH khác rồi nói chuyện qua stdin/stdout. multiprocessing là cửa còn lại: chạy một HÀM Python của chính bạn ở một process riêng. Hai module, hai bộ API, không phải hai tên của cùng một thứ.

- Process(target=hàm, args=(…)) rồi start(), join() — dùng y như Thread
- Worker phải là def ở cấp module: con tìm lại hàm theo tên, nên lambda không dùng được
- Queue là kênh IPC: con put(), cha get(); dữ liệu được pickle khi đi qua
- Trên Windows/macOS con khởi động bằng cách import lại file, nên cần if __name__ == "__main__":

Ví dụ: bộ nhớ không dùng chung

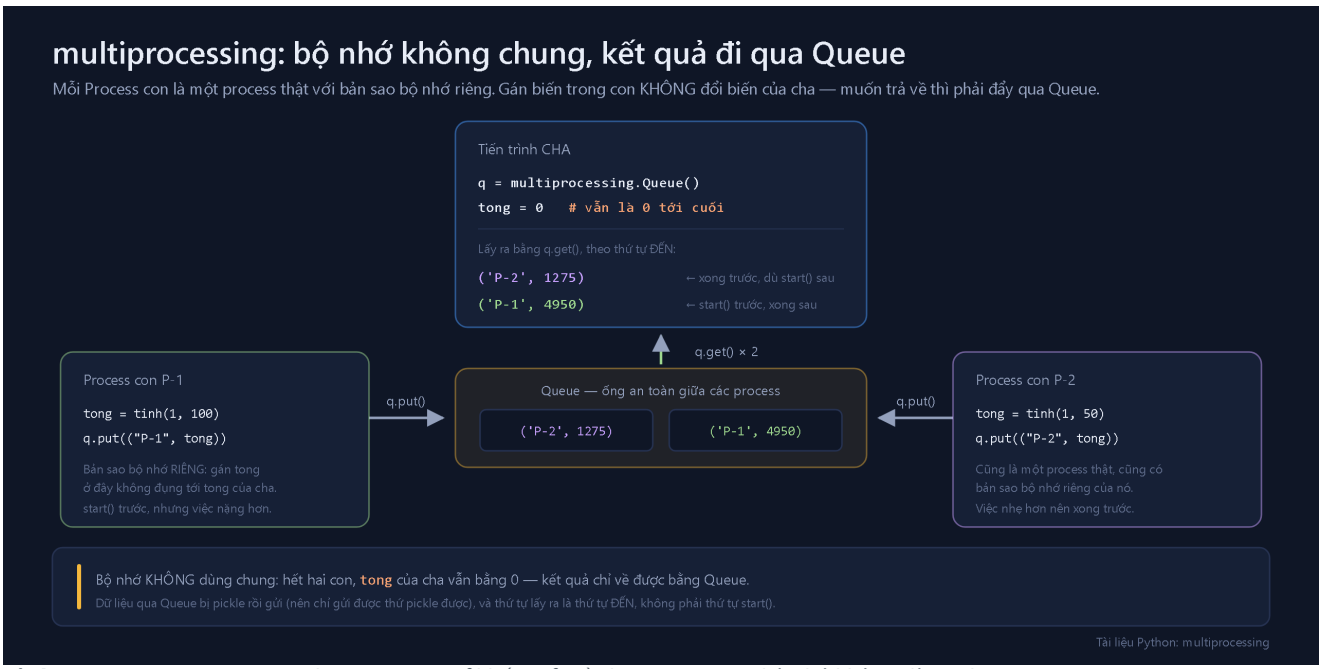
Slide mô tả fork() là "clone" process cha trừ process id — clone, nghĩa là BẢN SAO. Worker chạy counter += 1 hay ghi vào một dict toàn cục chỉ sửa bản của nó; cha đọc lại vẫn thấy số cũ, dict vẫn rỗng. Đây đúng là chỗ process khác thread: thread trong cùng process dùng chung bộ nhớ, process thì không.

Dùng ở đâu

- Việc CPU-bound: nhiều process chạy trên nhiều lõi thật, không bị GIL chặn như thread
- Chia một tập dữ liệu thành phần, mỗi process một phần, gom kết quả qua Queue
- Cô lập rủi ro: một worker chết không kéo sập cả chương trình
- Giá phải trả: tạo process đắt hơn tạo thread, và mọi thứ gửi đi đều phải pickle được
- Studio mô phỏng multiprocessing: target chạy khi start() trên bản sao, Queue là kênh chung

Đọc sơ đồ

Ba hộp: cha ở giữa, hai con hai bên, mỗi hộp có vùng nhớ riêng vẽ tách hẳn — không mũi tên nào nối thẳng vùng nhớ con sang vùng nhớ cha. Đường duy nhất đi qua là Queue: mỗi con put một cặp vào hàng, cha get ra đủ số cặp rồi mới join. Thứ tự lấy ra không nhất thiết theo thứ tự start — vì thế phải gom theo khoá n, không theo chỉ số.



Hình P07-process-queue. Hai process con gửi kết quả về cha qua Queue; bộ nhớ không dùng chung

Mô tả hình: Cha và hai process con, mỗi process có vùng nhớ riêng tách biệt; chỉ một Queue nối con put() với cha get()

Nguồn lý thuyết: SL 22-26, 49-53; DOC multiprocessing (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. subprocess chạy một CHƯƠNG TRÌNH khác và nói chuyện qua stdin/stdout; multiprocessing chạy một HÀM Python của bạn ở process riêng.
2. Process(target=hàm, args=(...)) dùng giống Thread: target là tên hàm (không ngoặc), args là tuple, start() rồi join().
3. Worker phải là hàm def ở CẤP MODULE (không lambda, không hàm lồng): process con phải tìm lại được nó theo tên.
4. Process con có bộ nhớ riêng: sửa biến toàn cục trong con KHÔNG đổi bản của cha. Gửi kết quả qua Queue: con put(), cha get(); dữ liệu được pickle.
5. Trên Windows/macOS con khởi động bằng cách import lại file của bạn, nên code tạo process phải nằm trong if __name__ == "__main__":. Lấy hết kết quả từ Queue rồi mới join.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
from multiprocessing import Process, Queue

counter = 0


def worker(value, box):
    global counter
    counter += 1
    box.put(value * 2)


if __name__ == "__main__":
    box = Queue()
    p = Process(target=worker, args=(4, box))
    p.start()
    print("Con gửi về:", box.get())
    p.join()
    print("counter của cha vẫn là", counter)
```

Kết quả dự kiến (theo JSON): run_workers([2, 3, 4]) → {2: 4, 3: 9, 4: 16}; run_workers([]) → {}

NGỘ NHẬN THƯỜNG GẶP: "Process cũng chia sẻ biến như thread"

Các thread trong cùng một process dùng chung bộ nhớ; các process thì không. Worker ghi vào dict toàn cục hay append vào list của cha chỉ sửa BẢN SAO trong con — cha nhận dict rỗng. Kết quả phải đi qua IPC: Queue, Pipe, Value/Manager. Và subprocess với multiprocessing không phải hai tên của cùng một API.

CÂU DỰ ĐOÁN

Worker chạy counter += 1 trong process con (counter là biến toàn cục bằng 0). Sau join, cha in counter được mấy?

ĐÚNG	0 — con sửa bản sao của nó, cha không thấy	Process có không gian địa chỉ riêng; gửi về phải qua Queue/Pipe.
sai	1	Đó là hành vi của thread (chung bộ nhớ), không phải process.
sai	Báo lỗi	Không lỗi — chỉ là hai bản biến khác nhau.

TỰ KIỂM TRA

1. subprocess và multiprocessing có phải hai tên cho cùng một API không?

✔ Không — subprocess chạy chương trình khác; multiprocessing chạy hàm Python ở process riêng — Cả hai liên quan process nhưng mục đích và API khác nhau.

• Có — Hai module, hai bộ API.

• multiprocessing là bản cũ của subprocess — Không có quan hệ cũ/mới.

Đáp án: Cả hai liên quan process nhưng mục đích và API khác nhau.

2. Vì sao không dùng target=lambda n: n * n cho Process?

✔ Process con phải tìm/pickle lại target theo tên ở cấp module; lambda không có tên như vậy — Định nghĩa worker bằng def ở cấp module.

• Process không nhận hàm — Process nhận hàm qua target.

• lambda chạy chậm hơn — Không phải chuyện tốc độ.

Đáp án: Định nghĩa worker bằng def ở cấp module.

3. Bỏ if __name__ == "__main__": trên Windows thì...

• Chỉ chạy chậm hơn — Không phải chuyện tốc độ.

✔ Process con import lại file và chạy lại đoạn tạo process — Python báo RuntimeError (bootstrapping) — Main guard chặn code tạo process chạy lại khi file bị import.

• Chạy bình thường — Trên Windows (spawn) thì không.

Đáp án: Main guard chặn code tạo process chạy lại khi file bị import.

TỰ GIẢI THÍCH

Vì sao worker phải put kết quả vào Queue thay vì ghi vào dict toàn cục?

Mẫu trả lời: Mỗi process con có bộ nhớ riêng (bản sao), nên ghi vào dict toàn cục chỉ sửa bản của con; cha vẫn thấy dict rỗng. Queue là kênh IPC: con put cặp (n, n * n), dữ liệu được pickle chuyển sang cha, cha get đủ len(numbers) lần. Trường hợp đặc biệt: numbers rỗng thì không tạo process nào và trả {}.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Queue() → kênh IPC dùng chung giữa cha và con
Bước 2	Process(target=square, args=(n, results)) → mô tả process, chưa chạy
Bước 3	start() → square chạy ở process con, put (n, n * n)
Bước 4	cha get() đủ len(numbers) lần → join hết → dict

CÂU PHẢN VÍ DỤ

run_workers([-3, 5]) trả gì?

Lựa chọn: {-3: -9, 5: 25} | {-3: 9, 5: 25} | {} · Đáp án: {-3: 9, 5: 25}

TEMPLATE LẮP MẢNH

```
from multiprocessing import Process, Queue


def square(n, results):
    {{slot1}}


def run_workers(numbers):
    results = Queue()
    workers = [{{slot2}} for n in numbers]
    for worker in workers:
        worker.start()
    squares = {}
    for _ in numbers:
        n, value = {{slot3}}
        squares[n] = value
    for worker in workers:
        worker.join()
    return squares
```

Slot	Mảnh đúng

[Slot 1]	`results.put((n, n * n))`
[Slot 2]	`Process(target=square, args=(n, results))`
[Slot 3]	`results.get()`

Mảnh bẫy	Vì sao sai
`results`	Queue là kênh, không phải kết quả — phải get() từng cặp.
`return n * n`	Giá trị return của target bị bỏ đi — cha không nhận được.
`Process(target=square(n, results))`	Gọi square ngay trong cha; Process nhận target=None.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: gửi kết quả, tạo Process, nhận kết quả.

Yêu cầu thêm: Thử một số, list rỗng và số âm; thử cho worker ghi vào biến toàn cục để thấy cha không nhận được gì.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết square(n, results): tính n * n và put cặp (n, n * n) vào Queue results. Viết run_workers(numbers): tạo một Queue, tạo một Process cho MỖI số (target=square, args=(n, results)), start hết, lấy đủ len(numbers) kết quả bằng results.get(), join hết, trả dict {n: n * n}. Lời gọi tạo process ở cấp ngoài cùng của file phải nằm trong if __name__ == "__main__":.

Thực hành: Worker cấp module chỉ tính rồi put vào Queue; cha tạo Process cho từng số, start hết, get đủ, join hết. Lời gọi tạo process ở cấp ngoài cùng nằm dưới main guard.

Gọi	Kết quả mong đợi
`run_workers([2, 3, 4])`	`{2: 4, 3: 9, 4: 16}`
`run_workers([])`	`{}`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`square(n, results)`	Worker chạy trong process con. · n: int; results: Queue · None (kết quả đi qua Queue) · results.put((n, n * n)).
`run_workers(numbers)`	Mỗi số một process, gom kết quả qua Queue. · numbers: list[int] · dict {n: n * n} · Tạo len(numbers) Process, start hết, get đủ, join hết.

BỘ CHẤM (6 ca)

Ca	Gọi	Mong đợi
run_workers([2, 3, 4])	`run_workers([2, 3, 4])`	`{2: 4, 3: 9, 4: 16}`
Dữ liệu khác, có số âm	`run_workers([7, -3, 10])`	`{7: 49, -3: 9, 10: 100}`
List rỗng → {}	`run_workers([])`	`{}`
square gửi đúng một cặp qua Queue	`_studio.check('from multiprocessing import Queue\nq = Queue()\nsquare(6, q)\nassert q.get() == (6, 36), "square phải put cặp (n, n * n)"')`	`True`
Mỗi số một Process; target là hàm; start hết rồi join hết	`(lambda r: (r.process_count, r.process_targets_callable, r.process_starts_before_joins, r.all_processes_joined))(_studio.call(run_workers, [1, 2, 3]))`	`(3, True, True, True)`
Import file không tự tạo process (main guard)	`(lambda r: (r.stdout, r.process_count))(_studio.run(entry='None'))`	`('', 0)`

MÔI TRƯỜNG CHẠY

Trình duyệt không tạo được process thật, nên Studio MÔ PHỎNG multiprocessing: target của mỗi Process chạy ngay khi start() trên một BẢN SAO dữ liệu (đổi biến toàn cục trong con không ảnh hưởng cha, đúng như process thật); Queue/Value/Manager là kênh dùng chung. Thời gian đo bằng đồng hồ ảo: các process chạy song song nên tổng thời gian ≈ process lâu nhất. Trên Windows nhớ đặt code tạo process trong if __name__ == "__main__": — chạy thật trên máy để thấy khác biệt.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. Queue() → kênh IPC dùng chung giữa cha và con
Khung 2 — Bước 2. Process(target=square, args=(n, results)) → mô tả process, chưa chạy
Khung 3 — Bước 3. start() → square chạy ở process con, put (n, n * n)
Khung 4 — Bước 4. cha get() đủ len(numbers) lần → join hết → dict
Khung đối chiếu — ngộ nhận "Process cũng chia sẻ biến như thread". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Các thread trong cùng một process dùng chung bộ nhớ; các process thì không. Worker ghi vào dict toàn cục hay append vào list của cha chỉ sửa BẢN SAO trong con — cha nhận dict rỗng. Kết quả phải đi qua IPC: Queue, Pipe, Value/Manager. Và subprocess với multiprocessing không phải hai tên của cùng một API.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: results, return n * n, Process(target=square(n, results))
Khung kết quả — 2 cặp vào/ra. run_workers([2, 3, 4]) → {2: 4, 3: 9, 4: 16}; run_workers([]) → {}

Nguồn: pe-v4 · python-learning-data.json#P07 · parser pe-v4/build · file build 007-P07.json

M2 · Đa luồng

6 bài: T01, T02, T03, T04, T05, T06

Ứng viên animation — ưu tiên cao (★★): không có · ưu tiên (★): T01, T02, T03, T04, T05, T06

T01 · Thread, tài nguyên và concurrency

Mã bài: T01 · Nâng cao · M2 · Đa luồng · 25 phút · 20 XP · thứ tự 8

MỤC TIÊU

Phân biệt phần thread dùng chung và phần mỗi thread có riêng.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

Một process có thể chạy nhiều thread. Viết scope_of(part): trả "shared" cho phần mọi thread trong process dùng chung ("code", "global", "heap", "file") và "per-thread" cho phần mỗi thread có riêng ("stack", "pc" — program counter, "registers"); tên khác → ValueError. Viết thêm who_am_i() trả tên của thread ĐANG chạy dòng lệnh đó (threading.current_thread().name).

VÍ DỤ MINH HOẠ

scope_of("heap") → "shared"; who_am_i() gọi ở chương trình chính → "MainThread", gọi trong Thread(..., name="saver") → "saver".

LÝ THUYẾT

Định nghĩa: thread là gì

Thread là MỘT dòng thực thi (single flow of execution) nằm trong một process; slide gọi nó là "lightweight process". Mặc định mỗi process chỉ có một thread. Multi-thread là nhiều dòng thực thi cùng chạy trong một process, dùng chung đúng một PCB.

- Chung (trong PCB): heap, biến toàn cục, code, file descriptor, scheduling, accounting
- Riêng từng thread: stack và stack pointer, program counter, registers
- Heap chung nên trao đổi dữ liệu không cần shm*() hay IPC như giữa hai process

Ví dụ: cùng mục tiêu, khác cách chia

Multi-process và multi-thread đặt ra cùng ba mục tiêu: làm nhiều việc một lúc, tăng mức dùng CPU, tăng responsiveness. Khác nhau ở cách chia tài nguyên — slide gọi tên thẳng: fork() là "resource cloning", process con nhận bản sao vùng nhớ; thread là "resource sharing", mọi thread nhìn cùng một vùng nhớ. Tạo, kết thúc và chuyển giữa các thread đều rẻ hơn process.

Dùng ở đâu

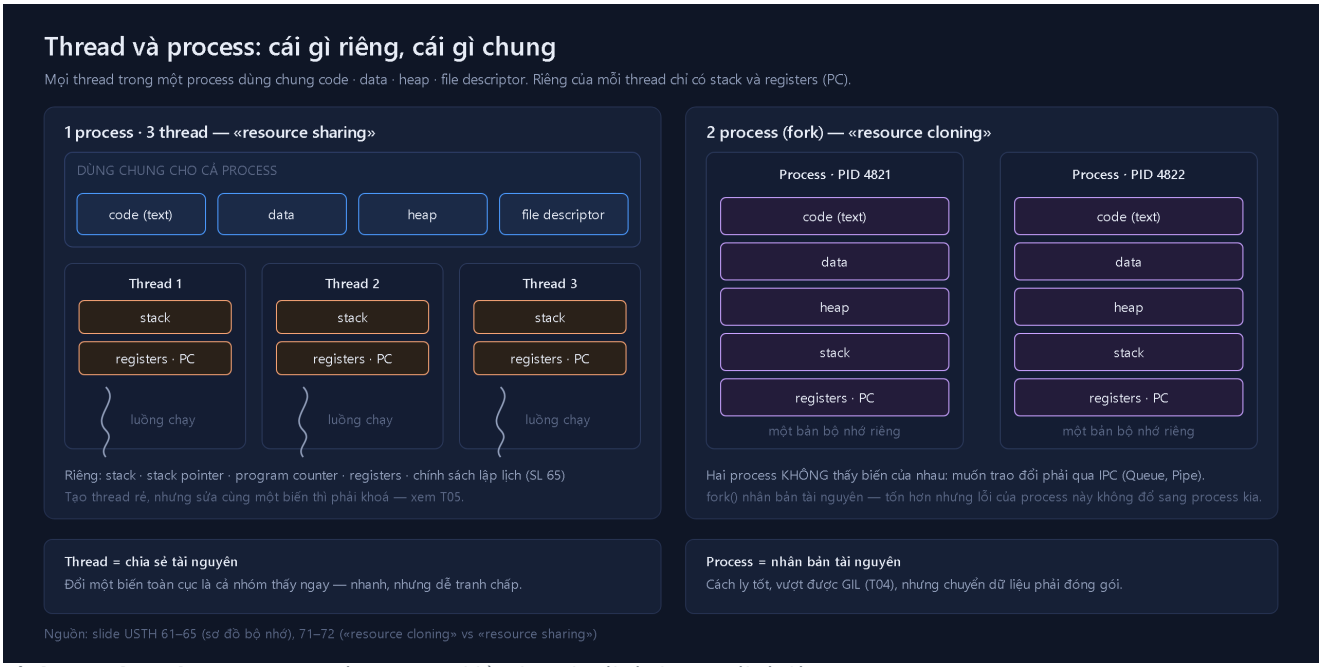
- Apache prefork: mỗi HTTP request một child process — cách ly tốt, tốn bộ nhớ
- Apache worker: ít process hơn, mỗi process nhiều thread, mỗi thread một request
- Chrome: mỗi tab một process; trong process có main thread, render thread, I/O thread
- Giao diện: một thread lo UI, thread khác lo việc chờ lâu để cửa sổ không đứng

Cái giá và giới hạn

- Nondeterministic: cùng một code, mỗi lần chạy thứ tự các dòng in có thể khác
- Phải tự đồng bộ: race condition (T05), deadlock, code khó đọc hơn
- Nhiều lõi thì tăng số thread, nhưng quá nhiều thì overhead và đồng bộ ăn hết lợi
- Studio chạy các target lần lượt; chạy file trên máy mới thấy thứ tự thật

Đọc sơ đồ

Khung lớn là MỘT process. Hàng trên — code, data, heap, file descs — vẽ một lần duy nhất: đó là phần dùng chung. Mỗi cột bên dưới là một thread, có ô stack và registers riêng, đường cong là dòng thực thi của nó. Đối chiếu phần multi-process bên cạnh: ở đó mỗi process có đủ một bộ ô riêng và không ô nào nối sang process khác.



Hình T01-thread-vs-process. Một process nhiều thread: cái gì chung, cái gì riêng

Mô tả hình: Process chứa code, data, heap, file descriptor dùng chung; ba thread mỗi thread một stack và registers riêng; bên cạnh là hai process tách rời, không chia gì

Nguồn lý thuyết: SL 61-72 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Process là chương trình đang chạy, có vùng nhớ riêng. Thread là một DÒNG THỰC THI bên trong process: process nào cũng có ít nhất main thread (tên "MainThread"); multi-thread là nhiều dòng thực thi trong cùng một process.
2. Mỗi thread có stack (biến cục bộ của các lời gọi hàm đang chạy), program counter (đang ở lệnh nào) và thanh ghi riêng. Hai thread chạy cùng một hàm vẫn không lẫn biến cục bộ của nhau.
3. Code, biến toàn cục, heap (mọi object: list, dict...) và file đang mở thì DÙNG CHUNG: trong ví dụ, worker append vào shared_log và main thread thấy ngay, không cần gửi lại.
4. current_thread() trả về thread đang chạy dòng lệnh gọi nó. Cùng một code, chạy trong worker thì ra "worker-1", chạy ở main thì ra "MainThread".
5. Process nhấn mạnh CÁCH LY, thread nhấn mạnh CHIA SẺ. Lợi: chương trình phản hồi nhanh (giao diện không đứng), chạy tác vụ nền, chia sẻ dữ liệu rẻ, tận dụng nhiều việc chờ cùng lúc. Giá phải trả: thứ tự chạy không xác định, race condition, deadlock, chi phí đồng bộ.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import threading

shared_log = []

def worker():
    step = "worker"
    shared_log.append(threading.current_thread().name)

t = threading.Thread(target=worker, name="worker-1")
t.start()
t.join()
shared_log.append(threading.current_thread().name)
print(shared_log)
```

Kết quả dự kiến (theo JSON): "heap" → "shared"; who_am_i() trong thread "saver" → "saver"

NGỘ NHẬN THƯỜNG GẶP: "Mỗi thread có bản sao heap và biến toàn cục riêng, giống process con"

Đó là hành vi của process: process con làm việc trên vùng nhớ riêng nên sửa list không ảnh hưởng cha. Thread trong cùng process dùng chung code, biến toàn cục, heap và file đang mở; chỉ stack, program counter và thanh ghi là của riêng từng thread. Vì dùng chung nên trao đổi dữ liệu rẻ — và cũng vì dùng chung mà sinh ra race condition (T05).

CÂU DỰ ĐOÁN

Worker thread append vào list toàn cục shared_log rồi kết thúc. Sau join(), main thread in shared_log thấy gì?

sai	Lúc thấy lúc không, tuỳ máy	Thứ tự chạy không xác định, nhưng sau join() worker chắc chắn đã xong.
sai	Không thấy — worker sửa trên bản sao của riêng nó	Đó là process con: nó có vùng nhớ riêng. Thread thì không.
ĐÚNG	Thấy phần tử worker vừa thêm — hai thread dùng chung heap của process	List là object trên heap; mọi thread trong process cùng trỏ tới nó.

TỰ KIỂM TRA

1. Hai thread trong cùng process có nhất thiết copy toàn bộ heap không?

• Có — mỗi thread một bản heap — Đó là process con, không phải thread.

• Chỉ copy khi thread ghi vào heap — Copy-on-write là chuyện của fork process.

✔ Không — chúng dùng chung heap của process — Chỉ stack, program counter và thanh ghi là riêng.

Đáp án: Chỉ stack, program counter và thanh ghi là riêng.

2. Hai thread cùng chạy hàm f; trong f có biến cục bộ total. Hai biến total đó…

• Gây NameError ở thread thứ hai — Mỗi thread có lời gọi f riêng, không lỗi gì.

• Là một biến dùng chung — Chỉ biến toàn cục và object trên heap mới dùng chung.

✔ Là hai biến khác nhau — mỗi lời gọi nằm trên stack của thread mình — Biến cục bộ sống trên stack.

Đáp án: Biến cục bộ sống trên stack.

3. Cái giá nào đi kèm multi-thread?

• Các thread không chia sẻ được dữ liệu — Ngược lại: chia sẻ là điểm mạnh của thread.

✔ Thứ tự chạy không xác định, race condition, deadlock — phải tự đồng bộ — Cái giá của việc dùng chung bộ nhớ.

• Mỗi thread tốn một bản sao bộ nhớ của process — Thread dùng chung bộ nhớ của process.

Đáp án: Cái giá của việc dùng chung bộ nhớ.

TỰ GIẢI THÍCH

Vì sao cùng một hàm who_am_i lại trả hai tên khác nhau ở hai thread, trong khi hai thread cùng thấy một list toàn cục?

Mẫu trả lời: Code của hàm dùng chung, nhưng mỗi thread là một dòng thực thi riêng với stack và program counter riêng; current_thread() trả về thread ĐANG chạy lệnh đó, nên worker "saver" nhận "saver", main nhận "MainThread". List toàn cục là object trên heap dùng chung, nên thay đổi của thread này thread kia thấy được. Trường hợp đặc biệt: tên lạ như "gpu" không thuộc nhóm nào → ValueError.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	scope_of("heap") → "shared": mọi thread thấy cùng object
Bước 2	scope_of("stack") → "per-thread": biến cục bộ không lẫn
Bước 3	who_am_i() ở chương trình chính → "MainThread"
Bước 4	who_am_i() trong Thread(name="saver") → "saver"

CÂU PHẢN VÍ DỤ

Thread(target=lambda: print(who_am_i()), name="bg") được start — dòng in ra là gì?

Lựa chọn: MainThread | bg | Thread-1 · Đáp án: bg

TEMPLATE LẮP MẢNH

```
import threading

SHARED = {{slot1}}
PER_THREAD = {{slot2}}


def scope_of(part):
    if part in SHARED:
        return "shared"
    if part in PER_THREAD:
        return "per-thread"
    raise ValueError("unknown part: " + str(part))


def who_am_i():
    return {{slot3}}
```

Slot	Mảnh đúng

[Slot 1]	`("code", "global", "heap", "file")`
[Slot 2]	`("stack", "pc", "registers")`
[Slot 3]	`threading.current_thread().name`

Mảnh bẫy	Vì sao sai
`("stack", "pc", "registers", "global")`	Biến toàn cục dùng chung — chỉ process con mới có bản riêng.
`threading.main_thread().name`	Luôn là main thread, không phải thread đang chạy.
`"MainThread"`	Viết cứng: gọi trong worker vẫn trả MainThread.
`("code", "global", "file")`	Thiếu heap: các thread dùng chung heap, không copy.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp hai nhóm tài nguyên và lời gọi đọc tên thread hiện tại.

Yêu cầu thêm: Gọi who_am_i trong hai thread đặt tên khác nhau; thử một tên tài nguyên lạ để thấy ValueError.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Một process có thể chạy nhiều thread. Viết scope_of(part): trả "shared" cho phần mọi thread trong process dùng chung ("code", "global", "heap", "file") và "per-thread" cho phần mỗi thread có riêng ("stack", "pc" — program counter, "registers"); tên khác → ValueError. Viết thêm who_am_i() trả tên của thread ĐANG chạy dòng lệnh đó (threading.current_thread().name).

Thực hành: Gom tên tài nguyên vào hai tuple, scope_of chỉ việc tra; who_am_i đọc tên thread LÚC ĐƯỢC GỌI, không viết cứng.

Gọi	Kết quả mong đợi
`scope_of("heap")`	`"shared"`
`scope_of("stack")`	`"per-thread"`
`who_am_i() trong Thread(name="saver")`	`"saver"`
`scope_of("gpu")`	`ValueError`

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
heap → "shared" (thread không copy heap)	`scope_of("heap")`	`'shared'`
stack → "per-thread"	`scope_of("stack")`	`'per-thread'`
global và file dùng chung	`[scope_of("global"), scope_of("file")]`	`['shared', 'shared']`
code dùng chung; pc, registers riêng	`[scope_of("code"), scope_of("pc"), scope_of("registers")]`	`['shared', 'per-thread', 'per-thread']`
Tên lạ → ValueError	`_studio.call(scope_of, "gpu").error`	`'ValueError'`
Gọi ở chương trình chính → "MainThread"	`who_am_i()`	`'MainThread'`
Gọi trong Thread(name="saver") → "saver"	`_studio.call(lambda box: [(lambda w: (w.start(), w.join()))(__import__("threading").Thread(target=lambda: box.append(who_am_i()), name=n)) for n in ['saver']] and box, []).value`	`['saver']`
Hai thread khác tên → hai kết quả	`_studio.call(lambda box: [(lambda w: (w.start(), w.join()))(__import__("threading").Thread(target=lambda: box.append(who_am_i()), name=n)) for n in ['a', 'b']] and box, []).value`	`['a', 'b']`

MÔI TRƯỜNG CHẠY

Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O. Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo. Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. scope_of("heap") → "shared": mọi thread thấy cùng object
Khung 2 — Bước 2. scope_of("stack") → "per-thread": biến cục bộ không lẫn
Khung 3 — Bước 3. who_am_i() ở chương trình chính → "MainThread"
Khung 4 — Bước 4. who_am_i() trong Thread(name="saver") → "saver"
Khung đối chiếu — ngộ nhận "Mỗi thread có bản sao heap và biến toàn cục riêng, giống process con". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Đó là hành vi của process: process con làm việc trên vùng nhớ riêng nên sửa list không ảnh hưởng cha. Thread trong cùng process dùng chung code, biến toàn cục, heap và file đang mở; chỉ stack, program counter và thanh ghi là của riêng từng thread. Vì dùng chung nên trao đổi dữ liệu rẻ — và cũng vì dùng chung mà sinh ra race condition (T05).
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: ("stack", "pc", "registers", "global"), threading.main_thread().name, "MainThread", ("code", "global", "file")
Khung kết quả — 4 cặp vào/ra. scope_of("heap") → "shared"; scope_of("stack") → "per-thread"; who_am_i() trong Thread(name="saver") → "saver"; scope_of("gpu") → ValueError

Nguồn: pe-v4 · python-learning-data.json#T01 · parser pe-v4/build · file build 008-T01.json

T02 · Thread target, start và join

Mã bài: T02 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 9

MỤC TIÊU

Tạo nhiều Thread với target/args, start hết rồi join hết.

CẦN NHỚ TRƯỚC

T01

BỐI CẢNH

download(url, sizes) đã cấp: "tải" một trang trong PAGES và ghi số ký tự vào dict sizes. Viết download_all(urls): mỗi url một threading.Thread(target=download, args=(url, sizes)); vòng thứ nhất start TẤT CẢ, vòng thứ hai join TẤT CẢ, rồi mới trả sizes. urls rỗng → {} và không tạo thread nào.

VÍ DỤ MINH HOẠ

download_all(["home.html", "news.html"]) → {"home.html": 11, "news.html": 13}, hai thread.

LÝ THUYẾT

Năm bước dùng thread

Slide rút cách dùng thread thành năm bước. Bài này đi đúng năm bước đó, nhưng cho nhiều worker một lúc chứ không phải một.

- 1. Dùng module: import threading
- 2. Mô tả việc: Thread(target=hàm, args=…), hoặc class con của Thread (T03)
- 3. Tạo instance — mới chỉ là object mô tả, chưa có dòng thực thi nào
- 4. Khởi chạy bằng start(): hệ điều hành cấp một dòng thực thi mới cho việc đó
- 5. [tuỳ chọn] join(): chờ thread kết thúc rồi mới đi tiếp

start() chứ không phải run()

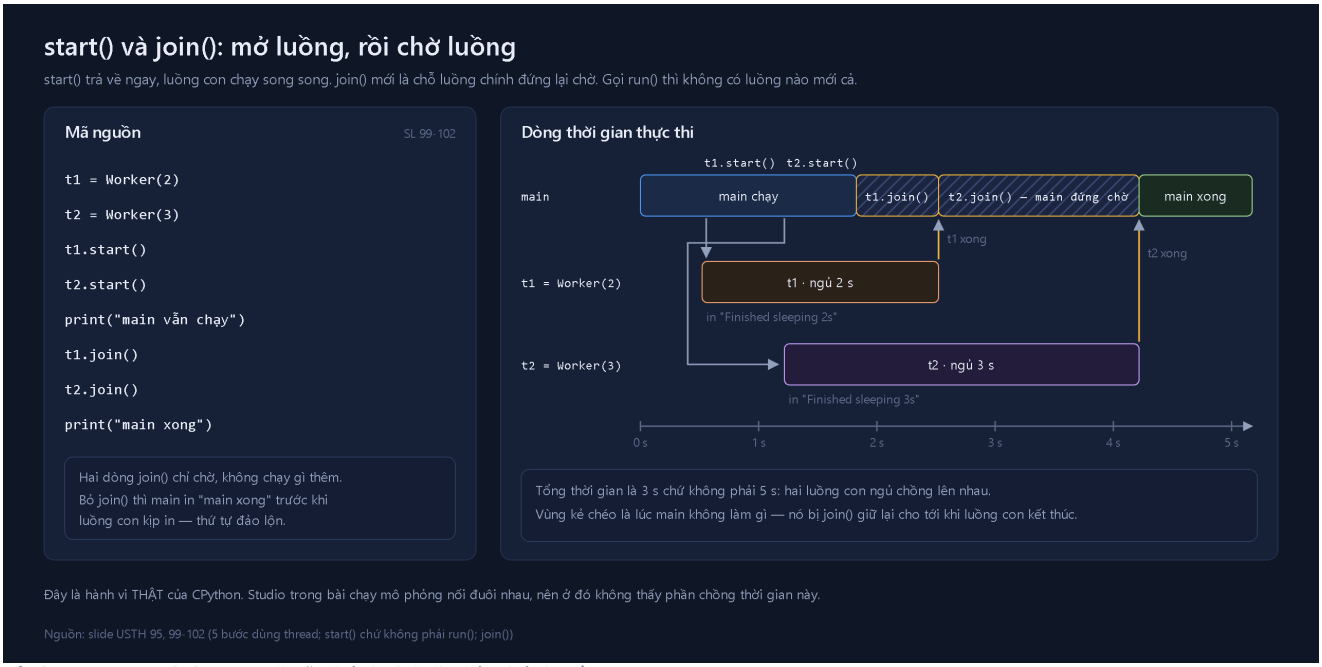
start() là chỗ DUY NHẤT sinh ra thread, và nó không nhận đối số — đối số đã đưa lúc tạo object. Gọi run() hay gọi thẳng hàm target chỉ là một lời gọi hàm bình thường trong thread hiện tại: kết quả vẫn đúng nhưng tuần tự, không được gì. Một Thread chỉ start được MỘT lần; muốn chạy lại phải tạo object mới.

Dùng ở đâu

- Tải nhiều URL, đọc nhiều file: phần lớn thời gian là chờ nên các lần chờ chồng lên nhau
- Giao diện: thread nền làm việc lâu, thread UI vẫn vẽ lại được (G06)
- Máy chủ: mỗi kết nối một thread, đúng mô hình Apache worker ở T01
- Chỉ đáng dùng khi CHỜ là chính — tính Python thuần thì vướng GIL (T04)

Đọc sơ đồ

Trục ngang là thời gian. Main thread chạy liền mạch; mỗi start() rẽ ra một đường mới bắt đầu đúng tại thời điểm đó, nên hai worker CHỒNG LÊN nhau. join() là đoạn main đứng chờ: đường main chỉ đi tiếp khi đường worker đã chạm vạch kết thúc. So hai cách xếp: start hết rồi join hết thì hai worker song song; start–join từng cái thì chúng nối đuôi, mất hết lợi ích. Studio luôn cho kiểu nối đuôi.



Hình T02-start-join. start() rẽ nhánh, join() chờ nhánh về

Mô tả hình: Dòng thời gian: main thread gọi start() hai lần tạo hai đường worker chạy chồng lên nhau, rồi join() chờ cả hai kết thúc trước khi main đi tiếp

Nguồn lý thuyết: SL 95-102 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Thread(target=greet, args=("Mai",)) chỉ TẠO đối tượng mô tả công việc, chưa chạy gì. target là tên hàm (không ngoặc); args là tuple đối số — một phần tử thì cần dấu phẩy ("Mai",); kwargs là dict đối số theo tên.
2. start() tạo dòng thực thi mới và cho target chạy trong đó; chương trình chính đi tiếp ngay. Gọi worker.run() thì target chạy NGAY trong thread hiện tại — không có thread mới nào.
3. join() CHỜ thread kết thúc; nó không khởi động thread (join khi chưa start → RuntimeError). Muốn nhiều worker chạy chồng lấp: vòng thứ nhất start hết, vòng thứ hai join hết.
4. time.sleep(x) chỉ tạm dừng thread gọi nó — các thread khác vẫn chạy. Đó là lý do nhiều thread cùng "chờ mạng" thì tổng thời gian gần bằng lần chờ lâu nhất.
5. Thread daemon=True bị bỏ dở khi chương trình chính thoát. Đừng giao việc phải hoàn tất (lưu dữ liệu) cho daemon thread mà không join.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import threading

def greet(name, punctuation):
    print("Hello " + name + punctuation)

worker = threading.Thread(target=greet, args=("Mai",), kwargs={"punctuation": "!"})
print("Trước start:", worker.is_alive())
worker.start()
worker.join()
print("Sau join:", worker.is_alive())
```

Kết quả dự kiến (theo JSON): ["home.html", "news.html"] → {"home.html": 11, "news.html": 13}; [] → {} và không tạo thread

NGỘ NHẬN THƯỜNG GẶP: "join() là lệnh cho thread chạy"

join chỉ CHỜ. Thread chỉ chạy khi start(); join một thread chưa start → RuntimeError: cannot join thread before it is started. Ngược lại, gọi run() trực tiếp thì target có chạy nhưng ở thread hiện tại — không có thread mới, không chồng lấp gì cả.

CÂU DỰ ĐOÁN

Ba trang, mỗi trang chờ mạng 1 giây. Viết "start rồi join ngay" trong CÙNG một vòng lặp thì tổng thời gian khoảng bao nhiêu?

sai	Gần 0 giây vì thread chạy nền	join vẫn chờ từng worker xong.
sai	Khoảng 1 giây	Chỉ đúng khi start hết rồi mới join.
ĐÚNG	Khoảng 3 giây — join chặn nên worker sau chỉ start khi worker trước xong	Thành chạy lần lượt; start hết rồi mới join thì khoảng 1 giây.

TỰ KIỂM TRA

1. Gọi t.run() có tạo thread mới không?

• Có, nhưng không join được — Không có thread mới nào để join.

• Có, run() và start() như nhau — Chỉ start() tạo dòng thực thi mới.

✔ Không — target chạy ngay trong thread đang gọi; muốn thread mới phải t.start() — run() chỉ là hàm gọi target.

Đáp án: run() chỉ là hàm gọi target.

2. Thread(target=save, args=("data.pkl")) — chuyện gì xảy ra khi start?

• Chạy bình thường — Thread gọi save(*args) = save("d", "a", "t", ...).

• Lỗi ngay lúc tạo Thread — Thread chỉ gọi target(*args) khi chạy.

✔ Lỗi TypeError trong thread: args là chuỗi nên bị tách thành 8 ký tự — ("data.pkl") chỉ là chuỗi trong ngoặc; cần ("data.pkl",).

Đáp án: ("data.pkl") chỉ là chuỗi trong ngoặc; cần ("data.pkl",).

3. Worker daemon=True đang ghi file thì chương trình chính kết thúc. Điều gì có thể xảy ra?

✔ Worker bị dừng giữa chừng, file ghi dở — Daemon không giữ chương trình sống; cần join nếu việc phải xong.

• Python báo lỗi DaemonError — Không có lỗi nào như vậy.

• Chương trình chờ worker ghi xong — Đó là thread thường (daemon=False).

Đáp án: Daemon không giữ chương trình sống; cần join nếu việc phải xong.

TỰ GIẢI THÍCH

Vì sao download_all phải start hết rồi mới join hết, và vì sao chỉ trả sizes sau vòng join?

Mẫu trả lời: start hết trước để các worker cùng chờ mạng chồng lấp nhau; join ngay sau từng start biến chúng thành chạy lần lượt. sizes là dict dùng chung mà các worker ghi vào; chỉ sau khi join hết mới chắc mọi worker đã ghi xong. Trường hợp đặc biệt: urls rỗng → không tạo thread nào, trả {}.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Thread(target=download, args=(url, sizes)) → mô tả việc, chưa chạy
Bước 2	worker.start() → download chạy trong thread mới
Bước 3	worker.join() → main chờ worker đó xong
Bước 4	return sizes → đọc kết quả sau khi join hết

CÂU PHẢN VÍ DỤ

download_all(["a.html", "b.html", "a.html"]) tạo mấy Thread?

Lựa chọn: 3 | 2 | 1 · Đáp án: 3

TEMPLATE LẮP MẢNH

```
def download_all(urls):
    sizes = {}
    threads = []
    for url in urls:
        worker = {{slot1}}
        threads.append(worker)
    for worker in threads:
        {{slot2}}
    for worker in threads:
        {{slot3}}
    return sizes
```

Slot	Mảnh đúng
[Slot 1]	`threading.Thread(target=download, args=(url, sizes))`
[Slot 2]	`worker.start()`
[Slot 3]	`worker.join()`

Mảnh bẫy	Vì sao sai
`threading.Thread(target=download, args=url)`	args phải là tuple đủ hai đối số (url, sizes).
`worker.run()`	run() chạy target trong thread hiện tại, không tạo thread mới.
`time.sleep(1)`	Ngủ không phải chờ: không biết worker đã xong chưa.
`threading.Thread(target=download(url, sizes))`	Gọi download ngay ở thread chính; Thread nhận target=None.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: tạo Thread đúng target/args, start, join.

Yêu cầu thêm: Đổi danh sách trang (có trang không tồn tại, có list rỗng); kiểm số thread bằng số url và kết quả chỉ đọc sau join.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

download(url, sizes) đã cấp: "tải" một trang trong PAGES và ghi số ký tự vào dict sizes. Viết download_all(urls): mỗi url một threading.Thread(target=download, args=(url, sizes)); vòng thứ nhất start TẤT CẢ, vòng thứ hai join TẤT CẢ, rồi mới trả sizes. urls rỗng → {} và không tạo thread nào.

Thực hành: Mỗi url một Thread MỚI (target=download, args=(url, sizes)); vòng thứ nhất start hết, vòng thứ hai join hết, rồi mới return sizes.

Gọi	Kết quả mong đợi
`["home.html", "news.html"]`	`{"home.html": 11, "news.html": 13}` — hai thread
`["home.html", "missing.html"]`	`{"home.html": 11, "missing.html": 0}`
`[]`	`{}` — không tạo thread nào

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`download_all(urls)`	Tải nhiều trang, mỗi trang một thread. · urls: list tên trang (có thể rỗng, có thể trùng) · dict {url: số ký tự} · Tạo len(urls) Thread, start hết rồi join hết; không in.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Hai trang → đúng kích thước	`download_all(["home.html", "news.html"])`	`{'home.html': 11, 'news.html': 13}`
Trang rỗng và trang không có → 0	`download_all(["empty.html", "missing.html"])`	`{'empty.html': 0, 'missing.html': 0}`
Mỗi url một Thread (kể cả url trùng)	`_studio.call(download_all, ["home.html", "news.html", "home.html"]).thread_count`	`3`
target là hàm download, không gọi sẵn	`(lambda r: (r.targets_callable, [t.target_name for t in r.threads]))(_studio.call(download_all, ["home.html", "news.html"]))`	`(True, ['download', 'download'])`
Start hết rồi mới join	`_studio.call(download_all, ["home.html", "news.html"]).starts_before_joins`	`True`
Join đủ mọi worker	`_studio.call(download_all, ["home.html", "news.html", "empty.html"]).all_joined`	`True`
urls rỗng → {} và không tạo thread	`(lambda r: (r.value, r.thread_count))(_studio.call(download_all, []))`	`({}, 0)`
Dữ liệu khác (PAGES khác)	`_studio.run(vars={'PAGES': {'x.html': 'abc', 'y.html': 'hello'}}, entry='download_all(["x.html", "y.html", "z.html"])').value`	`{'x.html': 3, 'y.html': 5, 'z.html': 0}`

MÔI TRƯỜNG CHẠY

Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O. Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo. Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. Thread(target=download, args=(url, sizes)) → mô tả việc, chưa chạy
Khung 2 — Bước 2. worker.start() → download chạy trong thread mới
Khung 3 — Bước 3. worker.join() → main chờ worker đó xong
Khung 4 — Bước 4. return sizes → đọc kết quả sau khi join hết
Khung đối chiếu — ngộ nhận "join() là lệnh cho thread chạy". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. join chỉ CHỜ. Thread chỉ chạy khi start(); join một thread chưa start → RuntimeError: cannot join thread before it is started. Ngược lại, gọi run() trực tiếp thì target có chạy nhưng ở thread hiện tại — không có thread mới, không chồng lấp gì cả.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: threading.Thread(target=download, args=url), worker.run(), time.sleep(1), threading.Thread(target=download(url, sizes))
Khung kết quả — 3 cặp vào/ra. ["home.html", "news.html"] → {"home.html": 11, "news.html": 13} — hai thread; ["home.html", "missing.html"] → {"home.html": 11, "missing.html": 0}; [] → {} — không tạo thread nào

Nguồn: pe-v4 · python-learning-data.json#T02 · parser pe-v4/build · file build 009-T02.json

T03 · Subclass Thread

Mã bài: T03 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 10

MỤC TIÊU

Tạo worker có trạng thái bằng class con của Thread.

CẦN NHỚ TRƯỚC

T02

BỐI CẢNH

Viết class SumWorker(threading.Thread): __init__(self, numbers) gọi super().__init__() TRƯỚC, rồi lưu self.numbers (list) và self.total = None; override run(self) để gán self.total = sum(self.numbers). Viết parallel_sum(chunks): mỗi khối một SumWorker, start hết, join hết, trả tổng các total. Không có khối nào → 0.

VÍ DỤ MINH HOẠ

parallel_sum([[1, 2, 3], [10, 20]]) → 36 (hai worker: 6 và 30). SumWorker([1, 2]).total trước khi chạy → None.

LÝ THUYẾT

Định nghĩa: hai cách tạo thread

Slide dạy cách OOP trước: định nghĩa một class con của threading.Thread, override run() — run() chính là việc sẽ chạy trong thread — và [tuỳ chọn] viết __init__() để nhận tham số. Cách Thread(target=hàm) nằm ở phần Extras, slide gọi là "simple threading without subclassing". Hai cách tương đương; chọn subclass khi worker cần mang dữ liệu và kết quả của riêng nó.

Ví dụ trong slide

class BackgroundThread(threading.Thread): __init__(self, sleepTime) gọi khởi tạo lớp cha rồi lưu self.__sleepTime; run() ngủ đúng chừng ấy giây rồi in. Nơi gọi viết BackgroundThread(10) — đối số đi vào __init__ — rồi .start() không đối số, rồi .join().

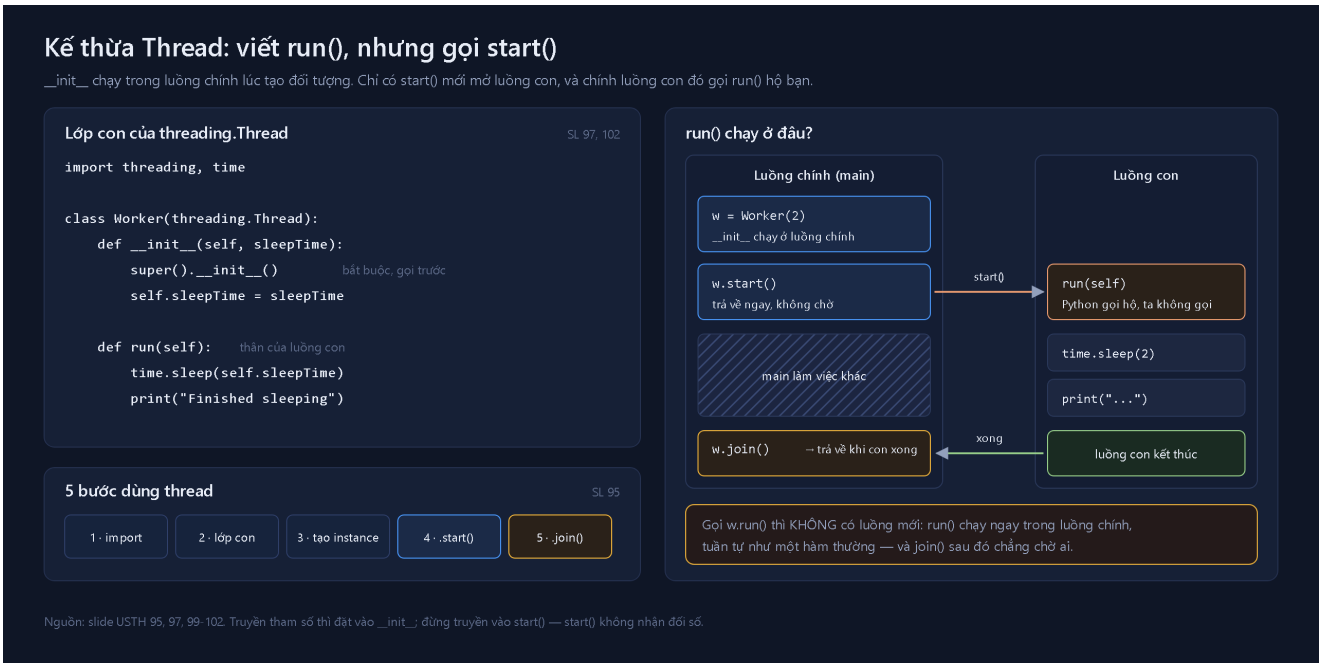
- threading.Thread.__init__(self) và super().__init__() là cùng một việc
- Bỏ lời gọi đó thì start() hỏng: trạng thái nội bộ của Thread chưa được dựng

Dùng ở đâu

- Worker có cấu hình riêng: đường dẫn file, phần dữ liệu được chia, bộ đếm riêng
- Cần lấy kết quả: run() ghi vào self.<thuộc tính>, nơi gọi đọc SAU join()
- Thread không trả giá trị — không có chỗ nào nhận được return của run()
- Việc một lần, không trạng thái thì target=hàm gọn hơn; đừng dựng class cho có

Đọc sơ đồ

Bên trái là class: __init__ nhận tham số, gọi super().__init__() rồi gán các self.<thuộc tính>; run() đọc chúng và ghi kết quả ngược lại vào self. Mũi tên giữa hình là điểm dễ nhầm: nơi gọi bấm start(), nhưng chạy trong thread con lại là run() — start() là người gọi run(), không ai gọi run() bằng tay. Bên phải: hai instance là hai object riêng, nên hai self.total không lẫn.



Hình T03-subclass-thread. Subclass Thread: __init__ nhận dữ liệu, run() là việc của thread

Mô tả hình: Class con của Thread với __init__ gọi super().__init__() và run() ghi kết quả vào self; start() ở thread cha kích hoạt run() chạy trong thread con

Nguồn lý thuyết: SL 97-102 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Cách thứ hai để tạo thread: viết class con của threading.Thread và override run() — run() là "việc của thread". Vẫn gọi start() để chạy: start() tạo thread mới rồi tự gọi run() trong thread đó.
2. __init__ của class con nhận tham số riêng (who, numbers) nhưng phải gọi super().__init__() trước khi dùng thread: phần khởi tạo của Thread tạo ra trạng thái mà start()/join() cần. Quên → start() báo lỗi (máy thật: RuntimeError: thread.__init__() not called; Studio mô phỏng báo AttributeError về thuộc tính started — cùng một nguyên nhân).
3. Trạng thái của worker nằm trong self: run() ghi kết quả vào self.message / self.total, nơi gọi đọc nó SAU join(). Trước khi chạy, ví dụ in None.
4. __init__ chạy ở thread TẠO object (thường là main thread); chỉ run() mới chạy trong thread mới. Việc nặng đặt trong __init__ là làm ở main thread.
5. Không bắt buộc dùng cách này: target=hàm đủ cho việc đơn giản. Subclass hợp khi worker mang dữ liệu và kết quả của riêng nó.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import threading

class Greeter(threading.Thread):
    def __init__(self, who):
        super().__init__()
        self.who = who
        self.message = None

    def run(self):
        self.message = "Hello " + self.who

worker = Greeter("Mai")
print(worker.message)
worker.start()
worker.join()
print(worker.message)
```

Kết quả dự kiến (theo JSON): SumWorker([1, 2]).total trước start → None; parallel_sum([[1, 2, 3], [10, 20]]) → 36

NGỘ NHẬN THƯỜNG GẶP: "Override start() cho tiện, vì start mới là thứ chạy thread"

start() là của Thread: nó dựng thread mới rồi gọi run(). Override start() thì không còn thread nào được tạo, và join() sau đó báo RuntimeError: cannot join thread before it is started. Việc của worker đặt trong run(); nơi dùng gọi start().

CÂU DỰ ĐOÁN

class W(threading.Thread) override run(). Chạy w = W(); w.start(); w.join() — run() chạy mấy lần, ở thread nào?

sai	Hai lần: ở start và ở join	join chỉ chờ, không chạy lại gì.
ĐÚNG	Một lần, trong thread mới do start() tạo	start() gọi run() trong thread con.
sai	Không lần nào — phải tự gọi w.run()	Gọi run() trực tiếp thì chạy ở thread hiện tại.

TỰ KIỂM TRA

1. Method nào được override, method nào được gọi để khởi chạy?

✔ Override run(); gọi start() — start() tạo thread rồi gọi run() bên trong nó.

• Override __init__; gọi join() — join chỉ chờ thread đã start.

• Override start(); gọi run() — Như vậy không có thread mới nào.

Đáp án: start() tạo thread rồi gọi run() bên trong nó.

2. class W(Thread) có __init__ riêng nhưng quên super().__init__(). Gọi W(...).start() thì...

✔ RuntimeError: thread.__init__() not called — Phần khởi tạo của Thread chưa chạy (Studio mô phỏng báo AttributeError … started — cùng nguyên nhân).

• Chạy bình thường — Thread kiểm tra và từ chối start.

• run() chạy ở main thread — Không chạy gì cả — start() báo lỗi trước.

Đáp án: Phần khởi tạo của Thread chưa chạy (Studio mô phỏng báo AttributeError … started — cùng nguyên nhân).

3. Đọc w.total ngay sau w.start(), chưa join, thì...

• Lỗi AttributeError — total đã gán None trong __init__.

✔ Có thể vẫn là None — worker chưa chắc đã chạy xong — Đọc kết quả sau join().

• Luôn đã có kết quả — start() không chờ worker.

Đáp án: Đọc kết quả sau join().

TỰ GIẢI THÍCH

Vì sao total khởi tạo None trong __init__ và chỉ được tính trong run()?

Mẫu trả lời: __init__ chạy ở thread tạo object (main thread), còn run() chạy trong thread mới sau start(). Tính tổng trong __init__ là làm việc ở main thread, worker không còn gì để làm. Để None tới khi run() xong giúp phân biệt "chưa chạy" với "tổng bằng 0"; nơi gọi đọc total sau join. Trường hợp đặc biệt: khối rỗng → total = 0, không phải None.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	SumWorker([1, 2]) → __init__: lưu numbers, total = None
Bước 2	worker.start() → Thread gọi run() trong thread mới
Bước 3	run() → self.total = 3
Bước 4	worker.join() → đọc total an toàn

CÂU PHẢN VÍ DỤ

parallel_sum([[2, 3], [], [5]]) trả về?

Lựa chọn: 5 | 10 | None · Đáp án: 10

TEMPLATE LẮP MẢNH

```
import threading


class SumWorker({{slot1}}):
    def __init__(self, numbers):
        {{slot2}}
        self.numbers = list(numbers)
        self.total = None

    def {{slot3}}(self):
        self.total = sum(self.numbers)


def parallel_sum(chunks):
    workers = [SumWorker(chunk) for chunk in chunks]
    for worker in workers:
        {{slot4}}
    for worker in workers:
        worker.join()
    return sum(worker.total for worker in workers)
```

Slot	Mảnh đúng
[Slot 1]	`threading.Thread`
[Slot 2]	`super().__init__()`
[Slot 3]	`run`
[Slot 4]	`worker.start()`

Mảnh bẫy	Vì sao sai
`worker.run()`	Chạy run() ngay trong main thread.
`self.total = sum(self.numbers)`	Tính ở __init__ = làm việc ở main thread; lại thiếu super().__init__().
`threading.Thread()`	Kế thừa CLASS Thread, không phải một object Thread.
`start`	Override start thì không còn thread nào được tạo.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp lớp cha, lời gọi khởi tạo của Thread, tên method được override và lệnh khởi chạy.

Yêu cầu thêm: Thử khối rỗng, nhiều khối, và đọc total trước khi start để thấy None.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết class SumWorker(threading.Thread): __init__(self, numbers) gọi super().__init__() TRƯỚC, rồi lưu self.numbers (list) và self.total = None; override run(self) để gán self.total = sum(self.numbers). Viết parallel_sum(chunks): mỗi khối một SumWorker, start hết, join hết, trả tổng các total. Không có khối nào → 0.

Thực hành: Kế thừa threading.Thread, gọi super().__init__() đầu tiên, lưu trạng thái vào self; việc tính đặt trong run(). parallel_sum tạo worker, start hết, join hết rồi cộng total.

Gọi	Kết quả mong đợi
`parallel_sum([[1, 2, 3], [10, 20]])`	`36`
`SumWorker([1, 2]).total trước start`	`None`
`parallel_sum([])`	`0 — không tạo worker`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`SumWorker(numbers) — class con của threading.Thread`	Một worker tự giữ dữ liệu và kết quả của nó. · numbers: iterable các số (list, tuple, range…) · object Thread; total = None tới khi run() chạy xong · run() gán self.total = sum(numbers); không in.
`parallel_sum(chunks)`	Cộng nhiều khối số, mỗi khối một SumWorker. · chunks: list các khối số (có thể rỗng) · int: tổng mọi total · Tạo len(chunks) worker, start hết rồi join hết.

BỘ CHẤM (7 ca)

Ca	Gọi	Mong đợi
SumWorker là một Thread	`isinstance(SumWorker([1]), __import__("threading").Thread)`	`True`
Chưa start → total là None	`SumWorker([1, 2]).total`	`None`
start + join → total đúng	`(lambda w: (w.start(), w.join(), w.total)[2])(SumWorker([4, 5, 6]))`	`15`
parallel_sum ba khối	`parallel_sum([[1, 2], [3], [4, 5, 6]])`	`21`
Mỗi khối một worker, start hết rồi join hết	`(lambda r: (r.thread_count, r.starts_before_joins, r.all_joined))(_studio.call(parallel_sum, [[1], [2], [3]]))`	`(3, True, True)`
Khối rỗng góp 0; nhận cả range/tuple	`parallel_sum([[], range(5), (1, 1)])`	`12`
Không có khối → 0, không tạo worker	`(lambda r: (r.value, r.thread_count))(_studio.call(parallel_sum, []))`	`(0, 0)`

MÔI TRƯỜNG CHẠY

Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O. Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo. Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. SumWorker([1, 2]) → __init__: lưu numbers, total = None
Khung 2 — Bước 2. worker.start() → Thread gọi run() trong thread mới
Khung 3 — Bước 3. run() → self.total = 3
Khung 4 — Bước 4. worker.join() → đọc total an toàn
Khung đối chiếu — ngộ nhận "Override start() cho tiện, vì start mới là thứ chạy thread". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. start() là của Thread: nó dựng thread mới rồi gọi run(). Override start() thì không còn thread nào được tạo, và join() sau đó báo RuntimeError: cannot join thread before it is started. Việc của worker đặt trong run(); nơi dùng gọi start().
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: worker.run(), self.total = sum(self.numbers), threading.Thread(), start
Khung kết quả — 3 cặp vào/ra. parallel_sum([[1, 2, 3], [10, 20]]) → 36; SumWorker([1, 2]).total trước start → None; parallel_sum([]) → 0 — không tạo worker

Nguồn: pe-v4 · python-learning-data.json#T03 · parser pe-v4/build · file build 010-T03.json

T04 · GIL, CPU-bound và I/O-bound

Mã bài: T04 · Nâng cao · M2 · Đa luồng · 30 phút · 20 XP · thứ tự 11

MỤC TIÊU

Ước lượng thời gian theo workload, chọn thread hay process.

CẦN NHỚ TRƯỚC

T03

BỐI CẢNH

Mỗi task là (kind, seconds): "cpu" = tính Python thuần, "io" = chờ (mạng, đĩa). Viết wall_time(tasks, strategy) theo mô hình lý tưởng (mỗi task một worker, đủ lõi, bỏ qua chi phí tạo): "serial" = cộng mọi task; "thread" (CPython có GIL) = max(tổng giây CPU, task I/O dài nhất); "process" = task dài nhất; strategy khác → ValueError; không có task → 0. Viết best_strategy(tasks): cách có wall_time nhỏ nhất, bằng nhau thì chọn cách đứng trước trong serial → thread → process (rẻ hơn).

VÍ DỤ MINH HOẠ

[("io", 2), ("io", 3), ("cpu", 4)]: serial 9, thread max(4, 3) = 4, process 4 → best_strategy = "thread".

LÝ THUYẾT

Định nghĩa: GIL

GIL (Global Interpreter Lock) là một mutex bên trong CPython: tại một thời điểm chỉ MỘT thread được điều khiển interpreter, tức chỉ một thread chạy bytecode Python. Đây là chi tiết của bản CPython, không phải quy định của ngôn ngữ.

- Thắt cổ chai với code Python tính toán thuần (CPU-bound)
- Không thành vấn đề với chương trình chờ I/O
- Không thành vấn đề với phần gọi xuống thư viện native — slide lấy NumPy làm ví dụ

Vì sao có GIL, bỏ đi thì mất gì

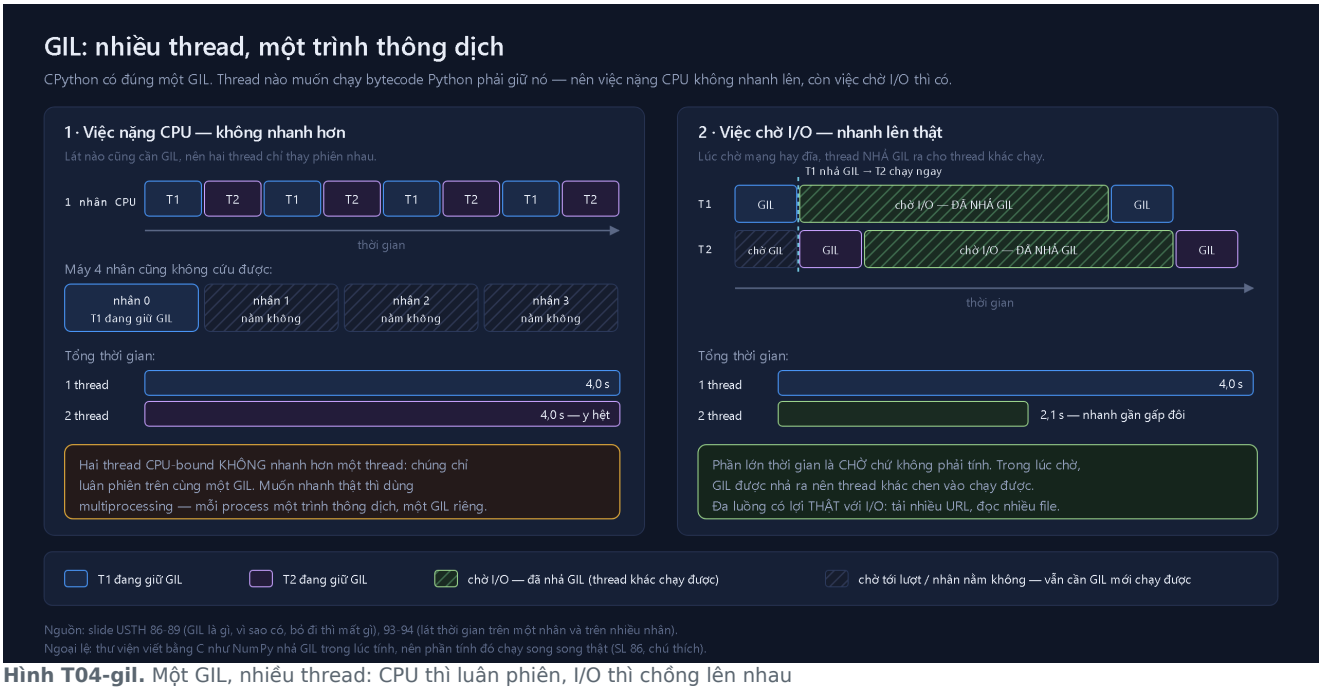
- Có: quản lý bộ nhớ — reference counting và garbage collector cần được bảo vệ
- Có: gọn thread-safety — một mutex trên interpreter thay cho mutex trên từng object
- Slide chốt: "that is not a bug but a feature"
- Bỏ: single-thread chậm đi, sinh nguy cơ deadlock, kém tương thích với extension cũ
- GIL KHÔNG thay Lock: trạng thái của ứng dụng vẫn phải tự bảo vệ (T05)

Dùng ở đâu

- Chờ mạng, đĩa, cơ sở dữ liệu, sleep → thread là đúng chỗ
- Tính Python thuần mà muốn dùng nhiều lõi → process: mỗi process một interpreter, một GIL
- Đo trước khi chọn bằng time.perf_counter(), đừng đoán theo số thread

Đọc sơ đồ

Hai dải thời gian trong slide là nền của hình. Một lõi: các lát T1, T2, T3… nối nhau trên cùng một hàng — concurrency là LUÂN PHIÊN, không phải cùng lúc. Nhiều lõi: mỗi core một hàng, các lát thật sự chồng nhau. GIL ép CPython về dáng hàng thứ nhất cho phần tính toán, dù máy có bao nhiêu lõi. Ô màu nhạt là lúc thread chờ I/O: nó nhả GIL nên lát của thread khác lấp vào được — chỗ đó mới là lãi.



Hình T04-gil. Một GIL, nhiều thread: CPU thì luân phiên, I/O thì chồng lên nhau

Mô tả hình: Dải thời gian một lõi với các lát thread nối nhau, so với nhiều lõi chạy song song; phần chờ I/O được tô nhạt để chỉ lúc thread nhả GIL cho thread khác chạy

Nguồn lý thuyết: SL 85-94 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. CPython có GIL (Global Interpreter Lock): tại một thời điểm chỉ MỘT thread được chạy bytecode Python. GIL giúp interpreter quản lý bộ nhớ an toàn (reference counting, garbage collector).
2. CPU-bound = thời gian đi vào tính toán. Nhiều thread tính Python thuần phải thay nhau giữ GIL nên phần CPU CỘNG DỒN: 2 + 3 giây vẫn ≈ 5 giây — thêm thread không nhanh hơn.
3. I/O-bound = thời gian đi vào chờ (mạng, đĩa, sleep). Thread đang chờ I/O nhả GIL, nên các lần chờ CHỒNG LÊN NHAU và chồng lên cả lúc thread khác tính: thread = max(tổng CPU, lần chờ dài nhất).
4. Mỗi process có interpreter và GIL riêng nên phần CPU chạy song song được trên nhiều lõi — đổi lại tốn chi phí tạo process và chép dữ liệu. Vì vậy khi bằng thời gian, chọn cách rẻ hơn.
5. Ngoại lệ cần nhớ: một số thư viện native (vd. phần lõi của NumPy) nhả GIL khi tính, nhưng không thể suy ra mọi code dùng NumPy đều song song; bản CPython free-threaded (3.13+) có thể tắt GIL. Và GIL KHÔNG thay Lock: race condition của ứng dụng vẫn còn (T05).

MẪU CODE · vi_du.py · Python 3.12 · demo

```
tasks = [("io", 2), ("io", 3), ("cpu", 4)]
cpu_total = sum(s for kind, s in tasks if kind == "cpu")
io_longest = max(s for kind, s in tasks if kind == "io")
serial = sum(s for kind, s in tasks)
threads = max(cpu_total, io_longest)
processes = max(s for kind, s in tasks)
print("serial:", serial, "| thread:", threads, "| process:", processes)
```

Kết quả dự kiến (theo JSON): [("io", 2), ("io", 3)], "thread" → 3; [("cpu", 2), ("cpu", 3)], "thread" → 5

NGỘ NHẬN THƯỜNG GẶP: "Thêm thread thì chương trình Python nào cũng nhanh hơn"

Chỉ đúng khi thời gian chủ yếu là CHỜ (I/O). Với tính toán Python thuần, GIL cho mỗi lúc một thread chạy bytecode nên thêm thread không giảm thời gian, có khi còn chậm hơn vì chuyển ngữ cảnh. Chọn theo loại workload, không theo số thread.

CÂU DỰ ĐOÁN

Hai task tính Python thuần, mỗi task 3 giây, chạy bằng 2 thread trên máy 4 lõi (CPython có GIL). Tổng thời gian khoảng bao nhiêu?

sai	≈ 1.5 giây	Không có cơ chế nào chia đôi một task.
sai	≈ 3 giây — mỗi thread một lõi	Đúng với 2 process, không đúng với thread khi có GIL.
ĐÚNG	≈ 6 giây — hai thread thay nhau giữ GIL	Phần CPU cộng dồn; muốn ≈ 3 giây phải dùng 2 process.

TỰ KIỂM TRA

1. Có GIL thì race condition của ứng dụng có tự biến mất không?

• Có — mỗi lúc chỉ một thread chạy — Thread có thể bị đổi giữa hai bytecode của cùng một câu lệnh.

✔ Không — GIL bảo vệ interpreter, không bảo vệ các bước đọc-sửa-ghi của bạn — counter += 1 vẫn có thể bị xen giữa; cần Lock (T05).

• Chỉ biến mất khi dùng list — List không có cơ chế khoá nào cho bạn.

Đáp án: counter += 1 vẫn có thể bị xen giữa; cần Lock (T05).

2. Tải 50 trang web, mỗi trang chủ yếu chờ mạng. Lựa chọn hợp lý?

• Không nên song song — Chờ tuần tự 50 trang là lãng phí.

• Process — vì thread bị GIL — GIL được nhả trong lúc chờ I/O; process tốn hơn mà không nhanh hơn.

✔ Thread — các lần chờ mạng chồng lên nhau — Thread đang chờ I/O nhả GIL.

Đáp án: Thread đang chờ I/O nhả GIL.

3. Phát biểu nào về NumPy đúng?

• NumPy không dùng được trong thread — Dùng được bình thường.

✔ Một số phép tính native nhả GIL, nhưng không suy ra được mọi code dùng NumPy đều chạy song song — Phải xét từng thao tác, đo thật trên máy.

• NumPy luôn bỏ qua GIL — Code Python bao quanh lời gọi NumPy vẫn cần GIL.

Đáp án: Phải xét từng thao tác, đo thật trên máy.

TỰ GIẢI THÍCH

Vì sao wall_time(..., "thread") cộng phần CPU nhưng lấy max phần I/O?

Mẫu trả lời: Với GIL, mỗi lúc chỉ một thread chạy bytecode nên các task CPU phải thay nhau: thời gian cộng dồn. Thread đang chờ I/O nhả GIL nên các lần chờ diễn ra cùng lúc và chồng lên cả lúc thread khác tính; vì vậy thời gian là max(tổng CPU, lần chờ dài nhất). Trường hợp đặc biệt: không có task nào → 0.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	serial → cộng mọi task: 2 + 3 + 4 = 9
Bước 2	thread → max(tổng CPU, I/O dài nhất) = max(4, 3) = 4
Bước 3	process → task dài nhất = 4
Bước 4	best_strategy → nhanh nhất; bằng nhau chọn cách rẻ hơn

CÂU PHẢN VÍ DỤ

best_strategy([("cpu", 2), ("cpu", 2), ("io", 1)]) trả về?

Lựa chọn: thread | serial | process · Đáp án: process

TEMPLATE LẮP MẢNH

```
def wall_time(tasks, strategy):
    cpu = [seconds for kind, seconds in tasks if kind == "cpu"]
    io = [seconds for kind, seconds in tasks if kind == "io"]
    if strategy == "serial":
        return {{slot1}}
    if strategy == "thread":
        return {{slot2}}
    if strategy == "process":
        return {{slot3}}
    raise ValueError("unknown strategy: " + str(strategy))
```

Slot	Mảnh đúng
[Slot 1]	`sum(cpu) + sum(io)`
[Slot 2]	`max(sum(cpu), max(io, default=0))`
[Slot 3]	`max(cpu + io, default=0)`

Mảnh bẫy	Vì sao sai
`max(sum(cpu), sum(io))`	Các lần chờ I/O chồng lên nhau: lấy lần dài nhất, không cộng.
`sum(cpu) + max(io, default=0)`	Lúc thread này chờ I/O, thread khác vẫn tính được — không nối tiếp.
`sum(cpu + io) / len(tasks)`	Không có phép chia đều: mỗi task chạy trọn thời gian của nó.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp công thức của ba cách chạy theo mô hình GIL.

Yêu cầu thêm: Thử workload toàn CPU, toàn I/O và trộn; xem khi nào thread thắng, khi nào cần process.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Mỗi task là (kind, seconds): "cpu" = tính Python thuần, "io" = chờ (mạng, đĩa). Viết wall_time(tasks, strategy) theo mô hình lý tưởng (mỗi task một worker, đủ lõi, bỏ qua chi phí tạo): "serial" = cộng mọi task; "thread" (CPython có GIL) = max(tổng giây CPU, task I/O dài nhất); "process" = task dài nhất; strategy khác → ValueError; không có task → 0. Viết best_strategy(tasks): cách có wall_time nhỏ nhất, bằng nhau thì chọn cách đứng trước trong serial → thread → process (rẻ hơn).

Thực hành: Tách giây CPU và giây I/O; serial cộng hết, thread lấy max(tổng CPU, I/O dài nhất), process lấy task dài nhất. best_strategy so lần lượt serial → thread → process, chỉ đổi khi NHANH HƠN hẳn.

Gọi	Kết quả mong đợi
`wall_time([("io", 2), ("io", 3), ("io", 1)], "thread")`	`3`
`wall_time([("cpu", 2), ("cpu", 3)], "thread")`	`5 — GIL, không nhanh hơn tuần tự`
`wall_time([("cpu", 2), ("cpu", 3)], "process")`	`3`
`best_strategy([("cpu", 1)])`	`"serial" — bằng nhau thì chọn cách đơn giản`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`wall_time(tasks, strategy)`	Ước lượng thời gian chạy lý tưởng của một workload. · tasks: list (kind, seconds), kind "cpu"/"io"; strategy: "serial" | "thread" | "process" · số giây (int/float); strategy lạ → ValueError · Không chạy gì thật, không in. Mô hình: mỗi task một worker, đủ lõi, bỏ qua chi phí tạo.
`best_strategy(tasks)`	Chọn cách chạy nhanh nhất; bằng nhau thì chọn cách đơn giản hơn. · tasks: như wall_time · "serial" | "thread" | "process" · So lần lượt serial → thread → process; chỉ đổi khi NHANH HƠN hẳn.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Toàn I/O, thread → lần chờ dài nhất	`wall_time([("io", 2), ("io", 3), ("io", 1)], "thread")`	`3`
Toàn CPU, thread → cộng dồn (GIL)	`wall_time([("cpu", 2), ("cpu", 3)], "thread")`	`5`
Toàn CPU, process → task dài nhất	`wall_time([("cpu", 2), ("cpu", 3)], "process")`	`3`
Trộn: serial và thread	`[wall_time([("cpu", 4), ("io", 3), ("io", 5)], s) for s in ("serial", "thread")]`	`[12, 5]`
Số thực: CPU 0.1 + 0.2 với I/O 0.25	`[round(wall_time([("cpu", 0.1), ("cpu", 0.2), ("io", 0.25)], s), 6) for s in ("serial", "thread", "process")]`	`[0.55, 0.3, 0.25]`
Không có task → 0	`[wall_time([], s) for s in ("serial", "thread", "process")]`	`[0, 0, 0]`
strategy lạ → ValueError	`_studio.call(wall_time, [("io", 1)], "gpu").error`	`'ValueError'`
best_strategy theo workload	`[best_strategy(t) for t in ([("io", 2), ("io", 2)], [("cpu", 2), ("cpu", 2)], [("cpu", 3), ("io", 2), ("io", 2)], [("cpu", 2), ("cpu", 2), ("io", 1)])]`	`['thread', 'process', 'thread', 'process']`
Bằng nhau → chọn cách đơn giản hơn	`[best_strategy([("cpu", 1)]), best_strategy([])]`	`['serial', 'serial']`

MÔI TRƯỜNG CHẠY

Bài này không chạy thread: bộ chấm dùng ĐỒNG HỒ ẢO (số giây ghi trong từng task), vì trình duyệt chạy Python trong một luồng duy nhất và thời gian đo thật thì mỗi lần một khác. Muốn thấy GIL thật, chạy trên máy: hai thread cùng tính sum(range(10**7)) rồi so với hai thread cùng time.sleep(1), đo bằng time.perf_counter().

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. serial → cộng mọi task: 2 + 3 + 4 = 9
Khung 2 — Bước 2. thread → max(tổng CPU, I/O dài nhất) = max(4, 3) = 4
Khung 3 — Bước 3. process → task dài nhất = 4
Khung 4 — Bước 4. best_strategy → nhanh nhất; bằng nhau chọn cách rẻ hơn
Khung đối chiếu — ngộ nhận "Thêm thread thì chương trình Python nào cũng nhanh hơn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Chỉ đúng khi thời gian chủ yếu là CHỜ (I/O). Với tính toán Python thuần, GIL cho mỗi lúc một thread chạy bytecode nên thêm thread không giảm thời gian, có khi còn chậm hơn vì chuyển ngữ cảnh. Chọn theo loại workload, không theo số thread.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: max(sum(cpu), sum(io)), sum(cpu) + max(io, default=0), sum(cpu + io) / len(tasks)
Khung kết quả — 4 cặp vào/ra. wall_time([("io", 2), ("io", 3), ("io", 1)], "thread") → 3; wall_time([("cpu", 2), ("cpu", 3)], "thread") → 5 — GIL, không nhanh hơn tuần tự; wall_time([("cpu", 2), ("cpu", 3)], "process") → 3; best_strategy([("cpu", 1)]) → "serial" — bằng nhau thì chọn cách đơn giản

Nguồn: pe-v4 · python-learning-data.json#T04 · parser pe-v4/build · file build 011-T04.json

T05 · Lock, race condition và deadlock

Mã bài: T05 · Nâng cao · M2 · Đa luồng · 40 phút · 20 XP · thứ tự 12

MỤC TIÊU

Bảo vệ dữ liệu chung bằng Lock và tránh deadlock.

CẦN NHỚ TRƯỚC

T04

BỐI CẢNH

Account có self.balance và self.lock = threading.Lock(); run_deposits (cấp sẵn) cho mỗi khoản nạp một thread. Viết deposit(amount): đọc-cộng-ghi balance trong with self.lock. Viết withdraw(amount): trong CÙNG một with self.lock, nếu amount > balance thì raise ValueError("insufficient funds"), ngược lại trừ. Viết safe_plan(plan, order): True nếu plan (các lock một thread khoá lồng nhau, theo thứ tự) đi đúng thứ tự chung order — tăng dần, không khoá lại lock đang giữ; tên không có trong order → ValueError.

VÍ DỤ MINH HOẠ

run_deposits(Account(0), [10, 20, 30]) → 60. Account(5).withdraw(9) → ValueError, balance vẫn 5. safe_plan(["log", "db"], ["db", "cache", "log"]) → False.

LÝ THUYẾT

Định nghĩa: race condition

Race condition là khi kết quả phụ thuộc THỨ TỰ các thread chạm vào dữ liệu chung: cùng một chương trình, lần này ra đúng, lần sau ra sai, và không lần nào báo lỗi. Đoạn code chạm vào dữ liệu chung và phải chạy trọn vẹn gọi là critical section.

Lock: đúng những gì slide cho

- threading.Lock(), còn gọi là mutex: mỗi lúc chỉ một thread giữ được
- lock.acquire() ... lock.release(): tự quản, và dễ quên release
- with lock: acquire/release tự động, kể cả khi khối ném exception
- Slide cảnh báo thẳng: có Lock rồi vẫn phải coi chừng race — Lock đặt sai chỗ là vô dụng

Dùng ở đâu

- Số dư, bộ đếm, tồn kho: mọi lần đọc-sửa-ghi một giá trị dùng chung
- Kiểm tra rồi hành động (đủ tiền thì trừ) phải nằm TRỌN trong một lần giữ khoá
- File hay kết nối dùng chung: cho một thread ghi một lúc
- Không cần Lock khi đã dùng queue.Queue (T06) — bên trong nó đã khoá sẵn

Deadlock: vì sao và tránh thế nào

Deadlock là khi các bên chờ nhau mãi, không ai tiến được: thread 1 giữ A chờ B, thread 2 giữ B chờ A. Slide xếp deadlock vào nhóm "vì sao KHÔNG nên đa luồng", cùng với nondeterministic và chi phí đồng bộ.

- Thống nhất một thứ tự khoá chung cho cả chương trình, mọi thread khoá theo đúng thứ tự đó
- Giữ khoá càng ngắn càng tốt; đừng đặt việc chậm (ghi file, in) trong critical section
- Đừng join một worker trong lúc đang giữ lock mà worker đó cần

Đọc sơ đồ

Nửa trên là lần chạy KHÔNG khoá: hai mũi tên đọc cùng chỉ vào một ô balance = 100, hai mũi tên ghi cùng đặt 101 — một lần cộng biến mất, đó là "lost update". Nửa dưới có Lock: thread thứ hai dừng ở hàng rào tới khi thread thứ nhất rời khối with, nên nó đọc 101 và ghi 102. Trong Studio thì nửa trên không bao giờ xảy ra vì các thread chạy lần lượt; chạy file trên máy nhiều lần mới thấy con số nhảy.



Hình T05-race-lock. Mất cập nhật khi không khoá, và hàng rào Lock sửa nó thế nào

Mô tả hình: Hai thread cùng đọc balance 100 rồi cùng ghi 101 nên mất một lần cộng; bên dưới, with lock buộc thread thứ hai chờ nên kết quả là 102

Nguồn lý thuyết: SL 103-105 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. balance += amount không phải một bước: đọc balance → cộng → ghi lại. Ví dụ minh hoạ xen kẽ bằng tay hai thread A, B cùng nạp 50: cả hai đọc 100 trước khi ai kịp ghi, nên một lần nạp bị mất. Kết quả phụ thuộc thứ tự truy cập = race condition.
2. Lock (mutex) chỉ cho MỘT thread giữ tại một thời điểm. Đoạn đọc-sửa-ghi đặt trong with self.lock: là critical section — thread khác tới đó phải chờ khối kết thúc. Lock phải DÙNG CHUNG (self.lock), tạo Lock mới mỗi lần gọi thì không ai phải chờ ai.
3. with lock tự release khi rời khối, kể cả khi có exception. acquire()/release() viết rời nhau thì một lỗi ở giữa làm lock kẹt mãi — mọi thread sau chờ vô hạn.
4. Kiểm tra rồi hành động (if amount > balance … balance -= amount) phải nằm TRỌN trong cùng một lần giữ khoá. Giữ khoá càng ngắn càng tốt: đừng đặt việc chậm (in, ghi file) vào trong.
5. Deadlock: các bên chờ nhau mãi. Thread 1 giữ A chờ B, thread 2 giữ B chờ A. Cách phòng: mọi thread khoá theo CÙNG một thứ tự đã thống nhất; Lock thường không khoá lại được khi chính mình đang giữ; đừng join một worker khi đang giữ lock mà worker đó cần.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
balance = 100
read_a = balance
read_b = balance
balance = read_a + 50
balance = read_b + 50
print("Mong đợi 200, thực tế", balance)
```

Kết quả dự kiến (theo JSON): run_deposits(Account(0), [10, 20, 30]) → 60; withdraw quá số dư → ValueError, khoá đã nhả

NGỘ NHẬN THƯỜNG GẶP: "Có GIL rồi nên balance += amount đã an toàn"

GIL chỉ bảo đảm mỗi lúc một thread chạy bytecode; += gồm nhiều bytecode (đọc, cộng, ghi) và thread có thể bị đổi giữa chúng. Trạng thái của ứng dụng phải tự bảo vệ bằng Lock. Lưu ý: Studio chạy thread lần lượt nên không tái hiện được race — bộ chấm kiểm cấu trúc: mọi lần đọc/ghi balance phải nằm trong lúc giữ self.lock.

CÂU DỰ ĐOÁN

Hai thread cùng chạy deposit(50) trên balance = 100, KHÔNG có lock. Kết quả có thể là?

sai	Luôn 200	Không có gì bảo đảm hai lần đọc-sửa-ghi không xen nhau.
ĐÚNG	150 hoặc 200 — tuỳ hai thread xen nhau thế nào	Đọc-sửa-ghi bị xen giữa thì mất một lần nạp.
sai	Luôn 150	Có lúc mỗi thread chạy trọn phần của mình thì ra 200.

TỰ KIỂM TRA

1. Vì sao with lock dễ quản lý hơn acquire/release rời nhau?

• Cho nhiều thread cùng giữ khoá — Lock luôn chỉ cho một thread giữ.

• Chạy nhanh hơn — Tốc độ như nhau; khác ở chỗ an toàn khi có lỗi.

✔ Tự release khi rời khối, kể cả khi có exception — Không bao giờ để lock kẹt vì quên release.

Đáp án: Không bao giờ để lock kẹt vì quên release.

2. Thread 1 khoá A rồi B; thread 2 khoá B rồi A. Rủi ro gì?

• Chỉ chậm hơn một chút — Hai bên chờ nhau mãi, không tiến triển.

✔ Deadlock — mỗi bên giữ một lock và chờ lock kia — Sửa: cả hai khoá theo cùng thứ tự A rồi B.

• Không sao vì cuối cùng đều release — Có thể không bên nào tới được lệnh release.

Đáp án: Sửa: cả hai khoá theo cùng thứ tự A rồi B.

3. Nên đặt gì vào critical section?

• Cả hàm, kể cả in và ghi file, cho chắc — Giữ lâu làm các thread khác chờ vô ích.

✔ Chỉ các bước đọc-sửa-ghi dữ liệu dùng chung — giữ khoá ngắn nhất có thể — Thread khác phải chờ suốt lúc bạn giữ khoá.

• Không cần, vì GIL đã khoá — GIL không bảo vệ trạng thái của ứng dụng.

Đáp án: Thread khác phải chờ suốt lúc bạn giữ khoá.

TỰ GIẢI THÍCH

Vì sao withdraw phải đặt cả phép kiểm tra lẫn phép trừ trong cùng một with self.lock?

Mẫu trả lời: Nếu kiểm tra ngoài khoá, hai thread có thể cùng thấy đủ tiền rồi lần lượt trừ, số dư âm. Đặt kiểm tra và trừ trong một lần giữ khoá thì thread kia chỉ kiểm tra sau khi lần trừ trước đã xong. with còn tự nhả khoá khi raise ValueError. Trường hợp đặc biệt: rút đúng bằng số dư thì được, số dư về 0.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	deposit(5) → with self.lock: đọc, cộng, ghi balance
Bước 2	withdraw(9) khi balance = 5 → ValueError, with nhả khoá
Bước 3	run_deposits(Account(0), [10, 20, 30]) → 60
Bước 4	safe_plan(["log", "db"], ["db", "cache", "log"]) → False

CÂU PHẢN VÍ DỤ

safe_plan(["cache", "log"], ["db", "cache", "log"]) trả về?

Lựa chọn: False | ValueError | True · Đáp án: True

TEMPLATE LẮP MẢNH

```
class Account:
    def __init__(self, balance=0):
        self.balance = balance
        self.lock = threading.Lock()

    def deposit(self, amount):
        {{slot1}}
            self.balance += amount

    def withdraw(self, amount):
        with self.lock:
            if {{slot2}}:
                raise ValueError("insufficient funds")
            self.balance -= amount


def safe_plan(plan, order):
    positions = [order.index(name) for name in plan]
    for i in range(1, len(positions)):
        if {{slot3}}:
            return False
    return True
```

Slot	Mảnh đúng
[Slot 1]	`with self.lock:`
[Slot 2]	`amount > self.balance`
[Slot 3]	`positions[i] <= positions[i - 1]`

Mảnh bẫy	Vì sao sai
`self.lock.acquire()`	Không có release đi kèm: khoá kẹt ngay sau lần gọi đầu.

`amount >= self.balance`	Rút đúng bằng số dư là hợp lệ.
`with threading.Lock():`	Mỗi lần gọi một Lock MỚI — không thread nào phải chờ ai.
`positions[i] < positions[i - 1]`	Cho qua ["db", "db"]: Lock thường không khoá lại được khi đang giữ.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp critical section của deposit, điều kiện của withdraw và phép so vị trí của safe_plan.

Yêu cầu thêm: Thử rút quá số dư, nạp dữ liệu sai kiểu, và một kế hoạch khoá ngược thứ tự; sau mỗi lỗi khoá phải được nhả.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Account có self.balance và self.lock = threading.Lock(); run_deposits (cấp sẵn) cho mỗi khoản nạp một thread. Viết deposit(amount): đọc-cộng-ghi balance trong with self.lock. Viết withdraw(amount): trong CÙNG một with self.lock, nếu amount > balance thì raise ValueError("insufficient funds"), ngược lại trừ. Viết safe_plan(plan, order): True nếu plan (các lock một thread khoá lồng nhau, theo thứ tự) đi đúng thứ tự chung order — tăng dần, không khoá lại lock đang giữ; tên không có trong order → ValueError.

Thực hành: Mọi lần đọc/ghi self.balance nằm trong with self.lock; withdraw kiểm tra và trừ trong CÙNG một khối. safe_plan đổi tên lock thành vị trí trong order rồi kiểm dãy vị trí tăng hẳn.

Gọi	Kết quả mong đợi
`run_deposits(Account(0), [10, 20, 30])`	`60`
`Account(5).withdraw(9)`	`ValueError; balance vẫn 5; khoá đã nhả`
`safe_plan(["db", "log"], ["db", "cache", "log"])`	`True`
`safe_plan(["log", "db"], ["db", "cache", "log"])`	`False — ngược thứ tự chung`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`Account.deposit(self, amount)`	Nạp tiền an toàn khi nhiều thread cùng gọi. · self; amount: số cần cộng · None · Đọc-cộng-ghi self.balance trong with self.lock; lỗi giữa chừng vẫn nhả khoá.
`Account.withdraw(self, amount)`	Rút tiền: kiểm tra rồi trừ như MỘT bước. · self; amount: số cần trừ · None; amount > balance → ValueError("insufficient funds") · Kiểm tra và trừ trong cùng một with self.lock; rút đúng bằng số dư là hợp lệ.
`safe_plan(plan, order)`	Kiểm một thread có khoá theo đúng thứ tự đã thống nhất không. · plan: list tên lock theo thứ tự thread khoá; order: list tên lock theo thứ tự chung · bool; tên lock không có trong order → ValueError · Không khoá gì thật, không in.

BỘ CHẤM (13 ca)

Ca	Gọi	Mong đợi
Ba thread nạp 10, 20, 30 → 60	`run_deposits(Account(0), [10, 20, 30])`	`60`
Không có khoản nạp → giữ số dư đầu	`run_deposits(Account(7), [])`	`7`
deposit đọc và ghi balance khi đang giữ self.lock	`_studio.check('class Probe:\n def __init__(self):\n self.held = False\n\n def acquire(self, blocking=True, timeout=-1):\n assert not self.held, "khoá lại self.lock khi đang giữ nó — Lock thật sẽ treo ở đây"\n self.held = True\n return True\n\n def release(self):\n self.held = False\n\n def locked(self):\n return self.held\n\n def __enter__(self):\n self.acquire()\n return self\n\n def __exit__(self, *exc):\n self.release()\n return False\n\n\nclass Spy(Account):\n log = None\n\n @property\n def balance(self):\n if self.log is not None:\n self.log.append(self.lock.held)\n return self._value\n\n @balance.setter\n def balance(self, value):\n if self.log is not None:\n self.log.append(self.lock.held)\n self._value = value\n\n\na = Spy(10)\na.lock = Probe()\na.log = []\na.deposit(5)\nassert a.log, "deposit không đọc/ghi self.balance"\nassert all(a.log), "deposit đọc hoặc ghi balance khi CHƯA giữ self.lock"\nassert not a.lock.held, "deposit xong mà chưa nhả self.lock"\nassert a._value == 15, "balance sau deposit(5) phải là 15"\n')`	`True`

withdraw: kiểm tra VÀ trừ cùng trong self.lock	`_studio.check('class Probe:\n def __init__(self):\n self.held = False\n\n def acquire(self, blocking=True, timeout=-1):\n assert not self.held, "khoá lại self.lock khi đang giữ nó — Lock thật sẽ treo ở đây"\n self.held = True\n return True\n\n def release(self):\n self.held = False\n\n def locked(self):\n return self.held\n\n def __enter__(self):\n self.acquire()\n return self\n\n def __exit__(self, *exc):\n self.release()\n return False\n\n\nclass Spy(Account):\n log = None\n\n @property\n def balance(self):\n if self.log is not None:\n self.log.append(self.lock.held)\n return self._value\n\n @balance.setter\n def balance(self, value):\n if self.log is not None:\n self.log.append(self.lock.held)\n self._value = value\n\n\na = Spy(10)\na.lock = Probe()\na.log = []\na.withdraw(4)\nassert len(a.log) >= 2, "withdraw phải đọc balance để kiểm tra rồi ghi lại"\nassert all(a.log), "phép KIỂM TRA số dư và phép TRỪ phải cùng nằm trong with self.lock"\nassert not a.lock.held, "withdraw xong mà chưa nhả self.lock"\nassert a._value == 6, "balance sau withdraw(4) phải là 6"\n')`	`True`
Rút quá số dư → ValueError, số dư không đổi	`(lambda a: (_studio.call(a.withdraw, 9).error, a.balance))(Account(5))`	`('ValueError', 5)`
Sau ValueError, khoá đã được nhả	`(lambda a: (_studio.call(a.withdraw, 9).error, a.lock.locked()))(Account(5))`	`('ValueError', False)`
Nạp sai kiểu → TypeError, khoá vẫn được nhả	`(lambda a: (_studio.call(a.deposit, "10").error, a.lock.locked()))(Account(0))`	`('TypeError', False)`
Rút đúng bằng số dư → 0	`(lambda a: (a.withdraw(5), a.balance)[1])(Account(5))`	`0`
safe_plan: đúng thứ tự (kể cả bỏ qua lock giữa, plan rỗng)	`[safe_plan(["db", "log"], ["db", "cache", "log"]), safe_plan(["cache"], ["db", "cache"]), safe_plan([], ["db"])]`	`[True, True, True]`
safe_plan: ngược thứ tự → False	`[safe_plan(["log", "db"], ["db", "cache", "log"]), safe_plan(["db", "log", "cache"], ["db", "cache", "log"])]`	`[False, False]`
safe_plan: khoá lại lock đang giữ → False	`safe_plan(["db", "db"], ["db", "log"])`	`False`
safe_plan theo order đã thống nhất, không theo bảng chữ cái	`safe_plan(["log", "cache"], ["log", "cache", "db"])`	`True`
safe_plan: lock lạ → ValueError	`_studio.call(safe_plan, ["db", "x"], ["db", "log"]).error`	`'ValueError'`

MÔI TRƯỜNG CHẠY

Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O. Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo. Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần. Vì các thread không bao giờ xen kẽ ở đây, race condition không tái hiện được: bộ chấm thay self.lock bằng một khoá "ghi nhật ký" để kiểm mọi lần đọc/ghi balance đều nằm trong lúc giữ khoá. Khoá lại một Lock đang giữ thì trên máy thật chương trình treo mãi.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. deposit(5) → with self.lock: đọc, cộng, ghi balance
Khung 2 — Bước 2. withdraw(9) khi balance = 5 → ValueError, with nhả khoá
Khung 3 — Bước 3. run_deposits(Account(0), [10, 20, 30]) → 60
Khung 4 — Bước 4. safe_plan(["log", "db"], ["db", "cache", "log"]) → False
Khung đối chiếu — ngộ nhận "Có GIL rồi nên balance += amount đã an toàn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. GIL chỉ bảo đảm mỗi lúc một thread chạy bytecode; += gồm nhiều bytecode (đọc, cộng, ghi) và thread có thể bị đổi giữa chúng. Trạng thái của ứng dụng phải tự bảo vệ bằng Lock. Lưu ý: Studio chạy thread lần lượt nên không tái hiện được race — bộ chấm kiểm cấu trúc: mọi lần đọc/ghi balance phải nằm trong lúc giữ self.lock.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: self.lock.acquire(), amount >= self.balance, with threading.Lock():, positions[i] < positions[i - 1]
Khung kết quả — 4 cặp vào/ra. run_deposits(Account(0), [10, 20, 30]) → 60; Account(5).withdraw(9) → ValueError; balance vẫn 5; khoá đã nhả; safe_plan(["db", "log"], ["db", "cache", "log"]) → True; safe_plan(["log", "db"], ["db", "cache", "log"]) → False — ngược thứ tự chung

Nguồn: pe-v4 · python-learning-data.json#T05 · parser pe-v4/build · file build 012-T05.json

T06 · Queue và tác vụ lưu nền

Mã bài: T06 · Nâng cao · M2 · Đa luồng · 40 phút · 20 XP · thứ tự 13

MỤC TIÊU

Giao việc lưu cho worker nền qua queue.Queue.

CẦN NHỚ TRƯỚC

T05

BỐI CẢNH

Hoàn thành class BackgroundSaver (self.jobs = queue.Queue(), self.lines = [] đã có). submit(record): put BẢN SAO record (dict(record)) vào self.jobs — chưa lưu gì. _work(): lấy từng job bằng self.jobs.get_nowait() tới khi gặp queue.Empty thì return; mỗi job thêm dòng "name,mark" vào self.lines rồi gọi self.jobs.task_done(). flush(): tạo Thread MỚI với target=self._work, start, join, trả len(self.lines).

VÍ DỤ MINH HOẠ

submit({"name": "An", "mark": 8}), submit({"name": "Binh", "mark": 7}) → flush() trả 2, lines = ["An,8", "Binh,7"] (đúng thứ tự FIFO).

LÝ THUYẾT

Định nghĩa: queue.Queue

queue.Queue là hàng đợi FIFO đã khoá sẵn bên trong, dựng riêng cho việc trao đổi giữa các thread: put() bỏ việc vào cuối, get() lấy ở đầu. Nhờ khoá nằm sẵn trong Queue mà hai bên không phải tự dựng Lock như T05 (tài liệu chính thức: module queue).

- get() mặc định CHỜ khi queue rỗng; get_nowait() không chờ, rỗng thì raise queue.Empty
- Mỗi get() thành công đi với đúng một task_done(); join() chờ tới khi mọi việc đã xong

Producer và consumer

Mẫu chuẩn chia chương trình làm hai vai: producer tạo việc và put vào queue rồi đi tiếp ngay; consumer (worker) lấy việc ra và làm phần chậm. Queue là ranh giới duy nhất giữa hai bên, nên chúng không cần biết gì về nhau — thêm worker chỉ là chạy thêm một thread cùng đọc một queue.

Dùng ở đâu

- Bài thực hành 8 của môn: nâng phần lưu dữ liệu lên dùng pickle + nén ở thread nền
- Giao diện: nút bấm chỉ submit, worker làm việc nặng, kết quả về qua một queue khác (G06)
- Máy chủ: một queue việc, vài worker cùng rút ra làm — thêm worker là tăng thông lượng
- Dữ liệu giao cho worker phải ổn định: put BẢN SAO, đừng put chính object còn đang sửa

Đọc sơ đồ

Giữa hình là queue, một hàng ô xếp theo thứ tự vào. Bên trái, producer put thêm vào cuối hàng rồi quay về việc của mình ngay — không chờ. Bên phải, worker lấy ở đầu hàng, làm việc chậm, rồi task_done() báo xong; bộ đếm việc chưa xong giảm một. Hai mũi tên không bao giờ chạm nhau: đó là điểm chính — producer và consumer chỉ gặp nhau qua queue, nên không cần Lock nào.



Hình T06-queue-worker. Producer đẩy việc vào queue, worker nền rút ra làm

Mô tả hình: Producer gọi put() thêm job vào cuối hàng đợi; worker thread gọi get() lấy ở đầu hàng, xử lý rồi task_done(); hai bên chỉ gặp nhau qua queue

Nguồn lý thuyết: SL 106-107 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. queue.Queue là hàng đợi an toàn cho nhiều thread: put() thêm vào cuối, get() lấy ở đầu (FIFO). Bên trong Queue đã có khoá nên producer và consumer không cần tự dùng Lock.
2. get() mặc định CHỜ khi queue rỗng — không trả None. get_nowait() không chờ: rỗng thì raise queue.Empty. Chọn cách chờ theo tình huống: worker chạy suốt (ví dụ minh hoạ) dùng get() + một sentinel None để báo dừng; xả hết việc đã có sẵn (bài này) dùng get_nowait() tới queue.Empty.
3. Mỗi get() thành công đi với đúng một task_done(); jobs.join() chờ tới khi mọi việc đã task_done. Quên task_done thì jobs.join() chờ mãi; gọi task_done nhiều hơn số việc → ValueError.
4. Tác vụ lưu nền: nơi gọi chỉ submit rồi đi tiếp, worker làm việc chậm (ghi file, pickle, nén — như bài thực hành lưu nền). Dữ liệu giao cho worker phải ổn định: put BẢN SAO dict(record), để nơi gọi sửa record sau đó không làm sai thứ sẽ được lưu.
5. Queue để trong self (self.jobs) để mọi method của object dùng chung. Mỗi lần flush tạo Thread MỚI — một Thread chỉ start được một lần.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import queue
import threading

jobs = queue.Queue()
for mark in [8, 6, 9]:
    jobs.put(mark)
jobs.put(None)

def consumer():
    while True:
        mark = jobs.get()
        if mark is None:
            jobs.task_done()
            break
        print("saved", mark)
        jobs.task_done()

worker = threading.Thread(target=consumer)
worker.start()
jobs.join()
worker.join()
print("queue rỗng:", jobs.empty())
```

Kết quả dự kiến (theo JSON): submit An 8, Binh 7 → flush() = 2, đúng thứ tự; sửa record sau submit → dòng đã lưu không đổi

NGỘ NHẬN THƯỜNG GẶP: "q.get() khi queue rỗng sẽ trả None"

get() mặc định CHỜ tới khi có phần tử — với queue không ai put thêm, chương trình treo. Muốn không chờ thì dùng get_nowait() và bắt queue.Empty; muốn worker dừng thì gửi một sentinel (None) có chủ đích.

CÂU DỰ ĐOÁN

Đoạn code sau in gì?

```
import queue

q = queue.Queue()
q.put("a")
q.put("b")
print(q.get(), q.get())
```

ĐÚNG	a b — vào trước ra trước (FIFO)	Queue lấy phần tử ở đầu hàng đợi.
sai	b a	Đó là LifoQueue (ngăn xếp), không phải Queue.
sai	a a	get() lấy phần tử RA khỏi queue.

TỰ KIỂM TRA

1. q.get() khi queue rỗng có mặc định trả None ngay không?

- Có, raise queue.Empty — Đó là get_nowait() (hoặc get(block=False)).
- ✔ Không; mặc định nó chờ — Chờ tới khi có thread khác put.
- Có — Đó là hiểu nhầm phổ biến — None chỉ có khi ai đó put None.

Đáp án: Chờ tới khi có thread khác put.

2. Worker get() 3 việc nhưng chỉ gọi task_done() 2 lần. jobs.join() sẽ...

- Raise ValueError — ValueError chỉ khi task_done nhiều hơn số việc.
- ✔ Chờ mãi — vẫn còn một việc chưa báo xong — Mỗi get() cần đúng một task_done().
- Trả về ngay — join chờ tới khi mọi việc đã task_done.

Đáp án: Mỗi get() cần đúng một task_done().

3. Vì sao submit đưa dict(record) chứ không đưa record?

- Queue không nhận dict — Queue nhận mọi object.
- Để chạy nhanh hơn — Sao chép còn tốn thêm một chút.
- ✔ Nơi gọi có thể sửa record trước khi worker lưu; bản sao giữ đúng dữ liệu lúc giao — Queue chỉ giữ tham chiếu tới object.

Đáp án: Queue chỉ giữ tham chiếu tới object.

TỰ GIẢI THÍCH

Vì sao _work dùng get_nowait() và dừng ở queue.Empty, còn worker chạy suốt lại cần get() và sentinel?

Mẫu trả lời: flush chỉ start worker khi mọi job đã được submit, nên queue rỗng nghĩa là hết việc: get_nowait() raise queue.Empty và worker kết thúc, không bao giờ phải chờ. Worker chạy suốt thì queue rỗng chỉ là "tạm chưa có việc", nên dùng get() để chờ và một sentinel None để biết khi nào dừng. Trường hợp đặc biệt: flush khi chưa submit gì → Empty ngay, trả 0.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	submit(record) → put bản sao vào self.jobs
Bước 2	flush() → Thread(target=self._work).start()
Bước 3	get_nowait() → lấy theo FIFO, lưu, task_done()
Bước 4	queue.Empty → worker kết thúc, join xong

CÂU PHẢN VÍ DỤ

Submit 2 record rồi flush; submit thêm 1 record rồi flush lần nữa. flush lần hai trả?

Lựa chọn: RuntimeError | 3 | 1 · Đáp án: 3

TEMPLATE LẮP MẢNH

```
    def submit(self, record):
        self.jobs.put({{slot1}})

    def _work(self):
        while True:
            try:
                record = {{slot2}}
            except queue.Empty:
                return
            self.lines.append(f"{record['name']},{record['mark']}")
            {{slot3}}

    def flush(self):
        worker = {{slot4}}
        worker.start()
        worker.join()
```

```
        return len(self.lines)
```

Slot	Mảnh đúng
[Slot 1]	`dict(record)`
[Slot 2]	`self.jobs.get_nowait()`
[Slot 3]	`self.jobs.task_done()`
[Slot 4]	`threading.Thread(target=self._work)`

Mảnh bẫy	Vì sao sai
`self.jobs.join()`	join chờ các task_done — gọi trong worker là tự chờ chính mình.
`self.jobs.get()`	Queue rỗng thì get() chờ mãi, không bao giờ tới queue.Empty.
`record`	Giao chính object của nơi gọi — sửa sau đó là đổi luôn dữ liệu sắp lưu.
`threading.Thread(target=self._work())`	Gọi _work ngay ở thread chính.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp bản sao khi submit, cách lấy việc không chờ, lời báo xong việc và Thread của flush.

Yêu cầu thêm: Thử flush khi chưa submit gì, flush hai lần liên tiếp, và sửa record sau khi submit.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành class BackgroundSaver (self.jobs = queue.Queue(), self.lines = [] đã có). submit(record): put BẢN SAO record (dict(record)) vào self.jobs — chưa lưu gì. _work(): lấy từng job bằng self.jobs.get_nowait() tới khi gặp queue.Empty thì return; mỗi job thêm dòng "name,mark" vào self.lines rồi gọi self.jobs.task_done(). flush(): tạo Thread MỚI với target=self._work, start, join, trả len(self.lines).

Thực hành: submit chỉ put bản sao; mọi việc lưu nằm trong _work chạy ở worker. Lấy bằng get_nowait() trong try/except queue.Empty; mỗi job xong gọi task_done(). flush tạo Thread mới mỗi lần.

Gọi	Kết quả mong đợi
`submit An 8, Binh 7; flush()`	`2; lines = ["An,8", "Binh,7"]`
`flush() khi chưa submit`	`0`
`sửa record sau submit rồi flush`	`dòng đã lưu vẫn là giá trị lúc submit`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`BackgroundSaver.submit(self, record)`	Giao một bản ghi cho worker nền. · record: dict có "name", "mark" · None · put BẢN SAO vào self.jobs; chưa lưu gì — việc lưu là của worker.
`BackgroundSaver.flush(self)`	Chạy worker nền xả hết việc trong queue. · self · int: tổng số dòng trong self.lines · Tạo Thread MỚI (target=self._work), start, join. Worker lấy bằng get_nowait() tới queue.Empty, mỗi job thêm "name,mark" vào self.lines rồi task_done().

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Lưu đúng thứ tự FIFO	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.submit({"name": 'Nga', "mark": 10}), s.flush(), s.lines,)[-1])(BackgroundSaver())`	`['Lan,9', 'Minh,6', 'Nga,10']`
flush trả số dòng đã lưu	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.submit({"name": 'Nga', "mark": 10}), s.flush(),)[-1])(BackgroundSaver())`	`3`
Chưa flush thì chưa lưu (việc nằm ở worker)	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), (s.lines, s.jobs.qsize()),)[-1])(BackgroundSaver())`	`([], 2)`
Sửa record sau submit không đổi dữ liệu đã giao	`(lambda s, r: (s.submit(r), r.update(mark=0), s.flush(), s.lines)[-1])(BackgroundSaver(), {"name": "Oanh", "mark": 8})`	`['Oanh,8']`
Chưa submit gì → flush trả 0	`(lambda s: (s.flush(), (s.lines, s.jobs.empty()),)[-1])(BackgroundSaver())`	`([], True)`
Mỗi get có đúng một task_done (gọi thêm → ValueError)	`(lambda s: (s.submit({"name": 'Lan', "mark": 9}), s.submit({"name": 'Minh', "mark": 6}), s.flush(), _studio.call(s.jobs.task_done).error,)[-1])(BackgroundSaver())`	`'ValueError'`

flush dùng một worker thread: start, join, target là method	`(lambda r: (r.thread_count, r.all_joined, r.targets_callable, r.value))((lambda s: (s.submit({"name": 'Phu', "mark": 5}), _studio.call(s.flush),)[-1])(BackgroundSaver()))`	`(1, True, True, 1)`
flush hai lần: mỗi lần một Thread mới	`(lambda s: (s.submit({"name": 'An', "mark": 1}), s.flush(), s.submit({"name": 'Binh', "mark": 2}), s.flush(), s.lines,)[-1])(BackgroundSaver())`	`['An,1', 'Binh,2']`

MÔI TRƯỜNG CHẠY

Trình duyệt không chạy được thread thật, nên Studio MÔ PHỎNG threading: target chạy lần lượt ngay khi start() và bộ chấm ghi lại thứ tự tạo / start / join. time.sleep() là đồng hồ ảo: các thread "ngủ" song song nên đo thời gian vẫn thấy lợi ích của thread với việc chờ I/O. Vì chạy lần lượt, producer phải start TRƯỚC consumer; một lệnh chờ không bao giờ được đáp (get() trên Queue rỗng, Event.wait() không ai set) sẽ dừng với thông báo deadlock thay vì treo. Hãy chạy thêm file trên máy — thứ tự dòng in của thread thật có thể khác mỗi lần. Vì worker chạy ngay lúc start(), bài này cho nơi gọi submit HẾT việc rồi mới flush, và worker xả queue bằng get_nowait() — không bao giờ phải chờ. Trên máy thật, worker chạy suốt thường dùng get() + sentinel None như ví dụ minh hoạ.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. submit(record) → put bản sao vào self.jobs
Khung 2 — Bước 2. flush() → Thread(target=self._work).start()
Khung 3 — Bước 3. get_nowait() → lấy theo FIFO, lưu, task_done()
Khung 4 — Bước 4. queue.Empty → worker kết thúc, join xong
Khung đối chiếu — ngộ nhận "q.get() khi queue rỗng sẽ trả None". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. get() mặc định CHỜ tới khi có phần tử — với queue không ai put thêm, chương trình treo. Muốn không chờ thì dùng get_nowait() và bắt queue.Empty; muốn worker dừng thì gửi một sentinel (None) có chủ đích.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: self.jobs.join(), self.jobs.get(), record, threading.Thread(target=self._work())
Khung kết quả — 3 cặp vào/ra. submit An 8, Binh 7; flush() → 2; lines = ["An,8", "Binh,7"]; flush() khi chưa submit → 0; sửa record sau submit rồi flush → dòng đã lưu vẫn là giá trị lúc submit

Nguồn: pe-v4 · python-learning-data.json#T06 · parser pe-v4/build · file build 013-T06.json

M3 · GUI Tkinter và dự án

7 bài: G01, G02, G03, G04, G05, G06, R02

Ứng viên animation — ưu tiên cao (★★): không có · ưu tiên (★): G01, G02, G03, G04, G05, G06, R02

G01 · GUI, CLI và cửa sổ

Mã bài: G01 · Nâng cao · M3 · GUI Tkinter và dự án · 25 phút · 20 XP · thứ tự 14

MỤC TIÊU

Tạo cửa sổ chính, cửa sổ phụ và chạy vòng lặp sự kiện.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

Cho app_title = "Student system" và size = "800x600". Tạo cửa sổ chính root = tk.Tk(), đặt tiêu đề app_title và kích thước size; tạo cửa sổ phụ details bằng tk.Toplevel(root) với tiêu đề "Details", kích thước "400x300"; dòng cuối gọi root.mainloop().

VÍ DỤ MINH HOẠ

app_title = "Student system" → root.title() trả "Student system"; details là Toplevel thuộc root.

LÝ THUYẾT

Định nghĩa: GUI và CLI

CLI (Command Line Interface) nhận lệnh gõ trong terminal rồi in kết quả ra màn hình. GUI (Graphical User Interface) vẽ cửa sổ, nút, ô nhập rồi CHỜ người dùng bấm hoặc gõ. Slide so sánh sáu tiêu chí: CLI khó học nhưng rất linh hoạt, tốn ít bộ nhớ, chạy nhanh, chỉ dùng bàn phím; GUI dễ học, kém linh hoạt hơn, tốn bộ nhớ hơn, chậm hơn, dùng được cả chuột.

Tkinter gồm bốn phần

- Window: tk.Tk() là cửa sổ chính, tk.Toplevel(root) là cửa sổ phụ
- Widgets: Frame, Label, Button, Entry… — mọi thứ người dùng nhìn thấy
- Layout: geometry manager xếp widget vào cửa sổ (pack, place, grid)
- Window event loop: mainloop() nhận sự kiện rồi gọi hàm xử lý
- Tkinter có sẵn trong Python (nền Tcl/Tk); PyQt, Kivy, wxPython phải cài thêm

Ví dụ: cửa sổ chính và cửa sổ phụ

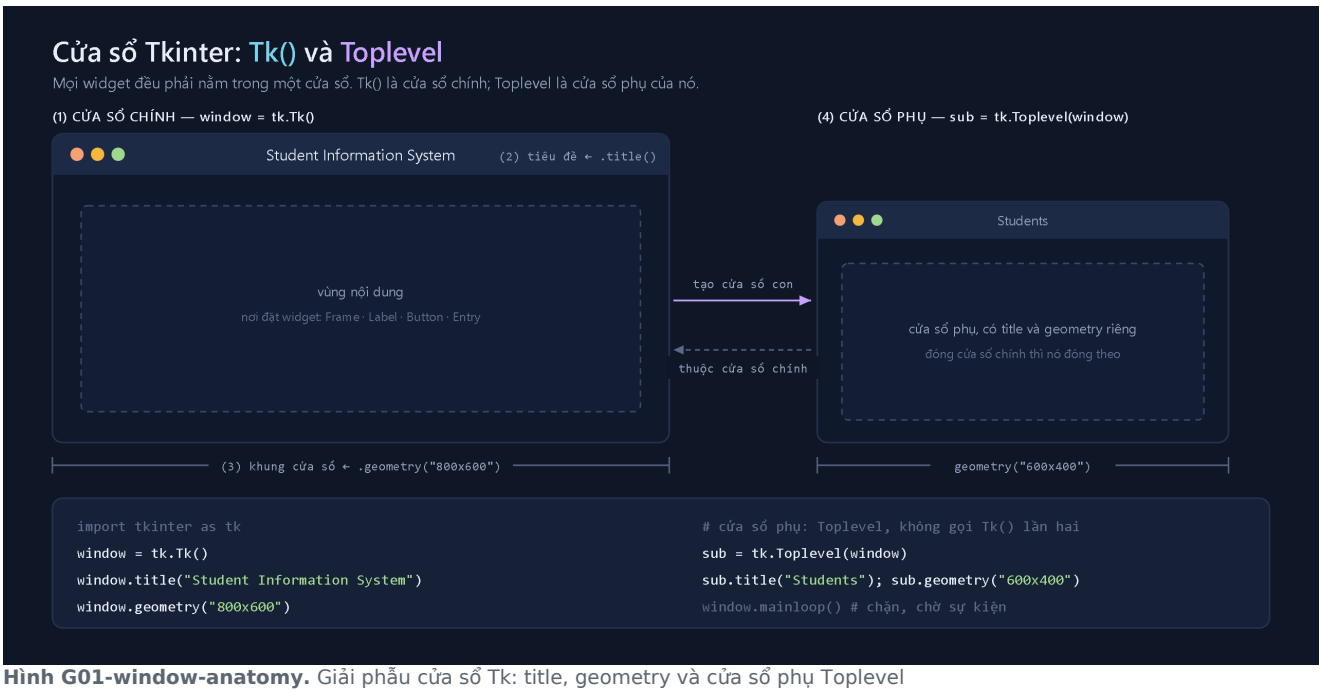
window = tk.Tk(); window.title("Student Information System"); window.geometry("800x600") dựng cửa sổ chính rộng 800, cao 600. sub = tk.Toplevel(window); sub.title("Students"); sub.geometry("600x400") mở thêm một cửa sổ phụ của CÙNG ứng dụng. Đóng cửa sổ chính thì cửa sổ phụ đóng theo, vì nó thuộc về root.

Dùng ở đâu

- Công cụ nội bộ: form nhập liệu, bảng điều khiển nhỏ cho người không quen terminal
- Tkinter đi kèm Python nên một file .py chạy được ngay, không phải cài thêm thư viện
- Cửa sổ phụ hợp với hộp Cài đặt, Trợ giúp, xem chi tiết một bản ghi
- Studio chạy tkinter GIẢ: không cửa sổ nào được vẽ, bộ chấm soi cây widget thay bạn
- Muốn thấy cửa sổ thật thì lưu file rồi chạy trên máy: python ten_file.py

Đọc sơ đồ

Hình vẽ một cửa sổ Tk: thanh tiêu đề mang chuỗi của title(), khung ngoài mang kích thước của geometry("800x600") — rộng x cao, chữ x thường — và vùng nội dung còn rỗng vì chưa có widget nào. Cửa sổ phụ Toplevel nằm bên cạnh, mũi tên trỏ về root nhắc quan hệ phụ thuộc. Cả hai chỉ hiện ra khi mainloop() chạy.



Hình G01-window-anatomy. Giải phẫu cửa sổ Tk: title, geometry và cửa sổ phụ Toplevel

Mô tả hình: Cửa sổ chính Tk với thanh tiêu đề, kích thước 800x600 và vùng nội dung rỗng; bên cạnh là cửa sổ phụ Toplevel có mũi tên trỏ về root

Nguồn lý thuyết: SL 108-117 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. CLI chạy lệnh từ trên xuống rồi kết thúc; GUI vẽ cửa sổ rồi CHỜ người dùng bấm, gõ (sự kiện) để gọi hàm xử lý.
2. Tkinter có sẵn trong Python nên các ví dụ dùng nó; PyQt, Kivy, wxPython là thư viện GUI khác cùng ý tưởng.
3. tk.Tk() tạo cửa sổ CHÍNH (một cái cho cả chương trình); tk.Toplevel(root) tạo cửa sổ PHỤ thuộc root.
4. title("...") và geometry("800x600") là lời gọi method: rộng x cao, chữ x thường. title() không đối số thì đọc lại tiêu đề.
5. mainloop() là vòng lặp sự kiện: nó chặn tại chỗ và chỉ trả về khi cửa sổ chính đóng — nên đặt ở cuối chương trình.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import tkinter as tk

root = tk.Tk()
root.title("Máy tính bỏ túi")
root.geometry("320x200")
print("Tiêu đề:", root.title())

help_window = tk.Toplevel(root)
help_window.title("Trợ giúp")
print("Cửa sổ phụ thuộc root:", help_window.master is root)

print("Gọi mainloop...")
root.mainloop()
print("mainloop đã trả về: cửa sổ đã đóng")
```

Kết quả dự kiến (theo JSON): Mặc định → root.title() = Student system, 800x600; app_title = "Quản lý điểm" → tiêu đề đổi theo

NGỘ NHẬN THƯỜNG GẶP: "Muốn thêm cửa sổ thì gọi tk.Tk() lần nữa"

Mỗi tk.Tk() khởi động một bộ thông dịch Tcl riêng: StringVar hay ảnh (PhotoImage) gắn với cửa sổ này không dùng được ở cửa sổ kia, lỗi rất khó hiểu. Cửa sổ phụ dùng tk.Toplevel(root). Cũng đừng viết root.title = "..." — đó là GHI ĐÈ method bằng một chuỗi: tiêu đề không đổi, còn root.title() sau đó nổ TypeError.

CÂU DỰ ĐOÁN

Trên máy thật, dòng print đặt SAU root.mainloop() chạy khi nào?

```
root = tk.Tk()
root.mainloop()
print("Bye")
```

sai	Ngay lập tức, cùng lúc cửa sổ hiện ra	Chỉ Studio mô phỏng mới cho mainloop trả về ngay.
sai	Không bao giờ chạy	Đóng cửa sổ thì mainloop trả về và code phía sau chạy tiếp.

ĐÚNG	Khi người dùng đóng cửa sổ chính — mainloop chặn tới lúc đó	Event loop chạy bên trong mainloop().

TỰ KIỂM TRA

1. Toplevel dùng để tạo gì?

✔ Một cửa sổ phụ thuộc ứng dụng Tk hiện tại — Đóng cửa sổ chính thì Toplevel đóng theo.
• Cửa sổ chính thứ hai, độc lập hoàn toàn — Cửa sổ chính là tk.Tk(); Toplevel phụ thuộc root.
• Một khung nằm bên trong cửa sổ — Đó là Frame; Toplevel là một cửa sổ riêng.

Đáp án: Đóng cửa sổ chính thì Toplevel đóng theo.

2. Khác biệt cốt lõi giữa chương trình CLI và GUI là gì?

✔ GUI chờ sự kiện trong event loop rồi gọi hàm xử lý; CLI chạy lệnh theo thứ tự — Vì thế GUI cần mainloop().
• CLI không nhận được dữ liệu từ người dùng — CLI nhận qua input() hoặc đối số dòng lệnh.
• GUI luôn chạy nhanh hơn CLI — Tốc độ không phải khác biệt cốt lõi; GUI thường tốn bộ nhớ hơn.

Đáp án: Vì thế GUI cần mainloop().

3. Viết root.title = "Student system" (không ngoặc) thì sao?

• Tiêu đề đổi như root.title("Student system") — Không có method nào được gọi.
• SyntaxError ngay khi chạy — Cú pháp gán hợp lệ — lỗi chỉ lộ ra lúc dùng.
✔ Tiêu đề không đổi; root.title giờ là chuỗi nên root.title() nổ TypeError — Đó là phép gán, không gọi method.

Đáp án: Đó là phép gán, không gọi method.

TỰ GIẢI THÍCH

Vì sao root.mainloop() phải nằm ở cuối chương trình?

Mẫu trả lời: mainloop() chặn tại chỗ để nhận và xử lý sự kiện cho tới khi cửa sổ chính đóng. Code đặt sau nó (tạo widget, cửa sổ phụ) chỉ chạy khi người dùng đã đóng cửa sổ — quá muộn. Vì vậy tạo xong mọi cửa sổ, widget rồi mới gọi mainloop(). Trường hợp đặc biệt: quên mainloop thì chạy bằng file, cửa sổ hiện rồi tắt ngay.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	tk.Tk() → tạo cửa sổ chính root
Bước 2	root.title(app_title) → đặt chữ trên thanh tiêu đề
Bước 3	tk.Toplevel(root) → tạo cửa sổ phụ thuộc root
Bước 4	root.mainloop() → chờ sự kiện tới khi cửa sổ đóng

CÂU PHẢN VÍ DỤ

Với app_title = "Quản lý điểm", root.title() trả gì?

Lựa chọn: Quản lý điểm | tk | Student system · Đáp án: Quản lý điểm

TEMPLATE LẮP MẢNH

```
import tkinter as tk

app_title = "Quản lý thư viện"
size = "640x480"

root = {{slot1}}
root.title(app_title)
{{slot2}}
details = {{slot3}}
details.title("Details")
{{slot4}}
```

Slot	Mảnh đúng
[Slot 1]	`tk.Tk()`
[Slot 2]	`root.geometry(size)`
[Slot 3]	`tk.Toplevel(root)`
[Slot 4]	`root.mainloop()`

Mảnh bẫy	Vì sao sai
`root.geometry("640*480")`	Kích thước dùng chữ x: "640x480" — dấu * làm Tk báo TclError.
`root.geometry = size`	Ghi đè method bằng chuỗi; kích thước không đổi.
`tk.Tk`	Thiếu ngoặc: root là class Tk chứ chưa phải cửa sổ.
`root.mainloop`	Thiếu ngoặc: chỉ nhắc tên method, vòng lặp sự kiện không chạy.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: "Quản lý thư viện", 640x480. Lắp bốn lời gọi.

Yêu cầu thêm: Đổi app_title và size (vd. "Quản lý điểm", "640x480"); bộ chấm chạy lại chương trình rồi đọc tiêu đề và kích thước của cửa sổ mới.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Cho app_title = "Student system" và size = "800x600". Tạo cửa sổ chính root = tk.Tk(), đặt tiêu đề app_title và kích thước size; tạo cửa sổ phụ details bằng tk.Toplevel(root) với tiêu đề "Details", kích thước "400x300"; dòng cuối gọi root.mainloop().

Thực hành: title và geometry là method nên phải có ngoặc và đối số; cửa sổ phụ là tk.Toplevel(root); root.mainloop() ở dòng cuối.

Gọi	Kết quả mong đợi
`app_title = "Student system", size = "800x600"`	`Cửa sổ chính 800x600 tên Student system; cửa sổ phụ Details 400x300`
`app_title = "Quản lý điểm", size = "640x480"`	`root.title() trả "Quản lý điểm", kích thước 640x480`

BỘ CHẤM (7 ca)

Ca	Gọi	Mong đợi
Tiêu đề cửa sổ chính	`root.title()`	`'Student system'`
Kích thước 800x600	`root.geometry().split('+')[0]`	`'800x600'`
Cửa sổ phụ là Toplevel, không phải Tk thứ hai	`type(details).__name__`	`'Toplevel'`
Cửa sổ phụ thuộc root	`details.master is root`	`True`
Cửa sổ phụ: Details, 400x300	`(details.title(), details.geometry().split('+')[0])`	`('Details', '400x300')`
Đã gọi mainloop()	`_studio.tk.mainloop_called`	`True`
Dữ liệu khác: "Quản lý điểm", 640x480	`(lambda w: (w.title(), w.geometry().split('+')[0]))(_studio.run(vars={'app_title': 'Quản lý điểm', 'size': '640x480'}).ns['root'])`	`('Quản lý điểm', '640x480')`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. tk.Tk() → tạo cửa sổ chính root
Khung 2 — Bước 2. root.title(app_title) → đặt chữ trên thanh tiêu đề
Khung 3 — Bước 3. tk.Toplevel(root) → tạo cửa sổ phụ thuộc root
Khung 4 — Bước 4. root.mainloop() → chờ sự kiện tới khi cửa sổ đóng
Khung đối chiếu — ngộ nhận "Muốn thêm cửa sổ thì gọi tk.Tk() lần nữa". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Mỗi tk.Tk() khởi động một bộ thông dịch Tcl riêng: StringVar hay ảnh (PhotoImage) gắn với cửa sổ này không dùng được ở cửa sổ kia, lỗi rất khó hiểu. Cửa sổ phụ dùng tk.Toplevel(root). Cũng đừng viết root.title = "..." — đó là GHI ĐÈ method bằng một chuỗi: tiêu đề không đổi, còn root.title() sau đó nổ TypeError.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: root.geometry("640*480"), root.geometry = size, tk.Tk, root.mainloop
Khung kết quả — 2 cặp vào/ra. app_title = "Student system", size = "800x600" → Cửa sổ chính 800x600 tên Student system; cửa sổ phụ Details 400x300; app_title = "Quản lý điểm", size = "640x480" → root.title() trả "Quản lý điểm", kích thước 640x480

Nguồn: pe-v4 · python-learning-data.json#G01 · parser pe-v4/build · file build 014-G01.json

G02 · Widget và callback

Mã bài: G02 · Nâng cao · M3 · GUI Tkinter và dự án · 30 phút · 20 XP · thứ tự 15

MỤC TIÊU

Nối nút với callback và đọc Entry đúng lúc bấm.

CẦN NHỚ TRƯỚC

G01

BỐI CẢNH

Hoàn thành form đăng nhập (đã có root, frame, user_entry): tạo password_entry che ký tự bằng show="*", Label status với chữ "Chưa đăng nhập", và Button chữ "Đăng nhập" với command=login. Hàm login() đọc hai Entry LÚC ĐƯỢC BẤM: tên bỏ khoảng trắng hai đầu; mật khẩu bằng PASSWORD thì status.config(text="Xin chào <tên>") và messagebox.showinfo("Đăng nhập", "Chào mừng <tên>!"); sai thì status.config(text="Sai mật khẩu").

VÍ DỤ MINH HOẠ

Gõ "Mai" và "usth2026", bấm Đăng nhập → status "Xin chào Mai" và hộp thông báo "Chào mừng Mai!".

LÝ THUYẾT

Định nghĩa: widget và callback

Trong Tkinter "everything is widget": Frame là khung chứa widget khác, Label hiện chữ, Button bấm được, Entry nhập một dòng. Widget nhận thuộc tính ngay lúc tạo (text, width, height, bg). Callback là hàm BẠN viết nhưng TKINTER gọi — khi sự kiện xảy ra. Với Button, tuỳ chọn command giữ hàm đó lại chờ người dùng bấm.

Ví dụ: nút gọi hàm

def onClick(): messagebox.showinfo(message="Button 1 clicked") rồi tk.Button(window, text="Button 1", command=onClick). Lúc dựng giao diện, onClick CHƯA chạy: nút chỉ cất tên hàm. Mỗi lần người dùng bấm, Tkinter mới gọi onClick() và hộp thông báo bật lên. Thêm ngoặc (command=onClick()) là tự tay gọi một lần rồi giao None cho nút.

Nối sự kiện: command và bind

- command=f: chỉ Button, Checkbutton, Radiobutton… — Tkinter gọi f() không đối số
- widget.bind("<Return>", f): mọi widget, mọi sự kiện — Tkinter gọi f(event)
- event mang thông tin: phím nào, chuột ở đâu, widget nào phát sinh
- messagebox.showinfo / showwarning / showerror bật hộp thông báo, cảnh báo, lỗi
- Trong callback: entry.get() đọc chữ, label.config(text=…) đổi chữ, cget("text") đọc lại

Dùng ở đâu

- Form đăng nhập, form nhập liệu: Entry thu dữ liệu, Button xác nhận, Label báo trạng thái
- Mọi thao tác người dùng đều đi qua callback — đó là cách GUI thay cho input() của CLI
- Hộp messagebox để báo việc đã xong hoặc dữ liệu sai, thay cho print()
- Studio không vẽ cửa sổ nên bộ chấm "bấm" bằng button.invoke() và đọc _studio.tk.messages
- Chạy file trên máy (python ten_file.py) để thật sự bấm nút và thấy hộp thông báo

Đọc sơ đồ

Hình chia làm hai nhịp. Nhịp dựng giao diện: Button cất tham chiếu tới hàm, chưa gọi. Nhịp người dùng bấm: sự kiện đi vào Tkinter, Tkinter gọi callback, callback đọc Entry rồi gọi label.config(text=…), và chữ trên Label đổi. Mũi tên nét liền là lời gọi, ô nét đứt là dữ liệu đọc tại thời điểm bấm — không phải lúc dựng giao diện.



Hình G02-widget-callback. Bấm Button → Tkinter gọi callback → Label đổi chữ

Mô tả hình: Sơ đồ: Button giữ hàm qua command; khi bấm, Tkinter gọi hàm, hàm đọc Entry bằng get() rồi gọi label.config(text=...) làm Label đổi chữ

Nguồn lý thuyết: SL 118-120 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Frame chứa widget khác; Label hiện chữ; Entry nhập một dòng (show="*" che mật khẩu); Button gọi callback khi bấm.
2. width, height, bg chỉnh kích thước và màu (tuỳ widget). Tạo xong phải pack() thì widget mới được xếp vào cửa sổ.
3. pack() trả về None: viết status = tk.Label(...) rồi status.pack() ở dòng riêng để còn giữ được widget.
4. command=login giao HÀM cho nút, Tkinter gọi login() mỗi lần bấm. command=login() gọi ngay lúc dựng giao diện.
5. Trong callback: entry.get() đọc chữ đang gõ, label.config(text=...) đổi chữ, cget("text") đọc lại chữ hiện tại.
6. messagebox.showinfo(tiêu đề, nội dung) bật hộp thông báo; showwarning, showerror là dạng cảnh báo và báo lỗi.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import tkinter as tk
from tkinter import messagebox

root = tk.Tk()
root.title("Đếm lượt bấm")
clicks = 0

counter = tk.Label(root, text="0 lượt", width=12, bg="lightyellow")
counter.pack()


def on_click():
    global clicks
    clicks += 1
    counter.config(text=f"{clicks} lượt")
    if clicks == 3:
        messagebox.showinfo("Info", "Đã bấm 3 lần")


button = tk.Button(root, text="Bấm tôi", command=on_click)
button.pack()

button.invoke()   # "bấm" bằng code — bộ chấm cũng làm vậy
button.invoke()
print(counter.cget("text"))
root.mainloop()
```

Kết quả dự kiến (theo JSON): Mai + usth2026 → Xin chào Mai, hộp "Chào mừng Mai!"; Sai mật khẩu → Sai mật khẩu, không hộp thông báo

NGỘ NHẬN THƯỜNG GẶP: "status = tk.Label(...).pack() cho gọn"

pack() xếp widget rồi trả về None, nên status thành None và status.config(...) trong callback nổ AttributeError — đúng lúc người dùng bấm nút, không phải lúc chạy chương trình. Giữ widget vào biến trước, pack ở dòng sau. Tương tự, command nhận TÊN hàm: command=login, không phải login().

CÂU DỰ ĐOÁN

Bấm nút này thì chuyện gì xảy ra?

```
button = tk.Button(root, text="Open", command=clicked())
```

sai	clicked chạy mỗi lần bấm như bình thường	Đó là command=clicked (không ngoặc).
sai	Báo lỗi mỗi lần bấm	command rỗng thì bấm không làm gì, cũng không báo lỗi.
ĐÚNG	Không có gì: clicked() đã chạy một lần lúc tạo nút, command nhận None	Dấu ngoặc là lời gọi ngay.

TỰ KIỂM TRA

1. Ở command=clicked có dùng dấu ngoặc gọi hàm không?

✔ Không — truyền tên hàm để Tkinter gọi khi bấm — Có ngoặc là gọi ngay lúc tạo nút.

• Có — phải viết command=clicked() — Khi đó clicked chạy một lần lúc dựng giao diện.

• Tuỳ phiên bản Python — Quy tắc callback không đổi theo phiên bản.

Đáp án: Có ngoặc là gọi ngay lúc tạo nút.

2. Entry mật khẩu cần tuỳ chọn nào để hiện dấu * thay cho ký tự?

✔ show="*" — Entry vẫn giữ chữ thật; get() trả đúng mật khẩu đã gõ.

• text="*" — Entry không dùng text để che chữ.

• password=True — Tkinter không có tuỳ chọn này.

Đáp án: Entry vẫn giữ chữ thật; get() trả đúng mật khẩu đã gõ.

3. entry.bind("<Return>", handler) khác command=handler ở điểm nào?

✔ bind gọi handler(event) — handler cần một tham số nhận event — command gọi handler() không đối số.

• bind chỉ dùng được cho Button — Mọi widget đều bind được sự kiện.

• Không khác gì — bind luôn truyền đối tượng event vào hàm.

Đáp án: command gọi handler() không đối số.

TỰ GIẢI THÍCH

Vì sao login() phải gọi user_entry.get() BÊN TRONG hàm, không đọc sẵn ở ngoài?

Mẫu trả lời: Dòng ngoài hàm chạy một lần lúc dựng giao diện, khi Entry còn rỗng, nên giá trị đọc được là "". Callback chạy mỗi lần bấm nút, lúc người dùng đã gõ xong, nên phải gọi get() trong callback để lấy chữ hiện tại. Trường hợp đặc biệt: mật khẩu rỗng vẫn phải ra "Sai mật khẩu".

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	tk.Entry(frame, show="*") → ô mật khẩu che ký tự
Bước 2	command=login → nút giữ hàm, chưa gọi
Bước 3	Bấm nút → Tkinter gọi login()
Bước 4	login() → get() hai Entry rồi status.config(text=...)

CÂU PHẢN VÍ DỤ

Gõ tên "An", mật khẩu "abc" rồi bấm Đăng nhập — status hiện gì?

Lựa chọn: Chưa đăng nhập | Xin chào An | Sai mật khẩu · Đáp án: Sai mật khẩu

TEMPLATE LẮP MẢNH

```
import tkinter as tk

root = tk.Tk()
name_entry = tk.Entry(root)
name_entry.pack()
greeting = {{slot1}}
greeting.pack()


def send():
    name = {{slot2}}
    greeting.config(text="Hello " + name)


tk.Button(root, text="Gửi", command={{slot3}}).pack()
root.mainloop()
```

Slot	Mảnh đúng
[Slot 1]	`tk.Label(root, text="")`
[Slot 2]	`name_entry.get()`
[Slot 3]	`send`

Mảnh bẫy	Vì sao sai
`send()`	Gọi send ngay lúc dựng giao diện; nút nhận None.
`tk.Label(root, text="").pack()`	pack() trả về None nên greeting thành None.
`name_entry`	Đó là widget, không phải chữ đã gõ — cần get().

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: nút "Gửi" chào theo tên trong Entry. Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm gõ tên và mật khẩu khác (đúng, sai, rỗng) vào hai Entry, bấm nút bằng invoke() rồi đọc status.cget("text") và hộp thông báo.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành form đăng nhập (đã có root, frame, user_entry): tạo password_entry che ký tự bằng show="*", Label status với chữ "Chưa đăng nhập", và Button chữ "Đăng nhập" với command=login. Hàm login() đọc hai Entry LÚC ĐƯỢC BẤM: tên bỏ khoảng trắng hai đầu; mật khẩu bằng PASSWORD thì status.config(text="Xin chào <tên>") và messagebox.showinfo("Đăng nhập", "Chào mừng <tên>!"); sai thì status.config(text="Sai mật khẩu").

Thực hành: login() gọi user_entry.get() và password_entry.get() ngay trong hàm, so với PASSWORD rồi đổi chữ status bằng config(text=...). Nút dùng command=login (không ngoặc).

Gọi	Kết quả mong đợi
`Tên "Mai", mật khẩu "usth2026"`	`status: Xin chào Mai; hộp thông báo "Chào mừng Mai!"`
`Mật khẩu sai hoặc rỗng`	`status: Sai mật khẩu; không bật hộp thông báo`

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Chưa bấm: status "Chưa đăng nhập"	`status.cget('text')`	`'Chưa đăng nhập'`
Có nút "Đăng nhập"	`_studio.tk.find(type='Button', text='Đăng nhập') is not None`	`True`
Ô mật khẩu che bằng *	`password_entry.cget('show')`	`'*'`
Mai + mật khẩu đúng	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1]`	`'Xin chào Mai'`
Mật khẩu sai	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), password_entry.delete(0, 'end'), password_entry.insert(0, 'usth'), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1]`	`'Sai mật khẩu'`
Tên khác có khoảng trắng " An "	`(user_entry.delete(0, 'end'), user_entry.insert(0, ' An '), password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1]`	`'Xin chào An'`
Mật khẩu rỗng	`(user_entry.delete(0, 'end'), user_entry.insert(0, 'Mai'), password_entry.delete(0, 'end'), password_entry.insert(0, ''), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1]`	`'Sai mật khẩu'`
Đúng mật khẩu → showinfo("Đăng nhập", "Chào mừng Lan!")	`((user_entry.delete(0, 'end'), user_entry.insert(0, 'Lan'), password_entry.delete(0, 'end'), password_entry.insert(0, 'usth2026'), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1], (lambda m: ('info' in str(m[0]), m[1], m[2]))(tuple(_studio.tk.messages[-1])))[1]`	`(True, 'Đăng nhập', 'Chào mừng Lan!')`

Sai mật khẩu → không bật hộp thông báo	`(lambda n: ((user_entry.delete(0, 'end'), user_entry.insert(0, 'Lan'), password_entry.delete(0, 'end'), password_entry.insert(0, 'sai'), _studio.tk.find(type='Button', text='Đăng nhập').invoke(), status.cget('text'))[-1], len(_studio.tk.messages) - n)[1])(len(_studio.tk.messages))`	`0`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. tk.Entry(frame, show="*") → ô mật khẩu che ký tự
Khung 2 — Bước 2. command=login → nút giữ hàm, chưa gọi
Khung 3 — Bước 3. Bấm nút → Tkinter gọi login()
Khung 4 — Bước 4. login() → get() hai Entry rồi status.config(text=...)
Khung đối chiếu — ngộ nhận "status = tk.Label(...).pack() cho gọn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. pack() xếp widget rồi trả về None, nên status thành None và status.config(...) trong callback nổ AttributeError — đúng lúc người dùng bấm nút, không phải lúc chạy chương trình. Giữ widget vào biến trước, pack ở dòng sau. Tương tự, command nhận TÊN hàm: command=login, không phải login().
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: send(), tk.Label(root, text="").pack(), name_entry
Khung kết quả — 2 cặp vào/ra. Tên "Mai", mật khẩu "usth2026" → status: Xin chào Mai; hộp thông báo "Chào mừng Mai!"; Mật khẩu sai hoặc rỗng → status: Sai mật khẩu; không bật hộp thông báo

Nguồn: pe-v4 · python-learning-data.json#G02 · parser pe-v4/build · file build 015-G02.json

G03 · Dữ liệu nhập và lựa chọn

Mã bài: G03 · Nâng cao · M3 · GUI Tkinter và dự án · 35 phút · 20 XP · thứ tự 16

MỤC TIÊU

Đọc Entry, Combobox, Radiobutton, Checkbutton bằng get() và kiểm dữ liệu trước khi dùng.

CẦN NHỚ TRƯỚC

G02

BỐI CẢNH

Form nhập điểm đã có name_var, mark_var (StringVar gắn hai Entry), course_box (ttk.Combobox), retake_var (BooleanVar của Checkbutton "Thi lại"), marks_list (Listbox) và status. (1) Tạo hai Radiobutton "Giữa kỳ", "Cuối kỳ" dùng CHUNG exam_var (value bằng đúng chữ trên nút). (2) Viết add_mark(): họ tên rỗng → messagebox.showwarning, dừng; điểm đổi sang float (chấp nhận "8,5"), không phải số hoặc ngoài 0..10 → messagebox.showerror, dừng; hợp lệ → thêm dòng "<tên> | <môn> | <kỳ thi> | <điểm>" (điểm là float vừa đổi, in thẳng: 8.5, 7.0, 9.25; thêm " | thi lại" nếu đánh dấu) vào cuối marks_list và đổi status thành "Đã nhập <số dòng> điểm".

VÍ DỤ MINH HOẠ

Mai, "8.5", Python, Giữa kỳ → dòng "Mai | Python | Giữa kỳ | 8.5"; "abc" → hộp lỗi, Listbox không đổi.

LÝ THUYẾT

Định nghĩa: widget nhập và biến Tk

Slide liệt kê năm widget thu dữ liệu: Entry (nhập chữ, show="*" cho mật khẩu), Checkbutton (bật/tắt), Radiobutton (chọn MỘT trong nhiều), Listbox (danh sách chọn được), ttk.Combobox (danh sách thả xuống). Chúng không giữ giá trị trong biến Python thường mà qua BIẾN ĐIỀU KHIỂN của Tk: StringVar, IntVar, DoubleVar, BooleanVar — đọc bằng get(), đặt bằng set().

Ví dụ: Radiobutton dùng chung một biến

radioValue = tk.StringVar(value="op1"); tk.Radiobutton(window, variable=radioValue, text="Radiobutton option 1", value="op1") và một nút nữa với value="op2". Slide ghi rõ "Text != Value": chữ hiện cho người đọc, value là thứ radioValue.get() trả về. Hai nút cùng variable mới thành một nhóm; khác variable thì bấm cái nào cũng không tắt cái kia.

Mỗi widget một cách đọc

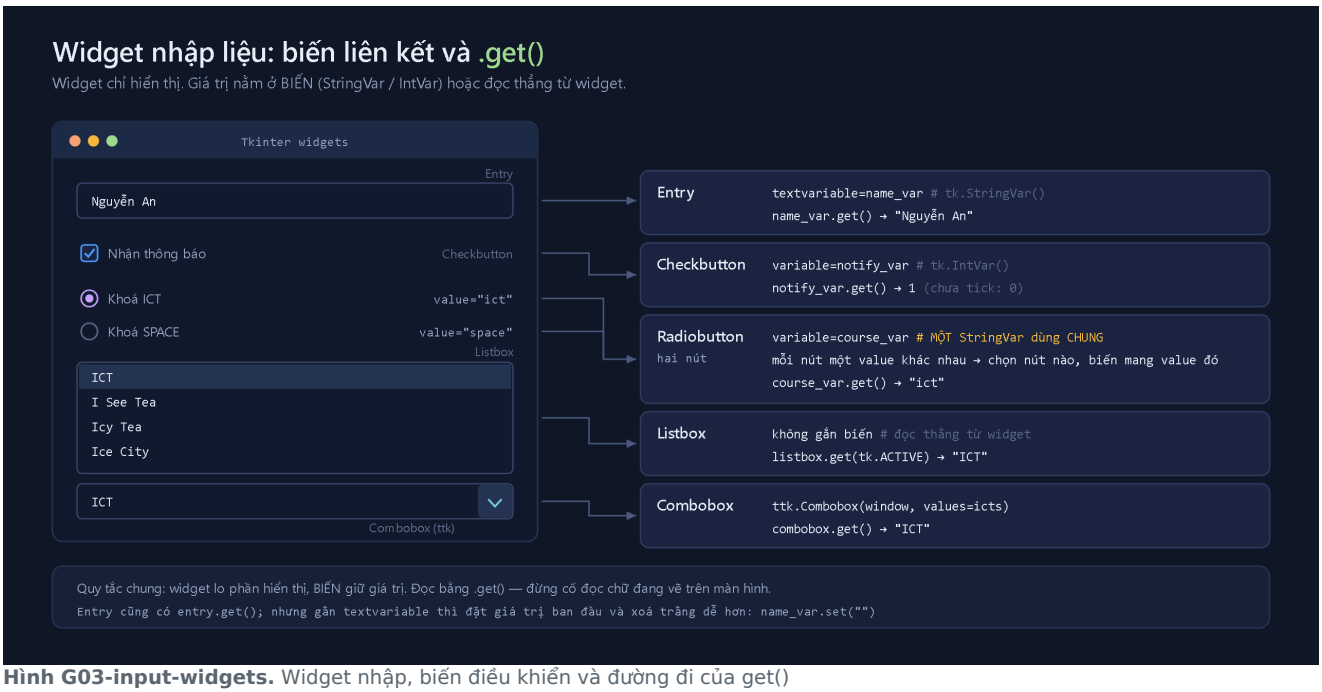
- Entry: entry.get() luôn trả CHUỖI — "8.5" chưa phải số, phải float() trong try/except
- Checkbutton: gắn BooleanVar, var.get() trả True/False
- Radiobutton: nhóm chung một StringVar, var.get() trả value của nút đang chọn
- Listbox: insert(tk.END, chuỗi) thêm cuối, size() đếm dòng, get(tk.ACTIVE) lấy mục active
- ttk.Combobox(values=[...]): get() lấy mục đang chọn; state="readonly" chặn gõ chữ lạ

Dùng ở đâu

- Form nhập điểm, phiếu khảo sát, hộp thoại lọc dữ liệu: mỗi ô là một widget nhập
- Combobox/Radiobutton thay cho gõ tay nên dữ liệu chỉ nằm trong tập hợp lệ cho trước
- Kiểm tra TRƯỚC khi dùng: sai thì messagebox rồi return, đừng ghi dữ liệu nửa vời
- Studio dựng widget giả: bộ chấm set() biến rồi invoke() nút thay cho người gõ
- Trên máy thật hãy tự gõ "abc" vào ô điểm để xem nhánh báo lỗi chạy đúng chưa

Đọc sơ đồ

Hình xếp năm widget nhập thành một cột, mỗi widget nối sang ô biến điều khiển của nó (StringVar, BooleanVar…). Từ biến có một mũi tên get() đi ra, luôn mang giá trị CHUỖI hoặc bool chứ chưa phải số. Hai Radiobutton chụm vào CÙNG một StringVar — đó là chi tiết hay bị vẽ sai nhất. Cuối sơ đồ là chốt kiểm tra: hợp lệ thì dùng, sai thì messagebox.



Hình G03-input-widgets. Widget nhập, biến điều khiển và đường đi của get()

Mô tả hình: Sơ đồ Entry, Checkbutton, hai Radiobutton chung một StringVar, Listbox và Combobox; mỗi cái nối tới biến điều khiển rồi tới get() và bước kiểm tra dữ liệu

Nguồn lý thuyết: SL 121-124 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. StringVar, IntVar, DoubleVar, BooleanVar giữ giá trị gắn với widget: get() đọc, set() đặt — không đọc thẳng object.
2. Entry luôn cho CHUỖI: "8.5" chưa phải số. float() đổi kiểu và nổ ValueError với "abc", nên bọc trong try/except.
3. Checkbutton bật/tắt một biến; các Radiobutton cùng nhóm phải dùng CHUNG một biến, mỗi nút một value.
4. Listbox.insert(tk.END, chuỗi) thêm cuối, size() đếm dòng; get(tk.ACTIVE) chỉ là mục active, chưa chắc là mục được chọn.
5. ttk.Combobox(values=[...]) chứa lựa chọn, get() lấy giá trị đang chọn; state="readonly" chặn gõ chữ lạ vào.
6. Kiểm tra trước, dùng sau: dữ liệu sai thì báo bằng messagebox và return — không thêm dòng nửa vời.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
size = tk.StringVar(value="M")
for text in ["S", "M", "L"]:
    tk.Radiobutton(root, text=text, value=text, variable=size).pack()

ice = tk.IntVar(value=0)
tk.Checkbutton(root, text="Thêm đá", variable=ice).pack()

drink = ttk.Combobox(root, values=["Trà", "Cà phê"], state="readonly")
drink.set("Trà")
drink.pack()

orders = tk.Listbox(root)
orders.pack()

ice.set(1)                      # như người dùng tích ô
drink.set("Cà phê")             # như người dùng chọn mục
orders.insert(tk.END, f"{drink.get()} size {size.get()}, đá = {ice.get()}")
print(orders.get(0, tk.END))

cups = "3"                      # Entry luôn trả chuỗi
print(int(cups) * 2, cups * 2)
```

Kết quả dự kiến (theo JSON): Mai, 8.5, Python, Giữa kỳ → "Mai | Python | Giữa kỳ | 8.5"; "abc" → showerror, Listbox không đổi

NGỘ NHẬN THƯỜNG GẶP: "Viết if retake_var: là kiểm ô đã tích"

retake_var là OBJECT BooleanVar — object luôn được coi là đúng, nên nhánh "thi lại" chạy cả khi ô chưa tích. Phải đọc giá trị: if retake_var.get():. Tương tự float(mark_var) nổ TypeError vì truyền object, không phải chuỗi. Và Radiobutton mỗi nút một biến riêng thì không còn là một nhóm: chọn "Cuối kỳ" không đổi exam_var.

CÂU DỰ ĐOÁN

Entry chứa "8.5" — mark_var.get() trả gì?

sai	Số thực 8.5	StringVar luôn trả str.
ĐÚNG	Chuỗi "8.5" — phải float() mới thành số 8.5	Entry chỉ biết chữ; đổi kiểu là việc của bạn.
sai	Số nguyên 8	Không có phép đổi kiểu tự động nào.

TỰ KIỂM TRA

1. Entry chứa "8.5" có tự là float không?
   - Có, Tkinter tự nhận ra số — Entry không biết kiểu dữ liệu.
   - ✔ Không — phải chuyển kiểu và xử lý dữ liệu không hợp lệ — get() luôn trả chuỗi.
   - Có, nếu dùng StringVar — StringVar cũng trả chuỗi.

Đáp án: get() luôn trả chuỗi.

2. Hai Radiobutton "Giữa kỳ", "Cuối kỳ" phải cấu hình thế nào để thành một nhóm?
   - ✔ Cùng variable=exam_var, mỗi nút một value khác nhau — Chọn nút nào thì exam_var nhận value của nút đó.
   - Mỗi nút một StringVar riêng — Khi đó chúng độc lập, không loại trừ nhau.
   - Cùng text, khác variable — text chỉ là chữ hiển thị.

Đáp án: Chọn nút nào thì exam_var nhận value của nút đó.

3. float("8,5") cho kết quả gì?
   - 85.0 — Dấu phẩy không bị bỏ qua.
   - ✔ ValueError — Python dùng dấu chấm thập phân — Thay "," bằng "." trước khi đổi kiểu.
   - 8.5 — float không hiểu dấu phẩy thập phân.

Đáp án: Thay "," bằng "." trước khi đổi kiểu.

4. marks_list.get(tk.ACTIVE) trả gì?
   - Mục cuối cùng — Mục cuối là get(tk.END).
   - Mọi mục trong Listbox — Muốn hết thì get(0, tk.END).
   - ✔ Mục đang active (có con trỏ), không nhất thiết là mọi mục được chọn — Mục được chọn: curselection().

Đáp án: Mục được chọn: curselection().

TỰ GIẢI THÍCH

Vì sao add_mark phải dừng (return) ngay khi điểm không hợp lệ, trước khi insert vào Listbox?

Mẫu trả lời: Dữ liệu từ giao diện là chuỗi người dùng gõ, có thể là "abc" hoặc "11". Đổi kiểu và kiểm khoảng trước; sai thì báo bằng messagebox rồi return để Listbox và status không nhận dòng hỏng. Trường hợp đặc biệt: "8,5" hợp lệ sau khi thay dấu phẩy; 0 và 10 nằm trong khoảng.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	name_var.get().strip() → họ tên đã bỏ khoảng trắng
Bước 2	float(raw.replace(",", ".")) → "8,5" thành 8.5
Bước 3	Không phải số hoặc ngoài 0..10 → showerror rồi return
Bước 4	marks_list.insert(tk.END, line) → thêm dòng cuối Listbox

CÂU PHẢN VÍ DỤ

Không thay "," bằng "." thì float("9,25") làm gì?

Lựa chọn: Trả 9.25 | Trả 925.0 | ValueError · Đáp án: ValueError

TEMPLATE LẮP MẢNH

```
import tkinter as tk

root = tk.Tk()
shift_var = tk.StringVar(value="Sáng")
laptop_var = tk.BooleanVar(value=False)

for text in ["Sáng", "Chiều"]:
    tk.Radiobutton(root, text=text, value=text, variable={{slot1}}).pack()
tk.Checkbutton(root, text="Có laptop", variable=laptop_var).pack()
summary = tk.Label(root, text="")
summary.pack()


def show():
    note = "mang laptop" if {{slot2}} else "không laptop"
    summary.config(text="Ca " + {{slot3}} + ", " + note)
```

```
tk.Button(root, text="Xem", command=show).pack()
root.mainloop()
```

Slot	Mảnh đúng
[Slot 1]	`shift_var`
[Slot 2]	`laptop_var.get()`
[Slot 3]	`shift_var.get()`

Mảnh bẫy	Vì sao sai
`laptop_var`	Object BooleanVar luôn được coi là đúng — phải gọi get().
`tk.StringVar(value="Sáng")`	Mỗi nút một biến mới: hai Radiobutton không còn chung nhóm.
`str(shift_var)`	str() của StringVar là TÊN biến Tcl (PY_VAR0), không phải giá trị.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: chọn ca học bằng Radiobutton và đánh dấu "Có laptop". Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm điền tên/điểm khác, chọn môn bằng course_box.set, bấm Radiobutton "Cuối kỳ", tích hoặc bỏ "Thi lại", rồi bấm Thêm và đọc dòng cuối Listbox.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Form nhập điểm đã có name_var, mark_var (StringVar gắn hai Entry), course_box (ttk.Combobox), retake_var (BooleanVar của Checkbutton "Thi lại"), marks_list (Listbox) và status.

(1) Tạo hai Radiobutton "Giữa kỳ", "Cuối kỳ" dùng CHUNG exam_var (value bằng đúng chữ trên nút).

(2) Viết add_mark(): họ tên rỗng → messagebox.showwarning, dừng; điểm đổi sang float (chấp nhận "8,5"), không phải số hoặc ngoài 0..10 → messagebox.showerror, dừng; hợp lệ → thêm dòng "<tên> | <môn> | <kỳ thi> | <điểm>" (điểm là float vừa đổi, in thẳng: 8.5, 7.0, 9.25; thêm " | thi lại" nếu đánh dấu) vào cuối marks_list và đổi status thành "Đã nhập <số dòng> điểm".

Thực hành: Mỗi giá trị đọc qua get(); điểm qua replace(",", ".") rồi float() trong try/except; kiểm 0..10; chỉ insert khi mọi thứ hợp lệ.

Gọi	Kết quả mong đợi
`Mai, "8.5", Python, Giữa kỳ, không thi lại`	`Dòng "Mai | Python | Giữa kỳ | 8.5"; status "Đã nhập 1 điểm"`
`Điểm "abc" hoặc "10.5"`	`showerror; Listbox giữ nguyên`
`Họ tên rỗng`	`showwarning; Listbox giữ nguyên`

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Chưa nhập: Listbox rỗng, status "Chưa có điểm"	`(marks_list.size(), status.cget('text'))`	`(0, 'Chưa có điểm')`
Mai, 8.5, Python, Giữa kỳ	`(name_var.set('Mai'), mark_var.set('8.5'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`	`'Mai | Python | Giữa kỳ | 8.5'`
Radiobutton Cuối kỳ + Data + thi lại	`(name_var.set('An'), mark_var.set('7'), course_box.set('Data'), _studio.tk.find(type='Radiobutton', text='Cuối kỳ').invoke(), retake_var.set(True), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`	`'An | Data | Cuối kỳ | 7.0 | thi lại'`
Dấu phẩy "9,25", tên có khoảng trắng	`(name_var.set(' Lan '), mark_var.set('9,25'), course_box.set('Web'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]`	`'Lan | Web | Giữa kỳ | 9.25'`

Biên 10 và 0 hợp lệ	`[(name_var.set('Huy'), mark_var.set('10'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1], (name_var.set('Huy'), mark_var.set('0'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Cuối kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1]]`	`['Huy | Python | Giữa kỳ | 10.0', 'Huy | Python | Cuối kỳ | 0.0']`
"abc" → showerror, không thêm dòng	`(lambda n, m: ((name_var.set('Mai'), mark_var.set('abc'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, len(_studio.tk.messages) - m, 'error' in str(_studio.tk.messages[-1][0]) if _studio.tk.messages else False))[1])(marks_list.size(), len(_studio.tk.messages))`	`(0, 1, True)`
"10.5" ngoài 0..10 → showerror, không thêm dòng	`(lambda n, m: ((name_var.set('Mai'), mark_var.set('10.5'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, len(_studio.tk.messages) - m, 'error' in str(_studio.tk.messages[-1][0]) if _studio.tk.messages else False))[1])(marks_list.size(), len(_studio.tk.messages))`	`(0, 1, True)`
Họ tên rỗng → showwarning, không thêm dòng	`(lambda n, m: ((name_var.set(' '), mark_var.set('8'), course_box.set('Python'), _studio.tk.find(type='Radiobutton', text='Giữa kỳ').invoke(), retake_var.set(False), _studio.tk.find(type='Button', text='Thêm').invoke(), marks_list.get(marks_list.size() - 1))[-1], (marks_list.size() - n, len(_studio.tk.messages) - m, 'warning' in str(_studio.tk.messages[-1][0]) if _studio.tk.messages else False))[1])(marks_list.size(), len(_studio.tk.messages))`	`(0, 1, True)`
status đếm đúng số dòng trong Listbox	`status.cget('text')`	`'Đã nhập %d điểm' % marks_list.size()`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. name_var.get().strip() → họ tên đã bỏ khoảng trắng
Khung 2 — Bước 2. float(raw.replace(",", ".")) → "8,5" thành 8.5
Khung 3 — Bước 3. Không phải số hoặc ngoài 0..10 → showerror rồi return
Khung 4 — Bước 4. marks_list.insert(tk.END, line) → thêm dòng cuối Listbox
Khung đối chiếu — ngộ nhận "Viết if retake_var: là kiểm ô đã tích". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. retake_var là OBJECT BooleanVar — object luôn được coi là đúng, nên nhánh "thi lại" chạy cả khi ô chưa tích. Phải đọc giá trị: if retake_var.get():. Tương tự float(mark_var) nổ TypeError vì truyền object, không phải chuỗi. Và Radiobutton mỗi nút một biến riêng thì không còn là một nhóm: chọn "Cuối kỳ" không đổi exam_var.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: laptop_var, tk.StringVar(value="Sáng"), str(shift_var)
Khung kết quả — 3 cặp vào/ra. Mai, "8.5", Python, Giữa kỳ, không thi lại → Dòng "Mai | Python | Giữa kỳ | 8.5"; status "Đã nhập 1 điểm"; Điểm "abc" hoặc "10.5" → showerror; Listbox giữ nguyên; Họ tên rỗng → showwarning; Listbox giữ nguyên

Nguồn: pe-v4 · python-learning-data.json#G03 · parser pe-v4/build · file build 016-G03.json

G04 · Layout: pack, place, grid

Mã bài: G04 · Nâng cao · M3 · GUI Tkinter và dự án · 30 phút · 20 XP · thứ tự 17

MỤC TIÊU

Bố trí form bằng grid trong Frame, khung bằng pack, nhãn góc bằng place.

CẦN NHỚ TRƯỚC

G03

BỐI CẢNH

Viết build_form(parent, fields): với field thứ row (đếm từ 0) đặt Label(text=field) ở grid(row=row, column=0, sticky="w") và một Entry ở grid(row=row, column=1, sticky="ew"); lưu Entry vào dict entries[field]; cuối cùng đặt Button "Lưu" ở grid(row=len(fields), column=0, columnspan=2); trả về entries. Trong root: form = Frame pack(fill="x") rồi entries = build_form(form, FIELDS); toolbar = Frame pack(side="bottom", fill="x") chứa nút "Mở" pack(side="left") và "Thoát" pack(side="right"); badge = Label "v1.0" đặt bằng place(relx=1.0, rely=0.0, anchor="ne").

VÍ DỤ MINH HOẠ

FIELDS = ["Họ tên", "Ngày sinh", "Điểm"] → ba hàng Label/Entry, nút Lưu ở hàng 3 chiếm 2 cột.

LÝ THUYẾT

Định nghĩa: geometry manager

Tạo widget xong nó vẫn chưa hiện: phải giao cho một GEOMETRY MANAGER quyết định widget nằm ở đâu và to bao nhiêu. Tk có ba manager, gọi như method của chính widget: .pack() xếp theo cạnh, .place() đặt theo toạ độ, .grid() đặt vào ô hàng–cột. Khung chứa chính là tk.Frame, nên một cửa sổ lớn thường là nhiều Frame, mỗi Frame xếp kiểu riêng.

Ví dụ: ba kiểu, ba cách mô tả vị trí

- pack: mặc định xếp dọc, canh giữa; side=tk.LEFT đổi hướng; fill=tk.X giãn ngang
- place: như position:absolute của HTML — x=10, y=10 hoặc relx=0.1, rely=0.1 (0 tới 1)
- grid: như table của HTML — column=0, row=2, sticky=tk.EW, padx=3, pady=3
- grid còn có columnspan=3, rowspan=2 để một widget chiếm nhiều ô
- columnconfigure(1, weight=1) cho cột 1 nuốt phần dư khi cửa sổ to ra

Trộn pack và grid là lỗi, không phải xấu

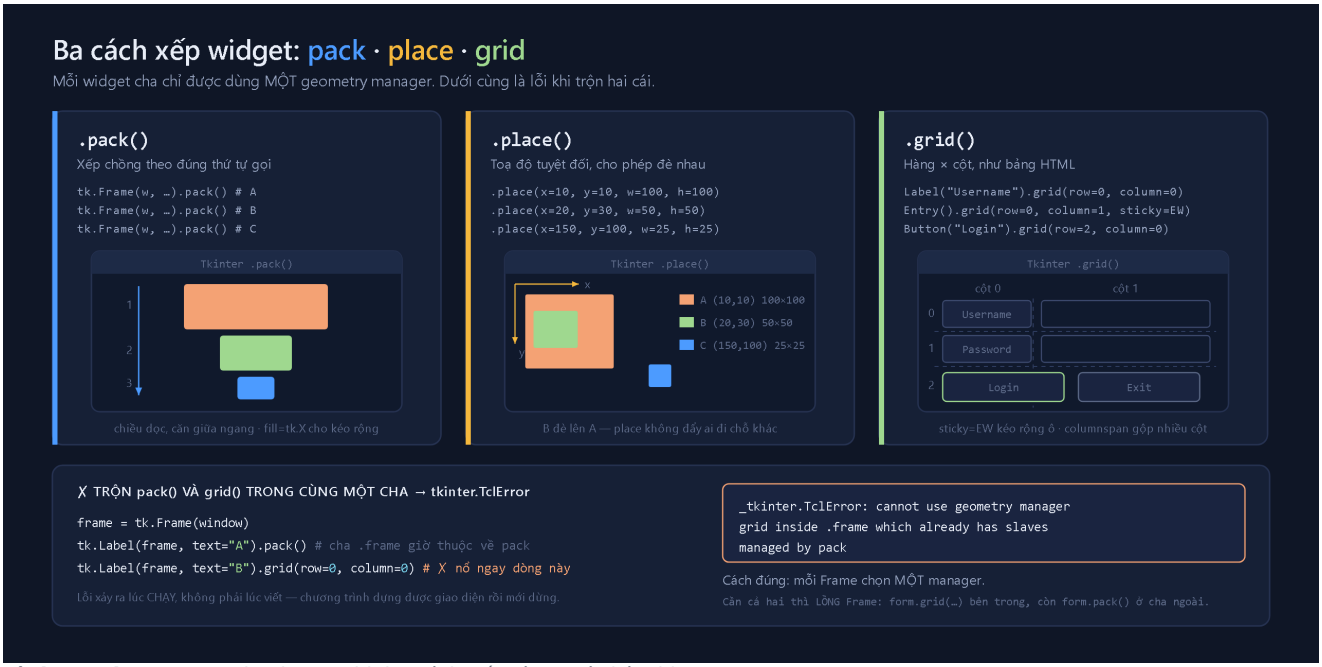
Hai manager tính kích thước theo hai cách khác nhau, nên nếu các con của CÙNG một parent vừa pack() vừa grid(), Tk không hội tụ được và báo TclError (có bản treo trước khi báo). Khác parent thì vô tư: một Frame bên trong dùng grid, Frame đó pack vào cửa sổ. place() trộn được với hai cái kia vì nó không tham gia tính chỗ trống.

Dùng ở đâu

- pack: thanh công cụ, cột nút, bố cục đơn giản xếp chồng hoặc nằm ngang
- grid: form nhãn–ô nhập nhiều dòng, bảng điều khiển — cột nhãn và cột nhập thẳng hàng
- place: chỉ khi cần đúng toạ độ (biểu đồ, ảnh nền), vì nó không tự co giãn theo nội dung
- Studio không vẽ nên bộ chấm đọc widget.manager và widget.layout để biết bạn xếp kiểu gì
- Kích thước thật chỉ thấy trên máy: chạy file rồi kéo giãn cửa sổ để kiểm fill/weight

Đọc sơ đồ

Hình đặt ba cửa sổ nhỏ cạnh nhau, CÙNG ba widget nhưng ba manager khác nhau: pack xếp thành chồng (hoặc hàng ngang khi side=LEFT), place ghim theo toạ độ nên có thể chồng lên nhau, grid chia lưới hàng–cột và các ô thẳng hàng. Dải đỏ phía dưới là ca lỗi: hai con của cùng một parent, một pack một grid → TclError.



Hình G04-layouts. pack, place, grid: ba cách xếp cùng một bộ widget

Mô tả hình: Ba cửa sổ mẫu cho pack xếp chồng, place theo toạ độ x/y, grid theo hàng và cột; dưới cùng là cảnh báo trộn pack với grid cùng parent gây TclError

Nguồn lý thuyết: SL 125-132 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. pack() xếp widget theo cạnh: side="top/bottom/left/right"; fill="x" giãn ngang; expand=True chiếm phần dư.
2. grid() đặt theo ô row, column (đếm từ 0); sticky="ew" cho widget giãn hết bề ngang ô; padx/pady chừa khoảng trống.
3. columnspan/rowspan cho widget chiếm nhiều ô; columnconfigure(1, weight=1) cho cột 1 giãn khi cửa sổ to ra.
4. place() đặt theo toạ độ x/y hoặc tỉ lệ relx/rely (0 tới 1); anchor="ne" chọn góc của widget bám vào điểm đó.
5. KHÔNG dùng pack và grid cho các con của CÙNG một parent — Tk báo TclError. Khác parent (Frame khác) thì được.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import tkinter as tk

root = tk.Tk()
header = tk.Label(root, text="Bảng điểm", bg="lightblue")
header.pack(fill="x")                 # root: dùng pack

table = tk.Frame(root)
table.pack(padx=8, pady=8)            # Frame vẫn là con của root: pack
for row, (name, mark) in enumerate([("Mai", 8.5), ("An", 7.0)]):
    tk.Label(table, text=name).grid(row=row, column=0, sticky="w")
    tk.Label(table, text=str(mark)).grid(row=row, column=1)

corner = tk.Label(root, text="beta")
corner.place(relx=1.0, rely=1.0, anchor="se")   # place không tranh chấp với pack

try:
    tk.Label(table, text="Thêm").pack()       # table đã có con dùng grid
except tk.TclError as error:
    print("TclError:", error)
root.mainloop()
```

Kết quả dự kiến (theo JSON): 3 field → Entry ở (0, 1), (1, 1), (2, 1); pack thêm Label vào form đang grid → TclError

NGỘ NHẬN THƯỜNG GẶP: "Thêm một Label bằng pack() vào Frame đang dùng grid cũng chẳng sao"

pack và grid cùng tính kích thước của parent và giành nhau quyền quyết định — Tk chặn bằng TclError "cannot use geometry manager pack inside ... which already has slaves managed by grid". Muốn phối hợp, tạo Frame riêng cho mỗi kiểu. Một nhầm lẫn khác: gọi grid() không có row/column và tưởng nó tự xếp như pack — Label và Entry sẽ không còn thẳng hàng.

CÂU DỰ ĐOÁN

Có thể pack frame ngoài, rồi grid các widget con BÊN TRONG frame đó không?

ĐÚNG	Có — frame (con của root) và widget con của frame có parent khác nhau	Luật chỉ áp dụng trong cùng một parent.
sai	Không — cả chương trình chỉ được dùng một kiểu	Mỗi parent chọn kiểu riêng cho các con của nó.
sai	Được, nhưng phải dùng place cho frame	pack frame là bình thường.

TỰ KIỂM TRA

1. Trong cùng một Frame, đã có widget dùng grid; thêm widget dùng pack thì...

✔ Tk báo TclError: không được trộn pack và grid trong cùng parent — Tách sang Frame khác nếu cần pack.

• Widget mới tự chuyển sang grid — Tk không tự đổi geometry manager.

• Chạy được, chỉ lệch vị trí — Tk chặn hẳn bằng lỗi.

Đáp án: Tách sang Frame khác nếu cần pack.

2. Entry ở cột 1 muốn giãn hết bề ngang ô thì dùng gì?

• grid(..., side="left") — side cũng là của pack.

• grid(..., fill="x") — fill là tuỳ chọn của pack, grid không nhận.

✔ grid(..., sticky="ew") — Bám cả cạnh trái (w) và phải (e).

Đáp án: Bám cả cạnh trái (w) và phải (e).

3. place(relx=1.0, rely=0.0, anchor="ne") đặt widget ở đâu?

• Góc dưới trái — Đó là relx=0.0, rely=1.0, anchor="sw".

✔ Góc trên phải: góc ne của widget bám vào điểm (100% ngang, 0% dọc) — relx/rely là tỉ lệ theo parent.

• Toạ độ (1, 0) pixel — Pixel là x/y; relx/rely là tỉ lệ.

Đáp án: relx/rely là tỉ lệ theo parent.

TỰ GIẢI THÍCH

Vì sao form dùng grid trong một Frame riêng thay vì grid thẳng vào root?

Mẫu trả lời: root còn toolbar dùng pack. Nếu grid các Label/Entry thẳng vào root thì root có con dùng cả grid lẫn pack → TclError. Đặt form vào Frame riêng: root chỉ pack hai Frame, còn bên trong form tự do dùng grid. Trường hợp đặc biệt: place không tranh chấp, nên badge đặt bằng place trong root vẫn được.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	form.pack(fill="x") → khung form giãn ngang trong root
Bước 2	enumerate(fields) → mỗi field một số hàng 0, 1, 2
Bước 3	Label cột 0, Entry cột 1 sticky="ew" → thẳng hàng
Bước 4	Nút Lưu grid(row=len(fields), columnspan=2) → hàng cuối

CÂU PHẢN VÍ DỤ

build_form(frame, ["Mã SV", "Lớp", "Email", "SĐT", "Ghi chú"]) đặt nút Lưu ở hàng nào?

Lựa chọn: 5 | 0 | 4 · Đáp án: 5

TEMPLATE LẮP MẢNH

```
import tkinter as tk

root = tk.Tk()
pad = tk.Frame(root)
pad.{{slot1}}
for index, text in enumerate(["7", "8", "4", "5"]):
    button = tk.Button(pad, text=text, width=4)
    button.grid(row={{slot2}}, column={{slot3}})
tk.Label(root, text="Bàn phím số").{{slot4}}
root.mainloop()
```

Slot	Mảnh đúng
[Slot 1]	`pack(padx=10, pady=10)`
[Slot 2]	`index // 2`
[Slot 3]	`index % 2`
[Slot 4]	`pack(side="bottom")`

Mảnh bẫy	Vì sao sai
`grid(row=0, column=0)`	pad dùng grid nhưng Label cùng root lại pack → TclError.
`grid(row=1, column=0)`	root đã có pad dùng pack → trộn với grid → TclError.

`index`	Mỗi nút một hàng: thành một cột dọc bốn nút.
`0`	Mọi nút dồn vào cột 0.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: bàn phím 2×2 trong một Frame được pack. Lắp bốn chỗ.

Yêu cầu thêm: Bộ chấm gọi build_form với danh sách field khác (5 field, 1 field) và đọc hàng/cột của từng widget; trộn pack với grid trong cùng parent sẽ làm chương trình dừng ngay bằng TclError.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết build_form(parent, fields): với field thứ row (đếm từ 0) đặt Label(text=field) ở grid(row=row, column=0, sticky="w") và một Entry ở grid(row=row, column=1, sticky="ew"); lưu Entry vào dict entries[field]; cuối cùng đặt Button "Lưu" ở grid(row=len(fields), column=0, columnspan=2); trả về entries.

Trong root: form = Frame pack(fill="x") rồi entries = build_form(form, FIELDS); toolbar = Frame pack(side="bottom", fill="x") chứa nút "Mở" pack(side="left") và "Thoát" pack(side="right"); badge = Label "v1.0" đặt bằng place(relx=1.0, rely=0.0, anchor="ne").

Thực hành: Một parent — một kiểu bố trí: form dùng grid cho con của nó, root dùng pack cho hai Frame, badge dùng place.

Gọi	Kết quả mong đợi
`FIELDS = ["Họ tên", "Ngày sinh", "Điểm"]`	`Entry ở (0, 1), (1, 1), (2, 1); nút Lưu ở hàng 3, columnspan=2`
`5 field`	`Entry ở hàng 0..4; nút Lưu ở hàng 5`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`build_form(parent, fields)`	Dựng form nhập bằng grid trong parent. · parent: Frame chưa có con nào; fields: list[str] · dict tên field → Entry, theo thứ tự fields · Field thứ i: Label (i, 0) sticky="w", Entry (i, 1) sticky="ew"; Button "Lưu" (len(fields), 0) columnspan=2.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
form pack(fill="x") trong root	`(form.manager, form.layout.get('fill'))`	`('pack', 'x')`
Mỗi Entry ở cột 1, mỗi field một hàng	`[(e.manager, e.layout.get('row'), e.layout.get('column')) for e in entries.values()]`	`[('grid', 0, 1), ('grid', 1, 1), ('grid', 2, 1)]`
Label ở cột 0, cùng hàng với Entry	`[(w.manager, w.layout.get('row'), w.layout.get('column')) for w in (_studio.tk.find(type='Label', text=t) for t in ['Họ tên', 'Ngày sinh', 'Điểm'])]`	`[('grid', 0, 0), ('grid', 1, 0), ('grid', 2, 0)]`
Entry giãn ngang: sticky có e và w	`[{'e', 'w'} <= set(''.join(e.layout.get('sticky', ''))) for e in entries.values()]`	`[True, True, True]`
Nút Lưu: hàng 3, columnspan=2	`(lambda b: (b.manager, b.layout.get('row'), b.layout.get('columnspan')))(_studio.tk.find(type='Button', text='Lưu'))`	`('grid', 3, 2)`
toolbar ở đáy; Mở bên trái, Thoát bên phải	`(toolbar.manager, toolbar.layout.get('side'), _studio.tk.find(type='Button', text='Mở').layout.get('side'), _studio.tk.find(type='Button', text='Thoát').layout.get('side'))`	`('pack', 'bottom', 'left', 'right')`
badge đặt bằng place ở góc trên phải	`(badge.manager, badge.layout.get('relx'), badge.layout.get('rely'), badge.layout.get('anchor'))`	`('place', 1.0, 0.0, 'ne')`
5 field khác: Entry hàng 0..4, nút Lưu hàng 5	`(lambda es: ([(e.layout.get('row'), e.layout.get('column')) for e in es.values()], _studio.tk.find_all(type='Button', text='Lưu')[-1].layout.get('row')))(build_form(tk.Frame(root), ['Mã SV', 'Lớp', 'Email', 'SĐT', 'Ghi chú']))`	`([(0, 1), (1, 1), (2, 1), (3, 1), (4, 1)], 5)`
1 field: Label hàng 0, nút Lưu ngay hàng 1	`(lambda es: (list(es), _studio.tk.find(type='Label', text='Ghi chú riêng').layout.get('row'), _studio.tk.find_all(type='Button', text='Lưu')[-1].layout.get('row')))(build_form(tk.Frame(root), ['Ghi chú riêng']))`	`(['Ghi chú riêng'], 0, 1)`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar…), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. form.pack(fill="x") → khung form giãn ngang trong root
Khung 2 — Bước 2. enumerate(fields) → mỗi field một số hàng 0, 1, 2
Khung 3 — Bước 3. Label cột 0, Entry cột 1 sticky="ew" → thẳng hàng
Khung 4 — Bước 4. Nút Lưu grid(row=len(fields), columnspan=2) → hàng cuối
Khung đối chiếu — ngộ nhận "Thêm một Label bằng pack() vào Frame đang dùng grid cũng chẳng sao". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. pack và grid cùng tính kích thước của parent và giành nhau quyền quyết định — Tk chặn bằng TclError "cannot use geometry manager pack inside ... which already has slaves managed by grid". Muốn phối hợp, tạo Frame riêng cho mỗi kiểu. Một nhầm lẫn khác: gọi grid() không có row/column và tưởng nó tự xếp như pack — Label và Entry sẽ không còn thẳng hàng.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: grid(row=0, column=0), grid(row=1, column=0), index, 0
Khung kết quả — 2 cặp vào/ra. FIELDS = ["Họ tên", "Ngày sinh", "Điểm"] → Entry ở (0, 1), (1, 1), (2, 1); nút Lưu ở hàng 3, columnspan=2; 5 field → Entry ở hàng 0..4; nút Lưu ở hàng 5

Nguồn: pe-v4 · python-learning-data.json#G04 · parser pe-v4/build · file build 017-G04.json

G05 · OOP và tách logic khỏi giao diện

Mã bài: G05 · Nâng cao · M3 · GUI Tkinter và dự án · 35 phút · 20 XP · thứ tự 18

MỤC TIÊU

Gói giao diện vào class App, tách logic thành hàm thuần.

CẦN NHỚ TRƯỚC

G04

BỐI CẢNH

Viết hai hàm THUẦN (không đụng widget, không messagebox): validate_mark(raw) đổi chuỗi thành float, không phải số hoặc ngoài 0..10 thì raise ValueError; summary(marks) trả "Chưa có điểm" khi list rỗng, ngược lại "<n> điểm, trung bình <tb, 2 chữ số thập phân>". Hoàn thành class App: __init__ tạo self.marks = [], self.mark_var rồi gọi create_widgets(); create_widgets tạo Entry gắn self.mark_var, nút "Thêm điểm" với command=self.add_mark và self.result = Label hiện summary(self.marks); add_mark gọi validate_mark — sai thì messagebox.showerror, đúng thì append vào self.marks và cập nhật self.result.

VÍ DỤ MINH HOẠ

validate_mark("8.5") → 8.5; gõ 8 rồi bấm "Thêm điểm" → nhãn "1 điểm, trung bình 8.00".

LÝ THUYẾT

Định nghĩa: hàm thuần và tầng giao diện

Hàm THUẦN nhận đối số, trả kết quả hoặc raise, và không đụng tới widget nào. Tầng GIAO DIỆN chỉ làm ba việc: đọc widget, gọi hàm thuần, hiện kết quả. Ranh giới này không phải chuyện đẹp xấu: hàm thuần test được bằng một lời gọi, còn code dính widget thì muốn test phải dựng cả cửa sổ. Bài tập PW9 chính là nâng giao diện của PW8 lên GUI mà giữ nguyên phần tính toán.

Ví dụ: tách một ô nhập điểm

validate_mark(raw) nhận chuỗi, trả float 0..10, raise ValueError nếu sai — không biết Entry là gì. Method add_mark của App gọi self.mark_var.get(), đưa vào validate_mark trong try, except ValueError thì messagebox.showerror. Đổi giao diện sang web hay CLI thì validate_mark giữ nguyên; chỉ phần bắt lỗi và hiện lỗi viết lại.

Vì sao gói giao diện vào class

- self.status, self.mark_var sống cùng App; biến cục bộ trong create_widgets() mất ngay
- __init__ dựng trạng thái rồi gọi create_widgets(); mỗi method là một việc người dùng làm
- Callback thành method: command=self.add_mark — vẫn KHÔNG ngoặc
- Nhiều cửa sổ cùng kiểu thì tạo nhiều đối tượng App, không phải chép code
- Tkinter không bắt buộc OOP; class chỉ đáng khi widget và trạng thái bắt đầu nhiều

Dùng ở đâu

- Mọi app có kiểm tra dữ liệu: quy tắc nghiệp vụ nằm ở hàm thuần, thông báo nằm ở GUI
- Dùng lại cùng một lõi cho GUI, cho script dòng lệnh và cho bài test tự động
- Hàm thuần báo lỗi bằng raise; GUI quyết định hiện bằng messagebox hay đổi màu ô nhập
- Studio chấm phần logic bằng cách gọi thẳng hàm thuần — không cần cửa sổ nào
- Đó cũng là lý do bài này chạy được trong Studio dù tkinter ở đây là đồ giả

Đọc sơ đồ

Hình chia đôi bằng một đường dọc. Bên trái là lớp App: widget, callback, messagebox — phần chỉ chạy được khi có cửa sổ. Bên phải là các hàm thuần: vào là dữ liệu, ra là dữ liệu hoặc exception. Mũi tên chỉ đi một chiều từ trái sang phải; không có mũi tên nào từ hàm thuần chạm vào widget. Bên phải còn có ô "test gọi thẳng" để thấy vì sao tách lại dễ kiểm.



Hình G05-logic-vs-ui. Ranh giới một chiều giữa lớp giao diện và hàm thuần

Mô tả hình: Sơ đồ chia đôi: bên trái lớp App với widget, callback và messagebox; bên phải hàm thuần nhận dữ liệu, trả kết quả hoặc raise, và được test gọi thẳng

Nguồn lý thuyết: SL 116, 135 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. self.widget giữ tham chiếu tới widget cần cập nhật; biến cục bộ trong create_widgets mất khi hàm kết thúc.
2. __init__ tạo trạng thái (self.marks, self.mark_var) rồi gọi create_widgets() để dựng giao diện.
3. Callback thành method: command=self.add_mark — vẫn không có ngoặc, Tkinter gọi khi bấm.
4. Logic tính/kiểm dữ liệu đặt trong hàm thuần: test được bằng gọi hàm, dùng lại được cho CLI hay web.
5. Hàm thuần báo lỗi bằng raise; GUI bắt lỗi và quyết định cách hiện (messagebox). Mỗi tầng một việc.
6. Tkinter cơ bản không bắt buộc OOP; class giúp quản lý khi ứng dụng có nhiều widget và trạng thái.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import tkinter as tk


def to_grade(mark):
    """Logic thuần: dùng được từ CLI, từ test, từ GUI."""
    if mark >= 8.5:
        return "A"
    if mark >= 7:
        return "B"
    return "C"


class GradeApp:
    def __init__(self, root):
        self.mark = tk.StringVar(value="9")
        tk.Entry(root, textvariable=self.mark).pack()
        tk.Button(root, text="Xếp loại", command=self.show).pack()
        self.answer = tk.Label(root, text="?")       # self: method khác còn dùng
        self.answer.pack()

    def show(self):
        self.answer.config(text=to_grade(float(self.mark.get())))


print([to_grade(m) for m in (9, 7.5, 5)])        # chạy được không cần cửa sổ
root = tk.Tk()
app = GradeApp(root)
app.show()
print(app.answer.cget("text"))
root.mainloop()
```

Kết quả dự kiến (theo JSON): validate_mark("8.5") → 8.5; "abc" → ValueError; Bấm "Thêm điểm" với 8 → "1 điểm, trung bình 8.00"

NGỘ NHẬN THƯỜNG GẶP: "Cho validate_mark tự bật messagebox là tiện nhất"

Khi đó validate_mark chỉ dùng được trong GUI: gọi từ CLI hay từ test sẽ bật hộp thoại và trả None thay vì báo lỗi rõ ràng, nơi gọi phải đoán vì sao có None. Hàm thuần raise ValueError; App (tầng giao diện) bắt lỗi và chọn cách hiển

thị. Đừng quên: command=self.add_mark() vẫn là gọi ngay, và result = tk.Label(...) không có self thì method khác không thấy.

CÂU DỰ ĐOÁN

Ô nhập đang là "abc", người dùng bấm "Thêm điểm". Chuyện gì xảy ra?

sai	self.marks thêm 0	Dữ liệu sai không bao giờ được thêm.
ĐÚNG	validate_mark raise ValueError; add_mark bắt lỗi, bật showerror; self.marks giữ nguyên	Hàm thuần báo lỗi, tầng giao diện quyết định cách hiện.
sai	Chương trình dừng hẳn với ValueError	add_mark bắt ValueError bằng try/except nên giao diện vẫn chạy.

TỰ KIỂM TRA

1. self.entry giúp gì so với một biến local mất sau hàm?

- Giúp widget tự lưu dữ liệu ra file — self chỉ là thuộc tính của object.
- Bắt buộc để widget hiện lên — Widget hiện nhờ pack/grid, không nhờ self.
- ✔ Giữ widget để các method khác truy cập — add_mark đọc self.mark_var và đổi self.result được vì chúng là thuộc tính của object.

Đáp án: add_mark đọc self.mark_var và đổi self.result được vì chúng là thuộc tính của object.

2. Vì sao validate_mark nên raise ValueError thay vì gọi messagebox.showerror?

- Vì messagebox không chạy trong hàm — messagebox gọi được ở đâu cũng được — vấn đề là phụ thuộc GUI.
- ✔ Để hàm dùng lại được ngoài GUI và nơi gọi tự quyết cách báo lỗi — Logic tách khỏi giao diện.
- Vì raise nhanh hơn — Không phải chuyện tốc độ.

Đáp án: Logic tách khỏi giao diện.

3. Khai marks = [] ở thân class (không trong __init__) thì hai App khác nhau...

- Báo lỗi khi chạy — Chạy được, chỉ sai âm thầm.
- Mỗi App có list riêng — Chỉ đúng khi tạo self.marks = [] trong __init__.
- ✔ Dùng CHUNG một list — điểm của cửa sổ này hiện ở cửa sổ kia — Thuộc tính class được mọi object chia sẻ.

Đáp án: Thuộc tính class được mọi object chia sẻ.

TỰ GIẢI THÍCH

Tách validate_mark/summary khỏi class App được lợi gì khi kiểm thử và khi đổi giao diện?

Mẫu trả lời: validate_mark và summary chỉ nhận dữ liệu và trả kết quả (hoặc raise), nên kiểm được bằng gọi hàm, không cần mở cửa sổ hay bấm nút. App chỉ lo đọc widget, gọi hàm và hiện kết quả. Đổi sang giao diện khác (CLI, web) vẫn dùng lại nguyên hai hàm. Trường hợp đặc biệt: list rỗng → "Chưa có điểm", "abc" → ValueError.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	App(root) → __init__ tạo self.marks, self.mark_var
Bước 2	create_widgets() → Entry, nút, self.result
Bước 3	Bấm "Thêm điểm" → add_mark() gọi validate_mark
Bước 4	ValueError → showerror; hợp lệ → append rồi summary()

CÂU PHẢN VÍ DỤ

summary([9, 8.5, 6.25]) trả gì?

Lựa chọn: 3 điểm, trung bình 7.92 | Chưa có điểm | 3 điểm, trung bình 7.91 · Đáp án: 3 điểm, trung bình 7.92

TEMPLATE LẮP MẢNH

```
import tkinter as tk


def to_fahrenheit(raw):
    celsius = float(raw)
    return round(celsius * 9 / 5 + 32, 1)


class App:
    def __init__(self, root):
        self.celsius = tk.StringVar(value="0")
        tk.Entry(root, textvariable=self.celsius).pack()
        tk.Button(root, text="Đổi", command={{slot1}}).pack()
        {{slot2}} = tk.Label(root, text="")
        self.answer.pack()
    def convert(self):
```

```
        value = to_fahrenheit({{slot3}})
        {{slot4}}


root = tk.Tk()
app = App(root)
root.mainloop()
```

Slot	Mảnh đúng
[Slot 1]	`self.convert`
[Slot 2]	`self.answer`
[Slot 3]	`self.celsius.get()`
[Slot 4]	`self.answer.config(text=str(value) + " °F")`

Mảnh bẫy	Vì sao sai
`print(value)`	In ra console; nhãn trên cửa sổ không đổi.
`self.celsius`	Truyền object StringVar; float() cần chuỗi → TypeError.
`self.convert()`	Gọi convert ngay trong __init__; nút nhận None.
`answer`	Biến cục bộ mất sau __init__; convert không thấy được.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: ứng dụng đổi °C sang °F. Hàm thuần to_fahrenheit đã có; lắp bốn chỗ trong class App.

Yêu cầu thêm: Bộ chấm gọi thẳng validate_mark/summary với dữ liệu mới (biên 0 và 10, "abc", list ba điểm), rồi tạo App mới trên tk.Tk() mới để kiểm self.marks riêng và nút "Thêm điểm".

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm THUẦN (không đụng widget, không messagebox): validate_mark(raw) đổi chuỗi thành float, không phải số hoặc ngoài 0..10 thì raise ValueError; summary(marks) trả "Chưa có điểm" khi list rỗng, ngược lại "<n> điểm, trung bình <tb, 2 chữ số thập phân>".

Hoàn thành class App: __init__ tạo self.marks = [], self.mark_var rồi gọi create_widgets(); create_widgets tạo Entry gắn self.mark_var, nút "Thêm điểm" với command=self.add_mark và self.result = Label hiện summary(self.marks); add_mark gọi validate_mark — sai thì messagebox.showerror, đúng thì append vào self.marks và cập nhật self.result.

Thực hành: Hai hàm thuần không chạm widget; App giữ widget trong self, bắt ValueError của validate_mark trong add_mark.

Gọi	Kết quả mong đợi
`validate_mark("8.5") / validate_mark("11")`	`8.5 / ValueError`
`Gõ 8 rồi bấm "Thêm điểm"`	`self.marks = [8.0]; nhãn "1 điểm, trung bình 8.00"`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`validate_mark(raw)`	Đổi chuỗi người dùng gõ thành điểm hợp lệ. · raw: str (có thể có khoảng trắng hai đầu) · float trong 0..10; không phải số hoặc ngoài khoảng → ValueError · Hàm thuần: không đọc/ghi widget, không bật messagebox.
`summary(marks)`	Tạo chữ tóm tắt cho nhãn kết quả. · marks: list[float] · "Chưa có điểm" nếu rỗng; ngược lại "<n> điểm, trung bình <tb 2 số lẻ>" · Hàm thuần, không sửa marks.
`App(root)`	Gói giao diện: Entry gắn self.mark_var, nút "Thêm điểm", nhãn self.result. · root: cửa sổ Tk · object có self.marks (list riêng), self.mark_var, self.result · add_mark(): hợp lệ → thêm vào self.marks và cập nhật nhãn; sai → showerror, không thêm.

BỘ CHẤM (11 ca)

Ca	Gọi	Mong đợi
validate_mark("8.5") → 8.5	`validate_mark("8.5")`	`8.5`
Biên 0 và 10, có khoảng trắng	`[validate_mark(" 0 "), validate_mark("10")]`	`[0.0, 10.0]`
Dữ liệu sai → ValueError	`[_studio.call(validate_mark, raw).error for raw in ['abc', '10.5', '-1', '']]`	`['ValueError', 'ValueError', 'ValueError', 'ValueError']`
validate_mark không tự bật messagebox	`(lambda n: (_studio.call(validate_mark, '11'), len(_studio.tk.messages) - n)[1])(len(_studio.tk.messages))`	`0`

summary([]) và summary([8, 7])	`(summary([]), summary([8, 7]))`	`('Chưa có điểm', '2 điểm, trung bình 7.50')`
summary làm tròn 2 chữ số	`summary([9, 8.5, 6.25])`	`'3 điểm, trung bình 7.92'`
App giữ nhãn trong self.result	`app.result.cget('text')`	`'Chưa có điểm'`
Bấm "Thêm điểm" với 8	`(app.marks.clear(), app.mark_var.set('8'), _studio.tk.find(type='Button', text='Thêm điểm').invoke(), list(app.marks), app.result.cget('text'))[3:]`	`([8.0], '1 điểm, trung bình 8.00')`
"abc" → showerror, không thêm	`(lambda n, m: (app.mark_var.set('abc'), _studio.tk.find(type='Button', text='Thêm điểm').invoke(), len(app.marks) - n, len(_studio.tk.messages) - m)[2:])(len(app.marks), len(_studio.tk.messages))`	`(0, 1)`
App mới: self.marks riêng, nhãn ban đầu	`(lambda a: (a.marks, a.result.cget('text')))(App(tk.Tk()))`	`([], 'Chưa có điểm')`
add_mark gọi trực tiếp hai lần	`(lambda a: (a.mark_var.set('7.5'), a.add_mark(), a.mark_var.set('9'), a.add_mark(), a.result.cget('text'))[-1])(App(tk.Tk()))`	`'2 điểm, trung bình 8.25'`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. App(root) → __init__ tạo self.marks, self.mark_var
Khung 2 — Bước 2. create_widgets() → Entry, nút, self.result
Khung 3 — Bước 3. Bấm "Thêm điểm" → add_mark() gọi validate_mark
Khung 4 — Bước 4. ValueError → showerror; hợp lệ → append rồi summary()
Khung đối chiếu — ngộ nhận "Cho validate_mark tự bật messagebox là tiện nhất". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Khi đó validate_mark chỉ dùng được trong GUI: gọi từ CLI hay từ test sẽ bật hộp thoại và trả None thay vì báo lỗi rõ ràng, nơi gọi phải đoán vì sao có None. Hàm thuần raise ValueError; App (tầng giao diện) bắt lỗi và chọn cách hiển thị. Đừng quên: command=self.add_mark() vẫn là gọi ngay, và result = tk.Label(...) không có self thì method khác không thấy.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: print(value), self.celsius, self.convert(), answer
Khung kết quả — 2 cặp vào/ra. validate_mark("8.5") / validate_mark("11") → 8.5 / ValueError; Gõ 8 rồi bấm "Thêm điểm" → self.marks = [8.0]; nhãn "1 điểm, trung bình 8.00"

Nguồn: pe-v4 · python-learning-data.json#G05 · parser pe-v4/build · file build 018-G05.json

G06 · Event loop và worker nền

Mã bài: G06 · Nâng cao · M3 · GUI Tkinter và dự án · 40 phút · 20 XP · thứ tự 19

MỤC TIÊU

Giữ giao diện phản hồi: việc dài ở worker, kết quả về qua queue + after().

CẦN NHỚ TRƯỚC

G05

BỐI CẢNH

Hoàn thành ứng dụng "Tác vụ nền" (slow_total(n) cộng 1..n đã có): worker(n, q) chạy ở thread nền, tính slow_total(n) rồi q.put(kết quả) — KHÔNG đụng widget. start_task(): n_var không phải số nguyên không âm → status "n phải là số nguyên không âm"; hợp lệ → status "Đang tính...", start một Thread(target=worker, args=(n, q)) (không join) rồi root.after(100, poll_result). poll_result(): lấy kết quả bằng q.get_nowait(); queue rỗng (Empty) → root.after(100, poll_result) hẹn lại; có kết quả → status "Tổng = <kết quả>".

VÍ DỤ MINH HOẠ

n = 10, bấm Tính → "Đang tính..." → event loop chạy poll_result → "Tổng = 55".

LÝ THUYẾT

Định nghĩa: mainloop là vòng lặp chặn

Slide gọi mainloop() là "a blocking method": nó không trả về mà quay vòng — lấy một sự kiện (bấm, gõ, chuột, hẹn giờ), gọi hàm xử lý tương ứng, vẽ lại, rồi lấy sự kiện tiếp theo. Mọi callback của bạn chạy BÊN TRONG vòng lặp đó, trên cùng một thread. Vòng lặp chỉ thoát khi cửa sổ chính đóng.

Vì sao cửa sổ "treo"

Vòng lặp đang ở trong callback của bạn thì không lấy được sự kiện nào khác: không vẽ lại, không nhận bấm, hệ điều hành báo "Not responding". Một vòng lặp tính 5 giây, một time.sleep, một lời gọi mạng — đều làm đúng như vậy. Cửa sổ không hỏng; nó chỉ đang xếp hàng chờ callback của bạn trả về.

Ba luật khi ghép thread vào Tkinter

- Việc dài chạy ở worker thread; callback chỉ start() rồi trả về ngay
- Worker KHÔNG đụng widget — Tk chỉ an toàn với thread đang chạy mainloop
- Kết quả đi về qua queue.Queue, cấu trúc an toàn giữa các thread
- Thread GUI đọc queue trong hàm được after(ms, f) hẹn: Empty thì hẹn lại, có thì cập nhật
- Không join() trong callback: join chặn thread GUI y hệt làm việc dài trực tiếp

Dùng ở đâu

- Lưu file lớn, nén, gọi API, quét thư mục — việc có thể mất vài giây
- Thanh tiến độ và nút Huỷ chỉ chạy được khi event loop còn rảnh để vẽ lại
- after(ms, f) cũng là cách làm đồng hồ, tự động làm mới, hoạt ảnh nhẹ — không cần thread
- Studio mô phỏng: worker chạy xong ngay lúc start(), after() chỉ xếp vào hàng đợi rồi bộ chấm chạy thay event loop — nên không thấy được cảnh treo
- Chạy file trên máy với n lớn: bản không thread sẽ đứng, bản có thread vẫn kéo được cửa sổ

Đọc sơ đồ

Hình có hai làn. Làn trên là thread GUI: mainloop quay vòng lấy sự kiện; callback Tính bắt đầu worker rồi TRẢ VỀ NGAY nên vòng lặp quay tiếp. Làn dưới là worker: tính lâu, xong thì put kết quả vào Queue. Nối hai làn chỉ có một mũi tên duy nhất đi qua Queue; chấm tròn trên làn GUI là các lần after(100) đọc queue, lần cuối mới đổi chữ Label.



Hình G06-event-loop. Event loop rảnh tay: worker tính, Queue chuyển, after() cập nhật

Mô tả hình: Hai làn thời gian: thread GUI chạy mainloop và các lần after(100) đọc Queue; worker thread tính lâu rồi put kết quả vào Queue, không chạm vào widget

Nguồn lý thuyết: SL 95-105, 133 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. Callback chạy TRÊN thread GUI: callback dài (vòng lặp nặng, sleep, chờ mạng) làm event loop đứng — cửa sổ treo.
2. Việc dài giao cho worker thread; worker chỉ đưa kết quả vào queue.Queue (an toàn giữa các thread), không đụng widget.
3. Thread GUI đọc queue bằng get_nowait() trong hàm được after() hẹn giờ: rỗng thì Empty → hẹn lại; có thì cập nhật widget.
4. Không join() worker trong callback: join chặn thread GUI y như làm việc dài trực tiếp.
5. Tkinter không an toàn khi nhiều thread cùng gọi widget: chỉ thread chạy mainloop được config/insert vào widget.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import threading
import tkinter as tk
from queue import Queue, Empty

q = Queue()


def work():
    q.put({"status": "done", "result": 5})      # worker chỉ gửi thông điệp


root = tk.Tk()
label = tk.Label(root, text="Đang chờ...")
label.pack()


def poll_result():
    try:
        message = q.get_nowait()
    except Empty:
        root.after(100, poll_result)             # chưa có: hẹn lượt sau
        return
    label.config(text=f"Kết quả: {message['result']}")
    print("Thread GUI cập nhật Label:", label.cget("text"))


threading.Thread(target=work).start()
root.after(100, poll_result)
root.mainloop()
```

Kết quả dự kiến (theo JSON): n = 10 → "Đang tính..." rồi "Tổng = 55"; Bấm Tính → 1 thread, không join trong callback

NGỘ NHẬN THƯỜNG GẶP: "Cho worker gọi status.config(...) luôn cho nhanh"

Tkinter chỉ an toàn khi widget được gọi từ thread chạy mainloop. Gọi từ worker có lúc chạy, có lúc lỗi "main thread is not in main loop" hoặc treo — tuỳ máy, rất khó tái hiện. Worker put vào queue, thread GUI poll bằng after(). Cũng đừng "chờ cho chắc" bằng join() trong callback: cửa sổ đứng tới khi worker xong, mất hết lợi ích của thread.

CÂU DỰ ĐOÁN

Trong mô hình worker + queue + after(), ai cập nhật Label?

sai	Queue tự cập nhật Label	Queue chỉ chứa dữ liệu.
ĐÚNG	Thread GUI, khi poll_result được event loop gọi qua after()	Worker chỉ put kết quả vào queue.
sai	Worker thread, ngay khi tính xong	Gọi widget từ thread khác là không an toàn.

TỰ KIỂM TRA

1. Trong mô hình này ai cập nhật Label?

• Worker thread — Worker chỉ gửi thông điệp vào queue.

• Hệ điều hành — Tkinter vẽ lại widget khi thread GUI đổi cấu hình.

✔ Thread GUI khi callback poll_result chạy qua event loop — after() hẹn poll_result; event loop gọi nó trên thread GUI.

Đáp án: after() hẹn poll_result; event loop gọi nó trên thread GUI.

2. Vì sao poll_result dùng q.get_nowait() chứ không dùng q.get()?

• get() không có trong queue.Queue — get() có, nhưng chặn.

✔ get() chặn khi queue rỗng — chặn thread GUI là cửa sổ treo — get_nowait() báo Empty ngay để hẹn lại.

• get_nowait() nhanh hơn khi queue có dữ liệu — Khác biệt là hành vi khi rỗng.

Đáp án: get_nowait() báo Empty ngay để hẹn lại.

3. Gọi worker_thread.join() ngay trong start_task thì...

• Worker chạy nhanh hơn — join không tăng tốc worker.

✔ Cửa sổ đứng tới khi worker xong — như làm việc dài trong callback — join chặn nơi gọi, ở đây là thread GUI.

• Không ảnh hưởng gì — join chặn thread GUI.

Đáp án: join chặn nơi gọi, ở đây là thread GUI.

TỰ GIẢI THÍCH

Mô tả đường đi của kết quả từ lúc bấm "Tính" tới lúc Label đổi chữ, và ai làm bước nào.

Mẫu trả lời: Thread GUI chạy start_task: kiểm n, đổi status "Đang tính...", start worker rồi hẹn poll_result bằng after(). Worker tính slow_total(n) rồi put kết quả vào queue. Event loop gọi poll_result trên thread GUI: queue rỗng thì hẹn lại, có kết quả thì status.config. Trường hợp đặc biệt: n = "abc" thì không tạo thread nào.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Bấm Tính → start_task() start worker, hẹn poll_result
Bước 2	Worker → slow_total(n) rồi q.put(kết quả)
Bước 3	poll_result() rỗng → after(100, poll_result) hẹn lại
Bước 4	poll_result() có kết quả → status.config trên thread GUI

CÂU PHẢN VÍ DỤ

n = 100, bấm Tính rồi event loop chạy poll_result — status hiện gì?

Lựa chọn: Tổng = 100 | Đang tính... | Tổng = 5050 · Đáp án: Tổng = 5050

TEMPLATE LẮP MẢNH

```
import threading
import tkinter as tk
from queue import Queue, Empty

root = tk.Tk()
box = Queue()
label = tk.Label(root, text="...")
label.pack()


def count_even(limit):
    box.put(sum(1 for k in range(limit) if k % 2 == 0))


def check_box():
    try:
```

```
        value = {{slot1}}
    except Empty:
        {{slot2}}
        return
    label.config(text=f"Có {value} số chẵn")


def start():
    {{slot3}}
    root.after(100, check_box)


tk.Button(root, text="Đếm", command=start).pack()
root.mainloop()
```

Slot	Mảnh đúng
[Slot 1]	`box.get_nowait()`
[Slot 2]	`root.after(100, check_box)`
[Slot 3]	`threading.Thread(target=count_even, args=(1000,)).start()`

Mảnh bẫy	Vì sao sai
`box.get()`	Chặn thread GUI tới khi có dữ liệu — cửa sổ đứng hình.
`pass`	Không hẹn lượt kiểm tra sau: kết quả đến muộn sẽ không bao giờ được hiện.
`count_even(1000)`	Chạy việc dài ngay trong callback; giao diện đứng tới khi xong.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: worker đếm số chẵn rồi gửi về qua queue box. Lắp ba chỗ.

Yêu cầu thêm: Bộ chấm bấm Tính với n khác (10, 100, "abc"), đọc nhãn ngay sau khi bấm, rồi chạy hàng đợi after() thay event loop; còn gọi poll_result() khi queue rỗng để xem nó có tự hẹn lại không.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành ứng dụng "Tác vụ nền" (slow_total(n) cộng 1..n đã có): worker(n, q) chạy ở thread nền, tính slow_total(n) rồi q.put(kết quả) — KHÔNG đụng widget.

start_task(): n_var không phải số nguyên không âm → status "n phải là số nguyên không âm"; hợp lệ → status "Đang tính...", start một Thread(target=worker, args=(n, q)) (không join) rồi root.after(100, poll_result).

poll_result(): lấy kết quả bằng q.get_nowait(); queue rỗng (Empty) → root.after(100, poll_result) hẹn lại; có kết quả → status "Tổng = <kết quả>".

Thực hành: Callback chỉ làm việc ngắn: kiểm n, đổi chữ, start thread, hẹn after. Worker chỉ put vào queue. Chỉ poll_result (thread GUI) đổi widget.

Gọi	Kết quả mong đợi
`n = 10, bấm Tính`	`Ngay sau khi bấm: "Đang tính..."; sau khi event loop chạy: "Tổng = 55"`
`n = "abc"`	`"n phải là số nguyên không âm"; không tạo thread`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`worker(n, q)`	Việc nền: tính tổng và gửi về. · n: int ≥ 0; q: Queue · None · q.put(slow_total(n)); không đọc/ghi widget.
`poll_result()`	Thread GUI kiểm queue. · không · None · Có kết quả → status "Tổng = ..."; rỗng → root.after(100, poll_result).

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Chưa bấm: status "Sẵn sàng"	`status.cget('text')`	`'Sẵn sàng'`
slow_total(10) = 55	`slow_total(10)`	`55`
worker đưa KẾT QUẢ vào queue	`(lambda box: (worker(4, box), box.get_nowait())[1])(__import__('queue').Queue())`	`10`
Bấm Tính: một thread được start, KHÔNG join trong callback	`(lambda r: (r.thread_count, r.threads[0].started, r.threads[0].joined, _studio.tk.run_after(n=5))[:3])((n_var.set('10'), _studio.call(_studio.tk.find(type='Button', text='Tính').invoke))[1])`	`(1, True, False)`

n = 10: "Đang tính..." rồi "Tổng = 55"	`(lambda r: (r[2], r[4]))((n_var.set('10'), _studio.tk.find(type='Button', text='Tính').invoke(), status.cget('text'), _studio.tk.run_after(n=5), status.cget('text')))`	`('Đang tính...', 'Tổng = 55')`
n = 100: "Tổng = 5050"	`(n_var.set('100'), _studio.tk.find(type='Button', text='Tính').invoke(), status.cget('text'), _studio.tk.run_after(n=5), status.cget('text'))[4]`	`'Tổng = 5050'`
n = "abc": báo lỗi, không tạo thread	`(lambda r: (r.thread_count, status.cget('text')))((n_var.set('abc'), _studio.call(_studio.tk.find(type='Button', text='Tính').invoke))[1])`	`(0, 'n phải là số nguyên không âm')`
Queue rỗng → poll_result tự hẹn lại, nhận kết quả ở lượt sau	`(poll_result(), q.put(42), _studio.tk.run_after(n=5), status.cget('text'))[-1]`	`'Tổng = 42'`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật. Thread cũng là mô phỏng: worker chạy xong ngay khi start(), còn after() chỉ xếp lời gọi vào hàng đợi — bộ chấm chạy hàng đợi đó thay cho event loop. Trên máy thật worker chạy song song và cửa sổ vẫn bấm, kéo được trong lúc chờ; hãy thử với n lớn để thấy khác biệt.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. Bấm Tính → start_task() start worker, hẹn poll_result
Khung 2 — Bước 2. Worker → slow_total(n) rồi q.put(kết quả)
Khung 3 — Bước 3. poll_result() rỗng → after(100, poll_result) hẹn lại
Khung 4 — Bước 4. poll_result() có kết quả → status.config trên thread GUI
Khung đối chiếu — ngộ nhận "Cho worker gọi status.config(...) luôn cho nhanh". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Tkinter chỉ an toàn khi widget được gọi từ thread chạy mainloop. Gọi từ worker có lúc chạy, có lúc lỗi "main thread is not in main loop" hoặc treo — tuỳ máy, rất khó tái hiện. Worker put vào queue, thread GUI poll bằng after(). Cũng đừng "chờ cho chắc" bằng join() trong callback: cửa sổ đứng tới khi worker xong, mất hết lợi ích của thread.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: box.get(), pass, count_even(1000)
Khung kết quả — 2 cặp vào/ra. n = 10, bấm Tính → Ngay sau khi bấm: "Đang tính..."; sau khi event loop chạy: "Tổng = 55"; n = "abc" → "n phải là số nguyên không âm"; không tạo thread

Nguồn: pe-v4 · python-learning-data.json#G06 · parser pe-v4/build · file build 019-G06.json

R02 · Dự án PW5–PW9: lưu, shell, thread và GUI

Mã bài: R02 · Nâng cao · M3 · GUI Tkinter và dự án · 60 phút · 40 XP · thứ tự 20

MỤC TIÊU

Ghép lưu file nén, process con, worker thread và Tkinter thành một ứng dụng.

CẦN NHỚ TRƯỚC

G06

BỐI CẢNH

Sổ điểm mini PW5–PW9. report.py đã cấp: đọc "tên,điểm" từ stdin, in báo cáo; không có dòng dữ liệu nào thì in lỗi ra stderr và thoát mã 1. Viết lần lượt: 1) validate(name, raw_mark) → (tên đã strip, float 0..10), nhận cả "7,5"; tên trống, không phải số hoặc ngoài khoảng → ValueError. 2) save_marks(marks, path) lưu dict bằng pickle NÉN gzip; load_marks(path) nạp lại, file chưa có → {}. 3) run_report(marks) chạy [sys.executable, "report.py"] bằng subprocess, gửi mỗi sinh viên một dòng "tên,điểm" qua stdin, trả stdout (str); con thoát mã khác 0 → RuntimeError(stderr của con). 4) start_save(marks, path, done) start một Thread lưu BẢN SAO marks; xong thì put ("ok", số sinh viên) vào done (OSError → ("error", thông báo)); trả Thread, không join. 5) App (giao diện đã dựng sẵn): __init__ nạp self.marks = load_marks(path); add → validate, self.marks[tên] = điểm, status "Có <n> sinh viên" (sai → showerror); save → status "Đang lưu...", self.saver = start_save(...), after(100, self.poll_save); poll_save → get_nowait, rỗng thì hẹn lại, có thì status "Đã lưu <n> sinh viên"; report → self.output hiện run_report(self.marks) (lỗi → showerror); wait_for_save → join self.saver nếu có.

VÍ DỤ MINH HOẠ

Thêm Mai 8.5, An 7 → bấm Lưu → "Đang lưu..." → "Đã lưu 2 sinh viên"; Báo cáo → "Mai: 8.5 / An: 7.0 / Average: 7.75".

LÝ THUYẾT

Bài cửa: ghép bốn kỹ thuật của PW5–PW9

Chuỗi bài thực hành của giảng viên là MỘT chương trình lớn dần: PW5–PW6 thêm lưu dữ liệu (pickle, nén), PW7 làm một shell chạy lệnh và nối vào/ra giữa các process, PW8 chuyển việc lưu sang thread nền, PW9 thay giao diện dòng lệnh bằng Tkinter. Bài này gộp lại: một sổ điểm có GUI, lưu nén, chạy báo cáo ở process con và lưu ở thread nền.

Bốn mảnh ghép, mỗi mảnh một lý do

- pickle + gzip: dict Python → bytes → file nén; gzip.open(path, "wb"/"rb"), luôn nhị phân
- subprocess: báo cáo là CHƯƠNG TRÌNH KHÁC, chạy bằng sys.executable để trúng Python hiện tại
- Thread: lưu file có thể lâu, nên nó không được chạy trên thread của mainloop
- Tkinter: App chỉ đọc widget, gọi hàm thuần, hiện kết quả — mọi quy tắc nằm ngoài giao diện
- Queue + after(100): đường duy nhất để worker báo kết quả về cho widget

Ví dụ: một vòng dữ liệu trọn vẹn

Gõ "Mai", "8,5" → validate trả ("Mai", 8.5), raise ValueError nếu tên trống hoặc điểm ngoài 0..10. Bấm Lưu → status "Đang lưu...", worker pickle+gzip BẢN SAO của marks rồi put ("ok", 2) vào Queue, after(100) đọc được và đổi status thành "Đã lưu 2 sinh viên". Bấm Báo cáo → report.py chạy ở process con, nhận "Mai,8.5" qua stdin, trả về "Average: 7.75" qua stdout.

Bốn chỗ dễ hỏng nhất

- Lưu thẳng self.marks thay vì bản sao: người dùng gõ tiếp trong lúc worker đang pickle
- join() ngay trong callback Lưu: cửa sổ đứng, worker thành vô nghĩa — join lúc thoát
- Quên text=True: stdout là bytes, so sánh chuỗi trượt hết
- Bỏ qua returncode: con thoát mã 1 vì không có dữ liệu mà GUI vẫn hiện báo cáo rỗng
- load_marks lần chạy đầu: FileNotFoundError là bình thường → trả {}, đừng để nổ

Đọc sơ đồ

Hình là một vòng khép kín quanh ô GUI. Đi xuống: dữ liệu đã validate → worker thread → pickle+gzip → file .gz trên đĩa, kết quả quay về GUI qua Queue và after(). Đi sang ngang: marks → stdin của process con report.py → stdout quay về ô kết quả. Ranh giới nét đứt là chỗ ĐỔI KHÔNG GIAN: qua nó là sang thread khác hoặc process khác, nên dữ liệu phải đi bằng Queue hoặc pipe chứ không dùng chung biến.



Hình R02-pipeline. Đường đi dữ liệu: GUI → thread lưu nén, GUI → process con báo cáo

Mô tả hình: Sơ đồ GUI ở giữa; nhánh dưới qua worker thread, pickle và gzip ra file, kết quả về qua Queue và after(); nhánh ngang qua stdin của process con report.py rồi stdout về GUI

Nguồn lý thuyết: SL 54-56, 106-107, 135 (xem chú giải ở bìa)

ĐIỂM KIẾN THỨC

1. PW5–PW6: pickle biến dict thành bytes, gzip nén lại. Mở file bằng gzip.open(path, "wb"/"rb") — chế độ nhị phân.
2. Khởi động: nạp lại file nếu có; FileNotFoundError nghĩa là lần chạy đầu → bắt đầu với {}. Chỉ load pickle từ nguồn tin cậy.
3. PW7: process con chạy bằng sys.executable (đúng Python đang chạy); input= đóng vai "<", capture_output như ">".
4. text=True để stdout là str; không có thì nhận bytes. returncode khác 0 → báo lỗi bằng stderr của con.
5. PW8: lưu ở worker thread trên BẢN SAO dữ liệu; báo kết quả qua Queue; join trước khi thoát chứ không join trong callback.
6. PW9: App chỉ đọc widget, gọi hàm thuần và hiện kết quả; chỉ lưu dữ liệu đã qua validate.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import gzip
import pickle
import threading

result = {}


def save():
    result["blob"] = gzip.compress(pickle.dumps({"marks": [8, 7]}))


worker = threading.Thread(target=save)
worker.start()
worker.join()

blob = result["blob"]
print(blob[:2], len(blob), "byte")
print(pickle.loads(gzip.decompress(blob)))
```

Kết quả dự kiến (theo JSON): run_report({"Mai": 8.5, "An": 7.0}) → ... Average: 7.75; Bấm Lưu → "Đang lưu..." rồi "Đã lưu 2 sinh viên"

NGỘ NHẬN THƯỜNG GẶP: Bốn cái bẫy của PW5–PW9

Quên text=True nên stdout là bytes (và input= str nổ TypeError); gọi "python" thay vì sys.executable nên chạy nhầm Python khác (hoặc không có); lưu pickle mà không nén dù đề yêu cầu nén; join() worker ngay trong nút Lưu làm cửa sổ đứng. Ngoài ra: worker không được tự đổi Label — chỉ put vào queue.

CÂU DỰ ĐOÁN

Vì sao nút Lưu không được join() worker ngay trong callback?

sai	join làm file bị lưu hai lần	join chỉ chờ, không chạy lại target.
ĐÚNG	join chặn thread GUI tới khi lưu xong — cửa sổ đứng, mất ý nghĩa của worker	Chờ worker để lúc thoát (wait_for_save).
sai	Không sao, join chạy rất nhanh	Lưu file lớn hoặc ổ chậm có thể mất nhiều giây.

TỰ KIỂM TRA

1. subprocess.run([...], capture_output=True) không có text=True thì result.stdout là gì?

• str như thường — Mặc định pipe trả bytes.

• None — capture_output=True đã thu stdout.

✔ bytes, ví dụ b"Mai: 8.5\n..." — text=True mới giải mã thành str.

Đáp án: text=True mới giải mã thành str.

2. Cần học threading trước khi tạo nút Tkinter đầu tiên không?

• Có, Tkinter bắt buộc dùng thread — Tkinter chạy tốt trên một thread.

✔ Không; cần khi kết hợp nút với tác vụ nền dài — Nút đầu tiên chỉ cần callback; thread chỉ cần khi callback phải chờ lâu.

• Có, mỗi nút một thread — Chỉ việc dài mới cần worker.

Đáp án: Nút đầu tiên chỉ cần callback; thread chỉ cần khi callback phải chờ lâu.

3. Vì sao gọi [sys.executable, "report.py"] thay vì ["python", "report.py"]?

• sys.executable chạy được lệnh shell — Nó chỉ là đường dẫn tới trình thông dịch.

• "python" chạy chậm hơn — Không phải chuyện tốc độ.

✔ sys.executable là đúng file Python đang chạy chương trình; "python" có thể là bản khác hoặc không tồn tại — Trên Windows "python" có thể là stub của Store.

Đáp án: Trên Windows "python" có thể là stub của Store.

TỰ GIẢI THÍCH

Kể đường đi của dữ liệu từ ô nhập tới file marks.dat và tới báo cáo, nêu thread/process nào làm từng bước.

Mẫu trả lời: Thread GUI: add đọc hai StringVar, validate, ghi vào self.marks. Bấm Lưu: thread GUI start worker với bản sao marks; worker pickle + gzip ra file rồi put ("ok", n) vào queue; poll_save (thread GUI, qua after) đổi status. Báo cáo: process con report.py nhận "tên,điểm" qua stdin, in báo cáo ra stdout cho cha hiện lên Label. Trường hợp đặc biệt: chưa có file → {}, dữ liệu rỗng → con thoát mã 1 → RuntimeError.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Ô nhập → validate() → self.marks[tên] = điểm
Bước 2	Bấm Lưu → start_save() → worker pickle + gzip → put ("ok", n)
Bước 3	poll_save() qua after() → status "Đã lưu n sinh viên"
Bước 4	Bấm Báo cáo → subprocess report.py → stdout lên self.output

CÂU PHẢN VÍ DỤ

run_report({}) (không có sinh viên nào) cho kết quả gì?

Lựa chọn: Chuỗi rỗng "" | RuntimeError với stderr của report.py | Average: 0.00 · Đáp án: RuntimeError với stderr của report.py

LUYỆN CÓ HỖ TRỢ · Lập kế hoạch trước khi tự viết

Đi lại đường dữ liệu của bài trước khi tự viết code.

Yêu cầu thêm: Bộ chấm gọi thẳng các hàm thuần với dữ liệu mới, kiểm file nén gzip, chạy process con với dữ liệu rỗng, rồi bấm Thêm/Lưu/Báo cáo trên App và chạy hàng đợi after() thay event loop.

BÀI TẬP ĐỘC LẬP · Boss — tự viết, nộp lần đầu trước khi xem lời giải

Sổ điểm mini PW5–PW9. report.py đã cấp: đọc "tên,điểm" từ stdin, in báo cáo; không có dòng dữ liệu nào thì in lỗi ra stderr và thoát mã 1. Viết lần lượt:

1) validate(name, raw_mark) → (tên đã strip, float 0..10), nhận cả "7,5"; tên trống, không phải số hoặc ngoài khoảng → ValueError.

2) save_marks(marks, path) lưu dict bằng pickle NÉN gzip; load_marks(path) nạp lại, file chưa có → {}.

3) run_report(marks) chạy [sys.executable, "report.py"] bằng subprocess, gửi mỗi sinh viên một dòng "tên,điểm" qua stdin, trả stdout (str); con thoát mã khác 0 → RuntimeError(stderr của con).

4) start_save(marks, path, done) start một Thread lưu BẢN SAO marks; xong thì put ("ok", số sinh viên) vào done (OSError → ("error", thông báo)); trả Thread, không join.

5) App (giao diện đã dựng sẵn): __init__ nạp self.marks = load_marks(path); add → validate, self.marks[tên] = điểm, status "Có <n> sinh viên" (sai → showerror); save → status "Đang lưu...", self.saver = start_save(...), after(100, self.poll_save); poll_save → get_nowait, rỗng thì hẹn lại, có thì status "Đã lưu <n> sinh viên"; report → self.output hiện run_report(self.marks) (lỗi → showerror); wait_for_save → join self.saver nếu có.

Thực hành: Viết và thử từng hàm thuần trước (validate → save/load → run_report → start_save), rồi mới nối vào các method của App.

Gọi	Kết quả mong đợi
`validate(" Mai ", "8.5")`	`("Mai", 8.5)`
`run_report({"Mai": 8.5, "An": 7.0})`	`"Mai: 8.5\nAn: 7.0\nAverage: 7.75\n"`
Bấm Lưu với 2 sinh viên	"Đang lưu..." rồi (event loop) "Đã lưu 2 sinh viên"; file nén gzip

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`validate(name, raw_mark)`	Kiểm và chuẩn hoá một dòng nhập trước khi đưa vào dữ liệu. · name, raw_mark: str như người dùng gõ (raw_mark có thể dùng dấu phẩy "7,5") · (tên đã strip, float 0..10); tên trống / không phải số / ngoài khoảng → ValueError · Hàm thuần.
`save_marks(marks, path) / load_marks(path)`	PW5–PW6: lưu dict điểm bằng pickle, nén gzip; nạp lại. · marks: dict tên → điểm; path: tên file · load_marks: dict; file chưa có → {} · Ghi/đọc file nhị phân nén gzip (2 byte đầu 1f 8b).
`run_report(marks)`	PW7: chạy report.py trong process con, gửi dữ liệu qua stdin. · marks: dict · str — stdout của process con; con thoát mã khác 0 → RuntimeError(stderr) · subprocess.run([sys.executable, "report.py"], input=..., capture_output=True, text=True, timeout=...)
`start_save(marks, path, done)`	PW8: lưu ở worker thread để giao diện không đứng. · marks: dict (worker nhận BẢN SAO); path; done: Queue · Thread đã start (KHÔNG join) · Worker gọi save_marks; xong → done.put(("ok", số sinh viên)); OSError → done.put(("error", thông báo)).
`App(root, path=DATA_FILE)`	PW9: giao diện Tkinter gọi các hàm trên. · root: Tk; path: file dữ liệu · self.marks (nạp từ path), self.status, self.output, self.saver · add: validate → self.marks; save: "Đang lưu..." + start_save + after(poll_save); poll_save: "Đã lưu <n> sinh viên"; report: self.output = run_report(...); wait_for_save: join worker trước khi thoát.

BỘ CHẤM (17 ca)

Ca	Gọi	Mong đợi
Khởi động chưa có marks.dat → "Đã nạp 0 sinh viên"	`app.status.cget('text')`	`'Đã nạp 0 sinh viên'`
validate chuẩn hoá tên và điểm	`[validate(" Mai ", "8.5"), validate("An", "7,25")]`	`[('Mai', 8.5), ('An', 7.25)]`
validate từ chối dữ liệu sai bằng ValueError	`[_studio.call(validate, n, m).error for n, m in [('An', 'abc'), (' ', '7'), ('Binh', '11'), ('Chi', '-0.5')]]`	`['ValueError', 'ValueError', 'ValueError', 'ValueError']`
save_marks rồi load_marks phục hồi đúng dict	`(save_marks({'Lan': 9.25, 'Huy': 6.0}, 't1.dat'), load_marks('t1.dat'))[1]`	`{'Lan': 9.25, 'Huy': 6.0}`
File lưu được nén gzip (2 byte đầu 1f 8b)	`(save_marks({'A': 1.0}, 't2.dat'), open('t2.dat', 'rb').read(2))[1]`	`b'\x1f\x8b'`
Chưa có file → load_marks trả {}	`load_marks('chua_co.dat')`	`{}`
run_report: process con in báo cáo	`run_report({'Mai': 8.5, 'An': 7.0})`	`'Mai: 8.5\nAn: 7.0\nAverage: 7.75\n'`
Process con lỗi (không có dữ liệu) → RuntimeError	`_studio.call(run_report, {}).error`	`'RuntimeError'`
start_save: một worker, ghi file, gửi ("ok", 2)	`(lambda box: (lambda r: (r.thread_count, load_marks('t3.dat'), box.get_nowait()))(_studio.call(start_save, {'Mai': 8.0, 'An': 6.5}, 't3.dat', box)))(__import__('queue').Queue())`	`(1, {'Mai': 8.0, 'An': 6.5}, ('ok', 2))`

Bấm Thêm: Mai 8.5 rồi An 7	`(app.marks.clear(), app.name_var.set('Mai'), app.mark_var.set('8.5'), _studio.tk.find(type='Button', text='Thêm').invoke(), app.status.cget('text'), app.name_var.set('An'), app.mark_var.set('7'), _studio.tk.find(type='Button', text='Thêm').invoke(), app.status.cget('text'), dict(app.marks))[-2:]`	`('Có 2 sinh viên', {'Mai': 8.5, 'An': 7.0})`
Dữ liệu sai → showerror, không thêm	`(lambda n, m: (app.name_var.set('Binh'), app.mark_var.set('12'), _studio.tk.find(type='Button', text='Thêm').invoke(), app.status.cget('text'), len(app.marks) - n, len(_studio.tk.messages) - m)[-2:])(len(app.marks), len(_studio.tk.messages))`	`(0, 1)`
Bấm Lưu: worker nền, không join trong callback, "Đang lưu..."	`(lambda r: (r.thread_count, r.threads[0].joined, app.status.cget('text'), _studio.tk.run_after(n=5))[:3])(_studio.call(_studio.tk.find(type='Button', text='Lưu').invoke))`	`(1, False, 'Đang lưu...')`
Event loop chạy poll_save → "Đã lưu 1 sinh viên", file có dữ liệu	`(app.marks.clear(), app.marks.update({'Kim': 9.0}), _studio.tk.find(type='Button', text='Lưu').invoke(), _studio.tk.run_after(n=5), app.status.cget('text'), load_marks(app.path))[-2:]`	`('Đã lưu 1 sinh viên', {'Kim': 9.0})`
Bấm Báo cáo → hiện stdout của process con	`(app.marks.clear(), app.marks.update({'Lan': 9.0, 'Huy': 6.0}), _studio.tk.find(type='Button', text='Báo cáo').invoke(), app.output.cget('text'))[-1]`	`'Lan: 9.0\nHuy: 6.0\nAverage: 7.50\n'`
Trước khi thoát: wait_for_save() join worker đang lưu	`(lambda r: (app.wait_for_save(), r.threads[0].joined, _studio.tk.run_after(n=5))[1])(_studio.call(_studio.tk.find(type='Button', text='Lưu').invoke))`	`True`
Khởi động lại: App mới nạp file đã lưu	`(save_marks({'Tam': 5.5}, 'boot.dat'), (lambda a: (a.marks, a.status.cget('text')))(App(tk.Tk(), 'boot.dat')))[1]`	`({'Tam': 5.5}, 'Đã nạp 1 sinh viên')`
Process con chạy bằng sys.executable	`[c['args'][0] == __import__('sys').executable for c in _studio.run(entry="run_report({'Mai': 8.0})").subprocess_calls]`	`[True]`

MÔI TRƯỜNG CHẠY

Trình duyệt không mở được cửa sổ Tkinter, nên Studio dùng một tkinter MÔ PHỎNG: widget, biến (StringVar...), pack/grid/place, command=, bind và after() được ghi lại để chấm; mainloop() trả về ngay sau khi chạy hết các lệnh after() đang chờ. Không có màn hình nên bạn không thấy cửa sổ — bộ chấm "bấm nút" thay bạn. Chạy file trên máy (python ten_file.py) để thấy giao diện thật. Thread và subprocess cũng là mô phỏng: worker chạy xong ngay khi start(); process con chỉ chạy được [sys.executable, "report.py"] hoặc [sys.executable, "-c", code], có stdin/stdout/stderr riêng — lệnh shell (dir, ls...) không chạy. Chạy trên máy: lưu report.py cùng thư mục với bài và chạy lệnh từ thư mục đó (đường dẫn "report.py" là tương đối); trên Windows dữ liệu qua pipe dùng bảng mã mặc định của hệ điều hành, nên thử với tên không dấu. File kèm bài: report.py (module). Bài có cửa tự kiểm tra (gate): lời giải mẫu chỉ mở sau khi hoàn thành.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước)
Khung 1 — Bước 1. Ô nhập → validate() → self.marks[tên] = điểm
Khung 2 — Bước 2. Bấm Lưu → start_save() → worker pickle + gzip → put ("ok", n)
Khung 3 — Bước 3. poll_save() qua after() → status "Đã lưu n sinh viên"
Khung 4 — Bước 4. Bấm Báo cáo → subprocess report.py → stdout lên self.output
Khung đối chiếu — ngộ nhận Bốn cái bẫy của PW5–PW9. Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Quên text=True nên stdout là bytes (và input= str nổ TypeError); gọi "python" thay vì sys.executable nên chạy nhầm Python khác (hoặc không có); lưu pickle mà không nén dù đề yêu cầu nén; join() worker ngay trong nút Lưu làm cửa sổ đứng. Ngoài ra: worker không được tự đổi Label — chỉ put vào queue.
Khung kết quả — 3 cặp vào/ra. validate(" Mai ", "8.5") → ("Mai", 8.5); run_report({"Mai": 8.5, "An": 7.0}) → "Mai: 8.5\nAn: 7.0\nAverage: 7.75\n"; Bấm Lưu với 2 sinh viên → "Đang lưu..." rồi (event loop) "Đã lưu 2 sinh viên"; file nén gzip

Nguồn: pe-v4 · python-learning-data.json#R02 · parser pe-v4/build · file build 020-R02.json

M4 · Nhập dữ liệu, SQL và tra cứu

5 bài: X01, X02, X03, X04, X05

Ứng viên animation — ưu tiên cao (★★): X04, X05 · ưu tiên (★): X01, X02, X03

X01 · Nhập dữ liệu bằng NumPy và pandas

Mã bài: X01 · Trung cấp · M4 · Nhập dữ liệu, SQL và tra cứu · 30 phút · 15 XP · thứ tự 21

MỤC TIÊU

Đọc CSV đúng sep, comment, na_values rồi kiểm schema.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

File marks.csv (đã cấp) dùng dấu chấm phẩy ";" để ngăn cột, dòng đầu là chú thích bắt đầu bằng "#", và điểm chưa có ghi là "-". Viết load_marks(path) đọc file bằng pd.read_csv với sep, comment và na_values phù hợp. Viết schema(df) trả dict gồm rows (số dòng), columns (list tên cột), missing (số ô thiếu ở cột mark) và mean_mark (trung bình cột mark, bỏ qua ô thiếu, làm tròn 2 chữ số).

VÍ DỤ MINH HOẠ

marks.csv → rows 4, columns ["name", "class", "mark"], missing 1, mean_mark 8.17.

ĐIỂM KIẾN THỨC

1. np.loadtxt đọc bảng SỐ sạch thành ndarray: delimiter="," tách cột, skiprows=1 bỏ dòng tiêu đề, usecols chọn cột cần. Gặp chữ hoặc ô trống là ValueError — khi đó dùng np.genfromtxt (học kỹ ở N06).
2. Mỗi array có shape (kích thước từng chiều) và dtype (kiểu chung của mọi phần tử). len(a) chỉ cho độ dài chiều đầu — số hàng, không phải số phần tử.
3. pd.read_csv đọc bảng có tiêu đề thành DataFrame. Tham số hay dùng: sep (ký tự ngăn cột), header (dòng nào là tiêu đề), comment (bỏ phần sau ký tự chú thích), na_values (chuỗi nào coi là thiếu — "NA", ô trống đã có sẵn), nrows (chỉ đọc vài dòng đầu để xem thử).
4. Khám phá trước khi tính: df.shape, df.columns, df.dtypes, df.head()/tail(), df.info(). Cột điểm mà dtype là object nghĩa là còn chữ lẫn trong số — mean sẽ báo lỗi hoặc cho số vô nghĩa.
5. df.values (hay df.to_numpy()) chuyển bảng sang ndarray khi cần tính bằng NumPy; np.info(...) và help(...) tra nhanh tài liệu ngay trong Python.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np
import pandas as pd
from io import StringIO

text = "x,y\n1,2\n3,4\n5,6"
a = np.loadtxt(StringIO(text), delimiter=",", skiprows=1)
print(a.shape, len(a), a.sum())

weather = StringIO("city|temp\nHue|31\nVinh|NA\nHanoi|29")
df = pd.read_csv(weather, sep="|")
print(df.shape, df.columns.tolist())
print(df.dtypes)
print(df["temp"].isna().sum(), df["temp"].mean())
print(df.head(2))
```

Kết quả dự kiến (theo JSON): marks.csv → 4 dòng, 3 cột, 1 ô thiếu; mean_mark → 8.17 (bỏ qua ô thiếu)

NGỘ NHẬN THƯỜNG GẶP: "read_csv chạy không báo lỗi là đã đọc đúng"

Sai delimiter cho ra MỘT cột; quên na_values để "-" biến cả cột điểm thành chữ (dtype object); quên comment thì dòng chú thích chiếm chỗ tiêu đề. Không lỗi nào báo đỏ ngay lúc đọc — chỉ lộ ra khi xem df.columns, df.dtypes và số dòng.

CÂU DỰ ĐOÁN

File ngăn cột bằng ";". Gọi pd.read_csv(path) mà không truyền sep thì DataFrame có mấy cột?

```
name;class;mark
Mai;A;8.5
Binh;A;7
```

sai	Ba cột name, class, mark	read_csv không tự đổi sang ";" khi sep mặc định là ",".
sai	Báo lỗi ngay	Không có lỗi nào — sai lặng lẽ mới là điều nguy hiểm.
ĐÚNG	Một cột duy nhất, tên là cả chuỗi "name;class;mark"	Mặc định sep=",", file không có dấu phẩy nên mỗi dòng là một ô.

TỰ KIỂM TRA

1. Cột mark đọc ra có dtype object thay vì float64. Nguyên nhân khả dĩ nhất?

✔ Còn ký hiệu thiếu như "-" chưa khai trong na_values nên cả cột bị đọc thành chữ — Chỉ một ô không phải số là đủ để cả cột thành object.

• Do file có chữ tiếng Việt có dấu — Mã hoá ảnh hưởng cột chữ, không biến cột số thành object.

• pandas luôn đọc số thực thành object — Cột toàn số được đọc thành float64 hoặc int64.

Đáp án: Chỉ một ô không phải số là đủ để cả cột thành object.

2. a = np.loadtxt(...) có shape (5, 3). len(a) bằng bao nhiêu?

• 3 — Đó là số cột a.shape[1].

✔ 5 — len là độ dài chiều thứ nhất (số hàng) — Tổng số phần tử là a.size = 15.

• 15 — Đó là a.size.

Đáp án: Tổng số phần tử là a.size = 15.

3. Muốn xem thử 3 dòng đầu của một file CSV rất lớn mà KHÔNG đọc hết file, dùng gì?

✔ pd.read_csv(path, nrows=3) — nrows dừng đọc sau 3 dòng dữ liệu.

• pd.read_csv(path, header=3) — header chọn dòng làm tiêu đề, không giới hạn số dòng đọc.

• pd.read_csv(path).head(3) — Vẫn đọc toàn bộ file rồi mới cắt.

Đáp án: nrows dừng đọc sau 3 dòng dữ liệu.

TỰ GIẢI THÍCH

Vì sao phải xem columns, dtypes và số dòng NGAY sau khi đọc file?

Mẫu trả lời: Vì read_csv thường không báo lỗi khi đọc sai: sai sep thì chỉ có một cột, thiếu na_values thì cột số thành object, thiếu comment thì dòng chú thích chiếm chỗ tiêu đề. Kiểm schema ngay giúp phát hiện sớm, trước khi mean hay biểu đồ cho ra kết quả sai mà trông vẫn hợp lý.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	read_csv(sep=";") → tách đúng 3 cột name, class, mark
Bước 2	comment="#" → dòng chú thích đầu file bị bỏ qua
Bước 3	na_values=["-"] → "-" thành NaN, cột mark là float64
Bước 4	schema(df) → rows 4, missing 1, mean_mark 8.17

CÂU PHẢN VÍ DỤ

File "name;class;mark\nHoa;C;6\nLan;C;-" → schema trả mean_mark bao nhiêu?

Lựa chọn: Lỗi TypeError | 3.0 | 6.0 · Đáp án: 6.0

TEMPLATE LẮP MẢNH

```
import pandas as pd


def load_marks(path):
    return pd.read_csv(path, sep={{slot1}}, comment={{slot2}}, na_values={{slot3}})


def schema(df):
    return {
        "rows": {{slot4}},
        "columns": df.columns.tolist(),
        "missing": int(df["mark"].isna().sum()),
        "mean_mark": round(float(df["mark"].mean()), 2),
    }
```

Slot	Mảnh đúng
[Slot 1]	`";"`
[Slot 2]	`"#"`
[Slot 3]	`["-"]`
[Slot 4]	`len(df)`

Mảnh bẫy	Vì sao sai
`","`	Dấu phẩy là mặc định của read_csv; file này ngăn cột bằng ";".
`["NA"]`	"NA" pandas đã tự coi là thiếu; file này ghi thiếu bằng "-".
`df.size`	size là tổng số ô = số dòng × số cột, không phải số dòng.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp các tham số đọc file và cách đếm số dòng.

Yêu cầu thêm: Đọc một file khác cùng quy ước (hai dòng chú thích, ô thiếu ở dòng cuối) mà không sửa code: load_marks phải dùng tham số path, không viết cứng "marks.csv".

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

File marks.csv (đã cấp) dùng dấu chấm phẩy ";" để ngăn cột, dòng đầu là chú thích bắt đầu bằng "#", và điểm chưa có ghi là "-". Viết load_marks(path) đọc file bằng pd.read_csv với sep, comment và na_values phù hợp. Viết schema(df) trả dict gồm rows (số dòng), columns (list tên cột), missing (số ô thiếu ở cột mark) và mean_mark (trung bình cột mark, bỏ qua ô thiếu, làm tròn 2 chữ số).

Thực hành: Truyền đủ sep, comment, na_values cho read_csv; schema lấy số dòng bằng len(df), số ô thiếu bằng isna().sum(), mean tự bỏ qua NaN.

Gọi	Kết quả mong đợi
`marks.csv`	`rows 4 · columns [name, class, mark] · missing 1 · mean_mark 8.17`
`File không có ô thiếu`	`missing 0; mean_mark là trung bình mọi dòng`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`load_marks(path)`	Đọc file điểm dùng dấu ";" ngăn cột, có dòng chú thích "#", ghi thiếu là "-". · path: str — đường dẫn file CSV · DataFrame cột name, class, mark (mark là float64, ô "-" là NaN) · Không in gì.
`schema(df)`	Tóm tắt bảng vừa đọc để kiểm trước khi tính. · df: DataFrame có cột mark · dict rows, columns, missing, mean_mark (làm tròn 2 chữ số) · Không sửa df.

BỘ CHẤM (6 ca)

Ca	Gọi	Mong đợi
marks.csv: đúng 3 cột	`load_marks('marks.csv').columns.tolist()`	`['name', 'class', 'mark']`
marks.csv: 4 dòng, bỏ dòng chú thích	`len(load_marks('marks.csv'))`	`4`
"-" thành ô thiếu, cột mark là số thực	`(int(load_marks('marks.csv')['mark'].isna().sum()), str(load_marks('marks.csv')['mark'].dtype))`	`(1, 'float64')`
schema của marks.csv	`schema(load_marks('marks.csv'))`	`{'rows': 4, 'columns': ['name', 'class', 'mark'], 'missing': 1, 'mean_mark': 8.17}`
File khác: hai dòng chú thích, ô thiếu ở cuối	`schema(load_marks(_studio.write('t1.csv', '# a\n# b\nname;class;mark\nHoa;C;6\nLan;C;-\n')))`	`{'rows': 2, 'columns': ['name', 'class', 'mark'], 'missing': 1, 'mean_mark': 6.0}`
File không có ô thiếu	`schema(load_marks(_studio.write('t2.csv', 'name;class;mark\nKa;A;10\nLy;A;7.5\n')))`	`{'rows': 2, 'columns': ['name', 'class', 'mark'], 'missing': 0, 'mean_mark': 8.75}`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy và pandas THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy pandas. Thư viện: numpy, pandas. File kèm bài: marks.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. read_csv(sep=";") → tách đúng 3 cột name, class, mark
Khung 2 — Bước 2. comment="#" → dòng chú thích đầu file bị bỏ qua
Khung 3 — Bước 3. na_values=["-"] → "-" thành NaN, cột mark là float64
Khung 4 — Bước 4. schema(df) → rows 4, missing 1, mean_mark 8.17
Khung đối chiếu — ngộ nhận "read_csv chạy không báo lỗi là đã đọc đúng". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Sai delimiter cho ra MỘT cột; quên na_values để "-" biến cả cột điểm thành chữ (dtype object); quên comment thì dòng chú thích chiếm chỗ tiêu đề. Không lỗi nào báo đỏ ngay lúc đọc — chỉ lộ ra khi xem df.columns, df.dtypes và số dòng.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: ",", ["NA"], df.size
Khung kết quả — 2 cặp vào/ra. marks.csv → rows 4 · columns [name, class, mark] · missing 1 · mean_mark 8.17; File không có ô thiếu → missing 0; mean_mark là trung bình mọi dòng

Nguồn: pe-v4 · python-learning-data.json#X01 · parser pe-v4/build · file build 021-X01.json

X02 · Excel, Stata, SAS, HDF5 và MATLAB

Mã bài: X02 · Trung cấp · M4 · Nhập dữ liệu, SQL và tra cứu · 30 phút · 15 XP · thứ tự 22

MỤC TIÊU

Chọn đúng thư viện theo đuôi file; ghi–đọc Stata không thêm index.

CẦN NHỚ TRƯỚC

X01

BỐI CẢNH

Hoàn thành READERS (đuôi file chữ thường → tên hàm đọc) cho đủ 8 đuôi: .csv → "pd.read_csv"; .xlsx, .xls → "pd.read_excel"; .dta → "pd.read_stata"; .sas7bdat → "pd.read_sas"; .h5, .hdf5 → "h5py.File"; .mat → "scipy.io.loadmat". Viết reader_for(path): lấy đuôi file (không phân biệt hoa/thường, tên có thể có nhiều dấu chấm), trả tên hàm đọc; đuôi không có trong READERS → raise ValueError. Viết stata_roundtrip(df, path): ghi df ra file Stata KHÔNG kèm index, đọc lại bằng pd.read_stata và trả DataFrame đọc được.

VÍ DỤ MINH HOẠ

reader_for("Lab/Marks.XLSX") → "pd.read_excel"; reader_for("notes.txt") → ValueError; stata_roundtrip giữ đúng hai cột name, mark.

ĐIỂM KIẾN THỨC

1. Mỗi định dạng có "cửa" riêng: Excel (.xlsx/.xls) → pd.read_excel hoặc pd.ExcelFile; Stata (.dta) → pd.read_stata; SAS (.sas7bdat) → pd.read_sas (slide cũ dùng thư viện sas7bdat: SAS7BDAT(path).to_data_frame()); HDF5 (.h5) → h5py.File; MATLAB (.mat) → scipy.io.loadmat.
2. Excel có nhiều sheet: pd.ExcelFile(path).sheet_names liệt kê tên sheet; book.parse("Marks") hoặc pd.read_excel(path, sheet_name="Marks") đọc một sheet (sheet_name=0 là sheet đầu). usecols=["mark"] chỉ lấy cột cần — thay cho parse_cols trong bảng cũ. Cần cài openpyxl.
3. h5py.File(path) mở file HDF5 như một dict lồng nhau: list(f.keys()) cho các group/dataset, f["group/data"][()] lấy dữ liệu (bảng cũ viết .value — đã bị bỏ). scipy.io.loadmat(path) trả dict: tên biến MATLAB → ndarray. File .mat bản v7.3 thực chất là HDF5: loadmat báo NotImplementedError thì mới mở bằng h5py.
4. to_excel / to_stata mặc định ghi cả index thành một cột. Muốn đọc lại đúng bảng gốc thì ghi index=False (Excel) hoặc write_index=False (Stata).
5. Đuôi file lấy bằng Path(path).suffix — phần sau dấu chấm CUỐI, kèm dấu chấm: "exp.v2.h5" → ".h5", "README" → "". Hạ chữ thường bằng .lower() để "Marks.XLSX" vẫn nhận ra Excel. API đổi theo phiên bản: hiểu chức năng trước, rồi tra tài liệu của phiên bản đang dùng.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd
from io import BytesIO

weather = pd.DataFrame({"city": ["Hue", "Vinh"], "temp": [31.5, 29.0]})

with_index = BytesIO()
weather.to_stata(with_index)
with_index.seek(0)
print(pd.read_stata(with_index).columns.tolist())

clean = BytesIO()
weather.to_stata(clean, write_index=False)
clean.seek(0)
back = pd.read_stata(clean)
print(back.columns.tolist())
print(back)
```

Kết quả dự kiến (theo JSON): "Lab/Marks.XLSX" → "pd.read_excel"; "notes.txt" → ValueError

NGỘ NHẬN THƯỜNG GẶP: "Mọi file dữ liệu đều đọc bằng read_csv hoặc một thư viện chung"

.xlsx là file nén chứa XML, .dta và .sas7bdat là định dạng nhị phân của Stata và SAS, .h5 là HDF5 có cây group/dataset, .mat là workspace của MATLAB. Đọc chúng bằng read_csv chỉ ra rác hoặc UnicodeDecodeError. Nhìn đuôi file để chọn đúng thư viện, rồi tra tài liệu phiên bản đang dùng — API cũ như parse_cols, .value đã bị bỏ.

CÂU DỰ ĐOÁN

Ghi rồi đọc lại bằng Stata như dưới đây. Bảng đọc lại có những cột nào?

```
df = pd.DataFrame({"x": [1, 2]})
df.to_stata("a.dta")
print(pd.read_stata("a.dta").columns.tolist())
```

ĐÚNG	["index", "x"] — to_stata mặc định ghi cả index thành cột	Muốn bỏ thì truyền write_index=False.

sai	["x"]	Chỉ đúng khi đã truyền write_index=False.
sai	Báo lỗi vì Stata không lưu được số nguyên	Stata lưu được số nguyên.

TỰ KIỂM TRA

1. Muốn đọc workspace.mat thì dùng gì?

✔ scipy.io.loadmat — trả dict tên biến → array — Nhóm scipy.io trong bảng tra cứu.

• pd.read_csv — .mat là file nhị phân, không phải văn bản.

• pd.read_stata — Dành cho file .dta của Stata.

Đáp án: Nhóm scipy.io trong bảng tra cứu.

2. book.xlsx có hai sheet "Q1" và "Q2". Đọc sheet thứ hai thế nào?

✔ pd.read_excel("book.xlsx", sheet_name="Q2") hoặc sheet_name=1 — Chọn sheet bằng tên hoặc chỉ số đếm từ 0.

• pd.read_excel("book.xlsx") rồi lấy dòng thứ hai — Mặc định chỉ đọc sheet đầu tiên.

• pd.read_excel("book.xlsx", sheet_name=2) — Chỉ số đếm từ 0: 2 là sheet thứ BA (ở đây không có).

Đáp án: Chọn sheet bằng tên hoặc chỉ số đếm từ 0.

3. Vừa mở file HDF5: f = h5py.File("exp.h5"). Muốn biết bên trong có gì, làm gì đầu tiên?

• f.read() — Không có method này; dữ liệu lấy qua key.

• f.head() — h5py.File không phải DataFrame.

✔ list(f.keys()) — xem các group/dataset ở gốc — HDF5 tổ chức như dict lồng nhau.

Đáp án: HDF5 tổ chức như dict lồng nhau.

4. Survey của khoa xuất ra file survey.dta. Hàm nào đọc được?

• pd.read_sas("survey.dta") — read_sas dành cho .sas7bdat/.xpt của SAS — dễ nhầm vì cùng là phần mềm thống kê.

✔ pd.read_stata("survey.dta") — .dta là định dạng của Stata.

• h5py.File("survey.dta") — h5py chỉ mở file HDF5.

Đáp án: .dta là định dạng của Stata.

TỰ GIẢI THÍCH

Vì sao nên ghi index=False / write_index=False khi lưu bảng để chương trình khác đọc lại?

Mẫu trả lời: Mặc định pandas ghi index thành một cột riêng (tên "index" hoặc "Unnamed: 0"). Đọc lại sẽ có thêm cột không có trong bảng gốc, làm lệch số cột và có thể bị tính nhầm như dữ liệu thật. Tắt ghi index thì vòng ghi–đọc trả lại đúng các cột ban đầu.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	Path("Lab/Marks.XLSX").suffix.lower() → ".xlsx"
Bước 2	READERS[".xlsx"] → "pd.read_excel"
Bước 3	Path("notes.txt").suffix không có trong READERS → ValueError
Bước 4	to_stata(write_index=False) + read_stata → đúng cột name, mark

CÂU PHẢN VÍ DỤ

reader_for("exp.v2.h5") trả gì?

Lựa chọn: "h5py.File" | "v2" | ValueError · Đáp án: "h5py.File"

TEMPLATE LẮP MẢNH

```
from pathlib import Path

import pandas as pd

READERS = {".csv": "pd.read_csv", ".dta": {{slot1}}, ".mat": {{slot2}}}


def reader_for(path):
    suffix = {{slot3}}
    if suffix not in READERS:
        raise ValueError("unknown data format: " + path)
    return READERS[suffix]


def stata_roundtrip(df, path):
    {{slot4}}
    return pd.read_stata(path)
```

Slot	Mảnh đúng

[Slot 1]	`"pd.read_stata"`
[Slot 2]	`"scipy.io.loadmat"`
[Slot 3]	`Path(path).suffix.lower()`
[Slot 4]	`df.to_stata(path, write_index=False)`

Mảnh bẫy	Vì sao sai
`"pd.read_sas"`	.dta là Stata; read_sas dành cho .sas7bdat của SAS.
`"h5py.File"`	h5py.File dành cho HDF5 (.h5); .mat thông thường đọc bằng scipy.io.loadmat.
`df.to_stata(path)`	Ghi thêm cột index — đọc lại thừa một cột.
`path.split(".")[1]`	Lấy phần sau dấu chấm ĐẦU TIÊN, không kèm dấu chấm: "exp.v2.h5" → "v2".

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp bảng tra, cách lấy đuôi file và lệnh ghi Stata.

Yêu cầu thêm: Thêm một định dạng mới mà không sửa reader_for: chỉ thêm một dòng vào READERS (ví dụ ".parquet": "pd.read_parquet").

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Hoàn thành READERS (đuôi file chữ thường → tên hàm đọc) cho đủ 8 đuôi: .csv → "pd.read_csv"; .xlsx, .xls → "pd.read_excel"; .dta → "pd.read_stata"; .sas7bdat → "pd.read_sas"; .h5, .hdf5 → "h5py.File"; .mat → "scipy.io.loadmat". Viết reader_for(path): lấy đuôi file (không phân biệt hoa/thường, tên có thể có nhiều dấu chấm), trả tên hàm đọc; đuôi không có trong READERS → raise ValueError. Viết stata_roundtrip(df, path): ghi df ra file Stata KHÔNG kèm index, đọc lại bằng pd.read_stata và trả DataFrame đọc được.

Thực hành: Lấy đuôi bằng Path(path).suffix.lower(); tra trong READERS; không có thì raise ValueError. Ghi Stata với write_index=False rồi đọc lại bằng pd.read_stata.

Gọi	Kết quả mong đợi
`reader_for("Lab/Marks.XLSX")`	`"pd.read_excel"`
`reader_for("notes.txt")`	`ValueError`
`stata_roundtrip(bảng name/mark, "m.dta")`	`DataFrame đúng hai cột name, mark`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`reader_for(path)`	Chọn hàm đọc phù hợp theo đuôi file. · path: str — tên hoặc đường dẫn file · str — tên hàm đọc, vd. "pd.read_excel"; đuôi lạ → ValueError · Không mở file.
`stata_roundtrip(df, path)`	Ghi df ra file Stata (.dta) rồi đọc lại. · df: DataFrame; path: str · DataFrame đọc lại — đúng các cột của df, không thêm cột index · Tạo/ghi đè file path.

BỘ CHẤM (10 ca)

Ca	Gọi	Mong đợi
Excel: marks.xlsx	`reader_for('marks.xlsx')`	`'pd.read_excel'`
Không phân biệt hoa thường, có thư mục	`reader_for('Lab/Marks.XLSX')`	`'pd.read_excel'`
Stata: survey.dta	`reader_for('survey.dta')`	`'pd.read_stata'`
MATLAB: workspace.mat	`reader_for('workspace.mat')`	`'scipy.io.loadmat'`
Đủ các đuôi còn lại (tên có nhiều dấu chấm)	`[reader_for(p) for p in ('exp.v2.h5', 'run.HDF5', 'census.sas7bdat', 'a.csv', 'old.xls')]`	`['h5py.File', 'h5py.File', 'pd.read_sas', 'pd.read_csv', 'pd.read_excel']`
Đuôi lạ → ValueError	`_studio.call(reader_for, 'notes.txt').error`	`'ValueError'`
Không có đuôi → ValueError	`_studio.call(reader_for, 'README').error`	`'ValueError'`
Stata: đúng cột, không thêm index	`stata_roundtrip(pd.DataFrame({'name': ['Mai', 'An'], 'mark': [8.5, 9.0]}), 'r1.dta').columns.tolist()`	`['name', 'mark']`
Stata: giữ nguyên giá trị (dữ liệu khác)	`(lambda d: (d['city'].tolist(), d['temp'].tolist()))(stata_roundtrip(pd.DataFrame({'city': ['Hue', 'Vinh', 'Hanoi'], 'temp': [31.5, 29.0, 27.25]}), 'r2.dta'))`	`(['Hue', 'Vinh', 'Hanoi'], [31.5, 29.0, 27.25])`
Ghi ra đúng path đã cho	`(stata_roundtrip(pd.DataFrame({'x': [1.0]}), 'r3.dta'), __import__('os').path.getsize('r3.dta') > 0)[1]`	`True`

MÔI TRƯỜNG CHẠY

Bài này chỉ nạp pandas trong trình duyệt (Pyodide): ghi/đọc Stata và CSV chạy thật. Excel cần thêm openpyxl, HDF5 cần h5py, file .mat cần scipy — phần đó học qua giải thích và trắc nghiệm; muốn thử trên máy: pip install openpyxl h5py scipy rồi chạy các dòng mẫu trong phần giải thích. Lần đầu mở bài, trình duyệt tải thêm thư viện nên chờ lâu hơn một chút. Thư viện: pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. Path("Lab/Marks.XLSX").suffix.lower() → ".xlsx"
Khung 2 — Bước 2. READERS[".xlsx"] → "pd.read_excel"
Khung 3 — Bước 3. Path("notes.txt").suffix không có trong READERS → ValueError
Khung 4 — Bước 4. to_stata(write_index=False) + read_stata → đúng cột name, mark
Khung đối chiếu — ngộ nhận "Mọi file dữ liệu đều đọc bằng read_csv hoặc một thư viện chung". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. .xlsx là file nén chứa XML, .dta và .sas7bdat là định dạng nhị phân của Stata và SAS, .h5 là HDF5 có cây group/dataset, .mat là workspace của MATLAB. Đọc chúng bằng read_csv chỉ ra rác hoặc UnicodeDecodeError. Nhìn đuôi file để chọn đúng thư viện, rồi tra tài liệu phiên bản đang dùng — API cũ như parse_cols, .value đã bị bỏ.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: "pd.read_sas", "h5py.File", df.to_stata(path), path.split(".")[1]
Khung kết quả — 3 cặp vào/ra. reader_for("Lab/Marks.XLSX") → "pd.read_excel"; reader_for("notes.txt") → ValueError; stata_roundtrip(bảng name/mark, "m.dta") → DataFrame đúng hai cột name, mark

Nguồn: pe-v4 · python-learning-data.json#X02 · parser pe-v4/build · file build 022-X02.json

X03 · SQL với sqlite3 và pandas

Mã bài: X03 · Trung cấp · M4 · Nhập dữ liệu, SQL và tra cứu · 35 phút · 15 XP · thứ tự 23

MỤC TIÊU

Truy vấn có tham số và lấy kết quả về pandas.

CẦN NHỚ TRƯỚC

X02

BỐI CẢNH

make_db(rows) đã cấp: tạo database trong RAM có bảng marks(name, class, mark). Viết find_student(conn, name) trả DataFrame các dòng có đúng tên đó (cột name, class, mark), dùng pd.read_sql_query với chỗ giữ ? và params. Viết class_average(conn, class_name) tính AVG(mark) NGAY TRONG SQL cho một lớp và trả con số (None nếu lớp không có ai). Không ghép giá trị người dùng vào chuỗi SQL.

VÍ DỤ MINH HOẠ

class_average(conn, "A") → 8.25; find_student(conn, "O'Brien") → 1 dòng; find_student(conn, "x' OR '1'='1") → DataFrame rỗng, không lộ cả bảng.

ĐIỂM KIẾN THỨC

1. sqlite3.connect(":memory:") mở một database SQLite nằm trong RAM (đưa đường dẫn file .db thì lưu ra đĩa). conn.execute(sql) trả cursor: cursor.description cho tên cột; fetchone(), fetchmany(size), fetchall() lấy một dòng / tối đa size dòng tiếp theo / tất cả.
2. pd.read_sql_query(sql, conn, params=...) chạy truy vấn và trả thẳng DataFrame, tên cột lấy từ SELECT. Slide cũ dùng SQLAlchemy: engine = create_engine("sqlite:///data.sqlite") rồi đưa engine vào read_sql_query; với SQLite, kết nối sqlite3 có sẵn là đủ. Các ví dụ cũ như engine.table_names() đã bị bỏ ở SQLAlchemy mới.
3. Giá trị từ người dùng KHÔNG ghép vào chuỗi SQL. Viết chỗ giữ ? rồi truyền tuple params=(name,): driver gửi giá trị tách khỏi câu lệnh, nên dấu nháy trong "O'Brien" hay chuỗi "x' OR '1'='1" chỉ là dữ liệu.
4. Để database làm phần tổng hợp: SELECT AVG(mark) ... WHERE class = ? trả về MỘT dòng thay vì kéo cả bảng về Python. Không dòng nào khớp thì AVG trả NULL — Python nhận None.
5. with sqlite3.connect(...) as conn: chỉ tự commit (hoặc rollback khi lỗi) giao dịch, KHÔNG đóng kết nối — vẫn phải conn.close(). Ghi dữ liệu xong nhớ conn.commit(), nếu không một lần rollback sẽ xoá mất các dòng vừa thêm.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import sqlite3
import pandas as pd

with sqlite3.connect(":memory:") as conn:
    conn.execute("CREATE TABLE sales (city TEXT, amount REAL)")
    conn.executemany("INSERT INTO sales VALUES (?, ?)",
                     [("Hue", 120.0), ("Vinh", 80.0), ("Hue", 40.0)])
    cur = conn.execute("SELECT city, amount FROM sales ORDER BY amount DESC")
    print([column[0] for column in cur.description])
    print(cur.fetchmany(2))
    print(cur.fetchmany(2))
    sql = "SELECT city, SUM(amount) AS total FROM sales WHERE amount > ? GROUP BY city"
    print(pd.read_sql_query(sql, conn, params=(50,)))
conn.close()
```

Kết quả dự kiến (theo JSON): find_student(conn, "Mai") → 1 dòng; "x' OR '1'='1" → DataFrame rỗng

NGỘ NHẬN THƯỜNG GẶP: "Ghép chuỗi SQL bằng f-string là được, miễn dữ liệu của mình sạch"

Chỉ cần một tên có dấu nháy (O'Brien) là câu SQL vỡ; một chuỗi cố ý như x' OR '1'='1 biến điều kiện lọc thành luôn đúng và lộ cả bảng — đó là SQL injection. Chỗ giữ ? + params không chỉ an toàn hơn mà còn đúng với MỌI giá trị.

CÂU DỰ ĐOÁN

Ghép giá trị người dùng vào SQL bằng f-string như dưới. Truy vấn trả về gì?

```
name = "x' OR '1'='1"
sql = f"SELECT * FROM marks WHERE name = '{name}'"
```

sai	Không dòng nào, vì không ai tên như vậy	Đúng nếu dùng chỗ giữ ? và params.
sai	Báo lỗi cú pháp SQL	Chuỗi được chọn khéo để câu SQL vẫn hợp lệ.
ĐÚNG	Mọi dòng của bảng — điều kiện thành name = 'x' OR '1'='1', luôn đúng	Chuỗi người dùng đã trở thành một phần câu lệnh SQL: đó là SQL injection.

TỰ KIỂM TRA

1. cursor.fetchmany(size=5) làm gì?

- Chạy truy vấn 5 lần — Truy vấn đã chạy ở execute; fetch chỉ lấy kết quả.
- ✔ Lấy tối đa 5 dòng tiếp theo của kết quả; gọi lại để lấy nhóm sau — Hết dòng thì trả list rỗng.
- Lấy đúng 5 dòng, thiếu thì báo lỗi — Ít hơn 5 dòng vẫn trả bình thường.

Đáp án: Hết dòng thì trả list rỗng.

2. params=(name) với name = "Mai" sẽ thế nào?

- ✔ Lỗi: (name) chỉ là chuỗi "Mai" — sqlite3 coi là 3 giá trị cho 1 dấu ? — Tuple một phần tử phải có dấu phẩy: (name,).
- Tìm lần lượt "M", "a", "i" — sqlite3 báo lỗi số lượng tham số chứ không tìm ba lần.
- Chạy bình thường — (name) không phải tuple, chỉ là name trong ngoặc.

Đáp án: Tuple một phần tử phải có dấu phẩy: (name,).

3. Ra khỏi khối with sqlite3.connect(...) as conn: thì kết nối...

- Bị xoá database — Database trong RAM chỉ mất khi kết nối đóng.
- ✔ Vẫn mở — with chỉ commit/rollback; cần gọi conn.close() — Khác với with open(...) của file.
- Tự đóng như file — Đó là hành vi của open(), không phải sqlite3.

Đáp án: Khác với with open(...) của file.

TỰ GIẢI THÍCH

Vì sao dùng chỗ giữ ? với params thay vì ghép giá trị vào chuỗi SQL?

Mẫu trả lời: Với ? và params, driver gửi câu lệnh và giá trị riêng nhau, nên giá trị chỉ được coi là dữ liệu: dấu nháy trong O'Brien không làm vỡ câu, chuỗi x' OR '1'='1 không thành điều kiện luôn đúng. Ghép chuỗi thì giá trị người dùng trở thành một phần lệnh SQL — vừa lỗi với dữ liệu thật vừa mở cửa SQL injection.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	make_db(ROWS) → bảng marks 4 dòng trong RAM
Bước 2	WHERE name = ? + params=(name,) → giá trị đi riêng, chỉ là dữ liệu
Bước 3	read_sql_query → DataFrame cột name, class, mark
Bước 4	AVG(mark) ... fetchone() → (8.25,), trả 8.25

CÂU PHẢN VÍ DỤ

class_average(conn, "Z") khi không có lớp Z → ?

Lựa chọn: None | 0 | Lỗi ZeroDivisionError · Đáp án: None

TEMPLATE LẮP MẢNH

```
import sqlite3

import pandas as pd


def city_sales(conn, city):
    sql = "SELECT city, amount FROM sales WHERE city = {{slot1}}"
    return pd.read_sql_query(sql, conn, params={{slot2}})


def city_total(conn, city):
    row = conn.execute("SELECT {{slot3}} FROM sales WHERE city = ?", (city,)).{{slot4}}
    return row[0]
```

Slot	Mảnh đúng
[Slot 1]	`?`
[Slot 2]	`(city,)`
[Slot 3]	`SUM(amount)`
[Slot 4]	`fetchone()`

Mảnh bẫy	Vì sao sai
`(city)`	Thiếu dấu phẩy: chỉ là chuỗi city trong ngoặc, không phải tuple.
`'{city}'`	Ghép giá trị vào chuỗi SQL — mở cửa injection (và thiếu chữ f nên chỉ là chữ "{city}").
`fetchall()`	Trả list các dòng; row[0] khi đó là cả tuple chứ không phải con số.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: bảng sales(city, amount). Lắp chỗ giữ, tham số và cách lấy kết quả tổng hợp.

Yêu cầu thêm: Thử tên có dấu nháy (O'Brien) và chuỗi tấn công x' OR '1'='1: truy vấn có tham số phải xử lý cả hai như dữ liệu bình thường.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

make_db(rows) đã cấp: tạo database trong RAM có bảng marks(name, class, mark). Viết find_student(conn, name) trả DataFrame các dòng có đúng tên đó (cột name, class, mark), dùng pd.read_sql_query với chỗ giữ ? và params. Viết class_average(conn, class_name) tính AVG(mark) NGAY TRONG SQL cho một lớp và trả con số (None nếu lớp không có ai). Không ghép giá trị người dùng vào chuỗi SQL.

Thực hành: Viết SQL có dấu ?, truyền giá trị qua params=(name,) / (class_name,); để AVG chạy trong SQL và lấy kết quả bằng fetchone()[0].

Gọi	Kết quả mong đợi
`find_student(conn, "Mai")`	`1 dòng: Mai, A, 8.5`
`find_student(conn, "x' OR '1'='1")`	`DataFrame rỗng`
`class_average(conn, "A")`	`8.25; lớp không có ai → None`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`find_student(conn, name)`	Lấy các dòng của bảng marks có đúng tên name. · conn: kết nối sqlite3; name: str (có thể chứa dấu nháy) · DataFrame cột name, class, mark (rỗng nếu không ai khớp) · Chỉ đọc database.
`class_average(conn, class_name)`	Tính AVG(mark) của một lớp ngay trong SQL. · conn: kết nối sqlite3; class_name: str · float; lớp không có ai → None · Chỉ đọc database.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Tìm đúng một người	`find_student(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), 'Mai').values.tolist()`	`[['Mai', 'A', 8.5]]`
Đủ ba cột name, class, mark	`find_student(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), 'An').columns.tolist()`	`['name', 'class', 'mark']`
Tên có dấu nháy: O'Brien	`find_student(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), "O'Brien")['mark'].tolist()`	`[7.5]`
Chuỗi tấn công không lộ bảng	`len(find_student(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), "x' OR '1'='1"))`	`0`
Hai người trùng tên (dữ liệu khác)	`find_student(make_db([('Hoa', 'C', 6.0), ('Hoa', 'D', 7.0), ('Ty', 'C', 9.0)]), 'Hoa')['class'].tolist()`	`['C', 'D']`
AVG lớp A	`round(class_average(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), 'A'), 2)`	`8.25`
AVG dữ liệu khác	`round(class_average(make_db([('Ty', 'C', 6.0), ('Lan', 'C', 7.0), ('Ka', 'C', 9.5)]), 'C'), 2)`	`7.5`
Lớp không có ai → None	`class_average(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), 'Z')`	`None`
Chuỗi tấn công vào tên lớp → None	`class_average(make_db([('Mai', 'A', 8.5), ('An', 'B', 9.5), ('Binh', 'A', 8.0), ("O'Brien", 'B', 7.5)]), "x' OR '1'='1")`	`None`

MÔI TRƯỜNG CHẠY

sqlite3 chạy thật trong trình duyệt (Pyodide): database ":memory:" hoặc file .db nằm trong thư mục tạm của lần chạy và mất khi chạy lại. Lần đầu dùng, trình duyệt tải thêm module sqlite3. Thư viện: pandas, sqlite3.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 4 slot)
Khung 1 — Bước 1. make_db(ROWS) → bảng marks 4 dòng trong RAM
Khung 2 — Bước 2. WHERE name = ? + params=(name,) → giá trị đi riêng, chỉ là dữ liệu
Khung 3 — Bước 3. read_sql_query → DataFrame cột name, class, mark
Khung 4 — Bước 4. AVG(mark) … fetchone() → (8.25,), trả 8.25
Khung đối chiếu — ngộ nhận "Ghép chuỗi SQL bằng f-string là được, miễn dữ liệu của mình sạch". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Chỉ cần một tên có dấu nháy (O'Brien) là câu SQL vỡ; một chuỗi cố ý như x' OR '1'='1 biến điều kiện lọc thành luôn đúng và lộ cả bảng — đó là SQL injection. Chỗ giữ ? + params không chỉ an toàn hơn mà còn đúng với MỌI giá trị.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: (city), '{city}', fetchall()
Khung kết quả — 3 cặp vào/ra. find_student(conn, "Mai") → 1 dòng: Mai, A, 8.5; find_student(conn, "x' OR '1'='1") → DataFrame rỗng; class_average(conn, "A") → 8.25; lớp không có ai → None

Nguồn: pe-v4 · python-learning-data.json#X03 · parser pe-v4/build · file build 023-X03.json

X04 · Matplotlib: dữ liệu và loại biểu đồ

Mã bài: X04 · Trung cấp · M4 · Nhập dữ liệu, SQL và tra cứu · 30 phút · 15 XP · thứ tự 24

MỤC TIÊU

Tạo Figure/Axes và chọn line hay histogram theo câu hỏi.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

Viết hai hàm, mỗi hàm tạo fig, ax = plt.subplots(), vẽ lên ax rồi RETURN fig (không gọi plt.show() trong hàm). (1) plot_weekly(weeks, scores): line plot có marker "o", nhãn trục x "Week", trục y "Score". (2) plot_histogram(scores, edges): histogram dùng đúng các biên bin edges (bins=edges), nhãn trục x "Score", trục y "Count".

VÍ DỤ MINH HOẠ

plot_weekly([1, 2, 3], [7, 8, 6]) → một Axes, một đường đi qua 7, 8, 6. plot_histogram([4, 5, 5, 7, 8, 9], [4, 6, 8, 10]) → 3 cột cao 3, 1, 2.

ĐIỂM KIẾN THỨC

1. Một biểu đồ đi qua các bước: chuẩn bị dữ liệu → tạo Figure/Axes → vẽ → tùy chỉnh (nhãn, tiêu đề) → lưu → hiển thị. plt.subplots() trả về CẶP (fig, ax): Figure là cả tấm hình, Axes là một vùng toạ độ bên trong để vẽ.
2. Vẽ bằng method của ax (ax.plot, ax.hist, ax.set…) để biết chắc nét vẽ rơi vào Axes nào. Figure là một object Python: fig.axes, ax.lines, ax.patches cho đọc lại những gì đã vẽ — bộ chấm của bài dùng chính các thuộc tính này, không so ảnh.
3. Chọn biểu đồ theo câu hỏi: line nối các điểm theo một trục có thứ tự (tuần, ngày); bar/barh so các category; scatter cho quan hệ giữa hai biến; histogram ĐẾM số quan sát rơi vào từng khoảng (bin) — đó là phân bố, không phải diễn tiến.
4. ax.hist(data, bins=[4, 6, 8, 10]) tạo 3 bin [4, 6), [6, 8), [8, 10] — bin cuối gồm cả biên phải. Không truyền bins thì Matplotlib tự chia 10 bin đều, số cột và chiều cao đổi theo, kết luận về phân bố cũng đổi.
5. Trên máy: script gọi plt.show() ở cuối để mở cửa sổ; notebook tự hiện figure cuối cell. Hàm vẽ nên RETURN fig để nơi gọi tự quyết định hiển thị, lưu file hay kiểm tra. NumPy (linspace, sin…) hay đi kèm để tạo dữ liệu vẽ thử.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 2, 3], [20, 30, 25], marker="o")
ax.set(xlabel="Day", ylabel="Visitors")

print(type(fig).__name__, len(fig.axes))
print(ax.get_xlabel(), "/", ax.get_ylabel())
print(ax.lines[0].get_ydata().tolist())

fig2, ax2 = plt.subplots()
counts, edges, bars = ax2.hist([1, 2, 2, 4, 5], bins=[0, 3, 6])
print(counts.tolist(), len(ax2.patches))
```

Kết quả dự kiến (theo JSON): plot_weekly([1, 2, 3], [7, 8, 6]) → 1 Axes, nhãn Week / Score; plot_histogram([0, 10, 10], [0, 5, 10]) → cột cao 1 và 2

NGỘ NHẬN THƯỜNG GẶP: "Histogram và line plot cùng một mục đích — đều là vẽ các con số"

Line nối điểm theo một trục có thứ tự (tuần 1 → 2 → 3); histogram đếm bao nhiêu quan sát rơi vào mỗi khoảng giá trị. Vẽ danh sách điểm thi bằng ax.plot(scores) cho ra một đường zig-zag theo THỨ TỰ NHẬP, không nói gì về việc điểm tập trung ở đâu.

CÂU DỰ ĐOÁN

ax.hist([4, 5, 5, 7, 8, 9], bins=[4, 6, 8, 10]) tạo mấy cột, cao bao nhiêu?

sai	4 cột: 3, 1, 1, 1	Bốn biên tạo ba bin, không phải bốn.
sai	6 cột, mỗi cột cao 1	hist không vẽ một cột cho mỗi giá trị; nó đếm theo bin.
ĐÚNG	3 cột: 3, 1, 2	[4, 6) có 4, 5, 5; [6, 8) có 7; [8, 10] có 8 và 9 — bin cuối gồm cả biên phải.
sai	3 cột: 3, 1, 1	8 và 9 cùng rơi vào bin cuối [8, 10].

TỰ KIỂM TRA

1. fig, ax = plt.subplots() — fig và ax khác nhau thế nào?
   - ax là file ảnh, fig là dữ liệu — Chưa có file nào cho tới khi gọi savefig.

✔ fig là cả tấm hình (Figure), ax là một vùng toạ độ (Axes) bên trong để vẽ — Một Figure có thể chứa nhiều Axes: fig.axes là list.

• Giống nhau, chỉ là hai tên gọi — Figure chứa Axes; dữ liệu được vẽ lên Axes.

Đáp án: Một Figure có thể chứa nhiều Axes: fig.axes là list.

2. Muốn xem điểm thi của lớp tập trung ở khoảng nào, chọn biểu đồ nào?

• Line plot — Line nối các điểm theo thứ tự; thứ tự nhập danh sách điểm không có ý nghĩa.

• Scatter — Scatter cần HAI biến (ví dụ giờ học và điểm).

✔ Histogram — Histogram đếm số quan sát theo khoảng — đó là phân bố.

Đáp án: Histogram đếm số quan sát theo khoảng — đó là phân bố.

3. Gọi ax.hist(scores) mà không truyền bins thì...

• Báo lỗi thiếu bins — bins có giá trị mặc định nên không lỗi.

✔ Matplotlib tự chia 10 bin đều — số cột và chiều cao phụ thuộc lựa chọn đó — Muốn so được giữa các lần vẽ thì chốt biên bin.

• Mỗi giá trị khác nhau thành một cột — Không — mặc định là 10 bin.

Đáp án: Muốn so được giữa các lần vẽ thì chốt biên bin.

TỰ GIẢI THÍCH

Vì sao hàm vẽ nên return fig thay vì gọi plt.show() bên trong?

Mẫu trả lời: Hàm trả fig thì nơi gọi tự quyết định: hiển thị (plt.show()), lưu file (fig.savefig) hay kiểm tra (fig.axes[0].get_xlabel()). plt.show() bên trong hàm khoá việc dùng lại: trên máy nó chặn chương trình tới khi đóng cửa sổ, và sau khi cửa sổ đóng thì figure đã bị huỷ, không lưu được nữa.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	fig, ax = plt.subplots() → một Figure chứa một Axes
Bước 2	ax.plot(weeks, scores, marker="o") → đường nối điểm theo tuần
Bước 3	ax.hist(scores, bins=edges) → đếm số điểm trong từng bin
Bước 4	ax.set(xlabel=..., ylabel=...) → trục có tên
Bước 5	return fig → nơi gọi tự show, lưu hoặc kiểm tra

CÂU PHẢN VÍ DỤ

plot_histogram([0, 10, 10], [0, 5, 10]) cho các cột cao bao nhiêu?

Lựa chọn: 1 và 0 | 3 | 1 và 2 · Đáp án: 1 và 2

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt


def plot_weekly(weeks, scores):
    fig, ax = plt.subplots()
    ax.plot(weeks, scores, marker="o")
    ax.set(xlabel="Week", ylabel="Score")
    return fig


def plot_histogram(scores, edges):
    {{slot1}}
    {{slot2}}
    {{slot3}}
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`fig, ax = plt.subplots()`
[Slot 2]	`ax.hist(scores, bins=edges)`
[Slot 3]	`ax.set(xlabel="Score", ylabel="Count")`

Mảnh bẫy	Vì sao sai
`plt.show()`	Không ghi nhãn; và hàm vẽ không nên tự show.
`fig = plt.subplots()`	subplots() trả về CẶP (fig, ax); fig sẽ thành tuple.
`ax.plot(scores)`	Nối các điểm theo thứ tự nhập — không đếm theo bin.
`ax.hist(scores)`	Không truyền bins → 10 bin tự động, không phải biên để cho.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

plot_weekly đã viết sẵn. Lắp ba chỗ của plot_histogram: tạo Figure/Axes, vẽ histogram với biên cho sẵn, ghi nhãn.

Yêu cầu thêm: Đổi dữ liệu: 4 tuần với điểm [5, 9, 4, 6]; điểm [0, 10, 10] với biên [0, 5, 10]. Tự đoán số cột và chiều cao trước khi chạy.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm, mỗi hàm tạo fig, ax = plt.subplots(), vẽ lên ax rồi RETURN fig (không gọi plt.show() trong hàm). (1) plot_weekly(weeks, scores): line plot có marker "o", nhãn trục x "Week", trục y "Score". (2) plot_histogram(scores, edges): histogram dùng đúng các biên bin edges (bins=edges), nhãn trục x "Score", trục y "Count".

Thực hành: Mỗi hàm: fig, ax = plt.subplots() → vẽ bằng ax → ax.set(...) → return fig. Histogram phải nhận bins=edges.

Gọi	Kết quả mong đợi
`plot_weekly([1, 2, 3], [7, 8, 6])`	`Figure 1 Axes; đường qua 7, 8, 6; nhãn Week / Score`
`plot_histogram([4, 5, 5, 7, 8, 9], [4, 6, 8, 10])`	`3 cột cao 3, 1, 2; nhãn Score / Count`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_weekly(weeks, scores)`	Vẽ diễn tiến điểm theo tuần. · weeks, scores: hai list số cùng độ dài · Figure có đúng 1 Axes · Line có marker "o"; nhãn Week / Score; không gọi plt.show().
`plot_histogram(scores, edges)`	Vẽ phân bố điểm theo các bin cho sẵn. · scores: list số; edges: list biên bin tăng dần · Figure có 1 Axes, len(edges) - 1 cột · Cột thứ i cao bằng số điểm trong bin i; bin cuối gồm cả biên phải.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
plot_weekly trả về Figure có đúng 1 Axes	`(lambda f: (type(f).__name__, len(f.axes)))(plot_weekly([1, 2, 3, 4], [5, 9, 4, 6]))`	`('Figure', 1)`
Line đi qua đúng dữ liệu mới	`[float(v) for v in plot_weekly([1, 2, 3, 4], [5, 9, 4, 6]).axes[0].lines[0].get_ydata()]`	`[5.0, 9.0, 4.0, 6.0]`
Line có marker "o"	`plot_weekly([1, 2, 3, 4], [5, 9, 4, 6]).axes[0].lines[0].get_marker()`	`'o'`
Nhãn trục Week / Score	`(lambda ax: (ax.get_xlabel(), ax.get_ylabel()))(plot_weekly([1, 2, 3, 4], [5, 9, 4, 6]).axes[0])`	`('Week', 'Score')`
Histogram: số cột = số bin	`len(plot_histogram([0, 10, 10, 3, 7], [0, 5, 10]).axes[0].patches)`	`2`
Histogram đếm đúng; 10 thuộc bin cuối	`[float(v) for v in (p.get_height() for p in plot_histogram([0, 10, 10, 3, 7], [0, 5, 10]).axes[0].patches)]`	`[2.0, 3.0]`
Histogram giữ cả bin rỗng	`[float(v) for v in (p.get_height() for p in plot_histogram([1, 1, 2, 9], [0, 2, 4, 6, 8, 10]).axes[0].patches)]`	`[2.0, 1.0, 0.0, 0.0, 1.0]`
Histogram có nhãn Score / Count	`(lambda ax: (ax.get_xlabel(), ax.get_ylabel()))(plot_histogram([0, 10, 10, 3, 7], [0, 5, 10]).axes[0])`	`('Score', 'Count')`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. fig, ax = plt.subplots() → một Figure chứa một Axes
Khung 2 — Bước 2. ax.plot(weeks, scores, marker="o") → đường nối điểm theo tuần
Khung 3 — Bước 3. ax.hist(scores, bins=edges) → đếm số điểm trong từng bin
Khung 4 — Bước 4. ax.set(xlabel=..., ylabel=...) → trục có tên
Khung 5 — Bước 5. return fig → nơi gọi tự show, lưu hoặc kiểm tra
Khung đối chiếu — ngộ nhận "Histogram và line plot cùng một mục đích — đều là vẽ các con số". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Line nối điểm theo một trục có thứ tự (tuần 1 → 2 → 3); histogram đếm bao nhiêu quan sát rơi vào mỗi khoảng giá trị. Vẽ danh sách điểm thi bằng ax.plot(scores) cho ra một đường zig-zag theo THỨ TỰ NHẬP, không nói gì về việc điểm tập trung ở đâu.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: plt.show(), fig = plt.subplots(), ax.plot(scores), ax.hist(scores)
Khung kết quả — 2 cặp vào/ra. plot_weekly([1, 2, 3], [7, 8, 6]) → Figure 1 Axes; đường qua 7, 8, 6; nhãn Week / Score; plot_histogram([4, 5, 5, 7, 8, 9], [4, 6, 8, 10]) → 3 cột cao 3, 1, 2; nhãn Score / Count

Nguồn: pe-v4 · python-learning-data.json#X04 · parser pe-v4/build · file build 024-X04.json

X05 · Matplotlib: tùy chỉnh và xuất

Mã bài: X05 · Trung cấp · M4 · Nhập dữ liệu, SQL và tra cứu · 30 phút · 15 XP · thứ tự 25

MỤC TIÊU

Làm biểu đồ đọc được và xuất đúng figure ra PNG.

CẦN NHỚ TRƯỚC

X04

BỐI CẢNH

Viết hai hàm. (1) plot_marks(names, marks, pass_mark): bar chart điểm từng sinh viên; đường ngang nét đứt ax.axhline(pass_mark, linestyle="--", label="Pass"); trục y cố định 0..10; ax.legend(); tiêu đề "Marks", nhãn "Student" / "Score"; return fig. (2) export_png(fig, dpi): lưu CHÍNH fig (không phải "figure hiện hành") vào io.BytesIO dạng PNG với dpi cho trước, sau đó đóng fig bằng plt.close(fig), trả về bytes.

VÍ DỤ MINH HOẠ

plot_marks(["An", "Binh"], [8, 6], 5) → 2 cột 8 và 6, mốc nét đứt ở 5, legend "Pass". export_png của figure 4×2 inch với dpi 50 → bytes PNG 200×100 pixel, figure đã đóng.

ĐIỂM KIẾN THỨC

1. Chữ giúp đọc biểu đồ: ax.set(title=..., xlabel=..., ylabel=...) cho tên và đơn vị. Nét vẽ có label=... thì ax.legend() mới có mục để hiện. ax.axhline(y, linestyle="--") vẽ một đường mốc ngang (vd. điểm đạt) — nó cũng là một Line2D trong ax.lines.

2. Màu, alpha, marker, linewidth, linestyle là tham số của từng lệnh vẽ; ax.text/ax.annotate đặt chữ tại một toạ độ. ax.set_ylim(0, 10) cố định vùng nhìn: cột bắt đầu từ 0 nên chiều dài cột so được với nhau, và thang không nhảy theo dữ liệu.

3. fig.savefig(buf, ..., dpi=50) ghi CHÍNH figure đó. Kích thước pixel = figsize (inch) × dpi: 4×3 inch ở dpi 50 → 200×150. io.BytesIO là "file trong bộ nhớ": getvalue() trả bytes, mở đầu bằng chữ ký PNG; rộng/cao nằm ở byte 16–24.

4. Ba lệnh dọn dẹp khác nhau: ax.cla() xoá nội dung một Axes, fig.clf() xoá nội dung figure nhưng figure VẪN mở, plt.close(fig) đóng hẳn figure và giải phóng bộ nhớ. Luôn savefig TRƯỚC close (và trước plt.show() trên máy).

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt
from io import BytesIO

fig, ax = plt.subplots(figsize=(4, 3))
ax.bar(["Q1", "Q2"], [7, 4], color="steelblue")
ax.axhline(5, color="gray", linestyle="--", label="Target")
ax.set_ylim(0, 10)
ax.legend()

buf = BytesIO()
fig.savefig(buf, format="png", dpi=50)
png = buf.getvalue()
print(png[:4])
print(int.from_bytes(png[16:20], "big"), int.from_bytes(png[20:24], "big"))
plt.close(fig)
print(plt.fignum_exists(fig.number))
```

Kết quả dự kiến (theo JSON): plot_marks(["An", "Binh"], [8, 6], 5) → mốc nét đứt ở 5, legend Pass; export_png(figure 4×2, 50) → PNG 200×100, figure đã đóng

NGỘ NHẬN THƯỜNG GẶP: "plt.close(fig) rồi plt.savefig(...) vẫn lưu được biểu đồ vừa vẽ"

plt.savefig lưu figure HIỆN HÀNH. Khi fig đã đóng (hoặc sau plt.show() trên máy, lúc cửa sổ đã tắt) không còn figure đó; pyplot tạo một figure MỚI, trống, khổ mặc định 6.4×4.8 inch — file ra là ảnh trắng, sai kích thước. Dùng fig.savefig(...) trước, plt.close(fig) sau.

CÂU DỰ ĐOÁN

Lưu figure 4×3 inch với dpi=50 ra PNG thì ảnh có kích thước bao nhiêu pixel?

sai	400 × 300	Đó là dpi=100 — dpi quyết định mật độ điểm ảnh.
ĐÚNG	200 × 150	pixel = inch × dpi: 4 × 50 và 3 × 50.
sai	4 × 3	figsize tính bằng inch, không phải pixel.

TỰ KIỂM TRA

1. fig.clf() khác plt.close(fig) thế nào?

• close chỉ xoá Axes đầu tiên — Xoá một Axes là việc của ax.cla().

✔ clf xoá nội dung nhưng figure vẫn mở; close đóng hẳn figure — Chạy hàng loạt mà chỉ clf thì figure vẫn chiếm bộ nhớ.

• Giống nhau — clf giữ figure mở; close mới đóng.

Đáp án: Chạy hàng loạt mà chỉ clf thì figure vẫn chiếm bộ nhớ.

2. ax.axhline(5, label="Pass") nhưng không gọi ax.legend() thì…

• Legend tự hiện — Matplotlib không tự thêm legend.

✔ Đường mốc vẫn được vẽ, nhưng không có hộp chú giải — label chỉ hiện khi legend() được gọi.

• Báo lỗi — Không có lỗi nào.

Đáp án: label chỉ hiện khi legend() được gọi.

3. Vì sao cố định ax.set_ylim(0, 10) cho bảng điểm thang 10?

• Để ảnh nét hơn — Độ nét do dpi quyết định, không phải ylim.

✔ Cột bắt đầu từ 0 và thang không đổi theo dữ liệu, nên chiều dài cột so được — Tự co trục có thể cắt đáy cột, phóng đại chênh lệch.

• Bắt buộc, nếu không bar báo lỗi — Không bắt buộc; đây là lựa chọn để đọc đúng.

Đáp án: Tự co trục có thể cắt đáy cột, phóng đại chênh lệch.

TỰ GIẢI THÍCH

Vì sao export_png dùng fig.savefig(...) rồi mới plt.close(fig), không đảo thứ tự hay dùng plt.savefig?

Mẫu trả lời: fig.savefig ghi đúng figure được truyền vào, không phụ thuộc "figure hiện hành" — nơi gọi có thể vừa tạo figure khác. Đóng trước thì plt.savefig không còn figure đó: pyplot tạo figure mới trống, ảnh ra trắng và sai kích thước. Lưu xong mới close để giải phóng bộ nhớ; clf() không đủ vì figure vẫn mở.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	ax.bar(names, marks) → mỗi sinh viên một cột
Bước 2	ax.axhline(pass_mark, linestyle="--") → đường mốc điểm đạt
Bước 3	ax.set_ylim(0, 10) → cột bắt đầu từ 0, thang cố định
Bước 4	fig.savefig(buf, dpi=dpi) → bytes PNG của đúng fig
Bước 5	plt.close(fig) → figure đóng, giải phóng bộ nhớ

CÂU PHẢN VÍ DỤ

Figure 5×2 inch, export_png(fig, 40) → PNG bao nhiêu pixel?

Lựa chọn: 500 × 200 | 200 × 80 | 5 × 2 · Đáp án: 200 × 80

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt
from io import BytesIO


def plot_marks(names, marks, pass_mark):
    fig, ax = plt.subplots()
    ax.bar(names, marks)
    ax.axhline(pass_mark, color="gray", linestyle="--", label="Pass")
    ax.set_ylim(0, 10)
    ax.legend()
    ax.set(title="Marks", xlabel="Student", ylabel="Score")
    return fig


def export_png(fig, dpi):
    buf = {{slot1}}
    {{slot2}}
    {{slot3}}
    return buf.getvalue()
```

Slot	Mảnh đúng
[Slot 1]	`BytesIO()`
[Slot 2]	`fig.savefig(buf, format="png", dpi=dpi)`
[Slot 3]	`plt.close(fig)`

Mảnh bẫy	Vì sao sai
`open("chart.png", "wb")`	Để cần bytes trong bộ nhớ, không ghi file.
`plt.savefig(buf, format="png", dpi=dpi)`	Lưu figure HIỆN HÀNH — chưa chắc là fig được truyền vào.

`fig.clf()`	Xoá nội dung nhưng figure vẫn mở.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

plot_marks đã viết sẵn. Lắp export_png: tạo bộ đệm, lưu đúng figure với dpi, đóng figure.

Yêu cầu thêm: Thử 3 sinh viên với mốc 4; xuất figure 4×2 inch ở dpi 50 rồi dpi 100 và so kích thước pixel.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm. (1) plot_marks(names, marks, pass_mark): bar chart điểm từng sinh viên; đường ngang nét đứt ax.axhline(pass_mark, linestyle="--", label="Pass"); trục y cố định 0..10; ax.legend(); tiêu đề "Marks", nhãn "Student" / "Score"; return fig. (2) export_png(fig, dpi): lưu CHÍNH fig (không phải "figure hiện hành") vào io.BytesIO dạng PNG với dpi cho trước, sau đó đóng fig bằng plt.close(fig), trả về bytes.

Thực hành: plot_marks: bar → axhline có label → set_ylim(0, 10) → legend → nhãn → return fig. export_png: BytesIO → fig.savefig(..., dpi=dpi) → plt.close(fig) → getvalue().

Gọi	Kết quả mong đợi
`plot_marks(["An", "Binh"], [8, 6], 5)`	`2 cột 8, 6; mốc nét đứt ở 5; legend Pass; trục y 0..10`
`export_png(plt.figure(figsize=(4, 2)), 50)`	`bytes PNG 200×100 pixel; figure đã đóng`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_marks(names, marks, pass_mark)`	Vẽ điểm từng sinh viên kèm mốc điểm đạt. · names: list tên; marks: list điểm 0..10; pass_mark: số · Figure 1 Axes · Bar + axhline nét đứt label "Pass"; ylim 0..10; legend; tiêu đề Marks.
`export_png(fig, dpi)`	Xuất một figure ra PNG trong bộ nhớ. · fig: Figure bất kỳ; dpi: số điểm ảnh mỗi inch · bytes PNG (rộng = inch × dpi) · Đóng fig sau khi lưu (plt.fignum_exists(fig.number) → False).

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Mỗi sinh viên một cột đúng điểm	`[float(v) for v in (p.get_height() for p in plot_marks(["An", "Binh", "Chi"], [9, 3, 6], 4).axes[0].patches)]`	`[9.0, 3.0, 6.0]`
Đường mốc nét đứt ở pass_mark	`[(l.get_linestyle(), [float(v) for v in l.get_ydata()]) for l in plot_marks(["An", "Binh", "Chi"], [9, 3, 6], 4).axes[0].lines]`	`[('--', [4.0, 4.0])]`
Trục y cố định 0..10	`tuple(float(v) for v in plot_marks(["An", "Binh", "Chi"], [9, 3, 6], 4).axes[0].get_ylim())`	`(0.0, 10.0)`
Legend có mục "Pass"	`[t.get_text() for t in plot_marks(["An", "Binh", "Chi"], [9, 3, 6], 4).axes[0].get_legend().get_texts()]`	`['Pass']`
Tiêu đề và nhãn trục	`(lambda ax: (ax.get_title(), ax.get_xlabel(), ax.get_ylabel()))(plot_marks(["An", "Binh", "Chi"], [9, 3, 6], 4).axes[0])`	`('Marks', 'Student', 'Score')`
export_png trả bytes PNG	`export_png(plt.figure(figsize=(4, 2)), 50)[:8]`	`b'\x89PNG\r\n\x1a\n'`
Lưu đúng fig được truyền vào, không phải figure hiện hành	`(lambda b: (int.from_bytes(b[16:20], "big"), int.from_bytes(b[20:24], "big")))((lambda f, other: export_png(f, 50))(plt.figure(figsize=(4, 2)), plt.figure(figsize=(3, 3))))`	`(200, 100)`
dpi quyết định số pixel	`(lambda b: (int.from_bytes(b[16:20], "big"), int.from_bytes(b[20:24], "big")))(export_png(plt.figure(figsize=(2, 1)), 100))`	`(200, 100)`
Figure đã đóng sau khi xuất	`(lambda f: (export_png(f, 30), plt.fignum_exists(f.number))[1])(plt.figure())`	`False`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. ax.bar(names, marks) → mỗi sinh viên một cột
Khung 2 — Bước 2. ax.axhline(pass_mark, linestyle="--") → đường mốc điểm đạt
Khung 3 — Bước 3. ax.set_ylim(0, 10) → cột bắt đầu từ 0, thang cố định
Khung 4 — Bước 4. fig.savefig(buf, dpi=dpi) → bytes PNG của đúng fig
Khung 5 — Bước 5. plt.close(fig) → figure đóng, giải phóng bộ nhớ
Khung đối chiếu — ngộ nhận "plt.close(fig) rồi plt.savefig(...) vẫn lưu được biểu đồ vừa vẽ". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. plt.savefig lưu figure HIỆN HÀNH. Khi fig đã đóng (hoặc sau plt.show() trên máy, lúc cửa sổ đã tắt) không còn figure đó; pyplot tạo một figure MỚI, trống, khổ mặc định 6.4×4.8 inch — file ra là ảnh trắng, sai kích thước. Dùng fig.savefig(...) trước, plt.close(fig) sau.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: open("chart.png", "wb"), plt.savefig(buf, format="png", dpi=dpi), fig.clf()
Khung kết quả — 2 cặp vào/ra. plot_marks(["An", "Binh"], [8, 6], 5) → 2 cột 8, 6; mốc nét đứt ở 5; legend Pass; trục y 0..10; export_png(plt.figure(figsize=(4, 2)), 50) → bytes PNG 200×100 pixel; figure đã đóng

Nguồn: pe-v4 · python-learning-data.json#X05 · parser pe-v4/build · file build 025-X05.json

M5 · NumPy

6 bài: N01, N02, N03, N04, N05, N06

Ứng viên animation — ưu tiên cao (★★): N01, N02, N04, N05, N06 · ưu tiên (★): N03

N01 · Array, dtype, shape và axis

Mã bài: N01 · Trung cấp · M5 · NumPy · 30 phút · 15 XP · thứ tự 26

MỤC TIÊU

Tạo array 2 chiều và tổng hợp đúng axis.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

Viết make_grid(n, n_cols): array các số 1..n xếp thành hàng, mỗi hàng n_cols cột (n không chia hết cho n_cols thì để reshape báo ValueError). Viết describe(a) — a là array hoặc list lồng nhau 2 chiều — trả dict: shape, ndim, size, col_sums (tổng TỪNG CỘT), row_sums (tổng từng hàng) và total (tổng tất cả).

VÍ DỤ MINH HOẠ

make_grid(12, 4) → shape (3, 4); describe(...) → col_sums [15, 18, 21, 24], row_sums [10, 26, 42], total 78.

ĐIỂM KIẾN THỨC

1. np.array([[...], [...]]) tạo ndarray 2 chiều. Mọi phần tử chung MỘT dtype: toàn số nguyên thì là int64 trên máy 64-bit (trong trình duyệt có thể là int32 — tuỳ nền tảng); trộn 1 với 2.5 thì cả array thành float64.
2. shape là kích thước từng chiều, ví dụ (2, 3) = 2 hàng × 3 cột; ndim là số chiều (2); size là tổng số phần tử (6). len(a) chỉ là độ dài chiều đầu — số hàng.
3. np.arange(1, 13) cho 1..12 — điểm dừng KHÔNG nằm trong kết quả. reshape(3, 4) xếp lại 12 phần tử thành 3 hàng × 4 cột; viết -1 để NumPy tự tính một chiều; số phần tử không khớp → ValueError. linspace, zeros, ones tạo nhanh dữ liệu thử.
4. axis là chiều bị GỘP: sum(axis=0) gộp các hàng → còn một giá trị cho mỗi cột; sum(axis=1) gộp các cột → còn một giá trị cho mỗi hàng. Không ghi axis thì gộp hết thành một số.
5. Trước khi tính, viết ra shape mong muốn và đơn vị của mỗi cột: tổng theo cột của array (3, 4) phải có shape (4,).

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np

temps = np.array([[30, 32, 31], [25, 27, 26]])   # 2 thành phố × 3 ngày
print(temps.shape, temps.ndim, temps.size, len(temps))
print(temps.sum(axis=0))   # gộp các HÀNG → còn một số cho mỗi ngày (cột)
print(temps.sum(axis=1))   # gộp các CỘT → còn một số cho mỗi thành phố (hàng)
print(temps.sum())

print(np.array([1, 2, 3.5]).dtype)
print(np.linspace(0, 1, 5))
print(np.zeros((2, 3)).shape, np.ones(2))
print(np.arange(6).reshape(3, -1))
```

Kết quả dự kiến (theo JSON): make_grid(12, 4) → col_sums [15, 18, 21, 24]; make_grid(10, 4) → ValueError

NGỘ NHẬN THƯỜNG GẶP: "axis=0 nghĩa là tính theo hàng nên được một số cho mỗi hàng"

axis là chiều bị GỘP mất, không phải chiều được giữ lại. Với array (3, 4), axis=0 gộp 3 hàng → kết quả shape (4,) — một số cho mỗi cột. Ma trận vuông còn nguy hiểm hơn: nhầm axis vẫn ra đúng shape nên không có lỗi nào báo, chỉ có số sai.

CÂU DỰ ĐOÁN

a = np.arange(1, 13).reshape(3, 4). a.sum(axis=0) có bao nhiêu phần tử?

sai	1 — tổng toàn bộ là 78	Đó là a.sum() không có axis.
sai	3 — một tổng cho mỗi hàng	Đó là axis=1.
ĐÚNG	4 — gộp 3 hàng lại, còn một tổng cho mỗi cột	axis=0 là chiều hàng bị gộp mất.

TỰ KIỂM TRA

1. a.shape là (5, 3). len(a) và a.size bằng bao nhiêu?

- len(a) = 15, a.size = 15 — len chỉ là số hàng.
- ✔ len(a) = 5, a.size = 15 — len đo chiều đầu; size đếm mọi phần tử.
- len(a) = 3, a.size = 8 — size là TÍCH các chiều, không phải tổng.

Đáp án: len đo chiều đầu; size đếm mọi phần tử.

2. np.arange(10).reshape(4, 3) thì sao?

- Cắt bớt còn 9 phần tử — NumPy không tự cắt.
- ✔ ValueError — 10 phần tử không xếp được thành 4 × 3 = 12 — reshape giữ nguyên số phần tử.
- Tự thêm 2 số 0 cho đủ — NumPy không tự đệm.

Đáp án: reshape giữ nguyên số phần tử.

3. np.array([1, 2, 3.5]).dtype là gì?

- object — Chỉ khi trộn những kiểu không phải số.
- int và float lẫn nhau — Mỗi array chỉ có một dtype.
- ✔ float64 — một số thực kéo cả array thành số thực — Array có một dtype chung.

Đáp án: Array có một dtype chung.

TỰ GIẢI THÍCH

Giải thích axis=0 và axis=1 với array điểm có shape (học viên, môn).

Mẫu trả lời: Với array shape (học viên, môn), axis=0 gộp chiều học viên nên kết quả còn một giá trị cho mỗi môn (tổng hay trung bình theo môn); axis=1 gộp chiều môn nên còn một giá trị cho mỗi học viên. Kiểm lại bằng shape kết quả: (số môn,) hay (số học viên,).

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	np.arange(1, 13) → 12 số từ 1 đến 12
Bước 2	reshape(-1, 4) → shape (3, 4)
Bước 3	sum(axis=0) → [15, 18, 21, 24], một tổng mỗi cột
Bước 4	sum(axis=1) → [10, 26, 42], một tổng mỗi hàng
Bước 5	sum() → 78

CÂU PHẢN VÍ DỤ

describe([[1, 2], [3, 4]])["col_sums"] là gì?

Lựa chọn: [4, 6] | [3, 7] | 10 · Đáp án: [4, 6]

TEMPLATE LẮP MẢNH

```
import numpy as np

temps = np.arange(1, {{slot1}}).reshape(2, 3)
info = {
    "shape": temps.shape,
    "size": {{slot2}},
    "per_day": temps.sum({{slot3}}),
    "per_city": temps.sum({{slot4}}),
}
```

Slot	Mảnh đúng
[Slot 1]	`7`
[Slot 2]	`temps.size`
[Slot 3]	`axis=0`
[Slot 4]	`axis=1`

Mảnh bẫy	Vì sao sai
`axis=2`	Array 2 chiều chỉ có axis 0 và 1 — AxisError.
`len(temps)`	len chỉ đếm số hàng (2), không phải số phần tử (6).
`6`	arange dừng TRƯỚC điểm dừng: arange(1, 6) chỉ có 5 số, reshape(2, 3) sẽ lỗi.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: 2 thành phố × 3 ngày. Lắp lệnh tạo dữ liệu, số phần tử và tổng theo axis.

Yêu cầu thêm: Thử array 2 × 3 nhập tay ([[5, 0, 1], [2, 2, 2]]) và một ma trận vuông 2 × 2: viết shape kết quả mong muốn TRƯỚC khi chạy.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết make_grid(n, n_cols): array các số 1..n xếp thành hàng, mỗi hàng n_cols cột (n không chia hết cho n_cols thì để reshape báo ValueError). Viết describe(a) — a là array hoặc list lồng nhau 2 chiều — trả dict: shape, ndim, size, col_sums (tổng TỪNG CỘT), row_sums (tổng từng hàng) và total (tổng tất cả).

Thực hành: make_grid dùng np.arange(1, n + 1).reshape(-1, n_cols); describe đổi đầu vào bằng np.asarray rồi dùng shape, ndim, size, sum(axis=0), sum(axis=1), sum().

Gọi	Kết quả mong đợi
`make_grid(12, 4)`	`shape (3, 4); hàng cuối [9, 10, 11, 12]`
`describe(make_grid(12, 4))`	`col_sums [15, 18, 21, 24] · row_sums [10, 26, 42] · total 78`
`make_grid(10, 4)`	`ValueError`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`make_grid(n, n_cols)`	Xếp các số 1..n thành array 2 chiều, mỗi hàng n_cols cột. · n, n_cols: int dương · ndarray shape (n // n_cols, n_cols); n không chia hết → ValueError · Không in gì.
`describe(a)`	Mô tả array 2 chiều và tổng theo từng axis. · a: ndarray hoặc list lồng nhau, 2 chiều · dict shape, ndim, size, col_sums, row_sums, total · Không sửa a.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
make_grid(12, 4): shape (3, 4)	`make_grid(12, 4).shape`	`(3, 4)`
make_grid(12, 4): hàng cuối là 9..12	`[int(v) for v in make_grid(12, 4)[-1]]`	`[9, 10, 11, 12]`
make_grid(8, 2): 4 hàng, bắt đầu từ 1	`(make_grid(8, 2).shape, [int(v) for v in make_grid(8, 2)[0]])`	`((4, 2), [1, 2])`
make_grid(10, 4) → ValueError	`_studio.call(make_grid, 10, 4).error`	`'ValueError'`
Tổng từng cột (axis=0)	`[int(v) for v in describe(make_grid(12, 4))['col_sums']]`	`[15, 18, 21, 24]`
Tổng từng hàng (axis=1)	`[int(v) for v in describe(make_grid(12, 4))['row_sums']]`	`[10, 26, 42]`
shape, ndim, size, total	`(lambda d: (tuple(d['shape']), int(d['ndim']), int(d['size']), int(d['total'])))(describe(make_grid(12, 4)))`	`((3, 4), 2, 12, 78)`
List lồng nhau 2 × 3 (dữ liệu khác)	`(lambda d: ([int(v) for v in d['col_sums']], [int(v) for v in d['row_sums']], int(d['size'])))(describe([[5, 0, 1], [2, 2, 2]]))`	`([7, 2, 3], [6, 6], 6)`
Ma trận vuông 2 × 2: không nhầm axis	`(lambda d: ([int(v) for v in d['col_sums']], [int(v) for v in d['row_sums']]))(describe([[1, 2], [3, 4]]))`	`([4, 6], [3, 7])`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Số nguyên mặc định của numpy trong trình duyệt là int32 (trên máy 64-bit thường là int64) — bộ chấm không phụ thuộc dtype đó. Thư viện: numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. np.arange(1, 13) → 12 số từ 1 đến 12
Khung 2 — Bước 2. reshape(-1, 4) → shape (3, 4)
Khung 3 — Bước 3. sum(axis=0) → [15, 18, 21, 24], một tổng mỗi cột
Khung 4 — Bước 4. sum(axis=1) → [10, 26, 42], một tổng mỗi hàng
Khung 5 — Bước 5. sum() → 78
Khung đối chiếu — ngộ nhận "axis=0 nghĩa là tính theo hàng nên được một số cho mỗi hàng". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. axis là chiều bị GỘP mất, không phải chiều được giữ lại. Với array (3, 4), axis=0 gộp 3 hàng → kết quả shape (4,) — một số cho mỗi cột. Ma trận vuông còn nguy hiểm hơn: nhầm axis vẫn ra đúng shape nên không có lỗi nào báo, chỉ có số sai.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: axis=2, len(temps), 6
Khung kết quả — 3 cặp vào/ra. make_grid(12, 4) → shape (3, 4); hàng cuối [9, 10, 11, 12]; describe(make_grid(12, 4)) → col_sums [15, 18, 21, 24] · row_sums [10, 26, 42] · total 78; make_grid(10, 4) → ValueError

Nguồn: pe-v4 · python-learning-data.json#N01 · parser pe-v4/build · file build 026-N01.json

N02 · Indexing, mask và vectorization

Mã bài: N02 · Trung cấp · M5 · NumPy · 30 phút · 15 XP · thứ tự 27

MỤC TIÊU

Lọc bằng mask nhiều điều kiện, tính đồng loạt.

CẦN NHỚ TRƯỚC

N01

BỐI CẢNH

Viết curve(scores, bonus): chỉ giữ các điểm hợp lệ trong đoạn 0..10 (bỏ điểm âm và điểm lớn hơn 10), cộng bonus cho từng điểm còn lại rồi chặn tối đa ở 10; trả array số thực. Viết grade_labels(scores, cutoff): trả array nhãn "pass" (điểm ≥ cutoff) hoặc "fail" cho TỪNG điểm, dùng np.where.

VÍ DỤ MINH HOẠ

curve([3, 5, 8, 12], 0.5) → [3.5, 5.5, 8.5]; grade_labels([4, 6, 9], 5) → ["fail", "pass", "pass"].

ĐIỂM KIẾN THỨC

1. a[i, j] lấy một ô; a[:, 1] lấy cả cột thứ hai; a[0:2] lấy hai hàng đầu. Slice dùng start:stop như list.
2. So sánh trên array cho ra array bool cùng shape (mask). a[mask] giữ các phần tử ứng với True — kết quả là array 1 chiều MỚI.
3. Ghép điều kiện dùng & (và), | (hoặc), ~ (phủ định), mỗi điều kiện đặt trong NGOẶC: (a >= 0) & (a <= 10). and/or đòi một giá trị True/False duy nhất nên báo ValueError với array; thiếu ngoặc thì & được tính trước >=.
4. Phép toán trên array chạy element-wise cho mọi phần tử (vectorization): a + 0.5, a * 1.8 — không cần vòng for. Nhưng đừng nhồi mọi thứ vào một dòng khó đọc: đặt tên bước trung gian như valid để pipeline rõ ràng.
5. np.where(điều kiện, x, y) chọn x hoặc y cho TỪNG phần tử; np.clip(a, lo, hi) kéo giá trị ngoài biên về biên — nó không loại phần tử nào.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np

temps = np.array([[28, 35], [31, 40], [22, 30]])
print(temps[1, 0], temps[:, 1])          # một ô; cả cột thứ hai
hot = (temps >= 30) & (temps < 40)
print(hot)
print(temps[hot])                        # chỉ các ô thỏa CẢ HAI điều kiện
print(temps * 1.8 + 32)                  # °C → °F cho mọi ô, không vòng lặp
print(np.where(temps > 32, "nóng", "ổn"))
print(np.clip(temps, 25, 35))            # kéo về biên, không bỏ ô nào
```

Kết quả dự kiến (theo JSON): [3, 5, 8, 12], 0.5 → [3.5, 5.5, 8.5]; [9.8, 7], 0.5 → [10.0, 7.5]

NGỘ NHẬN THƯỜNG GẶP: "np.clip loại bỏ điểm sai"

clip không bỏ phần tử nào: 12 bị kéo về 10 và vẫn nằm trong kết quả. Muốn LOẠI điểm ngoài 0..10 thì lọc bằng mask; clip chỉ dùng để chặn giá trị sau khi cộng thưởng.

CÂU DỰ ĐOÁN

a = np.array([4, 6, 8, 11]). Dòng dưới cho gì?

```
a[(a >= 5) and (a <= 10)]
```

sai	array([6, 8])	Đúng nếu dùng & thay cho and.
ĐÚNG	ValueError: truth value of an array ... is ambiguous	and cần MỘT bool; array có nhiều giá trị.
sai	array([4, 6, 8, 11])	Không phép lọc nào chạy được.

TỰ KIỂM TRA

1. a = np.array([[6, 8], [7, 9]]). a[:, 1] là gì?
   - [7, 9] — hàng thứ hai — Hàng thứ hai là a[1].
   - ✔ [8, 9] — cột thứ hai của mọi hàng — : là mọi hàng, 1 là cột có chỉ số 1.
   - 8 — Một ô cần hai chỉ số cụ thể, ví dụ a[0, 1].

   Đáp án: : là mọi hàng, 1 là cột có chỉ số 1.
2. np.where(np.array([120, 80]) > 100, "high", "low") trả gì?
   - ✔ array(["high", "low"]) — chọn cho từng phần tử — where với 3 đối số là "if/else theo từng phần tử".
   - "high" — Không gộp thành một giá trị.
   - (array([0]),) — Đó là np.where với MỘT đối số: trả vị trí các phần tử True.

Đáp án: where với 3 đối số là "if/else theo từng phần tử".

3. a[a > 5] = 0 làm gì?

✔ Gán 0 cho mọi phần tử > 5 ngay trong a — Mask ở bên trái dấu = sửa array gốc.

• Lỗi vì mask không gán được — Gán qua mask là cách làm phổ biến.

• Tạo array mới chỉ gồm số 0 — Gán qua mask sửa tại chỗ.

Đáp án: Mask ở bên trái dấu = sửa array gốc.

TỰ GIẢI THÍCH

Vì sao viết (a >= 0) & (a <= 10) mà không viết a >= 0 and a <= 10?

Mẫu trả lời: and/or của Python cần MỘT giá trị True/False, còn a >= 0 là cả array bool nên Python báo ValueError (ambiguous). & so từng cặp phần tử và trả mask mới. Ngoặc bắt buộc vì & được tính trước >= và <=; thiếu ngoặc thì biểu thức thành a >= (0 & a) <= 10, sai nghĩa hoặc lỗi.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	a = [3, 5, 8, 12]; a >= 0 → [True, True, True, True]
Bước 2	a <= 10 → [True, True, True, False]
Bước 3	(a >= 0) & (a <= 10) → giữ [3, 5, 8]
Bước 4	valid + 0.5 → [3.5, 5.5, 8.5]
Bước 5	np.clip(..., 0, 10) → chặn trên ở 10

CÂU PHẢN VÍ DỤ

curve([9.8, 12], 0.5) → ?

Lựa chọn: [10.0] | [10.3] | [10.0, 10.0] · Đáp án: [10.0]

TEMPLATE LẮP MẢNH

```
import numpy as np

temps = np.array([18.0, 25.0, 33.0, 41.0])
ok = temps[{{slot1}}]
fahrenheit = {{slot2}}
labels = np.where({{slot3}}, "hot", "mild")
```

Slot	Mảnh đúng
[Slot 1]	`(temps >= 20) & (temps <= 35)`
[Slot 2]	`ok * 1.8 + 32`
[Slot 3]	`ok > 30`

Mảnh bẫy	Vì sao sai
`np.clip(temps, 20, 35)`	clip kéo về biên chứ không lọc; chỉ số phải là mask bool.
`temps >= 20 & temps <= 35`	Thiếu ngoặc: & được tính trước >=.
`(temps >= 20) and (temps <= 35)`	and với array → ValueError.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: nhiệt độ. Giữ các ngày từ 20 đến 35 độ, đổi sang °F, gắn nhãn.

Yêu cầu thêm: Thử điểm âm, điểm đúng bằng 10 và bonus đẩy điểm vượt 10: lọc trước, cộng sau, chặn cuối.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết curve(scores, bonus): chỉ giữ các điểm hợp lệ trong đoạn 0..10 (bỏ điểm âm và điểm lớn hơn 10), cộng bonus cho từng điểm còn lại rồi chặn tối đa ở 10; trả array số thực. Viết grade_labels(scores, cutoff): trả array nhãn "pass" (điểm ≥ cutoff) hoặc "fail" cho TỪNG điểm, dùng np.where.

Thực hành: Tạo mask bằng & với hai điều kiện trong ngoặc, lọc, cộng bonus, rồi np.clip(..., 0, 10). grade_labels dùng np.where.

Gọi	Kết quả mong đợi
`curve([3, 5, 8, 12], 0.5)`	`[3.5, 5.5, 8.5]`
`curve([9.8, 7], 0.5)`	`[10.0, 7.5]`
`grade_labels([4, 6, 9], 5)`	`["fail", "pass", "pass"]`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng

`curve(scores, bonus)`	Bỏ điểm ngoài 0..10, cộng thưởng rồi chặn trên ở 10. · scores: list/array số; bonus: số · ndarray float các điểm hợp lệ sau khi cộng (có thể rỗng) · Không sửa scores.
`grade_labels(scores, cutoff)`	Gắn nhãn đạt/trượt cho từng điểm. · scores: list/array số; cutoff: số · ndarray chuỗi "pass" (≥ cutoff) hoặc "fail", cùng độ dài · Không in gì.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: [3, 5, 8, 12] + 0.5	`[round(float(v), 4) for v in curve([3, 5, 8, 12], 0.5)]`	`[3.5, 5.5, 8.5]`
Bỏ điểm âm, 10 + 1 chặn về 10	`[round(float(v), 4) for v in curve([-1, 4, 10], 1)]`	`[5.0, 10.0]`
Chặn ở 10 sau khi cộng	`[round(float(v), 4) for v in curve([9.8, 7], 0.5)]`	`[10.0, 7.5]`
Giữ đúng biên 0 và 10	`[round(float(v), 4) for v in curve([0, 10, 10.5], 0)]`	`[0.0, 10.0]`
Không còn điểm hợp lệ → rỗng	`len(curve([11, -2], 1))`	`0`
Không sửa array đầu vào	`(lambda s: (curve(s, 1), s.tolist())[1])(np.array([1.0, 2.0, 12.0]))`	`[1.0, 2.0, 12.0]`
Nhãn cho từng điểm	`[str(v) for v in grade_labels([4, 6, 9], 5)]`	`['fail', 'pass', 'pass']`
Đúng ngưỡng là pass (dữ liệu khác)	`[str(v) for v in grade_labels([7, 6.99, 10], 7)]`	`['pass', 'fail', 'pass']`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Thư viện: numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. a = [3, 5, 8, 12]; a >= 0 → [True, True, True, True]
Khung 2 — Bước 2. a <= 10 → [True, True, True, False]
Khung 3 — Bước 3. (a >= 0) & (a <= 10) → giữ [3, 5, 8]
Khung 4 — Bước 4. valid + 0.5 → [3.5, 5.5, 8.5]
Khung 5 — Bước 5. np.clip(..., 0, 10) → chặn trên ở 10
Khung đối chiếu — ngộ nhận "np.clip loại bỏ điểm sai". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. clip không bỏ phần tử nào: 12 bị kéo về 10 và vẫn nằm trong kết quả. Muốn LOẠI điểm ngoài 0..10 thì lọc bằng mask; clip chỉ dùng để chặn giá trị sau khi cộng thưởng.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: np.clip(temps, 20, 35), temps >= 20 & temps <= 35, (temps >= 20) and (temps <= 35)
Khung kết quả — 3 cặp vào/ra. curve([3, 5, 8, 12], 0.5) → [3.5, 5.5, 8.5]; curve([9.8, 7], 0.5) → [10.0, 7.5]; grade_labels([4, 6, 9], 5) → ["fail", "pass", "pass"]

Nguồn: pe-v4 · python-learning-data.json#N02 · parser pe-v4/build · file build 027-N02.json

N03 · Broadcasting và chuẩn hoá dữ liệu

Mã bài: N03 · Trung cấp · M5 · NumPy · 35 phút · 15 XP · thứ tự 28

MỤC TIÊU

Trừ mean và z-score theo cột bằng broadcasting.

CẦN NHỚ TRƯỚC

N02

BỐI CẢNH

Viết center_columns(a): trừ mean của TỪNG CỘT khỏi cột đó (dùng mean(axis=0, keepdims=True)) và trả array số thực cùng shape. Viết zscore_columns(a): (a − mean cột) / std cột; cột có std = 0 (mọi giá trị bằng nhau) phải ra toàn 0 thay vì nan. Không sửa array đầu vào.

VÍ DỤ MINH HOẠ

center_columns([[2, 20], [4, 40]]) → [[-1, -10], [1, 10]]; zscore_columns([[1, 5], [3, 5]]) → [[-1, 0], [1, 0]].

ĐIỂM KIẾN THỨC

1. Broadcasting: NumPy so shape từ PHẢI sang TRÁI; hai chiều hợp nhau khi bằng nhau, hoặc một bên bằng 1 (hoặc không có). (3, 2) * (2,) → (3, 2): vector giá được "kéo" cho từng hàng.
2. a.mean(axis=0) có shape (2,); thêm keepdims=True thì shape (1, 2) — viết rõ "một hàng mean" nên a − mean trừ đúng từng cột. Với phép theo hàng (axis=1) thì bắt buộc keepdims: (3, 1) mới trừ/chia được cho (3, 2).
3. Shape không hợp, như (3, 2) − (3,), báo ValueError: operands could not be broadcast. Nguy hiểm hơn là ma trận vuông: nhầm axis vẫn broadcast được nên không có lỗi, chỉ có số sai.
4. z-score = (x − mean) / std. Cột hằng có std = 0 → chia cho 0 ra nan; thay std bằng 1 ở những cột đó (np.where(std == 0, 1, std)) để kết quả là 0. Khi chuẩn hoá dữ liệu mới, dùng lại mean/std của tập tham chiếu.
5. Broadcast không có nghĩa là không tốn bộ nhớ: kết quả shape (10000, 10000) vẫn được cấp phát đầy đủ.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np

qty = np.array([[2, 1], [0, 3], [4, 2]])   # 3 đơn hàng × 2 sản phẩm
price = np.array([15.0, 40.0])             # giá từng sản phẩm, shape (2,)
revenue = qty * price                      # (3, 2) * (2,) → giá áp cho mỗi hàng
print(revenue)

order_total = revenue.sum(axis=1, keepdims=True)
print(order_total.shape)                   # (3, 1)
print(revenue / order_total)               # tỉ trọng từng sản phẩm trong đơn

try:
    revenue - revenue.sum(axis=1)          # (3, 2) - (3,) → không khớp
except ValueError as error:
    print("ValueError:", error)
```

Kết quả dự kiến (theo JSON): [[2, 20], [4, 40]] → [[-1, -10], [1, 10]]; [[1, 5], [3, 5]] → [[-1, 0], [1, 0]]

NGỘ NHẬN THƯỜNG GẶP: "Không báo lỗi broadcast nghĩa là đã trừ đúng chiều"

Với ma trận vuông (2, 2), a − a.mean(axis=1) vẫn chạy: mean theo hàng shape (2,) bị kéo theo CỘT, cho số sai mà không lỗi. Viết shape mong muốn trước khi tính và dùng keepdims=True để chiều bị gộp vẫn hiện ra — (1, n) là mean cột, (m, 1) là mean hàng.

CÂU DỰ ĐOÁN

a có shape (3, 2). a − a.mean(axis=1) chạy thế nào?

ĐÚNG	ValueError — mean theo hàng có shape (3,), không khớp chiều cuối 2	So từ phải: 2 với 3 không bằng nhau và không có 1.
sai	Trừ mean của từng cột	mean(axis=1) là mean theo hàng.
sai	Trừ mean của từng hàng	Muốn vậy phải keepdims=True để có shape (3, 1).

TỰ KIỂM TRA

1. Shape nào broadcast được với (4, 3)?

- (3, 4) — Cả hai chiều đều lệch.
- ✔ (1, 3) — chiều 1 được kéo thành 4 — Từ phải: 3 = 3, còn 1 kéo được.
- (4,) — Từ phải: 3 với 4 không khớp → ValueError.

Đáp án: Từ phải: 3 = 3, còn 1 kéo được.

2. Chuẩn hoá z-score cột [5, 5, 5] mà không xử lý gì thì ra gì?

✔ nan — std = 0 nên thành 0/0 — Cần thay std = 0 bằng 1 để ra 0.

• 5 — Mean đã bị trừ đi.

• 0 — Chỉ khi đã xử lý std = 0.

Đáp án: Cần thay std = 0 bằng 1 để ra 0.

3. a.mean(axis=0, keepdims=True) với a shape (3, 2) có shape gì?

✔ (1, 2) — Chiều bị gộp vẫn còn, với độ dài 1.

• (3, 1) — Đó là mean theo hàng với keepdims.

• (2,) — Đó là khi không có keepdims.

Đáp án: Chiều bị gộp vẫn còn, với độ dài 1.

TỰ GIẢI THÍCH

Vì sao cần keepdims=True và xử lý std = 0 khi chuẩn hoá theo cột?

Mẫu trả lời: keepdims=True giữ chiều bị gộp với độ dài 1 (mean cột có shape (1, n)), nên phép trừ broadcast rõ ràng theo cột và không vô tình khớp nhầm chiều khi ma trận vuông. Cột hằng có std = 0, chia sẽ ra nan; thay std bằng 1 ở cột đó để z-score bằng 0, đúng ý nghĩa "không lệch khỏi mean".

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	mean(axis=0, keepdims=True) → [[3., 30.]], shape (1, 2)
Bước 2	a − mean → [[-1., -10.], [1., 10.]]
Bước 3	[[1, 5], [3, 5]].std(axis=0) → [1., 0.]
Bước 4	np.where(std == 0, 1, std) → cột hằng chia cho 1, ra 0

CÂU PHẢN VÍ DỤ

zscore_columns([[0, 7], [10, 7]]) → ?

Lựa chọn: [[-5, 0], [5, 0]] | [[-1, 0], [1, 0]] | [[-1, nan], [1, nan]] · Đáp án: [[-1, 0], [1, 0]]

TEMPLATE LẮP MẢNH

```
import numpy as np

marks = np.array([[6.0, 8.0, 10.0], [5.0, 5.0, 5.0]])
row_mean = marks.mean({{slot1}})
row_std = marks.std(axis=1, keepdims=True)
safe = {{slot2}}
z = {{slot3}}
```

Slot	Mảnh đúng
[Slot 1]	`axis=1, keepdims=True`
[Slot 2]	`np.where(row_std == 0, 1, row_std)`
[Slot 3]	`(marks - row_mean) / safe`

Mảnh bẫy	Vì sao sai
`axis=0, keepdims=True`	Đó là mean từng cột (từng môn), không phải từng học viên.
`axis=1`	Shape (2,) không trừ được cho (2, 3) — cần (2, 1).
`(marks - row_mean) / row_std`	Hàng hằng có std = 0 → nan.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: chuẩn hoá theo HÀNG (mỗi hàng là một học viên, mỗi cột một môn). Lắp đúng chiều.

Yêu cầu thêm: Thử array 3 × 2 và 3 × 3 có một cột hằng: kiểm từng cột sau khi trừ có mean 0, cột hằng ra 0 chứ không nan.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết center_columns(a): trừ mean của TỪNG CỘT khỏi cột đó (dùng mean(axis=0, keepdims=True)) và trả array số thực cùng shape. Viết zscore_columns(a): (a − mean cột) / std cột; cột có std = 0 (mọi giá trị bằng nhau) phải ra toàn 0 thay vì nan. Không sửa array đầu vào.

Thực hành: Đổi đầu vào sang float, lấy mean/std với axis=0, keepdims=True, thay std = 0 bằng 1 trước khi chia.

Gọi	Kết quả mong đợi
`center_columns([[2, 20], [4, 40]])`	`[[-1, -10], [1, 10]]`
`zscore_columns([[1, 5], [3, 5]])`	`[[-1, 0], [1, 0]]` — cột hằng ra 0

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng

`center_columns(a)`	Trừ mean của từng cột khỏi cột đó. · a: array/list 2 chiều · ndarray float cùng shape; mỗi cột có mean 0 · Không sửa a.
`zscore_columns(a)`	Chuẩn hoá z-score theo cột: (x − mean) / std. · a: array/list 2 chiều · ndarray float cùng shape; cột hằng (std = 0) ra toàn 0 · Không sửa a.

BỘ CHẤM (7 ca)

Ca	Gọi	Mong đợi
Đề mẫu: trừ mean từng cột	`[[round(float(v), 4) for v in row] for row in center_columns([[2, 20], [4, 40]])]`	`[[-1.0, -10.0], [1.0, 10.0]]`
3 × 3: không nhầm axis	`[[round(float(v), 4) for v in row] for row in center_columns([[1, 10, 100], [3, 30, 300], [5, 50, 500]])]`	`[[-2.0, -20.0, -200.0], [0.0, 0.0, 0.0], [2.0, 20.0, 200.0]]`
3 × 2: giữ shape, mỗi cột có mean 0	`(lambda c: (tuple(c.shape), [round(float(v), 6) for v in c.mean(axis=0)]))(center_columns([[1, 2], [3, 4], [5, 9]]))`	`((3, 2), [0.0, 0.0])`
z-score có cột hằng → 0, không nan	`[[round(float(v), 4) for v in row] for row in zscore_columns([[1, 5], [3, 5]])]`	`[[-1.0, 0.0], [1.0, 0.0]]`
z-score dữ liệu khác (3 hàng)	`[[round(float(v), 4) for v in row] for row in zscore_columns([[2, 1], [4, 1], [6, 1]])]`	`[[-1.2247, 0.0], [0.0, 0.0], [1.2247, 0.0]]`
z-score không có cột hằng	`[[round(float(v), 4) for v in row] for row in zscore_columns([[0, 2], [10, 6]])]`	`[[-1.0, -1.0], [1.0, 1.0]]`
Không sửa array đầu vào	`(lambda x: (center_columns(x), x.tolist())[1])(np.array([[1.0, 2.0], [3.0, 6.0]]))`	`[[1.0, 2.0], [3.0, 6.0]]`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Thư viện: numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. mean(axis=0, keepdims=True) → [[3., 30.]], shape (1, 2)
Khung 2 — Bước 2. a − mean → [[-1., -10.], [1., 10.]]
Khung 3 — Bước 3. [[1, 5], [3, 5]].std(axis=0) → [1., 0.]
Khung 4 — Bước 4. np.where(std == 0, 1, std) → cột hằng chia cho 1, ra 0
Khung đối chiếu — ngộ nhận "Không báo lỗi broadcast nghĩa là đã trừ đúng chiều". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Với ma trận vuông (2, 2), a − a.mean(axis=1) vẫn chạy: mean theo hàng shape (2,) bị kéo theo CỘT, cho số sai mà không lỗi. Viết shape mong muốn trước khi tính và dùng keepdims=True để chiều bị gộp vẫn hiện ra — (1, n) là mean cột, (m, 1) là mean hàng.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: axis=0, keepdims=True, axis=1, (marks - row_mean) / row_std
Khung kết quả — 2 cặp vào/ra. center_columns([[2, 20], [4, 40]]) → [[-1, -10], [1, 10]]; zscore_columns([[1, 5], [3, 5]]) → [[-1, 0], [1, 0]] — cột hằng ra 0

Nguồn: pe-v4 · python-learning-data.json#N03 · parser pe-v4/build · file build 028-N03.json

N04 · Thống kê, NaN và trọng số

Mã bài: N04 · Trung cấp · M5 · NumPy · 35 phút · 15 XP · thứ tự 29

MỤC TIÊU

Thống kê bỏ NaN và GPA có trọng số.

CẦN NHỚ TRƯỚC

N03

BỐI CẢNH

Viết nan_summary(values) trả dict: valid (số quan sát hợp lệ — không phải NaN), mean (nanmean), median (nanmedian); KHÔNG thay NaN bằng 0. Viết weighted_gpa(marks, credits): điểm trung bình có trọng số theo số tín chỉ, bỏ các môn có điểm NaN (bỏ luôn tín chỉ của môn đó).

VÍ DỤ MINH HOẠ

nan_summary([4, nan, 8, 10]) → valid 3, mean 7.333..., median 8.0; weighted_gpa([7, 8], [2, 3]) → 7.6.

ĐIỂM KIẾN THỨC

1. NaN (np.nan) đánh dấu "thiếu" trong array số thực. Mọi phép tính dính NaN đều ra NaN: np.mean([7, nan, 9]) là nan.
2. NaN không bằng chính nó: x == np.nan luôn False. Kiểm bằng np.isnan(a) (hoặc np.isfinite(a) để loại cả inf); đếm ô thiếu bằng np.isnan(a).sum().
3. np.nanmean / np.nanmedian / np.nanstd bỏ NaN rồi mới tính. Thay NaN bằng 0 là BỊA dữ liệu: [4, 0, 8, 10] cho mean 5.5 thay vì 7.33.
4. Mean nhạy với ngoại lai; median và quantile mô tả vị trí theo cách khác: [1, 2, 3, 100] có mean 26.5 nhưng median 2.5.
5. np.average(a, weights=w) = Σ a·w / Σ w — GPA theo tín chỉ. std mặc định ddof=0 (chia n); ddof=1 cho độ lệch chuẩn mẫu (chia n − 1) — ghi rõ đang dùng định nghĩa nào. np.corrcoef đo tương quan, không chứng minh nhân quả.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np

wait = np.array([5.0, 7.0, np.nan, 6.0, 60.0])   # phút chờ: 1 ô thiếu, 1 ngoại lai
print(np.isnan(wait).sum(), (wait == np.nan).sum())
print(wait.mean(), np.nanmean(wait), np.nanmedian(wait))
print(round(np.nanstd(wait), 2), round(np.nanstd(wait, ddof=1), 2))
print(np.nanquantile(wait, [0.25, 0.5, 0.75]))
print(np.average([6.0, 9.0], weights=[1, 3]))
```

Kết quả dự kiến (theo JSON): [4, nan, 8, 10] → valid 3, mean 7.33; [7, 8], [2, 3] → 7.6

NGỘ NHẬN THƯỜNG GẶP: "Ô thiếu thì điền 0 cho dễ tính"

Điền 0 là thêm một quan sát GIẢ: [4, NaN, 8, 10] thành [4, 0, 8, 10], mean tụt từ 7.33 xuống 5.5 và số quan sát tăng từ 3 lên 4. Dùng nanmean/nanmedian để bỏ ô thiếu; nếu thật sự phải điền thì đó là một quyết định phân tích cần ghi rõ.

CÂU DỰ ĐOÁN

a = np.array([7.0, np.nan, 9.0]). (a == np.nan).sum() bằng mấy?

ĐÚNG	0 — NaN không bằng bất cứ thứ gì, kể cả chính nó	Phải dùng np.isnan(a).sum() → 1.
sai	1	Đó là kết quả của np.isnan(a).sum().
sai	Báo lỗi	So sánh chạy bình thường, chỉ là luôn False.

TỰ KIỂM TRA

1. Điểm [7, 8], tín chỉ [2, 3]. np.average(điểm, weights=tín chỉ) bằng bao nhiêu?
   - 38 — Đó là tổng có trọng số chưa chia Σ trọng số.
   - ✔ 7.6 = (7·2 + 8·3) / 5 — Trọng số lớn kéo kết quả về phía điểm đó.
   - 7.5 — Đó là mean thường, bỏ qua tín chỉ.

Đáp án: Trọng số lớn kéo kết quả về phía điểm đó.

2. a = [1, 2, 3, 100]. Thống kê nào mô tả "điểm giữa" ít bị 100 kéo lệch?
   - Mean = 26.5 — Mean bị ngoại lai kéo lên.
   - Max = 100 — Max chính là ngoại lai.

✔ Median = 2.5 — Median chỉ phụ thuộc thứ tự.

Đáp án: Median chỉ phụ thuộc thứ tự.

3. np.std(a) và np.std(a, ddof=1) khác nhau thế nào?

✔ ddof=0 chia n (tổng thể), ddof=1 chia n − 1 (mẫu) nên lớn hơn một chút — Ghi rõ định nghĩa đang dùng.

• Giống hệt nhau — Mẫu số khác nhau.

• ddof=1 bỏ qua NaN — Bỏ NaN là việc của np.nanstd.

Đáp án: Ghi rõ định nghĩa đang dùng.

TỰ GIẢI THÍCH

Vì sao không đếm ô thiếu bằng a == np.nan và không điền 0 vào chỗ thiếu?

Mẫu trả lời: NaN không bằng chính nó nên a == np.nan luôn False — đếm ra 0 dù có ô thiếu; phải dùng np.isnan. Điền 0 tạo ra quan sát giả: số quan sát tăng và mean bị kéo xuống. nanmean/nanmedian bỏ ô thiếu và tính trên dữ liệu thật.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	np.isfinite([4, nan, 8, 10]).sum() → 3 quan sát
Bước 2	np.nanmean → (4 + 8 + 10) / 3 = 7.33
Bước 3	np.nanmedian → 8.0
Bước 4	~np.isnan(marks) → bỏ môn thiếu điểm cùng tín chỉ của nó
Bước 5	np.average([7, 8], weights=[2, 3]) → 7.6

CÂU PHẢN VÍ DỤ

nan_summary([nan, 1, 2, nan, 6])["mean"] = ?

Lựa chọn: 1.8 | nan | 3.0 · Đáp án: 3.0

TEMPLATE LẮP MẢNH

```
import numpy as np

wait = np.array([5.0, np.nan, 6.0, 60.0])
missing = {{slot1}}
typical = {{slot2}}
marks = np.array([6.0, 9.0, np.nan])
credits = np.array([1, 3, 2])
ok = {{slot3}}
gpa = np.average(marks[ok], weights={{slot4}})
```

Slot	Mảnh đúng
[Slot 1]	`np.isnan(wait).sum()`
[Slot 2]	`np.nanmedian(wait)`
[Slot 3]	`~np.isnan(marks)`
[Slot 4]	`credits[ok]`

Mảnh bẫy	Vì sao sai
`np.median(wait)`	Có NaN → kết quả nan.
`(wait == np.nan).sum()`	NaN không bằng chính nó → luôn đếm ra 0.
`credits`	Thiếu lọc: 3 tín chỉ cho 2 điểm → lỗi lệch shape.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: thời gian chờ (phút) có ô thiếu, và GPA ba môn có một môn chưa có điểm.

Yêu cầu thêm: Thử dữ liệu có hai ô thiếu và dữ liệu không có ô thiếu nào; GPA có một môn thiếu điểm phải ra như khi xoá hẳn môn đó.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết nan_summary(values) trả dict: valid (số quan sát hợp lệ — không phải NaN), mean (nanmean), median (nanmedian); KHÔNG thay NaN bằng 0. Viết weighted_gpa(marks, credits): điểm trung bình có trọng số theo số tín chỉ, bỏ các môn có điểm NaN (bỏ luôn tín chỉ của môn đó).

Thực hành: Đổi về float, đếm bằng np.isfinite, tính bằng np.nanmean/np.nanmedian; GPA lọc mask ~np.isnan(marks) cho cả điểm lẫn tín chỉ.

Gọi	Kết quả mong đợi
`nan_summary([4, nan, 8, 10])`	`valid 3 · mean 7.333… · median 8.0`
`weighted_gpa([7, 8, nan], [2, 3, 4])`	`7.6 — môn thiếu điểm bị bỏ cùng tín chỉ`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`nan_summary(values)`	Đếm quan sát hợp lệ và tính mean/median bỏ qua NaN. · values: list/array số, có thể có nan · dict valid (int), mean (float), median (float) · Không sửa values; không điền 0 vào ô thiếu.
`weighted_gpa(marks, credits)`	Điểm trung bình có trọng số theo tín chỉ, bỏ môn thiếu điểm. · marks: điểm (có thể nan); credits: tín chỉ, cùng độ dài · float · Không sửa đầu vào.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: 3 quan sát hợp lệ	`nan_summary([4, float('nan'), 8, 10])['valid']`	`3`
Đề mẫu: mean bỏ NaN	`round(nan_summary([4, float('nan'), 8, 10])['mean'], 4)`	`7.3333`
Đề mẫu: median bỏ NaN	`round(nan_summary([4, float('nan'), 8, 10])['median'], 4)`	`8.0`
Dữ liệu khác: hai ô thiếu	`(lambda d: (d['valid'], round(d['mean'], 4), round(d['median'], 4)))(nan_summary([float('nan'), 1, 2, float('nan'), 6]))`	`(3, 3.0, 2.0)`
Không có ô thiếu (có ngoại lai)	`(lambda d: (d['valid'], round(d['mean'], 4), round(d['median'], 4)))(nan_summary([1, 2, 3, 100]))`	`(4, 26.5, 2.5)`
GPA có trọng số	`round(weighted_gpa([7, 8], [2, 3]), 4)`	`7.6`
GPA bỏ môn thiếu điểm cùng tín chỉ	`round(weighted_gpa([7, 8, float('nan')], [2, 3, 4]), 4)`	`7.6`
GPA dữ liệu khác	`round(weighted_gpa([10, 5, 6], [1, 1, 2]), 4)`	`6.75`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Thư viện: numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. np.isfinite([4, nan, 8, 10]).sum() → 3 quan sát
Khung 2 — Bước 2. np.nanmean → (4 + 8 + 10) / 3 = 7.33
Khung 3 — Bước 3. np.nanmedian → 8.0
Khung 4 — Bước 4. ~np.isnan(marks) → bỏ môn thiếu điểm cùng tín chỉ của nó
Khung 5 — Bước 5. np.average([7, 8], weights=[2, 3]) → 7.6
Khung đối chiếu — ngộ nhận "Ô thiếu thì điền 0 cho dễ tính". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Điền 0 là thêm một quan sát GIẢ: [4, NaN, 8, 10] thành [4, 0, 8, 10], mean tụt từ 7.33 xuống 5.5 và số quan sát tăng từ 3 lên 4. Dùng nanmean/nanmedian để bỏ ô thiếu; nếu thật sự phải điền thì đó là một quyết định phân tích cần ghi rõ.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: np.median(wait), (wait == np.nan).sum(), credits
Khung kết quả — 2 cặp vào/ra. nan_summary([4, nan, 8, 10]) → valid 3 · mean 7.333... · median 8.0; weighted_gpa([7, 8, nan], [2, 3, 4]) → 7.6 — môn thiếu điểm bị bỏ cùng tín chỉ

Nguồn: pe-v4 · python-learning-data.json#N04 · parser pe-v4/build · file build 029-N04.json

N05 · View, copy và sai số số thực

Mã bài: N05 · Trung cấp · M5 · NumPy · 30 phút · 15 XP · thứ tự 30

MỤC TIÊU

Tránh sửa nhầm array gốc qua view; so số thực bằng tolerance.

CẦN NHỚ TRƯỚC

N04

BỐI CẢNH

Viết scaled_head(a, k, factor): trả array MỚI gồm k phần tử đầu của a nhân với factor; array a của người gọi phải giữ nguyên, và kết quả không được dùng chung bộ nhớ với a. Viết totals_match(parts, total): True nếu tổng các phần tử của parts bằng total trong sai số cho phép (np.isclose), không so bằng ==.

VÍ DỤ MINH HOẠ

a = np.array([10, 20, 30]); scaled_head(a, 2, 2) → [20, 40] và a vẫn là [10, 20, 30]; totals_match([0.1, 0.2], 0.3) → True.

ĐIỂM KIẾN THỨC

1. Slice cơ bản (a[1:3], a[:, 0]) trả VIEW: một cửa sổ nhìn vào cùng bộ nhớ. Gán vào view (window[0] = 50, head *= 2) là sửa luôn array gốc.
2. Boolean mask và fancy indexing (a[a > 5], a[[0, 2]]) trả COPY. Muốn chắc chắn độc lập thì gọi .copy(); np.shares_memory(x, y) cho biết hai array có dùng chung bộ nhớ không.
3. Phép toán tạo kết quả mới (a[:2] * 2) cho array MỚI; còn *=, +=, gán vào chỉ số là sửa TẠI CHỖ và giữ nguyên dtype (head *= 1.5 trên array số nguyên báo UFuncTypeError — khi đó dùng a[:k] * 1.5). reshape cũng có thể trả view — đừng mặc định là nó đã sao chép.
4. astype(int) CẮT phần thập phân (2.7 → 2, −1.5 → −1), không làm tròn; muốn làm tròn thì np.round trước. dtype nhỏ hơn (float32, int8) tiết kiệm bộ nhớ nhưng có thể mất độ chính xác hoặc tràn số.
5. Số thực nhị phân không biểu diễn đúng 0.1: 0.1 + 0.2 = 0.30000000000000004. So bằng np.isclose / np.allclose (hoặc math.isclose) với tolerance phù hợp, không dùng ==.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np

scores = np.array([4, 5, 6, 7])
window = scores[1:3]          # slice → view, chung bộ nhớ
window[0] = 50
print(scores)

picked = scores[scores > 5]    # mask → copy
picked[0] = -1
print(scores, picked)

safe = scores[:2].copy()
safe[:] = 0
print(scores, np.shares_memory(scores, window), np.shares_memory(scores, safe))

print(np.array([2.7, -1.5]).astype(int))   # cắt phần thập phân, không làm tròn
print(0.1 + 0.2, 0.1 + 0.2 == 0.3, np.isclose(0.1 + 0.2, 0.3))
```

Kết quả dự kiến (theo JSON): [10, 20, 30], k = 2, × 2 → [20, 40]; a giữ nguyên; [0.1, 0.2] với 0.3 → True

NGỘ NHẬN THƯỜNG GẶP: "Cắt slice là đã có bản sao riêng, như list Python"

Với list Python, lst[:2] là bản sao; với NumPy, a[:2] là VIEW. Sửa tại chỗ trên view (b[0] = 9, b *= 2) làm array của người gọi đổi theo — lỗi khó thấy vì hàm vẫn trả đúng kết quả. Gọi .copy() khi cần độc lập.

CÂU DỰ ĐOÁN

Sau ba dòng dưới, a là gì?

```
a = np.array([1, 2, 3])
b = a[:2]
b[0] = 9
```

sai	[1, 2, 3]	Chỉ đúng nếu b = a[:2].copy().
sai	[9, 2]	a vẫn có 3 phần tử; b mới có 2.
ĐÚNG	[9, 2, 3] — b là view của a	Slice cơ bản dùng chung bộ nhớ với array gốc.

TỰ KIỂM TRA

1. Phép nào trả COPY chứ không phải view?

✔ a[a > 0] — Boolean mask luôn tạo array mới.

• a[:, 0] — Vẫn là slice cơ bản — view.

• a[1:4] — Slice cơ bản là view.

Đáp án: Boolean mask luôn tạo array mới.

2. np.array([1.9, 2.1]).astype(int) cho gì?

• [2, 2] — astype không làm tròn.

✔ [1, 2] — cắt phần thập phân — Muốn [2, 2] thì np.round trước rồi mới astype.

• Lỗi vì mất thông tin — NumPy cho phép, chỉ âm thầm cắt.

Đáp án: Muốn [2, 2] thì np.round trước rồi mới astype.

3. Kiểm 0.1 + 0.2 có "bằng" 0.3 không, cách nào đúng?

• 0.1 + 0.2 == 0.3 → True — Thực tế là False: tổng là 0.30000000000000004.

✔ np.isclose(0.1 + 0.2, 0.3) → True — Sai số làm tròn nằm trong tolerance.

• int(0.1 + 0.2) == int(0.3) → True rồi kết luận bằng nhau — Đúng là True, nhưng vì cả hai bị cắt về 0 — so như vậy vô nghĩa.

Đáp án: Sai số làm tròn nằm trong tolerance.

TỰ GIẢI THÍCH

Vì sao scaled_head phải copy trước khi nhân tại chỗ, và vì sao so số thực bằng np.isclose?

Mẫu trả lời: a[:k] là view dùng chung bộ nhớ với a; nhân tại chỗ (*=) trên view sẽ sửa array của người gọi. copy() tạo bộ nhớ riêng nên chỉ kết quả đổi. Số thực nhị phân có sai số làm tròn (0.1 + 0.2 = 0.30000000000000004) nên == có thể sai dù hai số "bằng nhau"; np.isclose so trong một sai số cho phép.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	a[:2] → view, chung bộ nhớ với a
Bước 2	a[:2].copy() → bộ nhớ riêng
Bước 3	head *= 2 → chỉ head đổi, a giữ [10, 20, 30]
Bước 4	np.sum([0.1, 0.2]) → 0.30000000000000004
Bước 5	np.isclose(tổng, 0.3) → True, còn == cho False

CÂU PHẢN VÍ DỤ

totals_match([1.1, 2.2], 3.3) → ?

Lựa chọn: False | Lỗi | True · Đáp án: True

TEMPLATE LẮP MẢNH

```
import numpy as np

prices = np.array([1.1, 2.2, 4.0, 9.9])
first = prices[:3]{{slot1}}
first *= 2
same = np.shares_memory(prices, first)
total_ok = {{slot2}}
```

Slot	Mảnh đúng
[Slot 1]	`.copy()`
[Slot 2]	`np.isclose(first.sum(), 14.6)`

Mảnh bẫy	Vì sao sai
`first.sum() == 14.6`	Tổng thực tế là 14.600000000000001 → == cho False.
`[:]`	prices[:3][:] vẫn là view của prices.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: nhân đôi giá 3 sản phẩm đầu mà không sửa bảng giá gốc, rồi kiểm tổng.

Yêu cầu thêm: Thử array số thực và k bằng cả độ dài array; sau mỗi lời gọi in lại array gốc để chắc nó không đổi.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết scaled_head(a, k, factor): trả array MỚI gồm k phần tử đầu của a nhân với factor; array a của người gọi phải giữ nguyên, và kết quả không được dùng chung bộ nhớ với a. Viết totals_match(parts, total): True nếu tổng các phần tử của parts bằng total trong sai số cho phép (np.isclose), không so bằng ==.

Thực hành: Lấy a[:k].copy() rồi mới nhân (hoặc a[:k] * factor — phép * tạo array mới); tổng dùng np.sum và so bằng np.isclose.

Gọi	Kết quả mong đợi
`scaled_head(np.array([10, 20, 30]), 2, 2)`	`[20, 40]; a vẫn [10, 20, 30]`
`totals_match([0.1, 0.2], 0.3)`	`True`
`totals_match([0.1, 0.2], 0.31)`	`False`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`scaled_head(a, k, factor)`	Nhân k phần tử đầu của a với factor, trả array MỚI. · a: ndarray; k: int ≥ 0; factor: số (cùng loại số với a) · ndarray k phần tử, không dùng chung bộ nhớ với a · KHÔNG sửa a của người gọi.
`totals_match(parts, total)`	Kiểm tổng các phần có bằng total trong sai số cho phép. · parts: list/array số thực; total: số · bool · Không in gì.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
2 phần tử đầu × 2	`[int(v) for v in scaled_head(np.array([10, 20, 30]), 2, 2)]`	`[20, 40]`
Array gốc giữ nguyên	`(lambda a: (scaled_head(a, 2, 2), a.tolist())[1])(np.array([10, 20, 30]))`	`[10, 20, 30]`
Số thực, k = 3 (dữ liệu khác)	`(lambda a: ([round(float(v), 4) for v in scaled_head(a, 3, 0.5)], a.tolist()))(np.array([2.0, 4.0, 6.0, 8.0]))`	`([1.0, 2.0, 3.0], [2.0, 4.0, 6.0, 8.0])`
k = 0 → rỗng	`len(scaled_head(np.array([1, 2]), 0, 3))`	`0`
Sửa kết quả không làm đổi a	`(lambda a: (lambda h: (h.__setitem__(0, 99), a.tolist())[1])(scaled_head(a, 2, 1)))(np.array([5, 6, 7]))`	`[5, 6, 7]`
0.1 + 0.2 so với 0.3	`totals_match([0.1, 0.2], 0.3)`	`True`
1.1 + 2.2 so với 3.3	`totals_match([1.1, 2.2], 3.3)`	`True`
Lệch thật → False	`totals_match([0.1, 0.2], 0.31)`	`False`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Thư viện: numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 2 slot)
Khung 1 — Bước 1. a[:2] → view, chung bộ nhớ với a
Khung 2 — Bước 2. a[:2].copy() → bộ nhớ riêng
Khung 3 — Bước 3. head *= 2 → chỉ head đổi, a giữ [10, 20, 30]
Khung 4 — Bước 4. np.sum([0.1, 0.2]) → 0.30000000000000004
Khung 5 — Bước 5. np.isclose(tổng, 0.3) → True, còn == cho False
Khung đối chiếu — ngộ nhận "Cắt slice là đã có bản sao riêng, như list Python". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Với list Python, lst[:2] là bản sao; với NumPy, a[:2] là VIEW. Sửa tại chỗ trên view (b[0] = 9, b *= 2) làm array của người gọi đổi theo — lỗi khó thấy vì hàm vẫn trả đúng kết quả. Gọi .copy() khi cần độc lập.
Khung lắp mảnh — 2 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: first.sum() == 14.6, [:]
Khung kết quả — 3 cặp vào/ra. scaled_head(np.array([10, 20, 30]), 2, 2) → [20, 40]; a vẫn [10, 20, 30]; totals_match([0.1, 0.2], 0.3) → True; totals_match([0.1, 0.2], 0.31) → False

Nguồn: pe-v4 · python-learning-data.json#N05 · parser pe-v4/build · file build 030-N05.json

N06 · Đọc dữ liệu và mô phỏng tái lập

Mã bài: N06 · Trung cấp · M5 · NumPy · 35 phút · 15 XP · thứ tự 31

MỤC TIÊU

Đọc CSV có ô trống và mô phỏng tái lập bằng seed.

CẦN NHỚ TRƯỚC

N05

BỐI CẢNH

readings.csv (đã cấp) có dòng tiêu đề, ba cột day, temp, humidity và vài ô trống. Viết load_readings(path) đọc HAI cột temp, humidity thành array số thực (ô trống → nan) bằng np.genfromtxt. Viết column_means(path) trả mean từng cột, bỏ qua nan. Viết roll_dice(seed, n): tung xúc xắc n lần bằng np.random.default_rng(seed), trả array số nguyên 1..6 — cùng seed phải cho cùng kết quả, và hàm không được đụng tới RNG toàn cục np.random.

VÍ DỤ MINH HOẠ

load_readings("readings.csv") có shape (4, 2) và 2 ô nan; column_means → [30.33, 69.0]; roll_dice(7, 5) gọi hai lần cho cùng 5 số.

ĐIỂM KIẾN THỨC

1. np.loadtxt đọc bảng SỐ SẠCH: delimiter tách cột, skiprows bỏ dòng tiêu đề, usecols chọn cột, dtype ép kiểu. Gặp ô trống hay chữ là ValueError.
2. np.genfromtxt linh hoạt hơn: ô trống hoặc không đọc được thành nan (bỏ tiêu đề bằng skip_header, không phải skiprows). Đọc xong kiểm shape và số nan, rồi tính bằng nanmean như bài N04.
3. np.random.default_rng(seed) tạo một generator RIÊNG. Cùng seed (và cùng phiên bản NumPy) → cùng dãy số: kết quả mô phỏng tái lập được. Không seed thì mỗi lần chạy một khác.
4. rng.integers(low, high) KHÔNG lấy high: xúc xắc là integers(1, 7). Đừng gọi np.random.seed(...) trong hàm — nó đặt lại RNG toàn cục và làm đổi kết quả của mọi phần khác trong chương trình đang dùng RNG đó.
5. np.save/np.load lưu array kèm dtype và shape (.npy); np.savetxt ghi ra text cho người đọc. Ghi lại seed và phiên bản thư viện cùng kết quả. Dữ liệu mô phỏng chỉ để minh hoạ — không phải dữ liệu quan sát hay bằng chứng.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import numpy as np
from io import BytesIO, StringIO

clean = "x,y\n1,2\n3,4"
print(np.loadtxt(StringIO(clean), delimiter=",", skiprows=1))

messy = "x,y\n1,\n3,4"
a = np.genfromtxt(StringIO(messy), delimiter=",", skip_header=1)
print(a, np.isnan(a).sum())

rng = np.random.default_rng(2026)
print(rng.integers(1, 7, size=5))
print(np.random.default_rng(2026).integers(1, 7, size=5))   # cùng seed → cùng dãy
print(rng.normal(0, 1, size=2).round(3))                    # rng đi tiếp, số mới

buffer = BytesIO()
np.save(buffer, a)
buffer.seek(0)
print(np.load(buffer).shape)
```

Kết quả dự kiến (theo JSON): readings.csv → shape (4, 2), mean [30.33, 69.0]; roll_dice(7, 5) hai lần → cùng 5 số

NGỘ NHẬN THƯỜNG GẶP: "Đặt np.random.seed(...) ở đầu chương trình là đủ tái lập"

RNG toàn cục bị MỌI đoạn code dùng chung: một hàm khác rút thêm số là dãy của bạn lệch theo, và hàm nào gọi seed lại sẽ đổi kết quả của phần khác. default_rng(seed) cho mỗi mô phỏng một generator riêng — nhận seed làm tham số và ghi seed ra cùng kết quả.

CÂU DỰ ĐOÁN

Hai dòng dưới in ra hai dãy thế nào?

```
print(np.random.default_rng(42).integers(1, 7, size=3))
print(np.random.default_rng(42).integers(1, 7, size=3))
```

sai	Khác nhau vì là số ngẫu nhiên	Chỉ đúng khi không truyền seed.
ĐÚNG	Giống hệt nhau — mỗi dòng tạo generator mới với cùng seed 42	Seed quyết định toàn bộ dãy.
sai	Lỗi vì một seed chỉ dùng được một lần	Seed dùng bao nhiêu lần cũng được.

TỰ KIỂM TRA

1. File có ô trống "1,\n3,4". Hàm nào đọc được mà không lỗi?
   - np.loadtxt(..., delimiter=",") — loadtxt báo ValueError với ô trống.
   - Cả hai như nhau — Chỉ genfromtxt xử lý ô thiếu.
   - ✔ np.genfromtxt(..., delimiter=",") — ô trống thành nan — genfromtxt xử lý giá trị thiếu.

Đáp án: genfromtxt xử lý giá trị thiếu.

2. rng.integers(1, 6, size=1000) có bao giờ ra 6 không?
   - Thỉnh thoảng — Không bao giờ.
   - ✔ Không — high = 6 bị loại; xúc xắc phải là integers(1, 7) — Giống range và arange.
   - Có, cả 1 và 6 đều được lấy — integers mặc định không lấy high (endpoint=False).

Đáp án: Giống range và arange.

3. Muốn lưu array để lần sau nạp lại đúng dtype và shape, dùng gì?
   - print(a) rồi chép lại — Mất độ chính xác và cấu trúc.
   - np.savetxt cho array 3 chiều — savetxt chỉ ghi array 1D/2D ra text.
   - ✔ np.save("a.npy", a) rồi np.load("a.npy") — File .npy giữ dtype và shape.

Đáp án: File .npy giữ dtype và shape.

TỰ GIẢI THÍCH

Vì sao dùng default_rng(seed) cục bộ thay vì np.random.seed toàn cục cho một mô phỏng?

Mẫu trả lời: default_rng(seed) tạo generator riêng: cùng seed cho cùng dãy, và không phần code nào khác rút số từ nó nên kết quả không bị lệch. np.random.seed đặt lại RNG dùng chung — vừa dễ bị code khác làm lệch dãy, vừa làm thay đổi kết quả của những phần khác đang dùng RNG toàn cục.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	genfromtxt(skip_header=1, usecols=(1, 2)) → array (4, 2)
Bước 2	Ô trống trong CSV → nan
Bước 3	np.nanmean(axis=0) → [30.33, 69.0]
Bước 4	default_rng(7).integers(1, 7, size=5) → 5 số trong 1..6
Bước 5	default_rng(7) tạo lại → đúng 5 số đó

CÂU PHẢN VÍ DỤ

column_means của "day,temp,humidity\n1,20,50\n2,22,54\n3,24,58" → ?

Lựa chọn: [22.0, 54.0] | [2.0, 22.0, 54.0] | [nan, nan] · Đáp án: [22.0, 54.0]

TEMPLATE LẮP MẢNH

```
import numpy as np
from io import StringIO

text = "id,quiz1,quiz2\n1,7,\n2,9,6\n"
marks = np.genfromtxt(StringIO(text), delimiter=",", {{slot1}}, usecols=(1, 2))
means = {{slot2}}
rng = {{slot3}}
coins = rng.integers(0, 2, size=10)
```

Slot	Mảnh đúng
[Slot 1]	`skip_header=1`
[Slot 2]	`np.nanmean(marks, axis=0)`
[Slot 3]	`np.random.default_rng(2026)`

Mảnh bẫy	Vì sao sai
`skiprows=1`	Đó là tên tham số của loadtxt; genfromtxt dùng skip_header → TypeError.
`np.random.default_rng()`	Không seed → mỗi lần chạy một khác.
`marks.mean(axis=0)`	Cột có nan → kết quả nan.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: điểm hai bài kiểm tra có ô trống, và mô phỏng tung đồng xu.

Yêu cầu thêm: Thử file không có ô trống và file có cột day khác; với roll_dice, đổi seed phải đổi dãy, giữ seed phải giữ dãy.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

readings.csv (đã cấp) có dòng tiêu đề, ba cột day, temp, humidity và vài ô trống. Viết load_readings(path) đọc HAI cột temp, humidity thành array số thực (ô trống → nan) bằng np.genfromtxt. Viết column_means(path) trả mean từng cột, bỏ qua nan. Viết roll_dice(seed, n): tung xúc xắc n lần bằng np.random.default_rng(seed), trả array số nguyên 1..6 — cùng seed phải cho cùng kết quả, và hàm không được đụng tới RNG toàn cục np.random.

Thực hành: genfromtxt(path, delimiter=",", skip_header=1, usecols=(1, 2)); np.nanmean(..., axis=0); rng = np.random.default_rng(seed); rng.integers(1, 7, size=n).

Gọi	Kết quả mong đợi
`load_readings("readings.csv")`	`shape (4, 2); 2 ô nan`
`column_means("readings.csv")`	`[30.33, 69.0]`
`roll_dice(7, 5) gọi hai lần`	`cùng 5 số trong 1..6`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`load_readings(path)`	Đọc hai cột temp, humidity của file CSV có tiêu đề và ô trống. · path: str — CSV cột day,temp,humidity · ndarray float shape (số dòng, 2); ô trống → nan · Không in gì.
`column_means(path)`	Mean từng cột, bỏ qua nan. · path: str · ndarray 2 phần tử [mean temp, mean humidity] · Không in gì.
`roll_dice(seed, n)`	Mô phỏng n lần tung xúc xắc tái lập được. · seed: int; n: int ≥ 0 · ndarray n số nguyên trong 1..6; cùng seed → cùng dãy · Dùng generator riêng, không đụng RNG toàn cục của np.random.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
readings.csv: shape (4, 2), 2 ô nan	`(lambda a: (tuple(a.shape), int(np.isnan(a).sum())))(load_readings('readings.csv'))`	`((4, 2), 2)`
Mean từng cột bỏ nan	`[round(float(v), 2) for v in column_means('readings.csv')]`	`[30.33, 69.0]`
Bỏ cột day (dữ liệu khác)	`[[round(float(v), 4) for v in row] for row in load_readings(_studio.write('r2.csv', 'day,temp,humidity\n7,1.5,2.5\n8,3.5,4.5\n'))]`	`[[1.5, 2.5], [3.5, 4.5]]`
File không có ô trống	`[round(float(v), 2) for v in column_means(_studio.write('r1.csv', 'day,temp,humidity\n1,20,50\n2,22,54\n3,24,58\n'))]`	`[22.0, 54.0]`
Cùng seed → cùng dãy	`roll_dice(7, 20).tolist() == roll_dice(7, 20).tolist()`	`True`
Seed khác → dãy khác	`roll_dice(1, 20).tolist() != roll_dice(2, 20).tolist()`	`True`
200 lần: đủ n số, đủ mặt 1..6	`(lambda r: (len(r), sorted(set(int(v) for v in r))))(roll_dice(3, 200))`	`(200, [1, 2, 3, 4, 5, 6])`
Không đụng RNG toàn cục np.random	`(np.random.seed(123), roll_dice(1, 5), float(np.random.random()))[2] == (np.random.seed(123), float(np.random.random()))[1]`	`True`

MÔI TRƯỜNG CHẠY

Bài này chạy numpy THẬT ngay trong trình duyệt (Pyodide) — không có phần nào là mô phỏng. Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Muốn chạy trên máy: pip install numpy. Thư viện: numpy. File kèm bài: readings.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. genfromtxt(skip_header=1, usecols=(1, 2)) → array (4, 2)
Khung 2 — Bước 2. Ô trống trong CSV → nan
Khung 3 — Bước 3. np.nanmean(axis=0) → [30.33, 69.0]
Khung 4 — Bước 4. default_rng(7).integers(1, 7, size=5) → 5 số trong 1..6
Khung 5 — Bước 5. default_rng(7) tạo lại → đúng 5 số đó
Khung đối chiếu — ngộ nhận "Đặt np.random.seed(...) ở đầu chương trình là đủ tái lập". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. RNG toàn cục bị MỌI đoạn code dùng chung: một hàm khác rút thêm số là dãy của bạn lệch theo, và hàm nào gọi seed lại sẽ đổi kết quả của phần khác. default_rng(seed) cho mỗi mô phỏng một generator riêng — nhận seed làm tham số và ghi seed ra cùng kết quả.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: skiprows=1, np.random.default_rng(), marks.mean(axis=0)
Khung kết quả — 3 cặp vào/ra. load_readings("readings.csv") → shape (4, 2); 2 ô nan; column_means("readings.csv") → [30.33, 69.0]; roll_dice(7, 5) gọi hai lần → cùng 5 số trong 1..6

Nguồn: pe-v4 · python-learning-data.json#N06 · parser pe-v4/build · file build 031-N06.json

M6 · pandas

9 bài: PD01, PD02, PD03, PD04, PD05, PD06, PD07, PD08, PD09

Ứng viên animation — ưu tiên cao (★★): PD02, PD03, PD04, PD05, PD06, PD07, PD08, PD09 · ưu tiên (★): PD01

PD01 · Series, DataFrame và index

Mã bài: PD01 · Nâng cao · M6 · pandas · 30 phút · 20 XP · thứ tự 32

MỤC TIÊU

Lấy dữ liệu theo nhãn (loc), theo vị trí (iloc) và cộng Series theo nhãn.

CẦN NHỚ TRƯỚC

JSON không ghi bài tiên quyết cho bài này.

BỐI CẢNH

Viết hai hàm. pick(df, sid): df có cột "mark" và index là mã sinh viên; trả tuple (điểm của sid lấy theo NHÃN bằng loc, điểm của hàng ĐẦU TIÊN lấy theo VỊ TRÍ bằng iloc), cả hai ép về int. add_bonus(marks, bonus): hai Series điểm cùng tập mã sinh viên nhưng có thể khác thứ tự; trả Series marks + bonus cộng đúng theo mã.

VÍ DỤ MINH HOẠ

Index S01, S02 với điểm 7, 9: pick(df, "S02") → (9, 7). marks {S01: 7, S02: 9} cộng bonus {S02: 1, S01: 0} → {S01: 7, S02: 10}.

ĐIỂM KIẾN THỨC

1. Series là một cột có nhãn; DataFrame là bảng nhiều cột, mỗi cột một dtype. Cả hai có index — tập NHÃN của hàng.
2. loc tra theo nhãn, iloc theo vị trí: với index [30, 10, 20], s.loc[10] là 9 còn s.iloc[0] là 8.
3. df["mark"] trả Series; df[["mark"]] (list tên cột) giữ DataFrame một cột, shape (n, 1).
4. Trước khi phân tích, nhìn shape, dtypes, head(), info(), describe() — biết bảng có bao nhiêu hàng, cột nào kiểu gì.
5. a + b căn theo NHÃN chứ không theo vị trí: x cộng x, y cộng y. Nhãn có ở bên này mà thiếu ở bên kia cho NaN — kiểm nhãn trước khi cộng hay ghép.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

s = pd.Series([8, 9, 6], index=[30, 10, 20])
print(s.loc[10], s.iloc[0])

df = pd.DataFrame({"name": ["Mai", "An"], "mark": [8, 9]}, index=["S01", "S02"])
print(df.shape)
print(df.dtypes.to_dict())
print(type(df["mark"]).__name__, type(df[["mark"]]).__name__)

a = pd.Series([1, 2], index=["x", "y"])
b = pd.Series([10, 20], index=["y", "x"])
print((a + b).to_dict())
```

Kết quả dự kiến (theo JSON): S01, S02 / 7, 9 → pick(df, "S02") = (9, 7); bonus đảo thứ tự → vẫn cộng đúng mã

NGỘ NHẬN THƯỜNG GẶP: "Index chỉ là số thứ tự hàng 0, 1, 2..."

Index là NHÃN. Sau khi lọc, sắp xếp hoặc đặt mã sinh viên làm index, nhãn không còn trùng vị trí: df.loc[0] có thể KeyError hoặc trả nhầm hàng, còn s[0] trên Series index số là tra NHÃN 0. Cần hàng đầu tiên thì iloc[0]; cần đúng mã thì loc[mã].

CÂU DỰ ĐOÁN

s.loc[20] và s.iloc[0] lần lượt là?

```
s = pd.Series([8, 9], index=[10, 20])
```

sai	8 và 8	loc theo nhãn, nên 20 trỏ tới phần tử thứ hai.
ĐÚNG	9 và 8	loc[20] tìm NHÃN 20 (giá trị 9); iloc[0] lấy VỊ TRÍ đầu (giá trị 8).
sai	KeyError ở loc[20] vì Series chỉ có 2 phần tử	loc tìm nhãn 20, không phải vị trí 20 — nhãn 20 có tồn tại.

TỰ KIỂM TRA

1. df["mark"] và df[["mark"]] khác nhau thế nào?
   - Giống hệt nhau — Kiểu trả về khác nhau: Series và DataFrame.
   - df[["mark"]] là cú pháp lỗi — Truyền list tên cột là cách chọn nhiều cột — ở đây list một phần tử.
   - ✔ Một cặp [] cho Series; hai cặp [] (list tên cột) cho DataFrame một cột — shape lần lượt là (n,) và (n, 1).

Đáp án: shape lần lượt là (n,) và (n, 1).

2. a = Series([1, 2], index=["x", "y"]); b = Series([10, 20], index=["y", "x"]). (a + b)["x"] = ?
   - ✔ 21 — x của a (1) cộng x của b (20).

- Lỗi vì index khác thứ tự — Cùng tập nhãn thì pandas tự căn.
- 11 — Đó là cộng theo vị trí — pandas căn theo nhãn.

Đáp án: x của a (1) cộng x của b (20).

3. Muốn biết bảng có bao nhiêu hàng, cột và kiểu từng cột, xem gì trước?

✔ df.shape và df.dtypes (hoặc df.info()) — Đây là bước kiểm tra trước khi phân tích.

- Chỉ cần df.describe() — describe chỉ tóm tắt cột số, không cho biết dtype của cột chữ.
- print(df) rồi đếm tay — Bảng lớn thì print bị rút gọn; shape cho con số chính xác.

Đáp án: Đây là bước kiểm tra trước khi phân tích.

TỰ GIẢI THÍCH

Vì sao pick dùng loc cho mã sinh viên nhưng iloc cho "hàng đầu"?

Mẫu trả lời: Mã sinh viên là NHÃN của index nên tra bằng loc[sid]. "Hàng đầu" là VỊ TRÍ 0, không phải nhãn 0 — với index S01, S02 thì nhãn 0 không tồn tại, với index [1, 0] thì nhãn 0 lại là hàng thứ hai. iloc[0] luôn lấy hàng đầu dù index là gì.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	df.loc["S02", "mark"] → tìm hàng có NHÃN S02 → 9.
Bước 2	df.iloc[0]["mark"] → lấy hàng ở VỊ TRÍ đầu → 7.
Bước 3	marks + bonus → căn theo mã sinh viên rồi mới cộng.
Bước 4	df[["mark"]] → DataFrame một cột, shape (n, 1).

CÂU PHẢN VÍ DỤ

df có index [30, 10, 20], mark [4, 9, 7]. pick(df, 10) = ?

Lựa chọn: (4, 9) | (9, 4) | KeyError · Đáp án: (9, 4)

TEMPLATE LẮP MẢNH

```
import pandas as pd


def pick(df, sid):
    by_label = {{slot1}}
    first = {{slot2}}
    return int(by_label), int(first)


def add_bonus(marks, bonus):
    return {{slot3}}
```

Slot	Mảnh đúng
[Slot 1]	`df.loc[sid, "mark"]`
[Slot 2]	`df.iloc[0]["mark"]`
[Slot 3]	`marks + bonus`

Mảnh bẫy	Vì sao sai
`marks.values + bonus.values`	.values bỏ nhãn: cộng theo vị trí, sai khi hai Series khác thứ tự.
`df.iloc[sid]["mark"]`	iloc nhận VỊ TRÍ số nguyên; sid là nhãn.
`df.loc[0, "mark"]`	Nhãn 0 có thể không tồn tại hoặc không phải hàng đầu.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: index số lộn xộn [30, 10, 20] và bonus đảo thứ tự. Lắp ba chỗ.

Yêu cầu thêm: Đổi sang index số [30, 10, 20] và bonus đảo thứ tự; kiểm loc/iloc và phép cộng vẫn đúng.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm. pick(df, sid): df có cột "mark" và index là mã sinh viên; trả tuple (điểm của sid lấy theo NHÃN bằng loc, điểm của hàng ĐẦU TIÊN lấy theo VỊ TRÍ bằng iloc), cả hai ép về int. add_bonus(marks, bonus): hai Series điểm cùng tập mã sinh viên nhưng có thể khác thứ tự; trả Series marks + bonus cộng đúng theo mã.

Thực hành: loc[nhãn, cột] cho mã sinh viên; iloc[0] cho hàng đầu; cộng hai Series trực tiếp để pandas căn nhãn.

Gọi	Kết quả mong đợi
`index S01, S02; mark 7, 9; sid "S02"`	`(9, 7)`
`index [30, 10, 20]; mark [4, 9, 7]; sid 10`	`(9, 4)`
`marks {S01: 7, S02: 9}, bonus {S02: 1, S01: 0}`	`{S01: 7, S02: 10}`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`pick(df, sid)`	Lấy điểm theo nhãn và điểm của hàng đầu theo vị trí. · df: DataFrame có cột "mark", index là mã sinh viên; sid: một nhãn có trong index · tuple (int, int) · Không sửa df.
`add_bonus(marks, bonus)`	Cộng điểm thưởng theo đúng mã sinh viên. · marks, bonus: Series cùng tập nhãn, thứ tự có thể khác · Series có index là mã sinh viên · Không sửa hai Series đầu vào.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: S02 theo nhãn, hàng đầu theo vị trí	`pick(pd.DataFrame({"mark": [7, 9]}, index=["S01", "S02"]), "S02")`	`(9, 7)`
Index số lộn xộn [30, 10, 20]	`pick(pd.DataFrame({"mark": [4, 9, 7]}, index=[30, 10, 20]), 10)`	`(9, 4)`
Index [1, 0]: nhãn 0 KHÔNG phải hàng đầu	`pick(pd.DataFrame({"mark": [5, 8]}, index=[1, 0]), 0)`	`(8, 5)`
Index chữ, hàng đầu không phải S01	`pick(pd.DataFrame({"mark": [5, 8, 6]}, index=["B07", "A01", "C03"]), "C03")`	`(6, 5)`
pick trả int thuần	`[type(v).__name__ for v in pick(pd.DataFrame({"mark": [7, 9]}, index=["S01", "S02"]), "S01")]`	`['int', 'int']`
add_bonus: đề mẫu	`add_bonus(pd.Series([7, 9], index=["S01", "S02"]), pd.Series([1, 0], index=["S02", "S01"])).to_dict()`	`{'S01': 7, 'S02': 10}`
add_bonus căn theo mã khi bonus đảo thứ tự	`add_bonus(pd.Series([7, 9, 5], index=["S01", "S02", "S03"]), pd.Series([1, 0, 2], index=["S03", "S01", "S02"])).to_dict()`	`{'S01': 7, 'S02': 11, 'S03': 6}`
add_bonus không sửa marks	`(lambda r: (add_bonus(r, pd.Series([1, 1], index=["b", "a"])), r.tolist())[1])(pd.Series([3, 4], index=["a", "b"]))`	`[3, 4]`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. df.loc["S02", "mark"] → tìm hàng có NHÃN S02 → 9.
Khung 2 — Bước 2. df.iloc[0]["mark"] → lấy hàng ở VỊ TRÍ đầu → 7.
Khung 3 — Bước 3. marks + bonus → căn theo mã sinh viên rồi mới cộng.
Khung 4 — Bước 4. df[["mark"]] → DataFrame một cột, shape (n, 1).
Khung đối chiếu — ngộ nhận "Index chỉ là số thứ tự hàng 0, 1, 2…". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Index là NHÃN. Sau khi lọc, sắp xếp hoặc đặt mã sinh viên làm index, nhãn không còn trùng vị trí: df.loc[0] có thể KeyError hoặc trả nhầm hàng, còn s[0] trên Series index số là tra NHÃN 0. Cần hàng đầu tiên thì iloc[0]; cần đúng mã thì loc[mã].
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: marks.values + bonus.values, df.iloc[sid]["mark"], df.loc[0, "mark"]
Khung kết quả — 3 cặp vào/ra. index S01, S02; mark 7, 9; sid "S02" → (9, 7); index [30, 10, 20]; mark [4, 9, 7]; sid 10 → (9, 4); marks {S01: 7, S02: 9}, bonus {S02: 1, S01: 0} → {S01: 7, S02: 10}

Nguồn: pe-v4 · python-learning-data.json#PD01 · parser pe-v4/build · file build 032-PD01.json

PD02 · CSV, Excel và schema

Mã bài: PD02 · Nâng cao · M6 · pandas · 35 phút · 20 XP · thứ tự 33

MỤC TIÊU

Đọc file theo schema khai rõ và xuất CSV không cột index.

CẦN NHỚ TRƯỚC

PD01

BỐI CẢNH

scores.csv dùng dấu ; và có ba cột id;mark;note. Viết load_scores(path): đọc bằng pd.read_csv với sep=";", chỉ lấy hai cột id, mark (usecols), giữ id là chuỗi (dtype "string" — không mất số 0 đầu) và coi chữ "missing" là ô thiếu (na_values). File thiếu cột id hoặc mark phải báo ValueError. Viết export_csv(df, path): ghi CSV dấu phẩy, KHÔNG có cột index.

VÍ DỤ MINH HOẠ

"007;9;retake" → id "007", mark 9.0; "010;missing;absent" → mark NaN. Xuất lại → dòng đầu "id,mark".

ĐIỂM KIẾN THỨC

1. Đọc một file là khai rõ: dấu phân cách (sep), header, lấy cột nào (usecols), kiểu từng cột (dtype) và chữ nào nghĩa là "thiếu" (na_values).
2. Không khai dtype thì pandas ĐOÁN: "007" thành số 7 — mất số 0 đầu, mã không còn khớp với bảng khác. dtype={"id": "string"} giữ nguyên chuỗi.
3. Một ô chữ lạ ("missing", "n/a") làm cả cột số thành object (chuỗi). na_values=["missing"] biến ô đó thành NaN, cột còn là số.
4. usecols vừa bỏ cột thừa vừa là phép kiểm schema: file thiếu cột yêu cầu thì read_csv báo ValueError ngay.
5. to_csv mặc định ghi cả index thành một cột không tên; index=False bỏ cột rác đó.
6. Excel: pd.read_excel(path, sheet_name="Q1", usecols=..., dtype=...) cùng ý tưởng, cần engine openpyxl (pip install openpyxl trên máy; trình duyệt của bài chưa có thư viện này). File rất lớn: read_csv(..., chunksize=100_000) đọc từng khúc; luôn in số dòng đã đọc trước khi tổng hợp.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd
from io import StringIO

text = "code|qty\n0042|3\n0107|n/a\n"
df = pd.read_csv(StringIO(text), sep="|", dtype={"code": "string"}, na_values=["n/a"])
print(df["code"].tolist())
print(int(df["qty"].isna().sum()), "ô thiếu")
print(df.dtypes.to_dict())

guess = pd.read_csv(StringIO(text), sep="|")
print(guess["code"].tolist(), guess["qty"].tolist())

print(df.to_csv(index=False).splitlines()[0])
print(df.to_csv().splitlines()[0])
```

Kết quả dự kiến (theo JSON): scores.csv → id ["001", "007", "010"], 1 ô thiếu; export_csv → dòng đầu "id,mark"

NGỘ NHẬN THƯỜNG GẶP: "read_csv tự hiểu file, không cần khai gì"

pandas chỉ ĐOÁN: mã có số 0 đầu thành số, một ô "missing" làm cả cột điểm thành chuỗi, file dấu ; đọc ra một cột duy nhất "id;mark;note". parse_dates gặp ngày hỏng cũng không báo lỗi mà để nguyên cột chữ. Khai schema rồi kiểm df.dtypes ngay sau khi đọc.

CÂU DỰ ĐOÁN

Đọc KHÔNG khai dtype. df.loc[0, "id"] là gì?

```
df = pd.read_csv(StringIO("id,mark\n007,8"))
```

sai	Lỗi vì 007 không phải số hợp lệ	"007" vẫn đọc được thành số 7.
sai	Chuỗi "007"	Chỉ khi khai dtype chuỗi.
ĐÚNG	Số 7 — pandas đoán cột id là số, mất số 0 đầu	Khai dtype={"id": "string"} để giữ "007".

TỰ KIỂM TRA

1. File dấu ; nhưng gọi pd.read_csv(path) không có sep. Kết quả?

- • ValueError — Không lỗi — chỉ ra bảng sai.
- ✔ Một cột duy nhất tên "id;mark;note" — Mặc định sep=",".

• pandas tự nhận ra dấu ; — Không — mặc định luôn là dấu phẩy.

Đáp án: Mặc định sep=",".

2. df.to_csv("out.csv") (không index=False) rồi đọc lại. Thấy gì?

• Mất cột cuối — Không mất cột — thừa cột.

✔ Thêm cột "Unnamed: 0" chứa index cũ — Ghi index=False để không ghi index.

• Không khác gì — Index được ghi thành cột đầu.

Đáp án: Ghi index=False để không ghi index.

3. Cột mark có 8, 9 và "missing". Đọc không có na_values thì dtype của mark là?

✔ object — cả cột thành chuỗi vì một ô chữ — na_values=["missing"] biến ô đó thành NaN, cột còn là số.

• float64, ô missing thành NaN — Chỉ khi khai na_values.

• int64, ô missing thành 0 — pandas không bao giờ tự đổi chữ thành 0.

Đáp án: na_values=["missing"] biến ô đó thành NaN, cột còn là số.

TỰ GIẢI THÍCH

Vì sao phải khai dtype cho cột id dù file "trông" đã đúng?

Mẫu trả lời: Không khai thì pandas đoán kiểu: "007" thành 7, mất số 0 đầu và không còn khớp với mã ở bảng khác. Khai dtype={"id": "string"} biến kiểu cột thành một phần của cách đọc file; kiểm df.dtypes ngay sau khi đọc để chắc chắn.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	sep=";" → tách đúng ba cột id, mark, note.
Bước 2	usecols=["id", "mark"] → bỏ note; thiếu cột thì ValueError.
Bước 3	dtype={"id": "string"} → "007" vẫn là "007".
Bước 4	na_values=["missing"] → ô thiếu thành NaN, mark là float.
Bước 5	to_csv(path, index=False) → dòng đầu là id,mark.

CÂU PHẢN VÍ DỤ

File "id;mark;note\n0010;missing;x\n". load_scores(...)["mark"].isna().sum() = ?

Lựa chọn: ValueError | 0 | 1 · Đáp án: 1

TEMPLATE LẮP MẢNH

```
import pandas as pd


def load_scores(path):
    return pd.read_csv(
        path,
        {{slot1}},
        usecols=["id", "mark"],
        {{slot2}},
        {{slot3}},
    )


def export_csv(df, path):
    df.to_csv(path, {{slot4}})
```

Slot	Mảnh đúng
[Slot 1]	`sep=";"`
[Slot 2]	`dtype={"id": "string"}`
[Slot 3]	`na_values=["missing"]`
[Slot 4]	`index=False`

Mảnh bẫy	Vì sao sai
`na_values=[0]`	Ô thiếu được ghi là chữ "missing", không phải số 0.
`sep=","`	File dùng dấu ; — đọc bằng dấu phẩy ra một cột duy nhất.
`index=True`	Ghi thêm cột index không tên.
`dtype={"id": "int"}`	Ép số làm mất số 0 đầu của mã.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: file dấu ; có cột note thừa và ô "missing". Lắp bốn chỗ.

Yêu cầu thêm: Thử file có cả ô trống lẫn ô "missing", và một file thiếu cột mark; kiểm số ô thiếu và lỗi schema.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

scores.csv dùng dấu ; và có ba cột id;mark;note. Viết load_scores(path): đọc bằng pd.read_csv với sep=";", chỉ lấy hai cột id, mark (usecols), giữ id là chuỗi (dtype "string" — không mất số 0 đầu) và coi chữ "missing" là ô thiếu (na_values). File thiếu cột id hoặc mark phải báo ValueError. Viết export_csv(df, path): ghi CSV dấu phẩy, KHÔNG có cột index.

Thực hành: Khai đủ bốn tham số của read_csv (sep, usecols, dtype, na_values); xuất bằng index=False.

Gọi	Kết quả mong đợi
`scores.csv (cấp sẵn)`	`id ["001", "007", "010"]; mark có 1 ô thiếu`
`File thiếu cột mark`	`ValueError (từ usecols)`
`export_csv(df, "out.csv")`	`Dòng đầu file là id,mark`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`load_scores(path)`	Đọc file điểm dấu ; theo schema cố định. · path: tên file CSV dấu ; có cột id, mark (có thể thêm cột khác) · DataFrame đúng hai cột: id (string), mark (float, NaN cho "missing") · Thiếu cột id/mark → ValueError.
`export_csv(df, path)`	Ghi DataFrame ra CSV dấu phẩy. · df: DataFrame; path: tên file · None · Tạo/ghi đè file; không ghi cột index.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
scores.csv: id giữ số 0 đầu	`load_scores("scores.csv")["id"].tolist()`	`['001', '007', '010']`
Chỉ còn hai cột id, mark	`load_scores("scores.csv").columns.tolist()`	`['id', 'mark']`
Dữ liệu khác: id "0042", "0107", "0200"	`load_scores(_studio.write('t2.csv', 'id;mark;note\n0042;missing;a\n0107;6;b\n0200;7.5;c\n'))["id"].tolist()`	`['0042', '0107', '0200']`
"missing" là thiếu, cột mark vẫn là số	`float(load_scores(_studio.write('t2.csv', 'id;mark;note\n0042;missing;a\n0107;6;b\n0200;7.5;c\n'))["mark"].sum())`	`13.5`
Ô trống và "missing" đều là thiếu	`int(load_scores(_studio.write('t3.csv', 'id;mark;note\n0042;missing;a\n0107;;b\n0200;7.5;c\n'))["mark"].isna().sum())`	`2`
File thiếu cột mark → ValueError	`_studio.call(load_scores, _studio.write('bad.csv', 'id;score\n001;8\n')).error`	`'ValueError'`
export_csv không ghi cột index	`(export_csv(pd.DataFrame({"id": ["007"], "mark": [9]}), "out.csv"), _studio.read("out.csv").splitlines())[1]`	`['id,mark', '007,9']`
Xuất rồi đọc lại (dtype string) vẫn giữ "007"	`(export_csv(load_scores("scores.csv"), "round.csv"), pd.read_csv("round.csv", dtype={"id": "string"})["id"].tolist())[1]`	`['001', '007', '010']`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas. File kèm bài: scores.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. sep=";" → tách đúng ba cột id, mark, note.
Khung 2 — Bước 2. usecols=["id", "mark"] → bỏ note; thiếu cột thì ValueError.
Khung 3 — Bước 3. dtype={"id": "string"} → "007" vẫn là "007".
Khung 4 — Bước 4. na_values=["missing"] → ô thiếu thành NaN, mark là float.
Khung 5 — Bước 5. to_csv(path, index=False) → dòng đầu là id,mark.
Khung đối chiếu — ngộ nhận "read_csv tự hiểu file, không cần khai gì". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. pandas chỉ ĐOÁN: mã có số 0 đầu thành số, một ô "missing" làm cả cột điểm thành chuỗi, file dấu ; đọc ra một cột duy nhất "id;mark;note". parse_dates gặp ngày hỏng cũng không báo lỗi mà để nguyên cột chữ. Khai schema rồi kiểm df.dtypes ngay sau khi đọc.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: na_values=[0], sep=",", index=True, dtype={"id": "int"}
Khung kết quả — 3 cặp vào/ra. scores.csv (cấp sẵn) → id ["001", "007", "010"]; mark có 1 ô thiếu; File thiếu cột mark → ValueError (từ usecols); export_csv(df, "out.csv") → Dòng đầu file là id,mark

Nguồn: pe-v4 · python-learning-data.json#PD02 · parser pe-v4/build · file build 033-PD02.json

PD03 · Làm sạch thiếu, trùng và sai kiểu

Mã bài: PD03 · Nâng cao · M6 · pandas · 40 phút · 20 XP · thứ tự 34

MỤC TIÊU

Làm sạch kiểu số, nhãn chữ và dòng trùng theo khoá.

CẦN NHỚ TRƯỚC

PD02

BỐI CẢNH

Viết hai hàm. score_stats(values): values là list chuỗi điểm có thể hỏng ("8", "bad", "9", ""); trả dict {"bad": số ô không đổi được thành số, "mean": trung bình các ô HỢP LỆ làm tròn 2 chữ số, hoặc None nếu không có ô hợp lệ}. clean_sales(raw): raw là DataFrame đọc thô từ CSV bán hàng (order_id, date, region, product, qty, unit_price); trả bản sạch: region bỏ khoảng trắng và viết thường, qty và unit_price thành số (ô hỏng thành NaN), bỏ dòng trùng order_id (giữ dòng đầu), bỏ dòng thiếu qty hoặc unit_price, index đánh lại từ 0. Không sửa raw. (Ngày hỏng và qty âm để các bài sau xử lý.)

VÍ DỤ MINH HOẠ

["8", "bad", "9"] → {"bad": 1, "mean": 8.5}. sales.csv 11 dòng → clean_sales còn 8 dòng (bỏ bản O05 gửi lại, O03 thiếu qty, O08 thiếu giá).

ĐIỂM KIẾN THỨC

1. Kiểm trước, sửa sau: isna().sum() đếm ô thiếu, duplicated(subset=khoá).sum() đếm dòng trùng. Thiếu (NaN) khác 0 và khác chuỗi rỗng.
2. pd.to_numeric(..., errors="coerce") biến ô hỏng ("bad", "two", "?") thành NaN để đếm và rà soát; astype(float) thì ValueError ngay ở ô hỏng đầu tiên.
3. mean() tự bỏ qua NaN: ["8", "bad", "9"] → 8.5. fillna(0) trước khi tính là bịa thêm một điểm 0 → 5.67.
4. str.strip().str.lower() gộp " North ", "NORTH", "north" thành một nhãn; không chuẩn hoá thì groupby đếm thành ba vùng.
5. Grain của bảng bán hàng là MỘT đơn = một order_id. drop_duplicates(subset="order_id") bỏ bản gửi lại; drop_duplicates() không khoá chỉ bỏ dòng giống hệt nhau từng ô.
6. dropna(subset=[...]) bỏ dòng thiếu số liệu bắt buộc; ghi lại đã bỏ bao nhiêu dòng (len trước − len sau) thay vì xoá âm thầm.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

marks = pd.Series(["7", "", "x", "10"])
nums = pd.to_numeric(marks, errors="coerce")
print(nums.tolist())
print(int(nums.isna().sum()), "ô hỏng; mean hợp lệ =", nums.mean())
print("Nếu coi hỏng là 0:", nums.fillna(0).mean())

labels = pd.Series([" Hanoi", "HANOI ", "hanoi"])
print(labels.nunique(), "->", labels.str.strip().str.lower().nunique())

orders = pd.DataFrame({"id": ["A1", "A1", "A2"], "qty": [2, 5, 2]})
print(len(orders.drop_duplicates()), len(orders.drop_duplicates(subset="id")))
```

Kết quả dự kiến (theo JSON): ["8", "bad", "9"] → {"bad": 1, "mean": 8.5}; sales.csv 11 dòng → 8 dòng sạch

NGỘ NHẬN THƯỜNG GẶP: "Ô hỏng hay ô thiếu thì cứ coi là 0 cho dễ tính"

0 là một GIÁ TRỊ: số lượng 0, điểm 0. Ô "bad" hay ô trống là "không biết". fillna(0) kéo trung bình xuống và tạo ra đơn hàng 0 sản phẩm không có thật. Đổi thành NaN, đếm, rồi chọn chính sách có lý do (bỏ dòng, cách ly, hỏi lại nguồn).

CÂU DỰ ĐOÁN

Kết quả của dòng dưới là?

```
pd.to_numeric(pd.Series(["8", "bad", "9"]), errors="coerce").mean()
```

ĐÚNG	8.5 — "bad" thành NaN và mean bỏ qua NaN	Trung bình của 8 và 9.
sai	ValueError vì có chữ "bad"	errors="coerce" không báo lỗi mà đổi thành NaN.
sai	5.67 — "bad" tính là 0	Chỉ khi fillna(0): bịa thêm một điểm 0.

TỰ KIỂM TRA

1. Hai đơn O11, O12 khác order_id nhưng cùng ngày, vùng, sản phẩm, qty, giá. drop_duplicates(subset="order_id") làm gì?

✔ Giữ cả hai — đó là hai giao dịch thật — Khoá order_id khác nhau nên không trùng.

• Bỏ cả hai — drop_duplicates luôn giữ một bản (keep="first").

• Bỏ một đơn vì giống nhau — Chỉ khi bỏ trùng theo các cột giá trị — sai grain.

Đáp án: Khoá order_id khác nhau nên không trùng.

2. Đơn O05 bị gửi lại lần 2 với qty khác. drop_duplicates() KHÔNG có subset thì…

• Báo lỗi trùng khoá — drop_duplicates không báo lỗi.

• Giữ dòng đầu — Đó là kết quả của subset="order_id".

✔ Giữ cả hai dòng O05 vì hai dòng khác nhau ở qty — Không khoá thì chỉ dòng giống hệt từng ô mới bị bỏ.

Đáp án: Không khoá thì chỉ dòng giống hệt từng ô mới bị bỏ.

3. pd.Series(["2", "two"]).astype(float) thì…

• [2.0, nan] — Đó là kết quả của to_numeric(errors="coerce").

• [2.0, 0.0] — pandas không tự đổi chữ thành 0.

✔ ValueError ở ô "two" — Dùng to_numeric(errors="coerce") để đổi ô hỏng thành NaN.

Đáp án: Dùng to_numeric(errors="coerce") để đổi ô hỏng thành NaN.

TỰ GIẢI THÍCH

Vì sao bỏ trùng theo order_id mà không theo cả dòng hay theo các cột giá trị?

Mẫu trả lời: Mỗi dòng của bảng là một đơn — khoá order_id xác định grain. Bản gửi lại có thể lệch vài ô nên drop_duplicates() cả dòng không bắt được; còn bỏ theo giá trị (vùng, sản phẩm, giá) thì xoá nhầm hai giao dịch thật chỉ vì giống nhau. Bỏ theo khoá giữ đúng một dòng cho mỗi đơn.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	str.strip().str.lower() → " North " và "NORTH" thành "north".
Bước 2	to_numeric(errors="coerce") → "two" và ô trống thành NaN.
Bước 3	drop_duplicates(subset="order_id") → mỗi đơn còn một dòng.
Bước 4	dropna(subset=["qty", "unit_price"]) → bỏ dòng thiếu số liệu.
Bước 5	reset_index(drop=True) → index 0, 1, 2… liền mạch.

CÂU PHẢN VÍ DỤ

score_stats(["10", "?", "", "6"]) = ?

Lựa chọn: {"bad": 1, "mean": 8.0} | {"bad": 2, "mean": 8.0} | {"bad": 2, "mean": 4.0} · Đáp án: {"bad": 2, "mean": 8.0}

TEMPLATE LẮP MẢNH

```
import pandas as pd


def clean_sales(raw):
    df = raw.copy()
    df["region"] = {{slot1}}
    df["qty"] = {{slot2}}
    df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")
    df = {{slot3}}
    df = df.dropna(subset=["qty", "unit_price"])
    return df.reset_index(drop=True)
```

Slot	Mảnh đúng
[Slot 1]	`df["region"].str.strip().str.lower()`
[Slot 2]	`pd.to_numeric(df["qty"], errors="coerce")`
[Slot 3]	`df.drop_duplicates(subset="order_id", keep="first")`

Mảnh bẫy	Vì sao sai
`pd.to_numeric(df["qty"], errors="coerce").fillna(0)`	Coi ô thiếu là 0 — tạo đơn 0 sản phẩm.
`df["region"].lower()`	Series không có .lower(); phải đi qua .str.
`df.drop_duplicates()`	Không khoá: bản gửi lại lệch một ô vẫn lọt.
`df["qty"].astype(float)`	ValueError ở ô "two".

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: qty có chữ "two", nhãn vùng lẫn hoa thường, một đơn bị gửi lại. Lắp ba chỗ.

Yêu cầu thêm: Thử bảng có đơn gửi lại với qty khác, hai đơn thật giống hệt giá trị, qty "two" và giá "?"; đếm số dòng còn lại.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm. score_stats(values): values là list chuỗi điểm có thể hỏng ("8", "bad", "9", ""); trả dict {"bad": số ô không đổi được thành số, "mean": trung bình các ô HỢP LỆ làm tròn 2 chữ số, hoặc None nếu không có ô hợp lệ}. clean_sales(raw): raw là DataFrame đọc thô từ CSV bán hàng (order_id, date, region, product, qty, unit_price); trả bản sạch: region bỏ khoảng trắng và viết thường, qty và unit_price thành số (ô hỏng thành NaN), bỏ dòng trùng order_id (giữ dòng đầu), bỏ dòng thiếu qty hoặc unit_price, index đánh lại từ 0. Không sửa raw. (Ngày hỏng và qty âm để các bài sau xử lý.)

Thực hành: copy() trước khi sửa; chuẩn hoá chữ qua .str; to_numeric(errors="coerce") cho cột số; bỏ trùng theo order_id; dropna theo cột bắt buộc.

Gọi	Kết quả mong đợi
`["8", "bad", "9"]`	`{"bad": 1, "mean": 8.5}`
`sales.csv (11 dòng)`	`8 dòng: bỏ bản O05 gửi lại, O03 thiếu qty, O08 thiếu giá`
`Đơn gửi lại khác qty`	`Giữ dòng đầu của order_id`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`score_stats(values)`	Đếm ô điểm hỏng và tính trung bình các ô hợp lệ. · values: list chuỗi (có thể "bad", "", "?") · dict {"bad": int, "mean": float | None} · Không coi ô hỏng là 0.
`clean_sales(raw)`	Làm sạch bảng bán hàng đọc thô. · raw: DataFrame cột order_id, date, region, product, qty, unit_price · DataFrame mới, mỗi order_id một dòng, qty/unit_price là số, index 0..n-1 · Không sửa raw.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: ["8", "bad", "9"]	`score_stats(["8", "bad", "9"])`	`{'bad': 1, 'mean': 8.5}`
Ô trống và "?" đều hỏng, không tính là 0	`score_stats(["10", "?", "", "6"])`	`{'bad': 2, 'mean': 8.0}`
Không có ô hợp lệ → mean là None	`score_stats(["x", ""])`	`{'bad': 2, 'mean': None}`
Dữ liệu bẩn: còn đúng các đơn hợp lệ	`clean_sales(pd.read_csv(_studio.write('raw_a.csv', 'order_id,date,region,product,qty,unit_price\nA1,2026-03-01, Hanoi ,Pen,2,5\nA2,2026-03-01,HANOI,Pen,two,5\nA3,2026-03-02,hue,Book,1,12\nA3,2026-03-02,Hue,Book,4,12\nA4,2026-03-03,hue ,Book,1,12\nA5,2026-03-03,Hanoi,Pen,,5\nA6,2026-03-04,hanoi,Pen,3,?\nA7,2026-03-03,hue,Book,1,12\n')))["order_id"].tolist()`	`['A1', 'A3', 'A4', 'A7']`
Nhãn vùng đã chuẩn hoá	`clean_sales(pd.read_csv(_studio.write('raw_a.csv', 'order_id,date,region,product,qty,unit_price\nA1,2026-03-01, Hanoi ,Pen,2,5\nA2,2026-03-01,HANOI,Pen,two,5\nA3,2026-03-02,hue,Book,1,12\nA3,2026-03-02,Hue,Book,4,12\nA4,2026-03-03,hue ,Book,1,12\nA5,2026-03-03,Hanoi,Pen,,5\nA6,2026-03-04,hanoi,Pen,3,?\nA7,2026-03-03,hue,Book,1,12\n')))["region"].tolist()`	`['hanoi', 'hue', 'hue', 'hue']`
Đơn gửi lại: giữ dòng đầu (qty 1), qty là số	`clean_sales(pd.read_csv(_studio.write('raw_a.csv', 'order_id,date,region,product,qty,unit_price\nA1,2026-03-01, Hanoi ,Pen,2,5\nA2,2026-03-01,HANOI,Pen,two,5\nA3,2026-03-02,hue,Book,1,12\nA3,2026-03-02,Hue,Book,4,12\nA4,2026-03-03,hue ,Book,1,12\nA5,2026-03-03,Hanoi,Pen,,5\nA6,2026-03-04,hanoi,Pen,3,?\nA7,2026-03-03,hue,Book,1,12\n')))["qty"].tolist()`	`[2.0, 1.0, 1.0, 1.0]`
sales.csv: 11 → 8 dòng, index liền 0..7	`clean_sales(pd.read_csv("sales.csv")).index.tolist()`	`[0, 1, 2, 3, 4, 5, 6, 7]`
Không sửa raw	`(lambda r: (clean_sales(r), r["region"].tolist()[:2])[1])(pd.read_csv("sales.csv"))`	`[' North ', 'south']`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas. File kèm bài: sales.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. str.strip().str.lower() → " North " và "NORTH" thành "north".
Khung 2 — Bước 2. to_numeric(errors="coerce") → "two" và ô trống thành NaN.
Khung 3 — Bước 3. drop_duplicates(subset="order_id") → mỗi đơn còn một dòng.
Khung 4 — Bước 4. dropna(subset=["qty", "unit_price"]) → bỏ dòng thiếu số liệu.
Khung 5 — Bước 5. reset_index(drop=True) → index 0, 1, 2… liền mạch.
Khung đối chiếu — ngộ nhận "Ô hỏng hay ô thiếu thì cứ coi là 0 cho dễ tính". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. 0 là một GIÁ TRỊ: số lượng 0, điểm 0. Ô "bad" hay ô trống là "không biết". fillna(0) kéo trung bình xuống và tạo ra đơn hàng 0 sản phẩm không có thật. Đổi thành NaN, đếm, rồi chọn chính sách có lý do (bỏ dòng, cách ly, hỏi lại nguồn).
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: pd.to_numeric(df["qty"], errors="coerce").fillna(0), df["region"].lower(), df.drop_duplicates(), df["qty"].astype(float)
Khung kết quả — 3 cặp vào/ra. ["8", "bad", "9"] → {"bad": 1, "mean": 8.5}; sales.csv (11 dòng) → 8 dòng: bỏ bản O05 gửi lại, O03 thiếu qty, O08 thiếu giá; Đơn gửi lại khác qty → Giữ dòng đầu của order_id

Nguồn: pe-v4 · python-learning-data.json#PD03 · parser pe-v4/build · file build 034-PD03.json

PD04 · Lọc, tính cột và cập nhật an toàn

Mã bài: PD04 · Nâng cao · M6 · pandas · 40 phút · 20 XP · thứ tự 35

MỤC TIÊU

Lọc, tính cột và cập nhật bằng loc trên bản copy.

CẦN NHỚ TRƯỚC

PD03

BỐI CẢNH

Bảng đơn đã đúng kiểu (order_id, region, qty, unit_price; qty có thể ≤ 0). Viết add_flags(df): trả bản COPY có thêm cột revenue = qty * unit_price và cột status — "ok" cho mọi dòng, riêng dòng qty <= 0 là "check", cập nhật bằng .loc[mask, "status"]; không được sửa df gốc. Viết top_orders(df, region, min_revenue): từ add_flags(df), giữ dòng đúng region, status "ok" và revenue >= min_revenue, sắp revenue giảm dần.

VÍ DỤ MINH HOẠ

qty [2, 3, -1], giá [10, 20, 10] → revenue [20, 60, -10], status [ok, ok, check]. top_orders(..., "north", 30) chỉ còn đơn revenue 60.

ĐIỂM KIẾN THỨC

1. Mask là Series True/False. Ghép điều kiện bằng & (và), | (hoặc), ~ (phủ định) và BỌC NGOẶC từng điều kiện: & ưu tiên cao hơn ==, >=.
2. `and`/`or` của Python cần MỘT giá trị True/False nên với Series báo "truth value of a Series is ambiguous".
3. Cột mới: df["revenue"] = df["qty"] * df["unit_price"] (hoặc df.assign(revenue=...)) tính cả cột một lần, không cần vòng for. revenue cùng đơn vị tiền với unit_price.
4. Cập nhật một phần cột: df.loc[mask, "status"] = "check" — MỘT lần lập chỉ mục, ghi thẳng vào df.
5. df[mask]["status"] = ... là chained assignment: df[mask] tạo bảng tạm, phép gán rơi vào bảng tạm và df không đổi (pandas cảnh báo SettingWithCopyWarning; pandas 3 với Copy-on-Write càng chắc chắn không đổi).
6. Hàm nhận df của người gọi: out = df.copy() trước khi thêm cột, nếu không bảng của người gọi bị đổi theo. Lọc ra bảng con rồi muốn sửa riêng thì cũng copy().

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

df = pd.DataFrame({"mark": [4, 8, 9], "cls": ["A", "A", "B"]})
passed_a = df.loc[(df["mark"] >= 5) & (df["cls"] == "A")]
print(passed_a["mark"].tolist())

df["grade"] = "pass"
df.loc[df["mark"] < 5, "grade"] = "fail"
print(df["grade"].tolist())

df[df["mark"] < 5]["grade"] = "retake"
print(df["grade"].tolist())

top = df.sort_values("mark", ascending=False).head(2)
print(top["mark"].tolist())
```

Kết quả dự kiến (theo JSON): qty [2, 3], giá [10, 20], ngưỡng 30 → [60]; qty ≤ 0 → status "check"; df gốc không đổi

NGỘ NHẬN THƯỜNG GẶP: "Có SettingWithCopyWarning thôi, kết quả vẫn đúng"

Cảnh báo đó báo phép gán có thể rơi vào bảng tạm: df[mask]["col"] = ... KHÔNG sửa df, cả ở pandas 2.2 trong trình duyệt lẫn pandas 3. Dạng df["col"][mask] = ... có khi vẫn "ăn" ở pandas 2.2 nhưng hỏng âm thầm ở pandas 3 (Copy-on-Write). Luôn viết một lần: df.loc[mask, "col"] = giá_trị.

CÂU DỰ ĐOÁN

Hai dòng dưới khác nhau thế nào?

```
df.loc[df["qty"] <= 0, "status"] = "check"
df[df["qty"] <= 0]["status"] = "check"
```

ĐÚNG	Dòng đầu sửa df; dòng sau gán vào bảng tạm nên df không đổi	df[mask] tạo bảng mới; ["status"] = ... gán vào bảng đó.
sai	Dòng đầu lỗi vì loc chỉ dùng để đọc	loc dùng được cả để đọc lẫn ghi.
sai	Giống nhau, dòng sau ngắn hơn	Dòng sau là chained assignment — không ghi vào df.

TỰ KIỂM TRA

1. df[df["qty"] > 0 & df["region"] == "north"] (thiếu ngoặc) thì...
   - pandas tự hiểu là and — Python không đoán ý.

✔ Lỗi hoặc ra sai — & được tính trước > và == — Viết (df["qty"] > 0) & (df["region"] == "north").

• Chạy đúng như ý — & ưu tiên cao hơn phép so sánh.

Đáp án: Viết (df["qty"] > 0) & (df["region"] == "north").

2. Hàm gán out = df rồi out["revenue"] = ... (không copy). Sau khi gọi, bảng của người gọi...

• Không đổi — Python truyền bản sao — Python truyền tham chiếu tới cùng đối tượng.

✔ Có thêm cột revenue — hàm đã sửa chính đối tượng được truyền vào — DataFrame là đối tượng mutable như list.

• Lỗi — Gán cột mới luôn được phép.

Đáp án: DataFrame là đối tượng mutable như list.

3. Lấy 3 đơn doanh thu cao nhất, cách nào đúng?

✔ df.sort_values("revenue", ascending=False).head(3) — Sắp giảm dần rồi lấy đầu.

• df.sort_values("revenue").head(3) — Mặc định tăng dần: đó là 3 đơn thấp nhất.

• df.head(3) — head lấy 3 dòng đầu theo thứ tự hiện tại.

Đáp án: Sắp giảm dần rồi lấy đầu.

TỰ GIẢI THÍCH

Vì sao add_flags phải copy() và cập nhật status bằng loc?

Mẫu trả lời: copy() để hàm không sửa bảng người gọi đưa vào — người gọi có thể còn dùng bảng gốc. Cập nhật bằng out.loc[mask, "status"] là một phép gán duy nhất vào chính bảng; viết out[mask]["status"] = ... chỉ gán vào bảng tạm do out[mask] tạo ra nên status không đổi.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	out = df.copy() → bảng riêng, bảng gốc không đổi.
Bước 2	out["qty"] * out["unit_price"] → cột revenue cho mọi dòng.
Bước 3	out.loc[out["qty"] <= 0, "status"] → "check" đúng dòng.
Bước 4	(a) & (b) & (c) → mask: đúng vùng, ok, đủ doanh thu.
Bước 5	sort_values(..., ascending=False) → đơn lớn nhất lên đầu.

CÂU PHẢN VÍ DỤ

qty [1, 0, 4], giá [50, 20, 10] → cột status = ?

Lựa chọn: ["ok", "check", "ok"] | ["check", "check", "ok"] | ["ok", "ok", "ok"] · Đáp án: ["ok", "check", "ok"]

TEMPLATE LẮP MẢNH

```
import pandas as pd


def add_flags(df):
    out = {{slot1}}
    out["revenue"] = out["qty"] * out["unit_price"]
    out["status"] = "ok"
    {{slot2}}
    return out


def top_orders(df, region, min_revenue):
    data = add_flags(df)
    mask = {{slot3}}
    return {{slot4}}
```

Slot	Mảnh đúng
[Slot 1]	`df.copy()`
[Slot 2]	`out.loc[out["qty"] <= 0, "status"] = "check"`
[Slot 3]	`(data["region"] == region) & (data["status"] == "ok") & (data["revenue"] >= min_revenue)`
[Slot 4]	`data.loc[mask].sort_values("revenue", ascending=False)`

Mảnh bẫy	Vì sao sai
`data.loc[mask].sort_values("revenue")`	Mặc định tăng dần — đơn nhỏ lên đầu.
`data["region"] == region and data["revenue"] >= min_revenue`	`and` không dùng được với Series.
`df`	Không copy: thêm cột vào chính bảng của người gọi.

`out[out["qty"] <= 0]["status"] = "check"`	Chained assignment: gán vào bảng tạm, out không đổi.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: có đơn qty 0, qty âm và hai vùng. Lắp bốn chỗ.

Yêu cầu thêm: Đổi vùng và ngưỡng doanh thu; thêm đơn qty 0 và qty âm; kiểm bảng gốc không có cột mới.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Bảng đơn đã đúng kiểu (order_id, region, qty, unit_price; qty có thể ≤ 0). Viết add_flags(df): trả bản COPY có thêm cột revenue = qty * unit_price và cột status — "ok" cho mọi dòng, riêng dòng qty <= 0 là "check", cập nhật bằng .loc[mask, "status"]; không được sửa df gốc. Viết top_orders(df, region, min_revenue): từ add_flags(df), giữ dòng đúng region, status "ok" và revenue >= min_revenue, sắp revenue giảm dần.

Thực hành: copy → tính revenue → status mặc định "ok" → loc[mask, "status"] = "check"; lọc bằng mask có ngoặc rồi sort_values giảm dần.

Gọi	Kết quả mong đợi
`qty [2, 3, -1], giá [10, 20, 10]`	`revenue [20, 60, -10]; status [ok, ok, check]`
`top_orders(df, "north", 30)`	`Chỉ đơn north, status ok, revenue ≥ 30, giảm dần`
`Sau khi gọi`	`df gốc không có cột revenue/status`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`add_flags(df)`	Thêm cột revenue và status trên bản sao. · df: DataFrame có qty, unit_price · DataFrame mới thêm revenue, status ("ok"/"check") · Không sửa df.
`top_orders(df, region, min_revenue)`	Lọc đơn tốt của một vùng, lớn nhất trước. · df như trên; region: str; min_revenue: số · DataFrame đã lọc, revenue giảm dần · Không sửa df.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: revenue ≥ 30, giảm dần	`top_orders(pd.DataFrame({"order_id": ["O1", "O2"], "region": ["north", "north"], "qty": [2, 3], "unit_price": [10, 20]}), "north", 30)["revenue"].tolist()`	`[60]`
revenue = qty × unit_price	`add_flags(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}))["revenue"].tolist()`	`[40.0, 40.0, 0.0, 50.0, -50.0, 80.0]`
qty ≤ 0 → status "check"	`add_flags(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}))["status"].tolist()`	`['ok', 'ok', 'check', 'ok', 'check', 'ok']`
Không sửa bảng gốc	`(lambda r: (add_flags(r), r.columns.tolist())[1])(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}))`	`['order_id', 'region', 'qty', 'unit_price']`
Lọc vùng + ngưỡng, sắp giảm dần	`top_orders(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}), "hue", 40)["order_id"].tolist()`	`['B4', 'B1']`
Ngưỡng 0: đơn status "check" vẫn bị loại	`top_orders(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}), "hue", 0)["order_id"].tolist()`	`['B4', 'B1']`
Vùng khác	`top_orders(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}), "hanoi", 50)["order_id"].tolist()`	`['B6']`

Không đơn nào đạt → bảng rỗng	`len(top_orders(pd.DataFrame({"order_id": ["B1", "B2", "B3", "B4", "B5", "B6"], "region": ["hue", "hanoi", "hue", "hue", "hue", "hanoi"], "qty": [5, 4, 0, 1, -2, 2], "unit_price": [8.0, 10.0, 30.0, 50.0, 25.0, 40.0]}), "hanoi", 1000))`	`0`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. out = df.copy() → bảng riêng, bảng gốc không đổi.
Khung 2 — Bước 2. out["qty"] * out["unit_price"] → cột revenue cho mọi dòng.
Khung 3 — Bước 3. out.loc[out["qty"] <= 0, "status"] → "check" đúng dòng.
Khung 4 — Bước 4. (a) & (b) & (c) → mask: đúng vùng, ok, đủ doanh thu.
Khung 5 — Bước 5. sort_values(..., ascending=False) → đơn lớn nhất lên đầu.
Khung đối chiếu — ngộ nhận "Có SettingWithCopyWarning thôi, kết quả vẫn đúng". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Cảnh báo đó báo phép gán có thể rơi vào bảng tạm: df[mask]["col"] = ... KHÔNG sửa df, cả ở pandas 2.2 trong trình duyệt lẫn pandas 3. Dạng df["col"][mask] = ... có khi vẫn "ăn" ở pandas 2.2 nhưng hỏng âm thầm ở pandas 3 (Copy-on-Write). Luôn viết một lần: df.loc[mask, "col"] = giá_trị.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: data.loc[mask].sort_values("revenue"), data["region"] == region and data["revenue"] >= min_revenue, df, out[out["qty"] <= 0]["status"] = "check"
Khung kết quả — 3 cặp vào/ra. qty [2, 3, -1], giá [10, 20, 10] → revenue [20, 60, -10]; status [ok, ok, check]; top_orders(df, "north", 30) → Chỉ đơn north, status ok, revenue ≥ 30, giảm dần; Sau khi gọi → df gốc không có cột revenue/status

Nguồn: pe-v4 · python-learning-data.json#PD04 · parser pe-v4/build · file build 035-PD04.json

PD05 · GroupBy và KPI có mẫu số rõ ràng

Mã bài: PD05 · Nâng cao · M6 · pandas · 40 phút · 20 XP · thứ tự 36

MỤC TIÊU

Tính KPI theo vùng với mẫu số rõ ràng.

CẦN NHỚ TRƯỚC

PD04

BỐI CẢNH

Bảng đơn (order_id, region, qty, unit_price; region có thể thiếu, qty có thể thiếu). Viết region_kpi(df) trả DataFrame index là region (region thiếu gom thành "unknown"), đúng thứ tự cột: orders (số DÒNG đơn của vùng — kể cả đơn thiếu qty), qty (tổng), revenue (tổng qty*unit_price), avg_price = revenue / qty của vùng, share = revenue vùng / TỔNG revenue mọi vùng. Trước khi trả, kiểm tổng revenue các vùng khớp tổng revenue của bảng nguồn.

VÍ DỤ MINH HOẠ

Vùng A có rev 10, 20; vùng B có 40 → revenue A 30, B 40; tổng 70; A chiếm 3/7 ≈ 0.4286.

ĐIỂM KIẾN THỨC

1. GroupBy là split-apply-combine: tách theo khoá, tính trên từng nhóm, ghép lại. agg trả MỘT dòng mỗi nhóm; transform trả kết quả dài bằng bảng gốc.
2. size đếm số dòng của nhóm; count chỉ đếm ô KHÔNG thiếu của một cột. Đơn thiếu qty vẫn là một đơn.
3. Named aggregation agg(orders=("order_id", "size"), revenue=("revenue", "sum")) đặt tên metric rõ ràng ngay trong kết quả.
4. Tỷ lệ phải nói rõ mẫu số: share = doanh thu vùng / TỔNG doanh thu; giá bình quân = tổng doanh thu / tổng số lượng.
5. Trung bình của các giá (hay trung bình của các trung bình) không bằng giá bình quân khi số lượng mỗi đơn khác nhau: đơn 9 cái và đơn 1 cái không nặng như nhau.
6. groupby mặc định BỎ nhóm có khoá thiếu (dropna=True). Gom thành "unknown" (hoặc dropna=False) khi đó là chính sách, và luôn kiểm tổng các nhóm = tổng nguồn để bắt dòng bị mất.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

df = pd.DataFrame({"cls": ["A", "A", "A", "B"], "score": [6, 8, None, 9]})
out = df.groupby("cls").agg(rows=("score", "size"), graded=("score", "count"), avg=("score", "mean"))
print(out)

df["share_in_class"] = df["score"] / df.groupby("cls")["score"].transform("sum")
print(df["share_in_class"].round(3).tolist())

shop = pd.DataFrame({"qty": [4, 1], "price": [5, 20]})
shop["rev"] = shop["qty"] * shop["price"]
print("mean các giá:", shop["price"].mean())
print("giá bình quân:", shop["rev"].sum() / shop["qty"].sum())
```

Kết quả dự kiến (theo JSON): A: 10, 20; B: 40 → revenue A 30, B 40; A chiếm 3/7 ≈ 0.4286 tổng doanh thu

NGỘ NHẬN THƯỜNG GẶP: "Muốn giá trung bình thì lấy mean của cột giá"

mean(unit_price) coi mọi đơn nặng như nhau. Giá bình quân thật là tổng doanh thu / tổng số lượng — ratio of sums, không phải mean of ratios. Tương tự, trung bình của các trung bình theo vùng không bằng trung bình chung khi các vùng có cỡ khác nhau. Mỗi tỷ lệ phải nói rõ tử số và mẫu số.

CÂU DỰ ĐOÁN

Vùng north có hai đơn: 1 cái giá 10 và 3 cái giá 20. Giá bình quân của vùng là?

sai	15 — trung bình của hai giá	Đó là mean các giá: coi đơn 1 cái nặng bằng đơn 3 cái.
ĐÚNG	17.5 — tổng doanh thu 70 chia tổng số lượng 4	Giá bình quân có trọng số theo số lượng.
sai	35 — doanh thu chia số đơn	Đó là doanh thu trung bình mỗi ĐƠN, không phải giá mỗi sản phẩm.

TỰ KIỂM TRA

1. Nhóm có 3 dòng, một dòng score thiếu. size() và count() lần lượt cho?
   - 3 và 3 — count không đếm NaN.
   - 2 và 3 — Ngược lại.

✔ 3 và 2 — size đếm dòng; count bỏ ô thiếu.

Đáp án: size đếm dòng; count bỏ ô thiếu.

2. agg và transform khác nhau thế nào?

• transform chỉ dùng cho cột chữ — transform dùng cho mọi phép tính theo nhóm.

• Giống nhau — Hình dạng kết quả khác nhau.

✔ agg một dòng mỗi nhóm; transform trả về đúng số dòng của bảng gốc — transform dùng để gắn chỉ số nhóm vào từng dòng (vd. tỷ trọng trong nhóm).

Đáp án: transform dùng để gắn chỉ số nhóm vào từng dòng (vd. tỷ trọng trong nhóm).

3. Có dòng region bị thiếu. df.groupby("region")["revenue"].sum().sum() so với df["revenue"].sum()?

• Lớn hơn — groupby không nhân dòng.

• Luôn bằng nhau — Dòng khoá thiếu không thuộc nhóm nào nên bị bỏ.

✔ Nhỏ hơn — nhóm khoá thiếu bị bỏ (dropna=True mặc định) — Gom thành "unknown" hoặc dropna=False, rồi đối chiếu tổng.

Đáp án: Gom thành "unknown" hoặc dropna=False, rồi đối chiếu tổng.

TỰ GIẢI THÍCH

Vì sao avg_price = revenue / qty của vùng mà không dùng mean(unit_price)?

Mẫu trả lời: Mỗi đơn có số lượng khác nhau: đơn 3 cái giá 20 đóng góp nhiều hơn đơn 1 cái giá 10. Tổng doanh thu chia tổng số lượng cân theo số lượng (17.5); mean các giá coi hai đơn như nhau (15). KPI phải nêu mẫu số — ở đây là tổng số sản phẩm đã bán.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	region.fillna("unknown") → không nhóm nào bị bỏ.
Bước 2	("order_id", "size") → đếm mọi dòng đơn của vùng.
Bước 3	revenue / qty theo vùng → giá bình quân có trọng số.
Bước 4	revenue / revenue.sum() → tỷ trọng, cộng lại bằng 1.
Bước 5	tổng các vùng = tổng nguồn → không mất dòng.

CÂU PHẢN VÍ DỤ

south: (2 cái, 15), (qty thiếu, 15), (2 cái, 30). orders và avg_price của south?

Lựa chọn: 2 và 22.5 | 3 và 20.0 | 3 và 22.5 · Đáp án: 3 và 22.5

TEMPLATE LẮP MẢNH

```
import pandas as pd


def region_kpi(df):
    data = df.assign(
        region={{slot1}},
        revenue=df["qty"] * df["unit_price"],
    )
    out = data.groupby("region").agg(
        orders={{slot2}},
        qty=("qty", "sum"),
        revenue=("revenue", "sum"),
    )
    out["avg_price"] = {{slot3}}
    out["share"] = {{slot4}}
    lost = data["revenue"].sum() - out["revenue"].sum()
    assert abs(lost) < 1e-9, "Tổng theo vùng lệch tổng nguồn: có dòng bị mất khi groupby"
    return out
```

Slot	Mảnh đúng
[Slot 1]	`df["region"].fillna("unknown")`
[Slot 2]	`("order_id", "size")`
[Slot 3]	`out["revenue"] / out["qty"]`
[Slot 4]	`out["revenue"] / out["revenue"].sum()`

Mảnh bẫy	Vì sao sai
`data.groupby("region")["unit_price"].mean()`	Mean các giá — không cân theo số lượng.
`("qty", "count")`	count bỏ đơn thiếu qty — đếm hụt đơn.
`df["region"]`	Vùng thiếu bị groupby bỏ — tổng lệch tổng nguồn.

`out["revenue"] / len(out)`	Mẫu số là số vùng, không phải tổng doanh thu.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: một đơn thiếu qty, một đơn thiếu region. Lắp bốn chỗ.

Yêu cầu thêm: Thêm một đơn thiếu qty và một đơn thiếu region; kiểm orders, avg_price, share và đối chiếu tổng.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Bảng đơn (order_id, region, qty, unit_price; region có thể thiếu, qty có thể thiếu). Viết region_kpi(df) trả DataFrame index là region (region thiếu gom thành "unknown"), đúng thứ tự cột: orders (số DÒNG đơn của vùng — kể cả đơn thiếu qty), qty (tổng), revenue (tổng qty*unit_price), avg_price = revenue / qty của vùng, share = revenue vùng / TỔNG revenue mọi vùng. Trước khi trả, kiểm tổng revenue các vùng khớp tổng revenue của bảng nguồn.

Thực hành: assign region đã gom thiếu và revenue; named aggregation với size/sum; avg_price và share là phép chia hai cột tổng; assert tổng khớp.

Gọi	Kết quả mong đợi
`Vùng A: 10, 20; B: 40 (qty 1)`	`revenue A 30, B 40; share A ≈ 0.4286`
`north: (1, 10), (3, 20)`	`avg_price north = 17.5`
`Đơn thiếu region`	`Vào nhóm "unknown"; tổng khớp tổng nguồn`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`region_kpi(df)`	KPI theo vùng với mẫu số rõ ràng. · df: DataFrame order_id, region, qty, unit_price (region/qty có thể thiếu) · DataFrame index region; cột orders, qty, revenue, avg_price, share · Không sửa df; tổng lệch → AssertionError.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: A = 30, B = 40	`region_kpi(pd.DataFrame({"order_id": ["1", "2", "3"], "region": ["A", "A", "B"], "qty": [1, 1, 1], "unit_price": [10, 20, 40]}))["revenue"].to_dict()`	`{'A': 30, 'B': 40}`
Đề mẫu: A chiếm 3/7 tổng	`round(float(region_kpi(pd.DataFrame({"order_id": ["1", "2", "3"], "region": ["A", "A", "B"], "qty": [1, 1, 1], "unit_price": [10, 20, 40]})).loc["A", "share"]), 4)`	`0.4286`
Doanh thu theo vùng, vùng thiếu → "unknown"	`region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]}))["revenue"].to_dict()`	`{'north': 70.0, 'south': 90.0, 'unknown': 5.0}`
orders đếm dòng (size), kể cả đơn thiếu qty	`region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]}))["orders"].to_dict()`	`{'north': 2, 'south': 3, 'unknown': 1}`
avg_price = tổng revenue / tổng qty	`region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]}))["avg_price"].round(2).to_dict()`	`{'north': 17.5, 'south': 22.5, 'unknown': 5.0}`
share có mẫu số là tổng doanh thu	`region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]}))["share"].round(4).to_dict()`	`{'north': 0.4242, 'south': 0.5455, 'unknown': 0.0303}`
Tổng các vùng khớp tổng nguồn	`round(float(region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]}))["revenue"].sum()), 6)`	`165.0`
Đúng thứ tự cột	`region_kpi(pd.DataFrame({"order_id": ["C1", "C2", "C3", "C4", "C5", "C6"], "region": ["north", "north", "south", "south", "south", None], "qty": [1, 3, 2, None, 2, 1], "unit_price": [10.0, 20.0, 15.0, 15.0, 30.0, 5.0]})).columns.tolist()`	`['orders', 'qty', 'revenue', 'avg_price', 'share']`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. region.fillna("unknown") → không nhóm nào bị bỏ.
Khung 2 — Bước 2. ("order_id", "size") → đếm mọi dòng đơn của vùng.
Khung 3 — Bước 3. revenue / qty theo vùng → giá bình quân có trọng số.
Khung 4 — Bước 4. revenue / revenue.sum() → tỷ trọng, cộng lại bằng 1.
Khung 5 — Bước 5. tổng các vùng = tổng nguồn → không mất dòng.
Khung đối chiếu — ngộ nhận "Muốn giá trung bình thì lấy mean của cột giá". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. mean(unit_price) coi mọi đơn nặng như nhau. Giá bình quân thật là tổng doanh thu / tổng số lượng — ratio of sums, không phải mean of ratios. Tương tự, trung bình của các trung bình theo vùng không bằng trung bình chung khi các vùng có cỡ khác nhau. Mỗi tỷ lệ phải nói rõ tử số và mẫu số.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: data.groupby("region")["unit_price"].mean(), ("qty", "count"), df["region"], out["revenue"] / len(out)
Khung kết quả — 3 cặp vào/ra. Vùng A: 10, 20; B: 40 (qty 1) → revenue A 30, B 40; share A ≈ 0.4286; north: (1, 10), (3, 20) → avg_price north = 17.5; Đơn thiếu region → Vào nhóm "unknown"; tổng khớp tổng nguồn

Nguồn: pe-v4 · python-learning-data.json#PD05 · parser pe-v4/build · file build 036-PD05.json

PD06 · Merge, concat và kiểm tra cardinality

Mã bài: PD06 · Nâng cao · M6 · pandas · 40 phút · 20 XP · thứ tự 37

MỤC TIÊU

Ghép bảng tra cứu không làm rơi hay nhân dòng.

CẦN NHỚ TRƯỚC

PD05

BỐI CẢNH

Viết attach_prices(orders, prices): orders (order_id, sku, qty) ghép bảng giá prices (sku, price) bằng LEFT merge theo sku, validate="many_to_one" (bảng giá trùng sku phải báo MergeError chứ không nhân dòng) và indicator=True (cột _merge). Kết quả có đúng len(orders) dòng, đúng thứ tự đơn. Viết stack(frames): chồng list DataFrame bằng concat với index mới 0..n-1; nếu một bảng không cùng tập cột với bảng đầu thì raise ValueError.

VÍ DỤ MINH HOẠ

orders sku A, B; bảng giá chỉ có A → 2 dòng, đơn B có price NaN và _merge = "left_only".

ĐIỂM KIẾN THỨC

1. merge ghép theo cột khoá (on=...); how chọn dòng giữ lại: inner (MẶC ĐỊNH) chỉ khoá có ở hai bên, left mọi dòng bên trái, outer tất cả.
2. Gắn thông tin từ bảng tra cứu (giá, phòng học) vào bảng giao dịch: how="left" để không đơn nào biến mất; dòng không ghép được có NaN.
3. Khoá lặp ở bảng tra cứu NHÂN dòng: một đơn ghép với hai giá thành hai dòng, doanh thu bị đếm đôi. validate="many_to_one" biến kỳ vọng "mỗi sku một giá" thành phép kiểm: sai là MergeError.
4. indicator=True thêm cột _merge (both / left_only / right_only) để đếm dòng không ghép được. Khác SQL, pandas ghép NaN với NaN — khoá thiếu cần chính sách riêng.
5. concat chồng các bảng cùng schema. ignore_index=True đánh lại index 0..n-1; không có thì index cũ lặp lại (0, 1, 0). Cột lệch tên thì concat không báo lỗi mà điền NaN — tự kiểm schema trước.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

students = pd.DataFrame({"sid": ["S1", "S2", "S3"], "class_id": ["K1", "K2", "K9"]})
classes = pd.DataFrame({"class_id": ["K1", "K2"], "room": ["A101", "B204"]})
out = students.merge(classes, on="class_id", how="left", validate="many_to_one", indicator=True)
print(out[["sid", "room", "_merge"]])
print(len(students), "->", len(out))

dup = pd.concat([classes, classes.head(1)], ignore_index=True)
print("bảng tra trùng K1:", len(students.merge(dup, on="class_id", how="left")), "dòng")

a = pd.DataFrame({"x": [1, 2]})
b = pd.DataFrame({"x": [3]})
print(pd.concat([a, b]).index.tolist(), pd.concat([a, b], ignore_index=True).index.tolist())
```

Kết quả dự kiến (theo JSON): sku A, B; giá chỉ có A → 2 dòng, 1 thiếu giá; Bảng giá trùng sku → MergeError

NGỘ NHẬN THƯỜNG GẶP: "merge xong thì số dòng vẫn như bảng đơn"

Không có gì bảo đảm điều đó: inner (mặc định) làm RƠI đơn không có giá, khoá lặp ở bảng giá làm NHÂN đơn. Chọn how="left", khai validate="many_to_one" và so len trước/sau merge.

CÂU DỰ ĐOÁN

orders có 3 đơn sku PEN; bảng giá có PEN hai lần (4.0 và 4.5). orders.merge(prices, on="sku") ra mấy dòng?

sai	3 — merge giữ số dòng bên trái	Chỉ đúng khi khoá bên phải duy nhất.
sai	MergeError	Chỉ khi có validate="many_to_one".
ĐÚNG	6 — mỗi đơn ghép với cả hai giá	Khoá lặp bên phải nhân số dòng.

TỰ KIỂM TRA

1. orders.merge(prices, on="sku") không ghi how. Đơn có sku không có trong bảng giá sẽ...

✔ Biến mất — mặc định how="inner" — Muốn giữ thì how="left".

• Có price = NaN — Đó là kết quả của how="left".

• Gây lỗi — Không lỗi — chỉ rơi dòng âm thầm.

Đáp án: Muốn giữ thì how="left".

2. pd.concat([a, b]) với a có index 0, 1 và b có index 0. Index kết quả là?

✔ [0, 1, 0] — Giữ index cũ; dùng ignore_index=True để thành [0, 1, 2].

• [0, 1, 2] — Chỉ khi ignore_index=True.

• Lỗi trùng index — concat không kiểm trùng index.

Đáp án: Giữ index cũ; dùng ignore_index=True để thành [0, 1, 2].

3. validate="many_to_one" kiểm điều gì?

✔ Khoá ở bảng PHẢI là duy nhất (mỗi sku một giá) — Bên trái được lặp (nhiều đơn cùng sku).

• Mọi đơn đều tìm được giá — Dòng không ghép được kiểm bằng indicator=True.

• Bảng trái không có khoá trùng — Đó là "one_to_many" hoặc "one_to_one".

Đáp án: Bên trái được lặp (nhiều đơn cùng sku).

TỰ GIẢI THÍCH

Vì sao attach_prices dùng how="left" và validate="many_to_one" cùng lúc?

Mẫu trả lời: how="left" bảo đảm không đơn nào rơi mất khi bảng giá thiếu sku — đơn đó có price NaN và _merge = left_only để đếm. validate="many_to_one" bảo đảm không đơn nào bị nhân lên khi bảng giá trùng sku — vi phạm thì MergeError ngay. Cả hai cùng giữ số dòng kết quả đúng bằng số đơn.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	how="left" → giữ mọi đơn, đơn thiếu giá có NaN.
Bước 2	validate="many_to_one" → bảng giá trùng sku thì MergeError.
Bước 3	indicator=True → cột _merge: both / left_only.
Bước 4	set(frame.columns) → so schema trước khi chồng.
Bước 5	concat(..., ignore_index=True) → index 0..n-1.

CÂU PHẢN VÍ DỤ

orders sku [PEN, CUP, PEN], bảng giá chỉ có PEN. Số dòng left_only?

Lựa chọn: 2 | 0 | 1 · Đáp án: 1

TEMPLATE LẮP MẢNH

```
import pandas as pd


def attach_prices(orders, prices):
    return orders.merge(
        prices,
        on="sku",
        {{slot1}},
        {{slot2}},
        indicator=True,
    )


def stack(frames):
    columns = set(frames[0].columns)
    for frame in frames[1:]:
        if set(frame.columns) != columns:
            raise ValueError("Các bảng không cùng schema")
    return {{slot3}}
```

Slot	Mảnh đúng
[Slot 1]	`how="left"`
[Slot 2]	`validate="many_to_one"`
[Slot 3]	`pd.concat(frames, ignore_index=True)`

Mảnh bẫy	Vì sao sai
`validate="one_to_many"`	Kiểm sai phía: bảng giá mới phải là phía "một".
`how="inner"`	Đơn không có giá bị rơi mất.
`pd.concat(frames)`	Giữ index cũ — 0, 1, 0.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: một sku không có giá, bảng giá có thể trùng sku. Lắp ba chỗ.

Yêu cầu thêm: Thêm sku không có giá và một bảng giá bị trùng sku; kiểm số dòng, _merge và lỗi MergeError.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết attach_prices(orders, prices): orders (order_id, sku, qty) ghép bảng giá prices (sku, price) bằng LEFT merge theo sku, validate="many_to_one" (bảng giá trùng sku phải báo MergeError chứ không nhân dòng) và indicator=True (cột _merge). Kết quả có đúng len(orders) dòng, đúng thứ tự đơn. Viết stack(frames): chồng list DataFrame bằng concat với index mới 0..n-1; nếu một bảng không cùng tập cột với bảng đầu thì raise ValueError.

Thực hành: Left merge + validate + indicator; so tập cột trước concat; ignore_index=True.

Gọi	Kết quả mong đợi
`orders D1–D4 (PEN, BOOK, PEN, CUP), giá PEN, BOOK`	`4 dòng; D4 left_only, price NaN`
`Bảng giá trùng PEN`	`MergeError`
`stack([x 1, 2], [x 3])`	`x [1, 2, 3], index [0, 1, 2]`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`attach_prices(orders, prices)`	Gắn giá vào từng đơn, không rơi và không nhân dòng. · orders: DataFrame có sku; prices: DataFrame sku, price (mỗi sku một dòng) · DataFrame len(orders) dòng, thêm price và _merge · Bảng giá trùng sku → MergeError.
`stack(frames)`	Chồng các bảng cùng schema. · frames: list DataFrame (ít nhất một) · DataFrame, index 0..n-1 · Tập cột khác nhau → ValueError.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: giữ cả hai đơn, một đơn thiếu giá	`(lambda o: (len(o), int(o["price"].isna().sum())))(attach_prices(pd.DataFrame({"sku": ["A", "B"]}), pd.DataFrame({"sku": ["A"], "price": [10]})))`	`(2, 1)`
Giữ đủ mọi đơn (left), đúng thứ tự	`attach_prices(pd.DataFrame({"order_id": ["D1", "D2", "D3", "D4"], "sku": ["PEN", "BOOK", "PEN", "CUP"], "qty": [2, 1, 5, 3]}), pd.DataFrame({"sku": ["PEN", "BOOK"], "price": [4.0, 12.0]}))["order_id"].tolist()`	`['D1', 'D2', 'D3', 'D4']`
Đơn CUP không có giá → left_only	`attach_prices(pd.DataFrame({"order_id": ["D1", "D2", "D3", "D4"], "sku": ["PEN", "BOOK", "PEN", "CUP"], "qty": [2, 1, 5, 3]}), pd.DataFrame({"sku": ["PEN", "BOOK"], "price": [4.0, 12.0]}))["_merge"].astype(str).tolist()`	`['both', 'both', 'both', 'left_only']`
Giá gắn đúng đơn	`attach_prices(pd.DataFrame({"order_id": ["D1", "D2", "D3", "D4"], "sku": ["PEN", "BOOK", "PEN", "CUP"], "qty": [2, 1, 5, 3]}), pd.DataFrame({"sku": ["PEN", "BOOK"], "price": [4.0, 12.0]}))["price"].fillna(-1).tolist()`	`[4.0, 12.0, 4.0, -1.0]`
Bảng giá trùng sku → MergeError, không nhân dòng	`_studio.call(attach_prices, pd.DataFrame({"order_id": ["D1", "D2", "D3", "D4"], "sku": ["PEN", "BOOK", "PEN", "CUP"], "qty": [2, 1, 5, 3]}), pd.DataFrame({"sku": ["PEN", "PEN", "BOOK"], "price": [4.0, 4.5, 12.0]})).error`	`'MergeError'`
stack: index mới 0..n-1	`stack([pd.DataFrame({"x": [1, 2]}), pd.DataFrame({"x": [3]})]).index.tolist()`	`[0, 1, 2]`
stack: cùng cột khác thứ tự vẫn chồng đúng	`stack([pd.DataFrame({"a": [1], "b": [2]}), pd.DataFrame({"b": [4], "a": [3]})])["a"].tolist()`	`[1, 3]`
stack: khác schema → ValueError	`_studio.call(stack, [pd.DataFrame({"x": [1]}), pd.DataFrame({"y": [2]})]).error`	`'ValueError'`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. how="left" → giữ mọi đơn, đơn thiếu giá có NaN.
Khung 2 — Bước 2. validate="many_to_one" → bảng giá trùng sku thì MergeError.
Khung 3 — Bước 3. indicator=True → cột _merge: both / left_only.
Khung 4 — Bước 4. set(frame.columns) → so schema trước khi chồng.
Khung 5 — Bước 5. concat(..., ignore_index=True) → index 0..n-1.
Khung đối chiếu — ngộ nhận "merge xong thì số dòng vẫn như bảng đơn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Không có gì bảo đảm điều đó: inner (mặc định) làm RƠI đơn không có giá, khoá lặp ở bảng giá làm NHÂN đơn. Chọn how="left", khai validate="many_to_one" và so len trước/sau merge.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: validate="one_to_many", how="inner", pd.concat(frames)
Khung kết quả — 3 cặp vào/ra. orders D1–D4 (PEN, BOOK, PEN, CUP), giá PEN, BOOK → 4 dòng; D4 left_only, price NaN; Bảng giá trùng PEN → MergeError; stack([x 1, 2], [x 3]) → x [1, 2, 3], index [0, 1, 2]

Nguồn: pe-v4 · python-learning-data.json#PD06 · parser pe-v4/build · file build 037-PD06.json

PD07 · Long, wide, pivot và melt

Mã bài: PD07 · Nâng cao · M6 · pandas · 35 phút · 20 XP · thứ tự 38

MỤC TIÊU

Đổi long ↔ wide mà không đổi tổng.

CẦN NHỚ TRƯỚC

PD06

BỐI CẢNH

Viết to_long(wide): wide có cột region và mỗi tháng một cột ("2026-01", "2026-02"…); trả bảng long ba cột region, month, revenue (melt với id_vars="region"). Viết to_wide(long): long có thể có NHIỀU dòng cùng (region, month) — trả bảng wide index region, cột là month, ô là TỔNG revenue (pivot_table với aggfunc="sum"); ô không có dữ liệu để NaN, KHÔNG điền 0.

VÍ DỤ MINH HOẠ

id 1 với A = 5, B = 8 → melt ra 2 dòng → pivot lại vẫn A = 5, B = 8. Hai dòng (hanoi, 2026-01) 30 và 20 → ô = 50.

ĐIỂM KIẾN THỨC

1. Long: mỗi dòng MỘT quan sát (vùng, tháng, doanh thu). Wide: mỗi tháng một cột. Cùng dữ liệu, hai hình dạng; groupby và vẽ theo nhóm thường cần long, bảng báo cáo thường là wide.
2. melt(id_vars=..., var_name=..., value_name=...) đổi wide → long: cột id giữ nguyên, các cột còn lại thành cặp (tên cột, giá trị). n hàng × k cột tháng → n·k dòng.
3. pivot(index, columns, values) đổi long → wide nhưng ĐÒI mỗi cặp (index, columns) duy nhất; trùng thì ValueError "Index contains duplicate entries".
4. pivot_table gộp các dòng trùng bằng aggfunc. Mặc định aggfunc là mean — muốn tổng doanh thu phải ghi aggfunc="sum".
5. fill_value=0 chỉ đúng khi ô trống thật sự nghĩa là "không có giao dịch". Chưa biết là không bán hay chưa thu thập thì để NaN. Đổi hình dạng không được làm đổi tổng: so sum trước và sau.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

wide = pd.DataFrame({"sid": ["S1", "S2"], "math": [7, 9], "art": [8, 6]})
long = wide.melt(id_vars="sid", var_name="subject", value_name="mark")
print(long)
print(len(wide), "->", len(long), "dòng; tổng", long["mark"].sum())

back = long.pivot(index="sid", columns="subject", values="mark")
print(back.loc["S2", "math"])

log = pd.DataFrame({"sid": ["S1", "S1"], "subject": ["math", "math"], "mark": [6, 8]})
print(log.pivot_table(index="sid", columns="subject", values="mark", aggfunc="max"))
print(log.pivot_table(index="sid", columns="subject", values="mark"))
```

Kết quả dự kiến (theo JSON): id 1: A = 5, B = 8 → melt → pivot lại A = 5, B = 8; Trùng (vùng, tháng) 30 + 20 → ô = 50

NGỘ NHẬN THƯỜNG GẶP: "pivot và pivot_table là một, cái nào cũng được"

pivot chỉ XẾP lại, không gộp: gặp (vùng, tháng) trùng là ValueError. pivot_table GỘP, nhưng mặc định bằng mean — tổng doanh thu lặng lẽ thành trung bình. Biết grain của dữ liệu trước, rồi chọn pivot (khoá duy nhất) hoặc pivot_table với aggfunc ghi rõ.

CÂU DỰ ĐOÁN

Bảng long có hai dòng (hanoi, 2026-01) doanh thu 30 và 20. Ô (hanoi, 2026-01) của bảng dưới bằng bao nhiêu?

```
long.pivot_table(index="region", columns="month", values="revenue")
```

sai	ValueError vì trùng khoá	Đó là pivot; pivot_table thì gộp.
ĐÚNG	25 — không ghi aggfunc thì mặc định là mean	Ghi aggfunc="sum" để được 50.
sai	50	Chỉ khi aggfunc="sum".

TỰ KIỂM TRA

1. wide có 3 vùng và 4 cột tháng. melt(id_vars="region") ra mấy dòng?

- 7 — melt nhân, không cộng.
- 4 — Mỗi vùng cho 4 dòng, có 3 vùng.
- ✔ 12 — 3 vùng × 4 tháng.

Đáp án: 3 vùng × 4 tháng.

2. Khi nào fill_value=0 trong pivot_table là hợp lý?

• Luôn luôn, để bảng đẹp — 0 là một giá trị: kéo lệch trung bình và che mất dữ liệu thiếu.

✔ Khi ô trống chắc chắn nghĩa là "không có giao dịch" — Chưa chắc thì để NaN.

• Không bao giờ — Có lúc 0 đúng nghĩa: đã ghi nhận đầy đủ và không bán.

Đáp án: Chưa chắc thì để NaN.

3. melt(var_name="month", value_name="revenue") quên id_vars="region" thì...

✔ Cột region cũng bị biến thành giá trị trong cột month — id_vars giữ cột định danh nguyên vẹn.

• pandas tự giữ region — Không có id_vars thì mọi cột đều bị melt.

• Lỗi — Không lỗi — chỉ ra bảng sai.

Đáp án: id_vars giữ cột định danh nguyên vẹn.

TỰ GIẢI THÍCH

Vì sao to_wide dùng pivot_table(aggfunc="sum") mà không dùng pivot?

Mẫu trả lời: Dữ liệu long có thể có nhiều dòng cùng (vùng, tháng) vì mỗi dòng là một đơn. pivot đòi cặp khoá duy nhất nên báo ValueError; pivot_table gộp các dòng đó, và phải ghi aggfunc="sum" vì mặc định là mean. Ô không có đơn để NaN vì chưa biết là không bán hay thiếu dữ liệu.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	melt(id_vars="region") → mỗi (vùng, tháng) một dòng.
Bước 2	var_name / value_name → đặt tên cột month, revenue.
Bước 3	pivot_table(..., aggfunc="sum") → gộp đơn trùng ô.
Bước 4	Không fill_value → ô chưa có dữ liệu là NaN.
Bước 5	sum() trước = sum() sau → hình dạng đổi, tổng giữ nguyên.

CÂU PHẢN VÍ DỤ

long có (hue, 2026-03) = 25 và (hue, 2026-03) = 5. to_wide(long).loc["hue", "2026-03"] = ?

Lựa chọn: 30 | ValueError | 15 · Đáp án: 30

TEMPLATE LẮP MẢNH

```
import pandas as pd


def to_long(wide):
    return wide.melt({{slot1}}, var_name="month", value_name="revenue")


def to_wide(long):
    table = long.{{slot2}}(index="region", columns="month", values="revenue", {{slot3}})
    return table
```

Slot	Mảnh đúng
[Slot 1]	`id_vars="region"`
[Slot 2]	`pivot_table`
[Slot 3]	`aggfunc="sum"`

Mảnh bẫy	Vì sao sai
`value_vars="region"`	Đó là cột bị melt, không phải cột giữ lại.
`fill_value=0`	Không ghi aggfunc thì mặc định mean; còn ô thiếu bị điền 0.
`pivot`	Không gộp được (vùng, tháng) trùng — ValueError.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: long có hai dòng cùng (hanoi, 2026-01) và vài ô không có dữ liệu. Lắp ba chỗ.

Yêu cầu thêm: Thêm dòng trùng (vùng, tháng) và một tháng chỉ một vùng có dữ liệu; kiểm ô gộp, ô NaN và tổng.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết to_long(wide): wide có cột region và mỗi tháng một cột ("2026-01", "2026-02"...); trả bảng long ba cột region, month, revenue (melt với id_vars="region"). Viết to_wide(long): long có thể có NHIỀU dòng cùng (region, month) — trả bảng wide index region, cột là month, ô là TỔNG revenue (pivot_table với aggfunc="sum"); ô không có dữ liệu để NaN, KHÔNG điền 0.

Thực hành: melt với id_vars; pivot_table với aggfunc="sum", không fill_value.

Gọi	Kết quả mong đợi
`wide hanoi 100/80, hue 40/60`	`long 4 dòng, tổng 280`
`long (hanoi, 2026-01) 30 và 20`	`Ô (hanoi, 2026-01) = 50`
`(hue, 2026-02) không có dòng nào`	`Ô đó là NaN, không phải 0`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`to_long(wide)`	Đổi bảng wide (mỗi tháng một cột) sang long. · wide: DataFrame cột region + các cột tháng · DataFrame cột region, month, revenue · Không sửa wide; tổng giữ nguyên.
`to_wide(long)`	Đổi long sang wide, gộp dòng trùng bằng tổng. · long: DataFrame region, month, revenue (có thể trùng cặp region, month) · DataFrame index region, cột month; ô thiếu là NaN · Không sửa long.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: melt rồi pivot lại vẫn A = 5, B = 8	`to_wide(to_long(pd.DataFrame({"region": ["r1"], "A": [5], "B": [8]}))).to_dict()`	`{'A': {'r1': 5}, 'B': {'r1': 8}}`
to_long: đúng ba cột	`to_long(pd.DataFrame({"region": ["hanoi", "hue"], "2026-01": [100, 40], "2026-02": [80, 60]})).columns.tolist()`	`['region', 'month', 'revenue']`
to_long: 2 vùng × 2 tháng, tổng giữ nguyên	`(lambda d: (len(d), int(d["revenue"].sum())))(to_long(pd.DataFrame({"region": ["hanoi", "hue"], "2026-01": [100, 40], "2026-02": [80, 60]})))`	`(4, 280)`
to_long: đúng từng quan sát	`to_long(pd.DataFrame({"region": ["hanoi", "hue"], "2026-01": [100, 40], "2026-02": [80, 60]})).sort_values(["region", "month"]).to_dict("records")`	`[{'region': 'hanoi', 'month': '2026-01', 'revenue': 100}, {'region': 'hanoi', 'month': '2026-02', 'revenue': 80}, {'region': 'hue', 'month': '2026-01', 'revenue': 40}, {'region': 'hue', 'month': '2026-02', 'revenue': 60}]`
Trùng (vùng, tháng) → cộng dồn	`float(to_wide(pd.DataFrame({"region": ["hanoi", "hanoi", "hue", "hanoi", "hue"], "month": ["2026-01", "2026-01", "2026-01", "2026-02", "2026-03"], "revenue": [30, 20, 15, 40, 25]})).loc["hanoi", "2026-01"])`	`50.0`
Ô không có dữ liệu giữ NaN (2 ô)	`int(to_wide(pd.DataFrame({"region": ["hanoi", "hanoi", "hue", "hanoi", "hue"], "month": ["2026-01", "2026-01", "2026-01", "2026-02", "2026-03"], "revenue": [30, 20, 15, 40, 25]})).isna().sum().sum())`	`2`
to_wide giữ tổng	`float(to_wide(pd.DataFrame({"region": ["hanoi", "hanoi", "hue", "hanoi", "hue"], "month": ["2026-01", "2026-01", "2026-01", "2026-02", "2026-03"], "revenue": [30, 20, 15, 40, 25]})).sum().sum())`	`130.0`
Hàng là vùng, cột là tháng	`(lambda w: (w.index.tolist(), w.columns.tolist()))(to_wide(pd.DataFrame({"region": ["hanoi", "hanoi", "hue", "hanoi", "hue"], "month": ["2026-01", "2026-01", "2026-01", "2026-02", "2026-03"], "revenue": [30, 20, 15, 40, 25]})))`	`(['hanoi', 'hue'], ['2026-01', '2026-02', '2026-03'])`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. melt(id_vars="region") → mỗi (vùng, tháng) một dòng.
Khung 2 — Bước 2. var_name / value_name → đặt tên cột month, revenue.
Khung 3 — Bước 3. pivot_table(..., aggfunc="sum") → gộp đơn trùng ô.
Khung 4 — Bước 4. Không fill_value → ô chưa có dữ liệu là NaN.
Khung 5 — Bước 5. sum() trước = sum() sau → hình dạng đổi, tổng giữ nguyên.
Khung đối chiếu — ngộ nhận "pivot và pivot_table là một, cái nào cũng được". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. pivot chỉ XẾP lại, không gộp: gặp (vùng, tháng) trùng là ValueError. pivot_table GỘP, nhưng mặc định bằng mean — tổng doanh thu lặng lẽ thành trung bình. Biết grain của dữ liệu trước, rồi chọn pivot (khoá duy nhất) hoặc pivot_table với aggfunc ghi rõ.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: value_vars="region", fill_value=0, pivot
Khung kết quả — 3 cặp vào/ra. wide hanoi 100/80, hue 40/60 → long 4 dòng, tổng 280; long (hanoi, 2026-01) 30 và 20 → Ô (hanoi, 2026-01) = 50; (hue, 2026-02) không có dòng nào → Ô đó là NaN, không phải 0

Nguồn: pe-v4 · python-learning-data.json#PD07 · parser pe-v4/build · file build 038-PD07.json

PD08 · Ngày giờ, resample và rolling

Mã bài: PD08 · Nâng cao · M6 · pandas · 40 phút · 20 XP · thứ tự 39

MỤC TIÊU

Tổng theo tháng và trung bình trượt trên ngày đã làm sạch.

CẦN NHỚ TRƯỚC

PD07

BỐI CẢNH

Viết monthly_revenue(df): df có cột date (chuỗi "YYYY-MM-DD", có thể hỏng) và revenue. Đổi date bằng pd.to_datetime(..., format="%Y-%m-%d", errors="coerce"), bỏ dòng NaT, đặt date làm index rồi resample("MS") lấy tổng revenue mỗi tháng — tháng không có dòng nào phải là NaN (sum(min_count=1)), không phải 0. Viết rolling_mean(s, window): s là Series có DatetimeIndex có thể chưa sắp xếp; trả trung bình trượt window quan sát trên chuỗi đã sort_index().

VÍ DỤ MINH HOẠ

Ngày 1/1 và 2/1 doanh thu 5 và 15 → tháng 1 = 20; rolling(2) ở ngày 2/1 = 10.

ĐIỂM KIẾN THỨC

1. pd.to_datetime(..., errors="coerce") biến chuỗi không phải ngày thành NaT (ngày thiếu) để đếm và loại; không có errors="coerce" thì một ô "bad" làm cả lệnh ValueError.
2. Ghi format="%Y-%m-%d" khi dữ liệu dễ nhập nhằng: "03/02/2026" là 3/2 hay 2/3? Có format thì chuỗi sai định dạng thành NaT thay vì bị đoán bừa.
3. resample cần DatetimeIndex: set_index("date") rồi resample("MS") gom theo tháng, nhãn là NGÀY ĐẦU tháng (MS = month start; "ME" là cuối tháng).
4. Tháng không có dòng nào: sum() trả 0, sum(min_count=1) trả NaN. Chưa biết là không bán hay chưa thu thập thì đừng để nó thành 0.
5. rolling(window) tính trên các quan sát LIỀN NHAU theo thứ tự hiện tại — chưa sort_index() thì "liền nhau" sai. min_periods quyết định cửa sổ thiếu quan sát ra NaN hay vẫn tính; rolling("7D") là cửa sổ 7 NGÀY, khác cửa sổ 7 dòng.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

raw = pd.Series(["2026-02-03", "03/02/2026", "oops"])
dates = pd.to_datetime(raw, format="%Y-%m-%d", errors="coerce")
print(dates.tolist())
print(int(dates.isna().sum()), "ngày không đọc được")

visits = pd.Series([3, 5, 2], index=pd.to_datetime(["2026-01-10", "2026-01-25", "2026-03-04"]))
print(visits.resample("MS").sum().tolist())
print(visits.resample("MS").sum(min_count=1).tolist())

steps = pd.Series([4, 6, 8, 2])
print(steps.rolling(2).mean().tolist())
print(steps.rolling(2, min_periods=1).mean().tolist())
```

Kết quả dự kiến (theo JSON): 1/1: 5, 2/1: 15 → tháng 1 = 20; rolling(2) = 10; Tháng 2 trống → NaN, không phải 0

NGỘ NHẬN THƯỜNG GẶP: "Tháng không có số liệu thì doanh thu là 0"

Có thể là không bán, cũng có thể hệ thống chưa ghi nhận. resample().sum() điền 0 âm thầm, kéo trung bình trượt xuống và vẽ ra một tháng "sụt giảm" không có thật. Để NaN bằng sum(min_count=1) rồi quyết định có lý do.

CÂU DỰ ĐOÁN

Doanh thu có ngày 10/1 và 4/3, không có ngày nào trong tháng 2. resample("MS").sum() cho tháng 2 bao nhiêu?

sai	Tháng 2 không xuất hiện	resample tạo đủ mọi tháng giữa đầu và cuối.
ĐÚNG	0 — sum của nhóm rỗng là 0	Dùng sum(min_count=1) để thành NaN.
sai	NaN	Chỉ khi sum(min_count=1).

TỰ KIỂM TRA

1. s.rolling(2).mean() khi s chưa sắp theo ngày thì...

- ✔ Tính trên hai dòng liền nhau theo thứ tự hiện tại — sai nghĩa thời gian — Gọi sort_index() trước.
- • Lỗi — Không lỗi với cửa sổ đếm dòng.
- • pandas tự sắp theo index — rolling không sắp lại dữ liệu.

Đáp án: Gọi sort_index() trước.

2. pd.to_datetime(pd.Series(["2026-01-01", "bad"])) không có errors=... thì...

• [2026-01-01, NaT] — Chỉ khi errors="coerce".

✔ ValueError ở "bad" — errors="coerce" để thành NaT.

• Bỏ qua "bad" — pandas không tự bỏ dòng.

Đáp án: errors="coerce" để thành NaT.

3. pd.Series([2, 4, 8]).rolling(2).mean().tolist() = ?

• [3.0, 6.0] — rolling giữ nguyên số dòng.

✔ [nan, 3.0, 6.0] — Cửa sổ đầu chỉ có một quan sát, chưa đủ 2 → NaN.

• [2.0, 3.0, 6.0] — Chỉ khi min_periods=1.

Đáp án: Cửa sổ đầu chỉ có một quan sát, chưa đủ 2 → NaN.

TỰ GIẢI THÍCH

Vì sao phải sort_index() trước rolling, còn resample thì không cần?

Mẫu trả lời: resample gom từng dòng vào thùng tháng theo GIÁ TRỊ ngày nên thứ tự dòng không đổi kết quả. rolling lấy các dòng LIỀN NHAU theo thứ tự hiện tại; chuỗi chưa sắp xếp thì cửa sổ ghép những ngày không kề nhau và trung bình trượt sai.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	to_datetime(errors="coerce") → "bad", "2026-13-01" thành NaT.
Bước 2	dropna(subset=["date"]) → bỏ dòng ngày hỏng.
Bước 3	set_index("date") → DatetimeIndex để resample.
Bước 4	resample("MS").sum(min_count=1) → tháng trống là NaN.
Bước 5	sort_index().rolling(2) → cửa sổ hai ngày liền nhau.

CÂU PHẢN VÍ DỤ

Ngày [2026-01-05, bad, 2026-02-10], revenue [10, 99, 30]. monthly_revenue(...).tolist() = ?

Lựa chọn: ValueError | [109.0, 30.0] | [10.0, 30.0] · Đáp án: [10.0, 30.0]

TEMPLATE LẮP MẢNH

```
import pandas as pd


def monthly_revenue(df):
    data = df.copy()
    data["date"] = pd.to_datetime(data["date"], format="%Y-%m-%d", {{slot1}})
    data = data.dropna(subset=["date"])
    monthly = data.set_index("date")["revenue"].resample({{slot2}})
    return {{slot3}}


def rolling_mean(s, window):
    return {{slot4}}
```

Slot	Mảnh đúng
[Slot 1]	`errors="coerce"`
[Slot 2]	`"MS"`
[Slot 3]	`monthly.sum(min_count=1)`
[Slot 4]	`s.sort_index().rolling(window).mean()`

Mảnh bẫy	Vì sao sai
`monthly.sum()`	Tháng không có dữ liệu thành 0.
`s.rolling(window).mean()`	Chưa sắp theo ngày.
`errors="raise"`	Một ô "bad" làm cả lệnh ValueError.
`"ME"`	Nhãn cuối tháng; đề yêu cầu đầu tháng.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: ngày lộn xộn, có "bad" và "2026-13-01", tháng 2 không có đơn. Lắp bốn chỗ.

Yêu cầu thêm: Thử ngày lộn xộn, có ngày hỏng và một tháng không có đơn; kiểm tổng tháng, tháng trống và rolling.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết monthly_revenue(df): df có cột date (chuỗi "YYYY-MM-DD", có thể hỏng) và revenue. Đổi date bằng pd.to_datetime(..., format="%Y-%m-%d", errors="coerce"), bỏ dòng NaT, đặt date làm index rồi resample("MS")

lấy tổng revenue mỗi tháng — tháng không có dòng nào phải là NaN (sum(min_count=1)), không phải 0. Viết rolling_mean(s, window): s là Series có DatetimeIndex có thể chưa sắp xếp; trả trung bình trượt window quan sát trên chuỗi đã sort_index().

Thực hành: to_datetime(errors="coerce") → dropna → set_index → resample("MS") → sum(min_count=1); rolling sau sort_index().

Gọi	Kết quả mong đợi
`1/1: 5, 2/1: 15`	`Tháng 1 = 20; rolling(2) ngày 2/1 = 10`
`Có "bad", "2026-13-01"`	`Hai dòng đó bị loại, không lỗi`
`Tháng 2 không có đơn`	`Tháng 2 = NaN, không phải 0`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`monthly_revenue(df)`	Tổng revenue theo tháng, bỏ ngày hỏng. · df: DataFrame cột date (chuỗi "YYYY-MM-DD", có thể hỏng), revenue · Series index là ngày đầu tháng; tháng trống là NaN · Không sửa df.
`rolling_mean(s, window)`	Trung bình trượt theo thứ tự ngày. · s: Series có DatetimeIndex (có thể chưa sắp xếp); window: int · Series đã sắp theo ngày; window − 1 giá trị đầu là NaN · Không sửa s.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Đề mẫu: 1/1 và 2/1 doanh thu 5, 15 → tháng 1 = 20	`monthly_revenue(pd.DataFrame({"date": ["2026-01-01", "2026-01-02"], "revenue": [5, 15]})).tolist()`	`[20.0]`
Đề mẫu: rolling(2) ở ngày cuối = 10	`float(rolling_mean(pd.Series([5, 15], index=pd.to_datetime(["2026-01-01", "2026-01-02"])), 2).iloc[-1])`	`10.0`
Ngày hỏng bị loại, không làm lỗi	`float(monthly_revenue(pd.DataFrame({"date": ["2026-03-02", "2026-01-05", "bad", "2026-01-20", "2026-03-30", "2026-13-01"], "revenue": [40, 10, 99, 20, 5, 7]})).sum())`	`75.0`
Tháng 2 không có dữ liệu → NaN, không phải 0	`monthly_revenue(pd.DataFrame({"date": ["2026-03-02", "2026-01-05", "bad", "2026-01-20", "2026-03-30", "2026-13-01"], "revenue": [40, 10, 99, 20, 5, 7]})).fillna(-1).tolist()`	`[30.0, -1.0, 45.0]`
Index là ngày đầu tháng (MS)	`[d.strftime("%Y-%m-%d") for d in monthly_revenue(pd.DataFrame({"date": ["2026-03-02", "2026-01-05", "bad", "2026-01-20", "2026-03-30", "2026-13-01"], "revenue": [40, 10, 99, 20, 5, 7]})).index]`	`['2026-01-01', '2026-02-01', '2026-03-01']`
sales.csv (bỏ trùng, qty > 0): tháng 1 = 110, tháng 2 = 45	`monthly_revenue(pd.read_csv("sales.csv").drop_duplicates(subset="order_id").query("qty > 0").assign(revenue=lambda d: d["qty"] * d["unit_price"])).tolist()`	`[110.0, 45.0]`
rolling(2) trên chuỗi chưa sắp xếp	`rolling_mean(pd.Series([8, 2, 4], index=pd.to_datetime(["2026-01-03", "2026-01-01", "2026-01-02"])), 2).fillna(-1).tolist()`	`[-1.0, 3.0, 6.0]`
rolling trả chuỗi theo ngày tăng dần	`[d.day for d in rolling_mean(pd.Series([8, 2, 4], index=pd.to_datetime(["2026-01-03", "2026-01-01", "2026-01-02"])), 2).index]`	`[1, 2, 3]`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas. File kèm bài: sales.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. to_datetime(errors="coerce") → "bad", "2026-13-01" thành NaT.
Khung 2 — Bước 2. dropna(subset=["date"]) → bỏ dòng ngày hỏng.
Khung 3 — Bước 3. set_index("date") → DatetimeIndex để resample.
Khung 4 — Bước 4. resample("MS").sum(min_count=1) → tháng trống là NaN.
Khung 5 — Bước 5. sort_index().rolling(2) → cửa sổ hai ngày liền nhau.
Khung đối chiếu — ngộ nhận "Tháng không có số liệu thì doanh thu là 0". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Có thể là không bán, cũng có thể hệ thống chưa ghi nhận. resample().sum() điền 0 âm thầm, kéo trung bình trượt xuống và vẽ ra một tháng "sụt giảm" không có thật. Để NaN bằng sum(min_count=1) rồi quyết định có lý do.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: monthly.sum(), s.rolling(window).mean(), errors="raise", "ME"
Khung kết quả — 3 cặp vào/ra. 1/1: 5, 2/1: 15 → Tháng 1 = 20; rolling(2) ngày 2/1 = 10; Có "bad", "2026-13-01" → Hai dòng đó bị loại, không lỗi; Tháng 2 không có đơn → Tháng 2 = NaN, không phải 0

Nguồn: pe-v4 · python-learning-data.json#PD08 · parser pe-v4/build · file build 039-PD08.json

PD09 · EDA, chất lượng dữ liệu và export

Mã bài: PD09 · Nâng cao · M6 · pandas · 45 phút · 20 XP · thứ tự 40

MỤC TIÊU

Báo cáo chất lượng rồi xuất bản sạch kèm nhật ký.

CẦN NHỚ TRƯỚC

PD08

BỐI CẢNH

Viết quality_report(raw) trả dict số nguyên: rows (số dòng), duplicate_ids (số dòng có order_id đã xuất hiện ở dòng trước), missing_qty (qty thiếu hoặc không phải số), negative_qty (qty < 0), missing_price (unit_price thiếu hoặc không phải số), bad_dates (date không đọc được theo "%Y-%m-%d"). Viết export_clean(raw, path): trên BẢN SAO, bỏ trùng order_id (giữ dòng đầu), bỏ dòng thiếu qty/giá/ngày và dòng qty <= 0, chuẩn hoá region (strip + lower), thêm cột revenue, ghi CSV không index ra path và trả nhật ký {"raw_rows", "clean_rows", "revenue"}. Cả hai hàm không được sửa raw.

VÍ DỤ MINH HOẠ

sales.csv → 1 dòng trùng, 1 qty thiếu, 1 qty âm, 1 giá thiếu, 1 ngày hỏng; export_clean → 11 dòng thô, 6 đơn hợp lệ, revenue 155.0.

ĐIỂM KIẾN THỨC

1. EDA bắt đầu bằng câu hỏi "một dòng là gì?" (grain), rồi schema (dtypes), số dòng, thiếu/trùng và phạm vi hợp lệ — trước mọi biểu đồ hay trung bình.
2. describe() cho min/max/mean cột số — qty âm hiện ngay ở min; value_counts() và nunique() cho cột phân loại; nhóm quá nhỏ thì đừng vội diễn giải.
3. duplicated() đánh dấu lần xuất hiện THỨ HAI trở đi của khoá (keep="first"); keep=False đánh dấu mọi bản sao — ra số dòng dính trùng, không phải số dòng thừa.
4. Sau read_csv, ô trống đã là NaN chứ không còn là chuỗi "": đếm thiếu bằng isna() sau to_numeric, không so với "".
5. Viết quy tắc chất lượng thành phép kiểm rõ: khoá duy nhất (is_unique), qty không âm (ge(0).all()), tổng sau groupby/merge khớp tổng nguồn. Outlier là tín hiệu để điều tra, không tự động xoá; tương quan không phải nhân quả.
6. Giữ raw riêng: báo cáo và làm sạch trên bản chuyển đổi, không gán ngược vào raw. Xuất bằng to_csv(index=False) kèm nhật ký: số dòng trước/sau, tổng doanh thu, chính sách đã áp.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import pandas as pd

df = pd.DataFrame({"id": [1, 2, 2, 3], "qty": [2, 5, 5, -1], "city": ["HN", "HN", "HN", "HUE"]})
print(df.shape)
print(df["qty"].describe()[["min", "max"]].tolist())
print(df["city"].value_counts().to_dict(), df["city"].nunique())

report = {
    "dup_ids": int(df["id"].duplicated().sum()),
    "rows_in_dup": int(df["id"].duplicated(keep=False).sum()),
    "negative": int(df["qty"].lt(0).sum()),
}
print(report)

summary = df.groupby("city")["qty"].sum()
print(summary.sum() == df["qty"].sum())
```

Kết quả dự kiến (theo JSON): sales.csv → 1 trùng, 1 thiếu qty, 1 qty âm, 1 thiếu giá, 1 ngày hỏng; export_clean → 11 dòng thô, 6 đơn, revenue 155

NGỘ NHẬN THƯỜNG GẶP: "Thấy dữ liệu sai thì sửa luôn vào bảng gốc cho gọn"

Sửa raw tại chỗ thì không còn gì để đối chiếu: không đếm lại được bao nhiêu dòng hỏng, không chạy lại được với chính sách khác. Báo cáo và làm sạch trên bản sao (copy, assign hoặc biến tạm), xuất file sạch riêng kèm nhật ký, raw giữ nguyên.

CÂU DỰ ĐOÁN

order_id ["O1", "O5", "O5", "O9"]. duplicated().sum() và duplicated(keep=False).sum() lần lượt là?

ĐÚNG	1 và 2	Mặc định chỉ đánh dấu bản thứ hai; keep=False đánh dấu cả hai bản.
sai	2 và 2	Mặc định keep="first" không đánh dấu bản đầu.

sai	1 và 1	keep=False đánh dấu mọi bản sao.

TỰ KIỂM TRA

1. Cột qty đọc từ CSV có ô trống. (raw["qty"] == "").sum() trả?

- Số ô trống — Ô trống không còn là "" sau read_csv.
- Lỗi — So sánh không lỗi, chỉ ra sai.
- ✔ 0 — read_csv đã đổi ô trống thành NaN, không phải chuỗi rỗng — Đếm thiếu bằng isna() sau to_numeric.

Đáp án: Đếm thiếu bằng isna() sau to_numeric.

2. describe() cho qty có min = -1. Nên làm gì?

- Bỏ qua vì chỉ một dòng — Một dòng âm vẫn làm sai tổng doanh thu.
- Xoá ngay mọi dòng lạ — Xoá tự động có thể mất dữ liệu thật (đơn trả hàng).
- ✔ Đếm, điều tra (trả hàng? nhập sai?) rồi ghi chính sách xử lý — Giá trị lạ là tín hiệu để điều tra.

Đáp án: Giá trị lạ là tín hiệu để điều tra.

3. Doanh số và nhiệt độ có tương quan mạnh. Kết luận nào đúng?

- ✔ Hai đại lượng cùng biến thiên; cần thêm bằng chứng mới nói được nhân quả — Correlation không phải causation.
- Nhiệt độ gây ra doanh số — Tương quan không chứng minh nhân quả.
- Tương quan mạnh nghĩa là dữ liệu sai — Không liên quan đến chất lượng dữ liệu.

Đáp án: Correlation không phải causation.

TỰ GIẢI THÍCH

Vì sao quality_report và export_clean phải làm trên bản chuyển đổi mà không gán ngược vào raw?

Mẫu trả lời: raw là bằng chứng gốc: còn nó thì đếm lại được số dòng trùng, thiếu, âm và chạy lại với chính sách khác. Gán to_numeric hay to_datetime ngược vào raw làm mất chữ gốc ("x", "2026/04/02") nên không còn biết ô nào đã hỏng. Làm trên biến tạm hoặc copy(), xuất bản sạch riêng kèm nhật ký.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	duplicated() → đánh dấu bản thứ hai trở đi của khoá.
Bước 2	to_numeric(errors="coerce").isna() → qty thiếu hoặc là chữ.
Bước 3	to_datetime(format=..., errors="coerce") → ngày hỏng thành NaT.
Bước 4	raw.copy() → làm sạch trên bản sao, raw giữ nguyên.
Bước 5	to_csv(path, index=False) + nhật ký → file sạch, số dòng trước/sau.

CÂU PHẢN VÍ DỤ

order_id [A, A, A, B]. quality_report(...)["duplicate_ids"] = ?

Lựa chọn: 2 | 3 | 1 · Đáp án: 2

TEMPLATE LẮP MẢNH

```
import pandas as pd


def _parse(raw):
    qty = pd.to_numeric(raw["qty"], errors="coerce")
    price = pd.to_numeric(raw["unit_price"], errors="coerce")
    dates = pd.to_datetime(raw["date"], format="%Y-%m-%d", errors="coerce")
    return qty, price, dates


def quality_report(raw):
    qty, price, dates = _parse(raw)
    return {
        "rows": int(len(raw)),
        "duplicate_ids": {{slot1}},
        "missing_qty": {{slot2}},
        "negative_qty": int((qty < 0).sum()),
        "missing_price": int(price.isna().sum()),
        "bad_dates": int(dates.isna().sum()),
    }


def export_clean(raw, path):
    qty, price, dates = _parse(raw)
    df = {{slot3}}
    df["qty"] = qty
    df["unit_price"] = price
```

```
    df["date"] = dates
    df = df.drop_duplicates(subset="order_id", keep="first")
    df = df.dropna(subset=["qty", "unit_price", "date"])
    df = df.loc[df["qty"] > 0].copy()
    df["region"] = df["region"].str.strip().str.lower()
    df["revenue"] = df["qty"] * df["unit_price"]
    {{slot4}}
    return {"raw_rows": int(len(raw)), "clean_rows": int(len(df)), "revenue": float(df["revenue"].sum())}
```

Slot	Mảnh đúng
[Slot 1]	`int(raw["order_id"].duplicated().sum())`
[Slot 2]	`int(qty.isna().sum())`
[Slot 3]	`raw.copy()`
[Slot 4]	`df.to_csv(path, index=False)`

Mảnh bẫy	Vì sao sai
`raw`	Gán cột vào df là sửa thẳng raw.
`df.to_csv(path)`	Thêm cột index không tên.
`int(raw["order_id"].duplicated(keep=False).sum())`	Đếm cả bản đầu — ra số dòng dính trùng.
`int((raw["qty"] == "").sum())`	Ô trống đã thành NaN sau read_csv.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Dữ liệu mới: qty có chữ "x", ngày sai định dạng và ngày 31/4, hai đơn gửi lại. Lắp bốn chỗ.

Yêu cầu thêm: Thử bảng có qty là chữ, ngày sai định dạng, ngày không tồn tại và đơn gửi lại; đối chiếu báo cáo với nhật ký xuất.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết quality_report(raw) trả dict số nguyên: rows (số dòng), duplicate_ids (số dòng có order_id đã xuất hiện ở dòng trước), missing_qty (qty thiếu hoặc không phải số), negative_qty (qty < 0), missing_price (unit_price thiếu hoặc không phải số), bad_dates (date không đọc được theo "%Y-%m-%d"). Viết export_clean(raw, path): trên BẢN SAO, bỏ trùng order_id (giữ dòng đầu), bỏ dòng thiếu qty/giá/ngày và dòng qty <= 0, chuẩn hoá region (strip + lower), thêm cột revenue, ghi CSV không index ra path và trả nhật ký {"raw_rows", "clean_rows", "revenue"}. Cả hai hàm không được sửa raw.

Thực hành: Một hàm phụ đổi kiểu (to_numeric, to_datetime đều coerce) dùng chung cho báo cáo và xuất; mọi thay đổi làm trên copy; to_csv(index=False).

Gọi	Kết quả mong đợi
`sales.csv`	`rows 11; trùng 1; qty thiếu 1; qty âm 1; giá thiếu 1; ngày hỏng 1`
`export_clean(sales.csv)`	`{"raw_rows": 11, "clean_rows": 6, "revenue": 155.0}`
`Sau khi gọi`	`raw không đổi; file sạch không có cột index`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`quality_report(raw)`	Đếm các lỗi chất lượng của bảng bán hàng thô. · raw: DataFrame order_id, date, region, product, qty, unit_price (đọc thô) · dict int: rows, duplicate_ids, missing_qty, negative_qty, missing_price, bad_dates · Không sửa raw.
`export_clean(raw, path)`	Áp chính sách làm sạch, ghi CSV sạch và trả nhật ký. · raw như trên; path: tên file đầu ra · dict {"raw_rows": int, "clean_rows": int, "revenue": float} · Ghi file path (không cột index); không sửa raw.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
sales.csv: báo cáo chất lượng	`quality_report(pd.read_csv("sales.csv"))`	`{'rows': 11, 'duplicate_ids': 1, 'missing_qty': 1, 'negative_qty': 1, 'missing_price': 1, 'bad_dates': 1}`

Dữ liệu khác: qty "x", ngày sai định dạng, ngày 31/4	`quality_report(pd.read_csv(_studio.write('raw_e.csv', 'order_id,date,region,product,qty,unit_price\nE1,2026-04-01,Hue,Pen,3,2.5\nE2,2026-04-01,hue ,Pen,x,2.5\nE2,2026-04-01,hue ,Pen,x,2.5\nE3,2026/04/02,Hanoi,Book,1,12\nE4,2026-04-03,hanoi,Book,-2,12\nE5,2026-04-31,HANOI,Pen,4,\nE6,2026-04-05,hue,Book,2,12\nE6,2026-04-05,hue,Book,5,12\n')))`	`{'rows': 8, 'duplicate_ids': 2, 'missing_qty': 2, 'negative_qty': 1, 'missing_price': 1, 'bad_dates': 2}`
Bảng sạch → báo cáo toàn 0	`quality_report(pd.DataFrame({"order_id": ["Z1"], "date": ["2026-05-01"], "region": ["hue"], "product": ["Pen"], "qty": [1], "unit_price": [3.0]}))`	`{'rows': 1, 'duplicate_ids': 0, 'missing_qty': 0, 'negative_qty': 0, 'missing_price': 0, 'bad_dates': 0}`
sales.csv: 11 dòng thô → 6 đơn, revenue 155	`export_clean(pd.read_csv("sales.csv"), "clean.csv")`	`{'raw_rows': 11, 'clean_rows': 6, 'revenue': 155.0}`
Dữ liệu khác: nhật ký làm sạch	`export_clean(pd.read_csv(_studio.write('raw_e.csv', 'order_id,date,region,product,qty,unit_price\nE1,2026-04-01,Hue,Pen,3,2.5\nE2,2026-04-01,hue ,Pen,x,2.5\nE2,2026-04-01,hue ,Pen,x,2.5\nE3,2026/04/02,Hanoi,Book,1,12\nE4,2026-04-03,hanoi,Book,-2,12\nE5,2026-04-31,HANOI,Pen,4,\nE6,2026-04-05,hue,Book,2,12\nE6,2026-04-05,hue,Book,5,12\n')), "clean_e.csv")`	`{'raw_rows': 8, 'clean_rows': 2, 'revenue': 31.5}`
export_clean không sửa raw	`(lambda r: (export_clean(r, "clean.csv"), r["date"].tolist()[:2] + r["region"].tolist()[:1])[1])(pd.read_csv("sales.csv"))`	`['2026-01-01', '2026-01-02', 'North ']`
File xuất không có cột index	`(export_clean(pd.read_csv("sales.csv"), "clean.csv"), _studio.read("clean.csv").splitlines()[0])[1]`	`'order_id,date,region,product,qty,unit_price,revenue'`
File xuất: 6 đơn, vùng đã chuẩn hoá	`(export_clean(pd.read_csv("sales.csv"), "clean.csv"), (lambda c: (len(c), sorted(set(c["region"]))))(pd.read_csv("clean.csv")))[1]`	`(6, ['north', 'south'])`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: numpy, pandas. File kèm bài: sales.csv (data).

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 4 slot)
Khung 1 — Bước 1. duplicated() → đánh dấu bản thứ hai trở đi của khoá.
Khung 2 — Bước 2. to_numeric(errors="coerce").isna() → qty thiếu hoặc là chữ.
Khung 3 — Bước 3. to_datetime(format=..., errors="coerce") → ngày hỏng thành NaT.
Khung 4 — Bước 4. raw.copy() → làm sạch trên bản sao, raw giữ nguyên.
Khung 5 — Bước 5. to_csv(path, index=False) + nhật ký → file sạch, số dòng trước/sau.
Khung đối chiếu — ngộ nhận "Thấy dữ liệu sai thì sửa luôn vào bảng gốc cho gọn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Sửa raw tại chỗ thì không còn gì để đối chiếu: không đếm lại được bao nhiêu dòng hỏng, không chạy lại được với chính sách khác. Báo cáo và làm sạch trên bản sao (copy, assign hoặc biến tạm), xuất file sạch riêng kèm nhật ký, raw giữ nguyên.
Khung lắp mảnh — 4 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: raw, df.to_csv(path), int(raw["order_id"].duplicated(keep=False).sum()), int((raw["qty"] == "").sum())
Khung kết quả — 3 cặp vào/ra. sales.csv → rows 11; trùng 1; qty thiếu 1; qty âm 1; giá thiếu 1; ngày hỏng 1; export_clean(sales.csv) → {"raw_rows": 11, "clean_rows": 6, "revenue": 155.0}; Sau khi gọi → raw không đổi; file sạch không có cột index

Nguồn: pe-v4 · python-learning-data.json#PD09 · parser pe-v4/build · file build 040-PD09.json

M7 · Matplotlib

6 bài: M01, M02, M03, M04, M05, M06

Ứng viên animation — ưu tiên cao (★★): M01, M02, M03, M04, M05 · ưu tiên (★): M06

M01 · Figure, Axes và chọn biểu đồ

Mã bài: M01 · Nâng cao · M7 · Matplotlib · 35 phút · 20 XP · thứ tự 41

MỤC TIÊU

Một Figure, hai Axes, hai câu hỏi — luôn vẽ qua biến Axes.

CẦN NHỚ TRƯỚC

X05

BỐI CẢNH

Viết revenue_dashboard(days, revenue, categories, orders) trả về Figure gồm 2 Axes cạnh nhau: fig, (ax_line, ax_bar) = plt.subplots(1, 2, layout="constrained"). Axes trái: line có marker "o" của doanh thu theo ngày, nhãn "Day" / "Revenue", tiêu đề "Daily revenue". Axes phải: barh số đơn theo category, SẮP để category nhiều đơn nhất nằm trên cùng, nhãn trục x "Orders", tiêu đề "Orders by category". Mọi lệnh vẽ gọi qua ax_line / ax_bar, không qua plt.plot.

VÍ DỤ MINH HOẠ

revenue_dashboard([1, 2, 3], [10, 15, 12], ["A", "B", "C"], [30, 15, 22]) → trái: đường qua 10, 15, 12; phải: các thanh từ dưới lên B (15), C (22), A (30).

ĐIỂM KIẾN THỨC

1. plt.subplots(1, 2) tạo MỘT Figure với HAI Axes xếp 1 hàng 2 cột; viết fig, (ax_line, ax_bar) = ... để đặt tên từng Axes ngay. layout="constrained" tự dàn khoảng cách để nhãn và tiêu đề hai Axes không chồng nhau.
2. pyplot giữ một "Axes hiện hành" — Axes được tạo hoặc chọn gần nhất. Sau subplots(1, 2) đó là Axes BÊN PHẢI, nên plt.plot(...) rơi vào ax_bar dù bạn định vẽ bên trái (ví dụ minh hoạ in ra 1 1). Gọi ax_line.plot(...) thì không thể nhầm.
3. Mỗi Axes một câu hỏi: diễn tiến theo ngày có thứ tự → line; so các category → bar/barh; quan hệ hai biến → scatter; phân bố → histogram. Không nối các category không có thứ tự bằng line — độ dốc giữa "A" và "B" không mang nghĩa gì.
4. barh vẽ thanh đầu tiên ở DƯỚI cùng. Muốn xếp hạng (nhiều nhất trên cùng) thì sắp TĂNG dần trước khi vẽ: sorted(zip(orders, categories)) sắp theo số đơn. Mỗi thanh phải có metric, đơn vị và mức tổng hợp rõ (tổng đơn theo category).
5. Notebook tự hiện figure cuối cell; script trên máy gọi plt.show(); chạy hàng loạt (server, CI) dùng backend Agg không có cửa sổ — Studio cũng chạy Agg.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt

fig, (left, right) = plt.subplots(1, 2, layout="constrained")
left.plot([1, 2, 3], [20, 30, 25], marker="o")
right.barh(["Pen", "Book"], [15, 30])

print(len(fig.axes))
print(plt.gca() is right)
plt.plot([1, 2], [5, 5])
print(len(left.lines), len(right.lines))
```

Kết quả dự kiến (theo JSON): Axes phải không có đường nào; orders [30, 15, 22] → thanh trên cùng là A

NGỘ NHẬN THƯỜNG GẶP: "plt.plot và ax.plot như nhau, dùng cái nào cũng được"

Với MỘT Axes thì kết quả giống nhau. Với nhiều Axes, plt.plot vẽ lên Axes hiện hành (thường là Axes tạo sau cùng), không phải Axes bạn đang nghĩ — biểu đồ sai mà không báo lỗi. Trong hàm vẽ, luôn gọi method của biến Axes.

CÂU DỰ ĐOÁN

Sau fig, (left, right) = plt.subplots(1, 2), lệnh plt.plot([1, 2], [5, 5]) vẽ vào đâu?

```
fig, (left, right) = plt.subplots(1, 2)
plt.plot([1, 2], [5, 5])
```

sai	Vào cả hai	Mỗi lệnh vẽ chỉ rơi vào một Axes.
sai	Báo lỗi vì có hai Axes	Không lỗi — chính vì im lặng nên dễ nhầm.
sai	Vào left — Axes đầu tiên	pyplot không chọn Axes đầu; nó dùng Axes hiện hành.
ĐÚNG	Vào right — Axes được tạo sau cùng là Axes hiện hành	pyplot vẽ lên plt.gca(); subplots tạo left trước, right sau.

TỰ KIỂM TRA

1. Doanh thu 7 ngày liên tiếp — loại biểu đồ nào hợp nhất?
   • Pie — Pie so phần của một tổng, không cho thấy diễn tiến theo thời gian.
   • Histogram — Histogram đếm phân bố giá trị, mất thứ tự ngày.
   ✔ Line có marker — Ngày có thứ tự; line cho thấy diễn tiến, marker đánh dấu từng quan sát.

Đáp án: Ngày có thứ tự; line cho thấy diễn tiến, marker đánh dấu từng quan sát.

2. Số đơn của 5 category không có thứ tự tự nhiên — nên vẽ thế nào?
   • Scatter category với số đơn — Scatter dành cho quan hệ giữa hai biến số.
   ✔ barh, sắp theo số đơn để đọc ngay thứ hạng — Sắp category theo câu hỏi (ở đây là xếp hạng).
   • Line nối 5 category theo thứ tự chữ cái — Độ dốc giữa hai category không có ý nghĩa.

Đáp án: Sắp category theo câu hỏi (ở đây là xếp hạng).

3. ax.barh(["B", "C", "A"], [15, 22, 30]) — thanh nào nằm trên cùng?
   • Tự sắp theo giá trị — barh giữ đúng thứ tự bạn đưa vào.
   ✔ A (30) — Thanh đầu tiên ở dưới cùng, thanh cuối ở trên cùng.
   • B (15) — B là thanh đầu tiên nên nằm dưới cùng.

Đáp án: Thanh đầu tiên ở dưới cùng, thanh cuối ở trên cùng.

TỰ GIẢI THÍCH

Vì sao hàm dashboard gọi ax_line.plot(...) thay vì plt.plot(...)?

Mẫu trả lời: subplots(1, 2) để Axes bên phải làm Axes hiện hành, nên plt.plot vẽ đường doanh thu đè lên biểu đồ barh còn Axes trái trống — không có lỗi nào báo. Gọi qua biến ax_line chỉ đúng vùng vẽ, và hàm vẫn đúng dù trước đó ai đã tạo thêm Axes hay figure khác.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	plt.subplots(1, 2, layout="constrained") → 1 Figure, 2 Axes
Bước 2	ax_line.plot(days, revenue, marker="o") → line bên trái
Bước 3	sorted(zip(orders, categories)) → cặp (số đơn, tên) tăng dần
Bước 4	ax_bar.barh(names, counts) → thanh lớn nhất nằm trên cùng
Bước 5	return fig → nơi gọi đọc fig.axes[0], fig.axes[1]

CÂU PHẢN VÍ DỤ

categories ["A", "B", "C"], orders [30, 15, 22] — thanh trên cùng là?

Lựa chọn: A | B | C · Đáp án: A

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt


def revenue_dashboard(days, revenue, categories, orders):
    fig, (ax_line, ax_bar) = {{slot1}}

    {{slot2}}
    ax_line.set(xlabel="Day", ylabel="Revenue", title="Daily revenue")

    ranked = {{slot3}}
    ax_bar.barh([name for count, name in ranked], [count for count, name in ranked])
    ax_bar.set(xlabel="Orders", title="Orders by category")
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`plt.subplots(1, 2, layout="constrained")`
[Slot 2]	`ax_line.plot(days, revenue, marker="o")`
[Slot 3]	`sorted(zip(orders, categories))`

Mảnh bẫy	Vì sao sai
`sorted(zip(orders, categories), reverse=True)`	Giảm dần → thanh lớn nhất nằm DƯỚI cùng.
`list(zip(orders, categories))`	Giữ thứ tự nhập, không xếp hạng.
`plt.subplots()`	Chỉ một Axes, không tách được thành hai.
`plt.plot(days, revenue, marker="o")`	Vẽ lên Axes hiện hành — tức ax_bar.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: tạo hai Axes, vẽ line đúng Axes, sắp category trước khi vẽ barh.

Yêu cầu thêm: Đổi sang 4 ngày và 4 category; kiểm Axes phải không có đường nào và thanh trên cùng là category lớn nhất.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết revenue_dashboard(days, revenue, categories, orders) trả về Figure gồm 2 Axes cạnh nhau: fig, (ax_line, ax_bar) = plt.subplots(1, 2, layout="constrained"). Axes trái: line có marker "o" của doanh thu theo ngày, nhãn "Day" / "Revenue", tiêu đề "Daily revenue". Axes phải: barh số đơn theo category, SẮP để category nhiều đơn nhất nằm trên cùng, nhãn trục x "Orders", tiêu đề "Orders by category". Mọi lệnh vẽ gọi qua ax_line / ax_bar, không qua plt.plot.

Thực hành: Đặt tên hai Axes khi tách; vẽ line bằng ax_line, barh bằng ax_bar; sắp tăng dần trước barh để lớn nhất ở trên.

Gọi	Kết quả mong đợi
`revenue_dashboard([1, 2, 3], [10, 15, 12], ["A", "B", "C"], [30, 15, 22])`	`2 Axes; trái: line 10, 15, 12; phải: barh từ dưới lên B, C, A`
`Một ngày, một category`	`Vẫn 2 Axes; barh một thanh`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`revenue_dashboard(days, revenue, categories, orders)`	Dashboard hai câu hỏi: diễn tiến doanh thu và xếp hạng category. · days, revenue: list cùng độ dài; categories, orders: list cùng độ dài · Figure: fig.axes[0] là line, fig.axes[1] là barh · Axes phải không có Line2D nào; thanh cuối (trên cùng) là category nhiều đơn nhất.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Một Figure, hai Axes	`len(revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes)`	`2`
Axes trái: đường qua doanh thu mới	`[float(v) for v in revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[0].lines[0].get_ydata()]`	`[40.0, 25.0, 35.0, 50.0]`
Axes trái: marker, nhãn và tiêu đề	`(lambda ax: (ax.lines[0].get_marker(), ax.get_xlabel(), ax.get_ylabel(), ax.get_title()))(revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[0])`	`('o', 'Day', 'Revenue', 'Daily revenue')`
Axes phải không có đường nào (không vẽ nhầm bằng plt.plot)	`len(revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[1].lines)`	`0`
barh: từ dưới lên tăng dần, lớn nhất trên cùng	`[float(v) for v in (p.get_width() for p in revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[1].patches)]`	`[7.0, 12.0, 25.0, 40.0]`
barh: tên category khớp thanh	`[t.get_text() for t in revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[1].get_yticklabels()]`	`['Bag', 'Pen', 'Ink', 'Book']`
Axes phải: nhãn Orders và tiêu đề	`(lambda ax: (ax.get_xlabel(), ax.get_title()))(revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).axes[1])`	`('Orders', 'Orders by category')`
layout="constrained"	`type(revenue_dashboard([1, 2, 3, 4], [40, 25, 35, 50], ["Pen", "Book", "Bag", "Ink"], [12, 40, 7, 25]).get_layout_engine()).__name__`	`'ConstrainedLayoutEngine'`
Một ngày, một category	`(lambda f: (len(f.axes), [float(v) for v in (p.get_width() for p in f.axes[1].patches)]))(revenue_dashboard([1], [9], ["Solo"], [3]))`	`(2, [3.0])`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. plt.subplots(1, 2, layout="constrained") → 1 Figure, 2 Axes
Khung 2 — Bước 2. ax_line.plot(days, revenue, marker="o") → line bên trái
Khung 3 — Bước 3. sorted(zip(orders, categories)) → cặp (số đơn, tên) tăng dần
Khung 4 — Bước 4. ax_bar.barh(names, counts) → thanh lớn nhất nằm trên cùng
Khung 5 — Bước 5. return fig → nơi gọi đọc fig.axes[0], fig.axes[1]
Khung đối chiếu — ngộ nhận "plt.plot và ax.plot như nhau, dùng cái nào cũng được". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Với MỘT Axes thì kết quả giống nhau. Với nhiều Axes, plt.plot vẽ lên Axes hiện hành (thường là Axes tạo sau cùng), không phải Axes bạn đang nghĩ — biểu đồ sai mà không báo lỗi. Trong hàm vẽ, luôn gọi method của biến Axes.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: sorted(zip(orders, categories), reverse=True), list(zip(orders, categories)), plt.subplots(), plt.plot(days, revenue, marker="o")
Khung kết quả — 2 cặp vào/ra. revenue_dashboard([1, 2, 3], [10, 15, 12], ["A", "B", "C"], [30, 15, 22]) → 2 Axes; trái: line 10, 15, 12; phải: barh từ dưới lên B, C, A; Một ngày, một category → Vẫn 2 Axes; barh một thanh

Nguồn: pe-v4 · python-learning-data.json#M01 · parser pe-v4/build · file build 041-M01.json

M02 · Phân bố, scatter và outlier

Mã bài: M02 · Nâng cao · M7 · Matplotlib · 35 phút · 20 XP · thứ tự 42

MỤC TIÊU

Đọc phân bố, so nhóm và giữ outlier.

CẦN NHỚ TRƯỚC

M01

BỐI CẢNH

Viết hai hàm trả về Figure. (1) plot_distribution(groups, edges): groups là dict tên nhóm → list điểm. fig, (ax_hist, ax_box) = plt.subplots(1, 2, layout="constrained"). ax_hist: histogram của TẤT CẢ điểm gộp lại với bins=edges, nhãn "Score" / "Count", tiêu đề f"All scores (n={n})" với n là tổng số quan sát. ax_box: boxplot mỗi nhóm một hộp theo thứ tự dict, GIỮ điểm ngoài whisker (không lọc, không tắt showfliers), tick là tên nhóm, nhãn y "Score". (2) plot_relation(hours, scores): scatter alpha=0.5, nhãn "Hours" / "Score".

VÍ DỤ MINH HOẠ

plot_distribution({"A": [4, 5, 6, 7], "B": [5, 6, 7, 20]}, [0, 5, 10, 25]) → histogram 3 cột 1, 6, 1; tiêu đề "All scores (n=8)"; hộp B có điểm ngoài 20.

ĐIỂM KIẾN THỨC

1. Histogram đếm số quan sát trong từng bin; biên bin là một quyết định: cùng dữ liệu, biên khác → hình khác → kết luận khác. Chốt bins=edges và ghi cỡ mẫu n lên tiêu đề để người đọc biết đang nhìn bao nhiêu quan sát.
2. Boxplot tóm tắt mỗi nhóm bằng median (vạch giữa), tứ phân vị (hộp) và whisker; điểm ngoài whisker được vẽ riêng (fliers). Điểm ngoài KHÔNG tự là lỗi — có thể là kết quả thật đáng chú ý nhất. Giữ nó trên hình rồi điều tra.
3. ax.boxplot(list_các_nhóm) đặt hộp ở x = 1, 2, ...; ghi tên bằng ax.set_xticks([1, 2], ["A", "B"]). Matplotlib 3.8 (trong Studio) còn nhận labels=...; từ 3.9 tham số đổi tên thành tick_labels= — set_xticks chạy được ở mọi phiên bản.
4. Scatter vẽ từng CẶP quan sát (giờ học, điểm). alpha=0.5 làm điểm trong suốt một nửa nên chỗ nhiều điểm chồng nhau hiện đậm hơn. ax.plot sẽ nối các cặp theo thứ tự nhập — đường đó không có nghĩa.
5. Scatter cho thấy tương quan, không chứng minh nhân quả: có thể có biến gây nhiễu hoặc dữ liệu thiếu. So phân bố các nhóm thì báo cỡ mẫu trước.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt

fig, (ax_hist, ax_box) = plt.subplots(1, 2, layout="constrained")
counts, edges, bars = ax_hist.hist([1, 2, 2, 4], bins=[0, 3, 5])
print(counts.tolist())

parts = ax_box.boxplot([[4, 5, 6, 7], [5, 6, 7, 20]])
ax_box.set_xticks([1, 2], ["A", "B"])
print([line.get_ydata().tolist() for line in parts["fliers"]])
print([t.get_text() for t in ax_box.get_xticklabels()])
```

Kết quả dự kiến (theo JSON): Ví dụ → cột 1, 6, 1; tiêu đề "All scores (n=8)"; Hộp B vẫn có điểm ngoài 20

NGỘ NHẬN THƯỜNG GẶP: "Điểm ngoài whisker là lỗi nhập liệu — bỏ đi cho biểu đồ gọn"

Boxplot chỉ ĐÁNH DẤU điểm xa so với phần lớn dữ liệu; nó không biết điểm đó sai hay thật. Bỏ điểm 20 làm nhóm B trông giống A và che mất thứ đáng điều tra nhất. Muốn loại dữ liệu thì phải có chính sách rõ ràng và ghi lại, không phải tắt showfliers.

CÂU DỰ ĐOÁN

groups = {"A": [4, 5, 6, 7], "B": [5, 6, 7, 20]}, edges = [0, 5, 10, 25]. Histogram gộp có mấy cột, cao bao nhiêu?

ĐÚNG	3 cột: 1, 6, 1	[0, 5) chỉ có 4; [5, 10) có 5, 6, 7, 5, 6, 7; [10, 25] có 20.
sai	3 cột: 3, 4, 1	5 thuộc bin [5, 10) — biên trái đóng, biên phải mở (trừ bin cuối).
sai	Hai histogram riêng cho A và B	Để gộp mọi điểm vào một histogram; so nhóm là việc của boxplot.

TỰ KIỂM TRA

1. ax.boxplot(data, showfliers=False) làm gì?

✔ Ẩn các điểm ngoài whisker khỏi hình — dữ liệu vẫn còn nhưng người đọc không thấy — Hình "gọn" hơn nhưng che outlier.

• Sửa outlier về giá trị whisker — Không sửa dữ liệu nào.

• Xoá outlier khỏi list data — List không đổi; chỉ là không vẽ.

Đáp án: Hình "gọn" hơn nhưng che outlier.

2. Scatter giờ học – điểm có xu hướng đi lên. Kết luận nào đúng mực?

• Học thêm 1 giờ chắc chắn tăng điểm — Scatter không chứng minh tác động.

• Không kết luận được gì vì có điểm chồng nhau — Điểm chồng xử lý bằng alpha; vẫn đọc được xu hướng.

✔ Hai biến có tương quan dương trong mẫu này; chưa đủ để nói học nhiều GÂY RA điểm cao — Tương quan khác nhân quả; còn biến gây nhiễu, cỡ mẫu.

Đáp án: Tương quan khác nhân quả; còn biến gây nhiễu, cỡ mẫu.

3. Vì sao tiêu đề histogram ghi n=...?

• Để đếm số bin — Số bin là len(edges) - 1.

✔ Để người đọc biết cỡ mẫu trước khi tin vào hình dạng phân bố — 8 quan sát và 8000 quan sát cho độ tin cậy rất khác nhau.

• Matplotlib bắt buộc — Không bắt buộc; đây là thói quen trình bày đúng.

Đáp án: 8 quan sát và 8000 quan sát cho độ tin cậy rất khác nhau.

TỰ GIẢI THÍCH

Vì sao bài giữ outlier trên boxplot và chốt biên bin, thay vì để mặc định hoặc lọc dữ liệu cho đẹp?

Mẫu trả lời: Biên bin mặc định đổi theo dữ liệu nên hai lần vẽ không so được; chốt edges thì cùng một thang. Outlier có thể là dữ liệu thật quan trọng nhất — boxplot vẽ riêng nó để người đọc thấy và điều tra; tắt showfliers hay lọc bỏ là đổi dữ liệu mà không báo cho ai.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	all_scores = [... của mọi nhóm] → gộp, n = len(all_scores)
Bước 2	ax_hist.hist(all_scores, bins=edges) → đếm theo biên cố định
Bước 3	ax_box.boxplot(list(groups.values())) → mỗi nhóm một hộp, giữ fliers
Bước 4	ax_box.set_xticks(range(1, k + 1), names) → tên nhóm dưới hộp
Bước 5	ax.scatter(hours, scores, alpha=0.5) → cặp quan sát, chỗ chồng đậm hơn

CÂU PHẢN VÍ DỤ

groups {"X": [1, 2], "Y": [3, 4, 5]} → tiêu đề histogram là?

Lựa chọn: All scores (n=3) | All scores (n=5) | All scores (n=2) · Đáp án: All scores (n=5)

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt


def plot_distribution(groups, edges):
    fig, (ax_hist, ax_box) = plt.subplots(1, 2, layout="constrained")
    all_scores = [score for scores in groups.values() for score in scores]
    {{slot1}}
    ax_hist.set(xlabel="Score", ylabel="Count", title={{slot2}})

    {{slot3}}
    ax_box.set_xticks(range(1, len(groups) + 1), list(groups))
    ax_box.set(ylabel="Score")
    return fig


def plot_relation(hours, scores):
    fig, ax = plt.subplots()
    ax.scatter(hours, scores, alpha=0.5)
    ax.set(xlabel="Hours", ylabel="Score")
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`ax_hist.hist(all_scores, bins=edges)`
[Slot 2]	`f"All scores (n={len(all_scores)})"`
[Slot 3]	`ax_box.boxplot(list(groups.values()))`

Mảnh bẫy	Vì sao sai
`f"All scores (n={len(groups)})"`	len(groups) là số NHÓM, không phải số quan sát.
`ax_hist.hist(all_scores)`	Bin tự động — không phải biên để cho.
`ax_box.boxplot(all_scores)`	Gộp mọi nhóm thành một hộp — mất phép so sánh.
`ax_box.boxplot(list(groups.values()), showfliers=False)`	Ẩn outlier khỏi hình.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

plot_relation đã viết sẵn. Lắp ba chỗ của plot_distribution: histogram biên cố định, tiêu đề có n, boxplot đủ nhóm.

Yêu cầu thêm: Thử ba nhóm, một nhóm có điểm rất thấp; kiểm boxplot vẫn vẽ điểm đó và n trên tiêu đề đúng tổng số điểm.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm trả về Figure. (1) plot_distribution(groups, edges): groups là dict tên nhóm → list điểm. fig, (ax_hist, ax_box) = plt.subplots(1, 2, layout="constrained"). ax_hist: histogram của TẤT CẢ điểm gộp lại với bins=edges, nhãn "Score" / "Count", tiêu đề f"All scores (n={n})" với n là tổng số quan sát. ax_box: boxplot mỗi nhóm một hộp theo thứ tự dict, GIỮ điểm ngoài whisker (không lọc, không tắt showfliers), tick là tên nhóm, nhãn y "Score". (2) plot_relation(hours, scores): scatter alpha=0.5, nhãn "Hours" / "Score".

Thực hành: Gộp mọi điểm cho histogram (bins=edges, n trên tiêu đề); boxplot nhận list các nhóm, tick là tên nhóm; scatter có alpha.

Gọi	Kết quả mong đợi
`plot_distribution({"A": [4, 5, 6, 7], "B": [5, 6, 7, 20]}, [0, 5, 10, 25])`	`histogram 1, 6, 1; tiêu đề All scores (n=8); hộp B giữ điểm 20`
`plot_relation([1, 2, 3, 4], [2, 3, 3, 5])`	`4 điểm scatter, alpha 0.5; nhãn Hours / Score`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_distribution(groups, edges)`	So phân bố: histogram gộp + boxplot theo nhóm. · groups: dict tên → list điểm; edges: list biên bin · Figure 2 Axes · Tiêu đề histogram ghi n; boxplot giữ mọi điểm ngoài whisker; tick là tên nhóm.
`plot_relation(hours, scores)`	Vẽ quan hệ giữa hai biến. · hours, scores: list cùng độ dài · Figure 1 Axes · Scatter alpha=0.5, không nối đường; nhãn Hours / Score.

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Hai Axes	`len(plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes)`	`2`
Histogram gộp mọi nhóm, đúng biên	`[float(v) for v in (p.get_height() for p in plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes[0].patches)]`	`[1.0, 9.0, 5.0]`
Tiêu đề ghi cỡ mẫu n	`plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes[0].get_title()`	`'All scores (n=15)'`
Histogram có nhãn Score / Count	`(lambda ax: (ax.get_xlabel(), ax.get_ylabel()))(plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes[0])`	`('Score', 'Count')`
Boxplot: mỗi nhóm một hộp, tick là tên nhóm	`[t.get_text() for t in plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes[1].get_xticklabels()]`	`['North', 'South', 'East']`
Boxplot giữ outlier của từng nhóm	`(lambda ax: [[float(v) for v in l.get_ydata()] for l in ax.lines if l.get_linestyle() == "None"])(plot_distribution({"North": [50, 55, 60, 62, 58], "South": [52, 57, 61, 59, 95], "East": [10, 56, 58, 60, 57]}, [0, 40, 60, 100]).axes[1])`	`[[], [95.0], [10.0]]`
Scatter: đúng các cặp quan sát	`plot_relation([1, 2, 2, 5], [3, 4, 4, 9]).axes[0].collections[0].get_offsets().tolist()`	`[[1.0, 3.0], [2.0, 4.0], [2.0, 4.0], [5.0, 9.0]]`
Scatter: alpha 0.5, nhãn Hours / Score, không nối đường	`(lambda ax: (ax.collections[0].get_alpha(), ax.get_xlabel(), ax.get_ylabel(), len(ax.lines)))(plot_relation([1, 2, 2, 5], [3, 4, 4, 9]).axes[0])`	`(0.5, 'Hours', 'Score', 0)`
Một nhóm duy nhất	`(lambda f: ([float(v) for v in (p.get_height() for p in f.axes[0].patches)], f.axes[0].get_title(), [t.get_text() for t in f.axes[1].get_xticklabels()]))(plot_distribution({"Solo": [1, 2, 3]}, [0, 2, 4]))`	`([1.0, 2.0], 'All scores (n=3)', ['Solo'])`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. all_scores = [... của mọi nhóm] → gộp, n = len(all_scores)
Khung 2 — Bước 2. ax_hist.hist(all_scores, bins=edges) → đếm theo biên cố định
Khung 3 — Bước 3. ax_box.boxplot(list(groups.values())) → mỗi nhóm một hộp, giữ fliers
Khung 4 — Bước 4. ax_box.set_xticks(range(1, k + 1), names) → tên nhóm dưới hộp
Khung 5 — Bước 5. ax.scatter(hours, scores, alpha=0.5) → cặp quan sát, chỗ chồng đậm hơn
Khung đối chiếu — ngộ nhận "Điểm ngoài whisker là lỗi nhập liệu — bỏ đi cho biểu đồ gọn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Boxplot chỉ ĐÁNH DẤU điểm xa so với phần lớn dữ liệu; nó không biết điểm đó sai hay thật. Bỏ điểm 20 làm nhóm B trông giống A và che mất thứ đáng điều tra nhất. Muốn loại dữ liệu thì phải có chính sách rõ ràng và ghi lại, không phải tắt showfliers.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: f"All scores (n={len(groups)})", ax_hist.hist(all_scores), ax_box.boxplot(all_scores), ax_box.boxplot(list(groups.values()), showfliers=False)
Khung kết quả — 2 cặp vào/ra. plot_distribution({"A": [4, 5, 6, 7], "B": [5, 6, 7, 20]}, [0, 5, 10, 25]) → histogram 1, 6, 1; tiêu đề All scores (n=8); hộp B giữ điểm 20; plot_relation([1, 2, 3, 4], [2, 3, 3, 5]) → 4 điểm scatter, alpha 0.5; nhãn Hours / Score

Nguồn: pe-v4 · python-learning-data.json#M02 · parser pe-v4/build · file build 042-M02.json

M03 · Subplot và heatmap đọc được

Mã bài: M03 · Nâng cao · M7 · Matplotlib · 40 phút · 20 XP · thứ tự 43

MỤC TIÊU

Heatmap có nhãn, colorbar có đơn vị và chung một thang.

CẦN NHỚ TRƯỚC

M02

BỐI CẢNH

Viết plot_heatmaps(tables, unit): tables là dict tiêu đề → DataFrame (index = region, columns = sản phẩm/tháng — như kết quả pivot_table). Tạo fig, axes = plt.subplots(1, len(tables), layout="constrained", squeeze=False). Mỗi bảng một Axes: ax.imshow(table.to_numpy(), cmap="viridis", vmin=0, vmax=vmax) với vmax là giá trị LỚN NHẤT của MỌI bảng; tick x là tên cột, tick y là tên hàng; tiêu đề là khoá của dict. Cuối cùng MỘT colorbar cho cả hàng: fig.colorbar(im, ax=list(axes[0]), label=unit). Return fig.

VÍ DỤ MINH HOẠ

plot_heatmaps({"Jan": jan, "Feb": feb}, "Orders") với jan = [[10, 20], [30, 40]], feb = [[5, 15], [25, 60]] → 2 heatmap cùng thang 0..60, 1 colorbar "Orders" (fig.axes có 3 Axes).

ĐIỂM KIẾN THỨC

1. Heatmap mã hoá GIÁ TRỊ bằng MÀU. ax.imshow(bảng) vẽ ô hàng i, cột j tại toạ độ (x = j, y = i): cột của bảng nằm ngang, hàng nằm dọc. Thiếu nhãn thì người đọc chỉ thấy ô màu vô danh: set_xticks(range(số cột), tên cột), set_yticks(range(số hàng), tên hàng).
2. Colorbar là "thước đo" của màu: fig.colorbar(im, ..., label="Orders") thêm một Axes nhỏ có đơn vị. Không có colorbar thì không ai biết màu vàng là bao nhiêu.
3. imshow tự co thang màu theo min/max của TỪNG bảng. Hai heatmap tự co thì cùng một màu có thể là 40 ở bảng này và 60 ở bảng kia — so màu giữa hai panel là sai. Đặt vmin/vmax giống nhau cho mọi panel rồi dùng MỘT colorbar chung.
4. Colormap sequential (viridis, Blues...) cho lượng đi một chiều từ thấp tới cao; diverging (coolwarm, RdBu) khi có mốc giữa có nghĩa như 0 của "thay đổi", với vmin = -m, vmax = m. Rainbow/jet tạo các dải sáng giả, mắt thấy ranh giới không có trong dữ liệu.
5. plt.subplots(1, k, squeeze=False) luôn trả mảng 2 chiều, kể cả k = 1 — axes[0] là hàng Axes duy nhất, nên một vòng for chạy đúng cho 1 hay nhiều bảng. Subplot để tách câu hỏi; dùng chung thang (sharey, vmin/vmax) khi người đọc cần so trực tiếp. Ghi số vào từng ô chỉ hữu ích khi bảng nhỏ.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt
import pandas as pd

table = pd.DataFrame([[10, 20, 15], [30, 40, 35]],
                     index=["North", "South"], columns=["Jan", "Feb", "Mar"])
fig, ax = plt.subplots(layout="constrained")
im = ax.imshow(table.to_numpy(), cmap="viridis", vmin=0, vmax=40)
ax.set_xticks(range(len(table.columns)), table.columns)
ax.set_yticks(range(len(table.index)), table.index)
fig.colorbar(im, ax=ax, label="Orders")

print(table.shape, [float(v) for v in im.get_clim()])
print([t.get_text() for t in ax.get_xticklabels()])
print(len(fig.axes), fig.axes[-1].get_ylabel())
```

Kết quả dự kiến (theo JSON): Jan + Feb → 3 Axes, mọi panel thang 0..max chung; Colorbar có nhãn "Orders"

NGỘ NHẬN THƯỜNG GẶP: "Mỗi heatmap tự có colorbar riêng là đủ để so sánh"

Hai colorbar với hai thang khác nhau buộc người đọc tra số từng ô — mắt vẫn so màu và kết luận sai. Muốn so thì chung vmin/vmax và một colorbar. Và đừng dùng rainbow cho dữ liệu một chiều: dùng sequential như viridis.

CÂU DỰ ĐOÁN

Hai heatmap vẽ bằng imshow KHÔNG đặt vmin/vmax: Jan có giá trị 10..40, Feb có 5..60. Ô 40 của Jan và ô 60 của Feb có màu gì?

sai	Matplotlib tự đồng bộ thang giữa các subplot	Không — mỗi imshow có thang riêng.
ĐÚNG	Cùng màu đậm nhất — mỗi panel tự co thang theo max của chính nó	Cùng màu nhưng khác giá trị: so màu giữa hai panel bị sai.
sai	Ô 60 sáng hơn ô 40	Chỉ đúng khi hai panel dùng chung vmin/vmax.

TỰ KIỂM TRA

1. Bảng "thay đổi doanh thu so với tháng trước" có cả số âm và dương — colormap nào hợp?
   - • Sequential viridis với thang tự động — Không thấy được đâu là tăng, đâu là giảm.
   - ✔ Diverging (vd. coolwarm) với vmin = -m, vmax = m để 0 nằm giữa — 0 là mốc có nghĩa; hai phía hai màu.
   - • jet để nhiều màu cho dễ phân biệt — Rainbow tạo ranh giới giả và không có mốc giữa.

Đáp án: 0 là mốc có nghĩa; hai phía hai màu.

2. ax.imshow(table.to_numpy()) với table 2 hàng × 3 cột — set_xticks cần mấy vị trí?
   - • 2 — số hàng — Hàng nằm trên trục y.
   - • 6 — số ô — Tick đánh dấu cột, không đánh dấu từng ô.
   - ✔ 3 — số CỘT nằm trên trục x — imshow đặt cột theo x, hàng theo y.

Đáp án: imshow đặt cột theo x, hàng theo y.

3. Vì sao chỉ cần MỘT colorbar cho cả hàng heatmap trong bài?
   - • Vì colorbar không quan trọng — Thiếu colorbar thì màu không đọc ra số.
   - ✔ Vì mọi panel dùng chung vmin/vmax nên một thước màu đúng cho tất cả — Chung thang mới dùng chung thước được.
   - • Vì Matplotlib chỉ cho một colorbar mỗi Figure — Một Figure có thể có nhiều colorbar.

Đáp án: Chung thang mới dùng chung thước được.

TỰ GIẢI THÍCH

Vì sao bài tính vmax từ TẤT CẢ bảng và đặt giống nhau cho mọi panel?

Mẫu trả lời: imshow mặc định co thang màu theo từng bảng, nên cùng một màu có thể là 40 ở panel này và 60 ở panel kia; người đọc so màu sẽ kết luận sai. Chốt vmin = 0 và vmax = max của mọi bảng thì cùng màu là cùng giá trị, và một colorbar có đơn vị đủ để đọc mọi panel.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	max(... for table in tables.values()) → vmax chung
Bước 2	plt.subplots(1, k, squeeze=False) → axes[0] luôn là hàng Axes
Bước 3	ax.imshow(..., vmin=0, vmax=vmax) → mọi panel cùng thang màu
Bước 4	ax.set_xticks(range(n_cột), table.columns) → tên cột trên trục x
Bước 5	fig.colorbar(im, ax=list(axes[0]), label=unit) → một thước màu có đơn vị

CÂU PHẢN VÍ DỤ

tables gồm 3 bảng → fig.axes có mấy Axes?

Lựa chọn: 3 | 6 | 4 · Đáp án: 4

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt
import pandas as pd


def plot_heatmaps(tables, unit):
    vmax = {{slot1}}
    fig, axes = plt.subplots(1, len(tables), layout="constrained", squeeze=False)
    for ax, (title, table) in zip(axes[0], tables.items()):
        im = ax.imshow(table.to_numpy(), cmap="viridis", {{slot2}})
        ax.set_xticks(range(len(table.columns)), table.columns)
        ax.set_yticks(range(len(table.index)), table.index)
        ax.set_title(title)
    {{slot3}}
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`max(table.to_numpy().max() for table in tables.values())`
[Slot 2]	`vmin=0, vmax=vmax`
[Slot 3]	`fig.colorbar(im, ax=list(axes[0]), label=unit)`

Mảnh bẫy	Vì sao sai
`max(tables)`	max của các KHOÁ (tiêu đề), không phải giá trị trong bảng.
`vmin=0`	Thiếu vmax → mỗi panel tự co thang theo max của nó.
`fig.colorbar(im, ax=ax)`	Colorbar chỉ gắn vào panel cuối và không có đơn vị.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: vmax chung, thang màu cố định cho mỗi panel, một colorbar có đơn vị.

Yêu cầu thêm: Thử bảng 3 hàng × 2 cột (không vuông) để chắc không đảo hàng/cột; thử dict chỉ một bảng.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết plot_heatmaps(tables, unit): tables là dict tiêu đề → DataFrame (index = region, columns = sản phẩm/tháng — như kết quả pivot_table). Tạo fig, axes = plt.subplots(1, len(tables), layout="constrained", squeeze=False). Mỗi bảng một Axes: ax.imshow(table.to_numpy(), cmap="viridis", vmin=0, vmax=vmax) với vmax là giá trị LỚN NHẤT của MỌI bảng; tick x là tên cột, tick y là tên hàng; tiêu đề là khoá của dict. Cuối cùng MỘT colorbar cho cả hàng: fig.colorbar(im, ax=list(axes[0]), label=unit). Return fig.

Thực hành: Tính vmax chung trước vòng lặp; trong vòng: imshow viridis với vmin=0, vmax=vmax, đặt tick cột/hàng, tiêu đề; sau vòng: một colorbar cho list(axes[0]).

Gọi	Kết quả mong đợi
`Jan [[10, 20], [30, 40]], Feb [[5, 15], [25, 60]], "Orders"`	`2 panel cùng thang 0..60; 1 colorbar "Orders"; 3 Axes`
`Một bảng 1×3, "Units"`	`1 panel + colorbar "Units"; tick x đủ 3 cột`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_heatmaps(tables, unit)`	Vẽ một hàng heatmap có thể so sánh với nhau. · tables: dict tiêu đề → DataFrame số; unit: chuỗi đơn vị · Figure: len(tables) panel + 1 Axes colorbar · Mọi panel clim = (0, max của mọi bảng); cmap viridis; tick là tên cột/hàng.

BỘ CHẤM (8 ca)

Ca	Gọi	Mong đợi
Hai panel + một colorbar = 3 Axes	`len(plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes)`	`3`
Mỗi panel vẽ đúng bảng của nó	`[ax.images[0].get_array().tolist() for ax in plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[:2]]`	`[[[10, 20], [30, 40], [5, 0]], [[15, 70], [25, 35], [8, 2]]]`
Mọi panel cùng thang 0..max của MỌI bảng	`[tuple(float(v) for v in ax.images[0].get_clim()) for ax in plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[:2]]`	`[(0.0, 70.0), (0.0, 70.0)]`
Tick x = tên cột, tick y = tên hàng (bảng không vuông)	`(lambda ax: ([t.get_text() for t in ax.get_xticklabels()], [t.get_text() for t in ax.get_yticklabels()]))(plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[0])`	`(['Pen', 'Ink'], ['N', 'S', 'E'])`
Tiêu đề panel là khoá của dict	`[ax.get_title() for ax in plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[:2]]`	`['Jan', 'Feb']`
Colorbar có nhãn đơn vị	`plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[-1].get_ylabel()`	`'Orders'`

Colormap sequential viridis	`[ax.images[0].get_cmap().name for ax in plot_heatmaps({"Jan": pd.DataFrame([[10, 20], [30, 40], [5, 0]], index=["N", "S", "E"], columns=["Pen", "Ink"]), "Feb": pd.DataFrame([[15, 70], [25, 35], [8, 2]], index=["N", "S", "E"], columns=["Pen", "Ink"])}, "Orders").axes[:2]]`	`['viridis', 'viridis']`
Một bảng duy nhất	`(lambda f: (len(f.axes), tuple(float(v) for v in f.axes[0].images[0].get_clim()), [t.get_text() for t in f.axes[0].get_xticklabels()], f.axes[-1].get_ylabel()))(plot_heatmaps({"Q1": pd.DataFrame([[1, 2, 3]], index=["All"], columns=["a", "b", "c"])}, "Units"))`	`(2, (0.0, 3.0), ['a', 'b', 'c'], 'Units')`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. max(... for table in tables.values()) → vmax chung
Khung 2 — Bước 2. plt.subplots(1, k, squeeze=False) → axes[0] luôn là hàng Axes
Khung 3 — Bước 3. ax.imshow(..., vmin=0, vmax=vmax) → mọi panel cùng thang màu
Khung 4 — Bước 4. ax.set_xticks(range(n_cột), table.columns) → tên cột trên trục x
Khung 5 — Bước 5. fig.colorbar(im, ax=list(axes[0]), label=unit) → một thước màu có đơn vị
Khung đối chiếu — ngộ nhận "Mỗi heatmap tự có colorbar riêng là đủ để so sánh". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Hai colorbar với hai thang khác nhau buộc người đọc tra số từng ô — mắt vẫn so màu và kết luận sai. Muốn so thì chung vmin/vmax và một colorbar. Và đừng dùng rainbow cho dữ liệu một chiều: dùng sequential như viridis.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: max(tables), vmin=0, fig.colorbar(im, ax=ax)
Khung kết quả — 2 cặp vào/ra. Jan [[10, 20], [30, 40]], Feb [[5, 15], [25, 60]], "Orders" → 2 panel cùng thang 0..60; 1 colorbar "Orders"; 3 Axes; Một bảng 1×3, "Units" → 1 panel + colorbar "Units"; tick x đủ 3 cột

Nguồn: pe-v4 · python-learning-data.json#M03 · parser pe-v4/build · file build 043-M03.json

M04 · Màu, annotation và kể chuyện dữ liệu

Mã bài: M04 · Nâng cao · M7 · Matplotlib · 35 phút · 20 XP · thứ tự 44

MỤC TIÊU

Nhấn đúng một category mà không bóp méo dữ liệu.

CẦN NHỚ TRƯỚC

M03

BỐI CẢNH

Viết plot_highlight(revenue) với revenue là pandas Series (index = category, giá trị = doanh thu — như kết quả df.groupby("category")["revenue"].sum()). Vẽ ax.bar(revenue.index, revenue.to_numpy(), color=colors) GIỮ nguyên thứ tự và đủ mọi category. pos = vị trí cột lớn nhất (int(revenue.to_numpy().argmax())); cột đó màu HIGHLIGHT, các cột khác màu BASE (hằng cho sẵn). Ghi giá trị cột lớn nhất: ax.annotate(str(value), xy=(pos, value), ha="center", va="bottom"). Trục y từ 0 tới value * 1.2 để chừa chỗ cho chữ. Tiêu đề nói kết quả: f"{top} has the highest revenue"; nhãn y "Revenue". Return fig.

VÍ DỤ MINH HOẠ

plot_highlight(pd.Series([12, 25, 18], index=["A", "B", "C"])) → B màu HIGHLIGHT, chữ "25" tại (1, 25), trục y 0..30, tiêu đề "B has the highest revenue".

ĐIỂM KIẾN THỨC

1. Tiêu đề nên nói KẾT QUẢ mà dữ liệu ủng hộ ("B has the highest revenue"), không chỉ tên biến ("Revenue by category"). Phạm vi, đơn vị, cỡ mẫu đưa vào nhãn trục hoặc dòng phụ.
2. Nhấn MỘT thứ quan trọng: cột cần chú ý một màu đậm, còn lại xám nhạt — mắt đi thẳng tới đó. color=[...] nhận một màu cho mỗi cột, theo đúng thứ tự cột.
3. Giữ màu nhất quán cho cùng nhóm giữa các biểu đồ, và đừng để màu là tín hiệu DUY NHẤT: thêm annotation, tiêu đề, marker hay linestyle (legend phân biệt Actual/Target bằng nét đứt) để người mù màu hoặc bản in đen trắng vẫn đọc được.
4. ax.annotate(text, xy=(x, y)) đặt chữ tại một điểm dữ liệu; trục category đặt cột thứ i ở x = i (0, 1, 2...). Chừa chỗ phía trên (ylim trên = max × 1.2) để chữ không bị cắt hay che dữ liệu.
5. Bar so CHIỀU DÀI nên baseline phải là 0. set_ylim(28, 42) làm cột 30 trông chỉ bằng 1/6 cột 40 dù thật ra là 3/4 — trục bị cắt phóng đại chênh lệch. Nếu buộc phải đổi thang thì nói rõ trên hình.
6. Không bỏ category nhỏ hay điểm "xấu" chỉ để câu chuyện đẹp. Muốn loại dữ liệu thì có chính sách và ghi lại; biểu đồ vẽ đủ những gì Series có, kèm mức độ không chắc chắn nếu có.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt
import pandas as pd

orders = pd.Series([20, 35, 15], index=["A", "B", "C"])
pos = int(orders.to_numpy().argmax())
colors = ["#bfc0c0"] * len(orders)
colors[pos] = "#bf4520"

fig, ax = plt.subplots()
ax.bar(orders.index, orders.to_numpy(), color=colors)
ax.annotate(str(orders.iloc[pos]), xy=(pos, orders.iloc[pos]), ha="center", va="bottom")
ax.set_ylim(0, orders.max() * 1.2)
print(orders.index[pos], pos, colors)
print(ax.texts[0].get_text(), [float(v) for v in ax.get_ylim()])
```

Kết quả dự kiến (theo JSON): [12, 25, 18] → B đậm, chữ "25" tại (1, 25); Trục y luôn bắt đầu từ 0

NGỘ NHẬN THƯỜNG GẶP: "Cắt trục y cho chênh lệch rõ hơn thì biểu đồ dễ đọc hơn"

Với bar, người đọc so CHIỀU DÀI cột. Cắt đáy trục làm chênh lệch nhỏ trông khổng lồ — biểu đồ gây hiểu nhầm dù mọi con số đều đúng. Bar luôn bắt đầu từ 0; muốn soi chênh lệch nhỏ thì vẽ chính chênh lệch đó, hoặc dùng line/dot plot và ghi rõ thang.

CÂU DỰ ĐOÁN

Doanh thu A = 30, B = 40, C = 34 vẽ bar với ax.set_ylim(28, 42). Người xem thấy gì?

sai	Tỷ lệ cột vẫn đúng vì số trên trục y vẫn đúng	Mắt so chiều dài cột, ít ai đọc số trên trục.
sai	Matplotlib tự giữ baseline 0	set_ylim đã ghi đè giới hạn.
ĐÚNG	Cột B trông gấp khoảng 6 lần cột A, dù thật ra chỉ hơn 1/3	Phần nhìn thấy là 40 − 28 = 12 so với 30 − 28 = 2.

TỰ KIỂM TRA

1. Muốn người xem thấy ngay category nổi bật nhất, cách nào tốt nhất?

• Xoá các category nhỏ cho gọn — Bỏ dữ liệu làm câu chuyện sai lệch.

✔ Một màu đậm cho cột đó, màu xám cho phần còn lại, kèm annotation giá trị — Nhấn một thứ, và không dựa vào màu đơn độc.

• Mỗi cột một màu cầu vồng — Màu nào cũng nổi thì không gì nổi.

Đáp án: Nhấn một thứ, và không dựa vào màu đơn độc.

2. Trục category ["A", "B", "C"] — muốn annotate cột "B" thì xy có x bằng bao nhiêu?

• 2 — vì B là chữ thứ hai — Vị trí đếm từ 0.

• Bằng giá trị của cột B — Giá trị là toạ độ y, không phải x.

✔ 1 — cột thứ i nằm ở x = i, đếm từ 0 — Category được đặt ở 0, 1, 2…

Đáp án: Category được đặt ở 0, 1, 2…

3. Vẽ Actual và Target trên cùng Axes — cách nào vẫn phân biệt được khi in đen trắng?

✔ label cho mỗi đường + ax.legend(); Target nét đứt linestyle="--", Actual có marker — Nét và marker vẫn còn khi mất màu.

• Chỉ đổi màu hai đường — Màu đơn độc mất khi in đen trắng hoặc với người mù màu.

• Ghi số lên mọi điểm — Chữ dày đặc che dữ liệu; legend và kiểu nét là đủ.

Đáp án: Nét và marker vẫn còn khi mất màu.

TỰ GIẢI THÍCH

Vì sao plot_highlight đặt ylim dưới = 0 và vẽ đủ mọi category, dù cắt trục hay bỏ category nhỏ làm hình "ấn tượng" hơn?

Mẫu trả lời: Người đọc bar so chiều dài cột; baseline khác 0 phóng đại chênh lệch nên hình nói sai dù số đúng. Bỏ category nhỏ là chọn dữ liệu cho hợp câu chuyện. Nhấn cột lớn nhất bằng màu + annotation + tiêu đề nói kết quả là đủ để kể chuyện mà không bóp méo dữ liệu.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	revenue.to_numpy().argmax() → vị trí cột lớn nhất
Bước 2	colors[pos] = HIGHLIGHT → chỉ một cột đậm, còn lại BASE
Bước 3	ax.annotate(str(value), xy=(pos, value)) → số ngay trên cột
Bước 4	ax.set_ylim(0, value * 1.2) → baseline 0, chừa chỗ cho chữ
Bước 5	title=f"{top} has the highest revenue" → tiêu đề nói kết quả

CÂU PHẢN VÍ DỤ

revenue = Series([12, 25, 18], index=["A", "B", "C"]) → giới hạn trục y là?

Lựa chọn: (12, 30) | (0, 25) | (0, 30) · Đáp án: (0, 30)

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt
import pandas as pd

HIGHLIGHT = "#bf4520"
BASE = "#bfc0c0"


def plot_highlight(revenue):
    pos = {{slot1}}
    top, value = revenue.index[pos], revenue.iloc[pos]
    colors = [BASE] * len(revenue)
    colors[pos] = HIGHLIGHT

    fig, ax = plt.subplots()
    ax.bar(revenue.index, revenue.to_numpy(), color=colors)
    ax.annotate(str(value), {{slot2}}, ha="center", va="bottom")
    {{slot3}}
    ax.set(title=f"{top} has the highest revenue", ylabel="Revenue")
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`int(revenue.to_numpy().argmax())`
[Slot 2]	`xy=(pos, value)`
[Slot 3]	`ax.set_ylim(0, value * 1.2)`

Mảnh bẫy	Vì sao sai
`xy=(value, pos)`	Đảo x và y: x là vị trí cột, y là giá trị.
`int(revenue.to_numpy().argmin())`	argmin là cột NHỎ nhất.
`1`	Viết cứng vị trí cột B của ví dụ — dữ liệu khác sẽ nhấn sai cột.
`ax.set_ylim(revenue.min() * 0.9, value * 1.2)`	Cắt đáy trục — phóng đại chênh lệch giữa các cột.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: tìm vị trí cột lớn nhất, đặt annotation đúng toạ độ, giữ baseline 0.

Yêu cầu thêm: Thử Series 4 category mà cột lớn nhất nằm cuối; kiểm màu, annotation và ylim theo dữ liệu mới, không theo ví dụ.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết plot_highlight(revenue) với revenue là pandas Series (index = category, giá trị = doanh thu — như kết quả df.groupby("category")["revenue"].sum()). Vẽ ax.bar(revenue.index, revenue.to_numpy(), color=colors) GIỮ nguyên thứ tự và đủ mọi category. pos = vị trí cột lớn nhất (int(revenue.to_numpy().argmax())); cột đó màu HIGHLIGHT, các cột khác màu BASE (hằng cho sẵn). Ghi giá trị cột lớn nhất: ax.annotate(str(value), xy=(pos, value), ha="center", va="bottom"). Trục y từ 0 tới value * 1.2 để chừa chỗ cho chữ. Tiêu đề nói kết quả: f"{top} has the highest revenue"; nhãn y "Revenue". Return fig.

Thực hành: Tính pos bằng argmax trên dữ liệu (không viết cứng); colors theo pos; annotate tại (pos, value); ylim (0, value * 1.2).

Gọi	Kết quả mong đợi
`Series([12, 25, 18], index=["A", "B", "C"])`	`B đậm; chữ "25" tại (1, 25); trục y 0..30; "B has the highest revenue"`
`Series([50, 20, 5], index=["X", "Y", "Z"])`	`X đậm (cột đầu); chữ "50" tại (0, 50); đủ 3 cột`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_highlight(revenue)`	Kể chuyện "category nào cao nhất" mà không bóp méo dữ liệu. · revenue: pandas Series số dương, index = tên category · Figure 1 Axes, đủ len(revenue) cột theo thứ tự Series · Một cột HIGHLIGHT; một annotation tại (pos, value); ylim (0, value * 1.2).

BỘ CHẤM (7 ca)

Ca	Gọi	Mong đợi
Đủ 4 cột, đúng thứ tự Series	`[float(v) for v in (p.get_height() for p in plot_highlight(pd.Series([18, 9, 31, 22], index=["Pen", "Ink", "Bag", "Book"])).axes[0].patches)]`	`[18.0, 9.0, 31.0, 22.0]`
Chỉ cột lớn nhất (Bag) có màu HIGHLIGHT	`[__import__("matplotlib.colors", fromlist=["to_hex"]).to_hex(p.get_facecolor()) for p in plot_highlight(pd.Series([18, 9, 31, 22], index=["Pen", "Ink", "Bag", "Book"])).axes[0].patches]`	`['#bfc0c0', '#bfc0c0', '#bf4520', '#bfc0c0']`
Annotation ghi giá trị tại cột lớn nhất	`[(t.get_text(), tuple(float(v) for v in t.xy)) for t in plot_highlight(pd.Series([18, 9, 31, 22], index=["Pen", "Ink", "Bag", "Book"])).axes[0].texts]`	`[('31', (2.0, 31.0))]`
Baseline 0, chừa chỗ phía trên	`tuple(round(float(v), 6) for v in plot_highlight(pd.Series([18, 9, 31, 22], index=["Pen", "Ink", "Bag", "Book"])).axes[0].get_ylim())`	`(0.0, 37.2)`
Tiêu đề nói kết quả; nhãn y	`(lambda ax: (ax.get_title(), ax.get_ylabel()))(plot_highlight(pd.Series([18, 9, 31, 22], index=["Pen", "Ink", "Bag", "Book"])).axes[0])`	`('Bag has the highest revenue', 'Revenue')`
Cột lớn nhất đứng đầu Series	`(lambda ax: ([__import__("matplotlib.colors", fromlist=["to_hex"]).to_hex(p.get_facecolor()) for p in ax.patches], [(t.get_text(), tuple(float(v) for v in t.xy)) for t in ax.texts]))(plot_highlight(pd.Series([50, 20, 5], index=["X", "Y", "Z"])).axes[0])`	`(['#bf4520', '#bfc0c0', '#bfc0c0'], [('50', (0.0, 50.0))])`

Một category duy nhất	`(lambda ax: ([float(v) for v in (p.get_height() for p in ax.patches)], ax.get_title(), tuple(round(float(v), 6) for v in ax.get_ylim())))(plot_highlight(pd.Series([7], index=["Only"])).axes[0])`	`([7.0], 'Only has the highest revenue', (0.0, 8.4))`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. revenue.to_numpy().argmax() → vị trí cột lớn nhất
Khung 2 — Bước 2. colors[pos] = HIGHLIGHT → chỉ một cột đậm, còn lại BASE
Khung 3 — Bước 3. ax.annotate(str(value), xy=(pos, value)) → số ngay trên cột
Khung 4 — Bước 4. ax.set_ylim(0, value * 1.2) → baseline 0, chừa chỗ cho chữ
Khung 5 — Bước 5. title=f"{top} has the highest revenue" → tiêu đề nói kết quả
Khung đối chiếu — ngộ nhận "Cắt trục y cho chênh lệch rõ hơn thì biểu đồ dễ đọc hơn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Với bar, người đọc so CHIỀU DÀI cột. Cắt đáy trục làm chênh lệch nhỏ trông khổng lồ — biểu đồ gây hiểu nhầm dù mọi con số đều đúng. Bar luôn bắt đầu từ 0; muốn soi chênh lệch nhỏ thì vẽ chính chênh lệch đó, hoặc dùng line/dot plot và ghi rõ thang.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: xy=(value, pos), int(revenue.to_numpy().argmin()), 1, ax.set_ylim(revenue.min() * 0.9, value * 1.2)
Khung kết quả — 2 cặp vào/ra. Series([12, 25, 18], index=["A", "B", "C"]) → B đậm; chữ "25" tại (1, 25); trục y 0..30; "B has the highest revenue"; Series([50, 20, 5], index=["X", "Y", "Z"]) → X đậm (cột đầu); chữ "50" tại (0, 50); đủ 3 cột

Nguồn: pe-v4 · python-learning-data.json#M04 · parser pe-v4/build · file build 044-M04.json

M05 · Ticks, scale và ngày trên trục

Mã bài: M05 · Nâng cao · M7 · Matplotlib · 40 phút · 20 XP · thứ tự 45

MỤC TIÊU

Trục thời gian đúng thứ tự, trục % đúng thang, trục log đúng chỗ.

CẦN NHỚ TRƯỚC

M04

BỐI CẢNH

Viết hai hàm trả về Figure. (1) plot_rate(df): df có cột "date" (chuỗi "YYYY-MM-DD", CHƯA sắp) và "rate" (tỷ lệ 0..1). Chuyển date bằng pd.to_datetime, sắp theo ngày, vẽ line marker "o"; trục y dùng PercentFormatter(xmax=1) và ylim 0..1; trục x dùng mdates.DateFormatter("%d/%m"); nhãn y "Conversion rate". (2) plot_log_sizes(names, sizes): bar với trục y log (ax.set_yscale("log")), nhãn y "Users (log scale)" để người đọc biết khoảng cách đã đổi nghĩa; nếu có size <= 0 thì raise ValueError TRƯỚC khi tạo figure.

VÍ DỤ MINH HOẠ

plot_rate với ngày 03/01, 01/01, 02/01 và rate 0.5, 0.25, 0.4 → đường đi qua 0.25, 0.4, 0.5 theo ngày; trục y ghi 0%, 20%, …, 100%. plot_log_sizes(["A", "B"], [10, 0]) → ValueError.

ĐIỂM KIẾN THỨC

1. Line nối điểm theo THỨ TỰ HÀNG trong dữ liệu, không theo thứ tự ngày. Dữ liệu chưa sắp thì đường đi tới, lùi, rồi tới — một hình zig-zag giả. Chuyển cột ngày bằng pd.to_datetime rồi sort_values("date") trước khi vẽ (chuỗi kiểu "2026-1-10" so theo chữ còn đứng trước "2026-1-9").
2. Formatter đổi CÁCH VIẾT nhãn, không đổi dữ liệu. PercentFormatter(xmax=1) hiểu 1.0 là 100%: 0.25 → "25%". Để mặc định xmax=100 thì 0.25 thành "0.25%" — sai 100 lần mà không có lỗi nào. Tỷ lệ 0..1 và số phần trăm 0..100 là hai thứ khác nhau.
3. FuncFormatter(lambda x, pos: f"{x/1000:g}k") cho nhãn tự chọn (1000 → "1k"); DateFormatter("%d/%m") viết ngày gọn, fig.autofmt_xdate() xoay nhãn ngày để không chồng. Tick label phải đi cùng vị trí tick — đổi nhãn bằng set_xticklabels mà không đặt vị trí là gán chữ cho sai chỗ.
4. Trục log (ax.set_yscale("log")) hợp với dải rất rộng (120 và 300 000): mỗi nấc là GẤP 10, không phải cộng thêm một khoảng bằng nhau — nên phải nói ra trên nhãn trục. log không định nghĩa cho 0 và số âm: Matplotlib lặng lẽ bỏ các cột đó khỏi hình, vì vậy hàm phải tự kiểm và raise ValueError.
5. Giới hạn trục phải giữ dữ liệu cần thấy: ylim(0, 1) cho tỷ lệ giữ cả mốc 0% và 100%. Trước khi xuất, đọc lại nhãn và thử diễn giải hình như người xem.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
from matplotlib.ticker import PercentFormatter

df = pd.DataFrame({"date": ["2026-01-03", "2026-01-01", "2026-01-02"],
                   "rate": [0.3, 0.1, 0.2]})
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values("date")

fig, ax = plt.subplots()
ax.plot(df["date"], df["rate"], marker="o")
ax.yaxis.set_major_formatter(PercentFormatter(xmax=1))
ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m"))
ax.set_ylim(0, 1)

print(df["rate"].tolist())
print(ax.yaxis.get_major_formatter()(0.3))
print(ax.xaxis.get_major_formatter()(mdates.date2num(df["date"].iloc[0])))
```

Kết quả dự kiến (theo JSON): Ngày xáo trộn → đường đi theo thứ tự ngày; 0.25 trên trục y → "25%"

NGỘ NHẬN THƯỜNG GẶP: "PercentFormatter() mặc định là đủ cho mọi dữ liệu phần trăm"

Mặc định xmax=100: nó coi 100 là 100%. Dữ liệu tỷ lệ 0..1 (0.25) sẽ hiện "0.25%". Với tỷ lệ phải ghi PercentFormatter(xmax=1). Formatter chỉ đổi chữ trên trục; nhầm xmax là sai thông điệp 100 lần mà không có lỗi nào báo.

CÂU DỰ ĐOÁN

Đã pd.to_datetime nhưng QUÊN sort: ngày 03/01, 01/01, 02/01 với rate 0.5, 0.25, 0.4. Đường line nối các điểm theo thứ tự nào?

ĐÚNG	03/01 → 01/01 → 02/01: đi lùi rồi tới, thành hình zig-zag giả	Line nối theo thứ tự hàng trong DataFrame.
sai	Báo lỗi vì ngày chưa sắp	Không có lỗi — chính vì im lặng nên dễ sót.
sai	01/01 → 02/01 → 03/01 vì trục ngày tự sắp	Điểm nằm đúng vị trí ngày, nhưng đường nối theo thứ tự hàng.

TỰ KIỂM TRA

1. Số người dùng 5 cửa hàng: 120, 900, 4 000, 45 000, 300 000. Vẽ bar trục y tuyến tính thì…

• Ổn, bar luôn dùng trục tuyến tính — Có thể, nhưng mất thông tin về các cửa hàng nhỏ.

✔ Mấy cửa hàng nhỏ gần như không thấy — cân nhắc trục log và ghi rõ "(log scale)" — Dải quá rộng; log cho thấy cấp độ lớn.

• Dùng trục log, không cần ghi gì — Log đổi nghĩa khoảng cách — phải nói rõ.

Đáp án: Dải quá rộng; log cho thấy cấp độ lớn.

2. ax.set_yscale("log") với một cột có giá trị 0 thì…

✔ Cột đó biến mất khỏi hình mà không báo lỗi — log(0) không xác định; Matplotlib che giá trị không dương.

• Python báo ZeroDivisionError — Không có lỗi nào — vì vậy hàm phải tự kiểm.

• Cột cao 0 như bình thường — Trên trục log không có vị trí cho 0.

Đáp án: log(0) không xác định; Matplotlib che giá trị không dương.

3. Vì sao chuyển cột ngày dạng chuỗi bằng pd.to_datetime trước khi sort và vẽ?

✔ Chuỗi so theo từng ký tự; ngày thật so theo thời gian và giữ đúng khoảng cách trên trục — Với "2026-1-9" và "2026-1-10", sắp chuỗi cho thứ tự sai.

• Để chạy nhanh hơn — Lý do là đúng thứ tự và đúng khoảng cách.

• Matplotlib không vẽ được chuỗi — Vẽ được — nhưng thành category, mất khoảng cách thời gian.

Đáp án: Với "2026-1-9" và "2026-1-10", sắp chuỗi cho thứ tự sai.

TỰ GIẢI THÍCH

Vì sao plot_log_sizes kiểm size <= 0 và raise ValueError thay vì cứ vẽ?

Mẫu trả lời: Trên trục log không có vị trí cho 0 hay số âm; Matplotlib lặng lẽ không vẽ các cột đó và không báo lỗi, nên người đọc tưởng dữ liệu chỉ có những cột còn lại. Raise ValueError trước khi tạo figure buộc nơi gọi xử lý dữ liệu không dương một cách có chủ đích.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	pd.to_datetime(df["date"]) → chuỗi thành Timestamp
Bước 2	data.sort_values("date") → hàng theo thứ tự thời gian
Bước 3	PercentFormatter(xmax=1) → 0.25 hiện "25%"
Bước 4	DateFormatter("%d/%m") → nhãn ngày gọn
Bước 5	any(size <= 0 ...) → ValueError trước khi vẽ log

CÂU PHẢN VÍ DỤ

Với PercentFormatter(xmax=1) trên trục 0..1, giá trị 0.05 hiện là?

Lựa chọn: 0.05% | 5% | 50% · Đáp án: 5%

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd
from matplotlib.ticker import PercentFormatter


def plot_rate(df):
    data = df.assign(date=pd.to_datetime(df["date"]))
    data = {{slot1}}
    fig, ax = plt.subplots()
    ax.plot(data["date"], data["rate"], marker="o")
    {{slot2}}
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m"))
    ax.set_ylim(0, 1)
    ax.set_ylabel("Conversion rate")
    return fig


def plot_log_sizes(names, sizes):
    if {{slot3}}:
        raise ValueError("log scale needs positive sizes")
```

```
    fig, ax = plt.subplots()
    ax.bar(names, sizes)
    ax.set_yscale("log")
    ax.set_ylabel("Users (log scale)")
    return fig
```

Slot	Mảnh đúng
[Slot 1]	`data.sort_values("date")`
[Slot 2]	`ax.yaxis.set_major_formatter(PercentFormatter(xmax=1))`
[Slot 3]	`any(size <= 0 for size in sizes)`

Mảnh bẫy	Vì sao sai
`data.sort_values("rate")`	Sắp theo giá trị — đường không còn đi theo thời gian.
`ax.yaxis.set_major_formatter(PercentFormatter())`	xmax mặc định 100: 0.25 hiện thành 0.25%.
`all(size <= 0 for size in sizes)`	Chỉ chặn khi MỌI giá trị không dương; một số 0 vẫn lọt.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: sắp theo thời gian, formatter phần trăm đúng thang, điều kiện chặn trục log.

Yêu cầu thêm: Thử 4 ngày xáo trộn vắt qua tháng 2 và một bộ sizes có số âm; đoán trước nhãn và lỗi rồi mới chạy.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm trả về Figure. (1) plot_rate(df): df có cột "date" (chuỗi "YYYY-MM-DD", CHƯA sắp) và "rate" (tỷ lệ 0..1). Chuyển date bằng pd.to_datetime, sắp theo ngày, vẽ line marker "o"; trục y dùng PercentFormatter(xmax=1) và ylim 0..1; trục x dùng mdates.DateFormatter("%d/%m"); nhãn y "Conversion rate". (2) plot_log_sizes(names, sizes): bar với trục y log (ax.set_yscale("log")), nhãn y "Users (log scale)" để người đọc biết khoảng cách đã đổi nghĩa; nếu có size <= 0 thì raise ValueError TRƯỚC khi tạo figure.

Thực hành: plot_rate: chuyển ngày → sort → vẽ → formatter % (xmax=1) và formatter ngày → ylim → nhãn. plot_log_sizes: kiểm any(size <= 0) TRƯỚC khi tạo figure.

Gọi	Kết quả mong đợi
`date 03/01, 01/01, 02/01; rate 0.5, 0.25, 0.4`	`line qua 0.25, 0.4, 0.5; trục y ghi 0%, 20%, …, 100%; nhãn x dạng 01/01`
`plot_log_sizes(["A", "B"], [10, 0])`	`ValueError, chưa tạo figure`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`plot_rate(df)`	Vẽ tỷ lệ chuyển đổi theo ngày. · df: DataFrame cột "date" (chuỗi YYYY-MM-DD) và "rate" (0..1) · Figure 1 Axes · Line theo thứ tự ngày; trục y PercentFormatter(xmax=1), 0..1; trục x DateFormatter("%d/%m").
`plot_log_sizes(names, sizes)`	So các cỡ chênh nhau nhiều bậc. · names: list tên; sizes: list số DƯƠNG · Figure 1 Axes, trục y log · size <= 0 → ValueError trước khi tạo figure; nhãn y có chữ "log".

BỘ CHẤM (9 ca)

Ca	Gọi	Mong đợi
Đường đi theo thứ tự ngày	`[float(v) for v in plot_rate(pd.DataFrame({"date": ["2026-02-02", "2026-01-30", "2026-02-01", "2026-01-31"], "rate": [0.6, 0.2, 0.45, 0.3]})).axes[0].lines[0].get_ydata()]`	`[0.2, 0.3, 0.45, 0.6]`
Trục x tăng dần theo thời gian	`[str(v)[:10] for v in plot_rate(pd.DataFrame({"date": ["2026-02-02", "2026-01-30", "2026-02-01", "2026-01-31"], "rate": [0.6, 0.2, 0.45, 0.3]})).axes[0].lines[0].get_xdata()]`	`['2026-01-30', '2026-01-31', '2026-02-01', '2026-02-02']`
Trục y là phần trăm của tỷ lệ 0..1	`[plot_rate(pd.DataFrame({"date": ["2026-02-02", "2026-01-30", "2026-02-01", "2026-01-31"], "rate": [0.6, 0.2, 0.45, 0.3]})).axes[0].yaxis.get_major_formatter()(v) for v in (0.25, 0.05, 1)]`	`['25%', '5%', '100%']`
ylim 0..1 và nhãn y	`(lambda ax: (tuple(float(v) for v in ax.get_ylim()), ax.get_ylabel()))(plot_rate(pd.DataFrame({"date": ["2026-02-02", "2026-01-30", "2026-02-01", "2026-01-31"], "rate": [0.6, 0.2, 0.45, 0.3]})).axes[0])`	`((0.0, 1.0), 'Conversion rate')`

Nhãn ngày dạng ngày/tháng	`plot_rate(pd.DataFrame({"date": ["2026-02-02", "2026-01-30", "2026-02-01", "2026-01-31"], "rate": [0.6, 0.2, 0.45, 0.3]})).axes[0].xaxis.get_major_formatter()(__import__("matplotlib.dates", fromlist=["date2num"]).date2num(pd.Timestamp("2026-01-31")))`	`'31/01'`
Trục log có ghi chú "log" trên nhãn	`(lambda ax: (ax.get_yscale(), "log" in ax.get_ylabel().lower()))(plot_log_sizes(["A", "B", "C"], [120, 4000, 300000]).axes[0])`	`('log', True)`
Trục log: cột đúng dữ liệu	`[float(v) for v in (p.get_height() for p in plot_log_sizes(["A", "B", "C"], [120, 4000, 300000]).axes[0].patches)]`	`[120.0, 4000.0, 300000.0]`
Có giá trị 0 → ValueError, chưa tạo figure	`(lambda n: (_studio.call(plot_log_sizes, ["A", "B"], [10, 0]).error, len(plt.get_fignums()) - n))(len(plt.get_fignums()))`	`('ValueError', 0)`
Số âm → ValueError	`_studio.call(plot_log_sizes, ["A"], [-5]).error`	`'ValueError'`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy, pandas.

KỊCH BẢN MINH HOẠ — ƯU TIÊN CAO (trace 5 bước, có scaffold 3 slot)
Khung 1 — Bước 1. pd.to_datetime(df["date"]) → chuỗi thành Timestamp
Khung 2 — Bước 2. data.sort_values("date") → hàng theo thứ tự thời gian
Khung 3 — Bước 3. PercentFormatter(xmax=1) → 0.25 hiện "25%"
Khung 4 — Bước 4. DateFormatter("%d/%m") → nhãn ngày gọn
Khung 5 — Bước 5. any(size <= 0 ...) → ValueError trước khi vẽ log
Khung đối chiếu — ngộ nhận "PercentFormatter() mặc định là đủ cho mọi dữ liệu phần trăm". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Mặc định xmax=100: nó coi 100 là 100%. Dữ liệu tỷ lệ 0..1 (0.25) sẽ hiện "0.25%". Với tỷ lệ phải ghi PercentFormatter(xmax=1). Formatter chỉ đổi chữ trên trục; nhầm xmax là sai thông điệp 100 lần mà không có lỗi nào báo.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: data.sort_values("rate"), ax.yaxis.set_major_formatter(PercentFormatter()), all(size <= 0 for size in sizes)
Khung kết quả — 2 cặp vào/ra. date 03/01, 01/01, 02/01; rate 0.5, 0.25, 0.4 → line qua 0.25, 0.4, 0.5; trục y ghi 0%, 20%, ..., 100%; nhãn x dạng 01/01; plot_log_sizes(["A", "B"], [10, 0]) → ValueError, chưa tạo figure

Nguồn: pe-v4 · python-learning-data.json#M05 · parser pe-v4/build · file build 045-M05.json

M06 · Layout, xuất hình và kiểm tra chất lượng

Mã bài: M06 · Nâng cao · M7 · Matplotlib · 40 phút · 20 XP · thứ tự 46

MỤC TIÊU

Xuất hình đúng kích thước và dọn figure khi chạy hàng loạt.

CẦN NHỚ TRƯỚC

M05

BỐI CẢNH

Viết hai hàm. (1) make_chart(title, values): fig, ax = plt.subplots(figsize=(6, 3), layout="constrained"); line marker "o" của values theo ngày 1, 2, …, len(values); tiêu đề title, nhãn "Day" / "Revenue"; return fig. (2) export_all(charts, dpi): charts là dict tiêu đề → list giá trị. Với từng mục: fig = make_chart(...), lưu fig vào io.BytesIO dạng PNG với dpi cho sẵn (KHÔNG dùng bbox_inches="tight" — để cần ảnh đúng 6·dpi × 3·dpi pixel), rồi plt.close(fig). Trả dict tiêu đề → bytes PNG. Sau export_all, số figure đang mở không đổi.

VÍ DỤ MINH HOẠ

make_chart("Week 1", [10, 20, 15]) → figure 6×3 inch, layout constrained. export_all({"Week 1": [10, 20, 15]}, 150) → {"Week 1": bytes PNG 900×450}, không figure nào còn mở.

ĐIỂM KIẾN THỨC

1. figsize tính bằng INCH, dpi là số điểm ảnh mỗi inch: PNG 6×3 inch ở dpi 150 có 900×450 pixel. Tăng dpi làm ảnh mịn hơn khi in hay phóng to nhưng KHÔNG thêm chi tiết dữ liệu nào.
2. layout="constrained" (đặt khi tạo figure) tự dàn lề để tiêu đề, nhãn trục, colorbar không chồng nhau. tight_layout() là cách cũ, chạy một lần mỗi khi gọi; chọn một trong hai, đừng dùng cả hai trên cùng figure.
3. PNG là ảnh raster (điểm ảnh) — hợp cho web, slide. SVG/PDF là vector — phóng to không vỡ, hợp cho báo cáo in. savefig đoán kiểu ảnh từ đuôi tên file ("chart.svg"); ghi vào BytesIO/StringIO thì phải nói rõ format="png"/"svg". bbox_inches="tight" cắt/thêm lề theo nội dung nên kích thước ảnh ra KHÁC figsize × dpi.
4. Kiểm FILE ĐÃ LƯU, không chỉ cửa sổ trên màn hình: đọc lại bytes (chữ ký PNG, rộng/cao ở byte 16–24) hoặc mở file ra xem. Chốt font, màu, nền, đơn vị trước khi xuất hàng loạt.
5. Thứ tự đúng: vẽ → savefig → close. Vòng lặp xuất 100 biểu đồ mà không plt.close(fig) giữ 100 figure trong bộ nhớ (Matplotlib cảnh báo khi quá 20). Script cần hiển thị thì gọi plt.show() ở CUỐI, sau khi đã lưu.

MẪU CODE · vi_du.py · Python 3.12 · demo

```
import matplotlib.pyplot as plt
from io import BytesIO, StringIO

fig, ax = plt.subplots(figsize=(4, 2), layout="constrained")
ax.bar(["A", "B"], [3, 4])
ax.set_title("Orders")

png = BytesIO()
fig.savefig(png, format="png", dpi=100)
data = png.getvalue()
print(int.from_bytes(data[16:20], "big"), int.from_bytes(data[20:24], "big"))

svg = StringIO()
fig.savefig(svg, format="svg")
print("<svg" in svg.getvalue())

before = len(plt.get_fignums())
plt.close(fig)
print(before, len(plt.get_fignums()))
```

Kết quả dự kiến (theo JSON): dpi 20 → mỗi PNG 120×60; Sau export_all không còn figure nào mở

NGỘ NHẬN THƯỜNG GẶP: "Tăng dpi là biểu đồ có nhiều thông tin hơn"

dpi chỉ là số điểm ảnh mỗi inch: cùng đường, cùng nhãn, chỉ mịn hơn và file nặng hơn. Muốn biểu đồ nói nhiều hơn phải đổi dữ liệu hoặc cách trình bày. Và bbox_inches="tight" đổi kích thước canvas — để cần đúng kích thước thì đừng dùng.

CÂU DỰ ĐOÁN

Figure figsize=(6, 3) lưu PNG với dpi=150 (không bbox_inches) — ảnh bao nhiêu pixel?

ĐÚNG	900 × 450	6 × 150 và 3 × 150.
sai	Tuỳ nội dung hình	Chỉ khi có bbox_inches="tight" kích thước mới phụ thuộc nội dung.
sai	6 × 3	figsize tính bằng inch, không phải pixel.
sai	600 × 300	Đó là dpi=100.

TỰ KIỂM TRA

1. Báo cáo in A4, người đọc sẽ phóng to biểu đồ. Nên xuất kiểu nào?

• PNG dpi=72 — Raster dpi thấp sẽ vỡ hạt khi phóng to.

✔ SVG hoặc PDF (vector) — Vector phóng to không vỡ.

• Chụp màn hình cửa sổ plt.show() — Kích thước phụ thuộc màn hình, không lặp lại được.

Đáp án: Vector phóng to không vỡ.

2. Vòng lặp xuất 200 biểu đồ, mỗi vòng plt.subplots() + savefig nhưng không close. Chuyện gì xảy ra?

✔ 200 figure vẫn mở, chiếm bộ nhớ; Matplotlib cảnh báo khi quá 20 figure — Batch phải plt.close(fig) sau mỗi lần lưu.

• savefig tự đóng figure — savefig không đóng gì cả.

• Figure cũ tự bị ghi đè — Mỗi plt.subplots() tạo một figure mới.

Đáp án: Batch phải plt.close(fig) sau mỗi lần lưu.

3. fig.savefig(buf, dpi=50, bbox_inches="tight") với figure 6×3 inch cho ảnh...

• Đúng 300×150 — Chỉ đúng khi không có bbox_inches="tight".

• Lỗi vì thiếu layout — Không có lỗi nào.

✔ Gần nhưng không đúng 300×150 — kích thước theo vùng nội dung — tight cắt/thêm lề nên canvas xuất thay đổi.

Đáp án: tight cắt/thêm lề nên canvas xuất thay đổi.

TỰ GIẢI THÍCH

Vì sao export_all phải gọi plt.close(fig) sau khi lưu, và vì sao gọi SAU savefig chứ không trước?

Mẫu trả lời: Mỗi make_chart tạo một figure mới mà pyplot giữ lại tới khi close; xuất hàng loạt không close thì figure dồn lại, tốn bộ nhớ. Close trước rồi plt.savefig thì không còn figure đó: pyplot tạo figure trống mới, ảnh ra trắng, sai kích thước và lại thêm một figure mở. Đúng thứ tự: fig.savefig → plt.close(fig).

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	plt.subplots(figsize=(6, 3), layout="constrained") → 6×3 inch
Bước 2	fig.savefig(buf, format="png", dpi=dpi) → PNG 6·dpi × 3·dpi
Bước 3	plt.close(fig) → figure đóng ngay sau khi lưu
Bước 4	result[title] = buf.getvalue() → bytes để kiểm hoặc ghi file

CÂU PHẢN VÍ DỤ

export_all({"A": [1, 2]}, 40) — ảnh "A" bao nhiêu pixel?

Lựa chọn: 600 × 300 | 240 × 120 | 6 × 3 · Đáp án: 240 × 120

TEMPLATE LẮP MẢNH

```
import matplotlib.pyplot as plt
from io import BytesIO


def make_chart(title, values):
    fig, ax = {{slot1}}
    ax.plot(range(1, len(values) + 1), values, marker="o")
    ax.set(title=title, xlabel="Day", ylabel="Revenue")
    return fig


def export_all(charts, dpi):
    result = {}
    for title, values in charts.items():
        fig = make_chart(title, values)
        buf = BytesIO()
        {{slot2}}
        {{slot3}}
        result[title] = buf.getvalue()
    return result
```

Slot	Mảnh đúng
[Slot 1]	`plt.subplots(figsize=(6, 3), layout="constrained")`
[Slot 2]	`fig.savefig(buf, format="png", dpi=dpi)`
[Slot 3]	`plt.close(fig)`

Mảnh bẫy	Vì sao sai

`plt.show()`	Không đóng figure; trên máy còn chặn vòng lặp tới khi tắt cửa sổ.
`fig.savefig(buf, format="png", dpi=dpi, bbox_inches="tight")`	Cắt theo nội dung → không đúng 6·dpi × 3·dpi.
`fig.clf()`	Xoá nội dung nhưng figure vẫn mở.
`plt.subplots(figsize=(600, 300))`	figsize tính bằng inch, không phải pixel.

LUYỆN CÓ HỖ TRỢ · Luyện có hỗ trợ

Lắp ba chỗ: kích thước + layout của figure, lưu PNG đúng dpi, đóng figure trong vòng lặp.

Yêu cầu thêm: Xuất 3 biểu đồ ở dpi 100 rồi dpi 40; kiểm kích thước từng PNG và số figure đang mở trước/sau.

BÀI TẬP ĐỘC LẬP · Thử thách độc lập

Viết hai hàm. (1) make_chart(title, values): fig, ax = plt.subplots(figsize=(6, 3), layout="constrained"); line marker "o" của values theo ngày 1, 2, ..., len(values); tiêu đề title, nhãn "Day" / "Revenue"; return fig. (2) export_all(charts, dpi): charts là dict tiêu đề → list giá trị. Với từng mục: fig = make_chart(...), lưu fig vào io.BytesIO dạng PNG với dpi cho sẵn (KHÔNG dùng bbox_inches="tight" — để cần ảnh đúng 6·dpi × 3·dpi pixel), rồi plt.close(fig). Trả dict tiêu đề → bytes PNG. Sau export_all, số figure đang mở không đổi.

Thực hành: make_chart chốt figsize (inch) và layout ngay khi tạo; export_all theo đúng thứ tự make_chart → savefig → close cho từng mục.

Gọi	Kết quả mong đợi
`make_chart("Week 1", [10, 20, 15])`	`figure 6×3 inch, layout constrained; line qua 10, 20, 15`
`export_all({"Week 1": [10, 20, 15]}, 150)`	`{"Week 1": PNG 900×450}; không figure nào còn mở`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`make_chart(title, values)`	Dựng một biểu đồ doanh thu khổ cố định. · title: chuỗi; values: list số · Figure 6×3 inch, layout constrained · Line marker "o" theo ngày 1..n; nhãn Day / Revenue.
`export_all(charts, dpi)`	Xuất hàng loạt biểu đồ ra PNG trong bộ nhớ. · charts: dict tiêu đề → list số; dpi: int · dict tiêu đề → bytes PNG (6·dpi × 3·dpi) · Đóng mọi figure đã tạo; số figure mở trước/sau như nhau.

BỘ CHẤM (6 ca)

Ca	Gọi	Mong đợi
make_chart: figure 6×3 inch	`make_chart("W", [1, 2, 3]).get_size_inches().tolist()`	`[6.0, 3.0]`
make_chart: layout constrained	`type(make_chart("W", [1, 2, 3]).get_layout_engine()).__name__`	`'ConstrainedLayoutEngine'`
make_chart: dữ liệu theo ngày, tiêu đề, nhãn	`(lambda ax: ([float(v) for v in ax.lines[0].get_xdata()], [float(v) for v in ax.lines[0].get_ydata()], ax.get_title(), ax.get_xlabel(), ax.get_ylabel()))(make_chart("Week 2", [8, 3, 5, 9]).axes[0])`	`([1.0, 2.0, 3.0, 4.0], [8.0, 3.0, 5.0, 9.0], 'Week 2', 'Day', 'Revenue')`
export_all: đủ khoá, mỗi giá trị là PNG 6·dpi × 3·dpi (dpi 20)	`{k: (v[:8], (lambda b: (int.from_bytes(b[16:20], "big"), int.from_bytes(b[20:24], "big")))(v)) for k, v in export_all({"A": [1, 2], "B": [3, 1, 2]}, 20).items()}`	`{'A': (b'\x89PNG\r\n\x1a\n', (120, 60)), 'B': (b'\x89PNG\r\n\x1a\n', (120, 60))}`
Pyplot trống: dpi 50 → PNG 300×150 và không figure nào còn mở	`(plt.close("all"), (lambda b: (int.from_bytes(b[16:20], "big"), int.from_bytes(b[20:24], "big")))(export_all({"Q": [5, 1, 4]}, 50)["Q"]), len(plt.get_fignums()))[1:]`	`((300, 150), 0)`
Dict rỗng → {} và không tạo figure	`(lambda n: (export_all({}, 50), len(plt.get_fignums()) - n))(len(plt.get_fignums()))`	`({}, 0)`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 4 bước, có scaffold 3 slot)
Khung 1 — Bước 1. plt.subplots(figsize=(6, 3), layout="constrained") → 6×3 inch
Khung 2 — Bước 2. fig.savefig(buf, format="png", dpi=dpi) → PNG 6·dpi × 3·dpi
Khung 3 — Bước 3. plt.close(fig) → figure đóng ngay sau khi lưu
Khung 4 — Bước 4. result[title] = buf.getvalue() → bytes để kiểm hoặc ghi file
Khung đối chiếu — ngộ nhận "Tăng dpi là biểu đồ có nhiều thông tin hơn". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. dpi chỉ là số điểm ảnh mỗi inch: cùng đường, cùng nhãn, chỉ mịn hơn và file nặng hơn. Muốn biểu đồ nói nhiều hơn phải đổi dữ liệu hoặc cách trình bày. Và bbox_inches="tight" đổi kích thước canvas — đề cần đúng kích thước thì đừng dùng.
Khung lắp mảnh — 3 slot. Thả từng mảnh vào chỗ trống trong template; mảnh bẫy phải diễn cảnh hỏng: plt.show(), fig.savefig(buf, format="png", dpi=dpi, bbox_inches="tight"), fig.clf(), plt.subplots(figsize=(600, 300))
Khung kết quả — 2 cặp vào/ra. make_chart("Week 1", [10, 20, 15]) → figure 6×3 inch, layout constrained; line qua 10, 20, 15; export_all({"Week 1": [10, 20, 15]}, 150) → {"Week 1": PNG 900×450}; không figure nào còn mở

Nguồn: pe-v4 · python-learning-data.json#M06 · parser pe-v4/build · file build 046-M06.json

M8 · Dự án DA

2 bài: DA01, DA02

Ứng viên animation — ưu tiên cao (★★): không có · ưu tiên (★): DA01, DA02

DA01 · Mini-project: doanh thu từ raw tới chart

Mã bài: DA01 · Nâng cao · M8 · Dự án DA · 60 phút · 40 XP · thứ tự 47

MỤC TIÊU

Tự dựng pipeline doanh thu từ raw tới chart.

CẦN NHỚ TRƯỚC

M06

BỐI CẢNH

Boss cuối khoá, phần 1. sales.csv (cấp sẵn) là 11 giao dịch mô phỏng có lỗi cố ý; grain: một dòng = một đơn, khoá order_id. Tự viết năm hàm thuần, không hàm nào được sửa dữ liệu đầu vào. (1) load_sales(path): đọc mọi cột dưới dạng chữ (dtype=str). (2) clean_sales(raw) → (clean, rejected): region bỏ khoảng trắng hai đầu và viết thường; date phải là ngày YYYY-MM-DD có thật; qty và unit_price phải là số > 0; mỗi dòng lỗi mang MỘT lý do theo thứ tự bad_date → bad_qty → bad_price; trong các dòng hợp lệ, order_id gặp lại thì giữ dòng đầu, dòng sau là duplicate. clean thêm month ("YYYY-MM") và revenue = qty × unit_price; rejected là các dòng raw bị loại kèm cột reason. (3) quality_report(raw, clean, rejected): đếm dòng vào/ra và từng lý do; tổng không khớp thì ValueError. (4) kpi_table(clean, by) theo "region" hoặc "month": orders, qty, revenue và avg_price = revenue / qty. (5) revenue_chart(clean) → Figure hai Axes: barh doanh thu theo vùng (tăng dần) và line doanh thu theo tháng, có tiêu đề ghi rõ dữ liệu mô phỏng và nhãn trục.

VÍ DỤ MINH HOẠ

sales.csv: 11 dòng raw → 6 đơn hợp lệ, revenue = 155 (north 65, south 90; 2026-01: 110, 2026-02: 45).

ĐIỂM KIẾN THỨC

1. Định nghĩa grain trước khi viết code: một dòng là một đơn hàng, nên order_id phải duy nhất. Bỏ trùng, đếm đơn, cộng doanh thu đều dựa trên câu này.
2. Đọc raw dưới dạng chữ (dtype=str) để chưa mất gì trước khi kiểm: mã "0101" không thành số 101, " North " còn nguyên dấu cách để bạn thấy dữ liệu bẩn ra sao. Ép kiểu là việc của bước làm sạch: pd.to_numeric và pd.to_datetime với errors="coerce" biến giá trị hỏng thành NaN/NaT để rà soát, không làm sập chương trình.
3. Loại dòng phải có lý do, và mỗi dòng đúng một lý do: nhờ vậy rows_in = rows_out + tổng các lý do. Phép đối chiếu này bắt được dòng "bốc hơi" giữa đường. rejected giữ text RAW (date vẫn là "bad") để người khác kiểm lại được.
4. KPI phải nói rõ mẫu số: giá bán bình quân = tổng revenue / tổng qty (tỷ số của tổng), không phải trung bình cột unit_price — đơn mua 5 cái phải nặng hơn đơn mua 1 cái.
5. Biểu đồ vẽ từ bảng đã tổng hợp: barh để so vùng (sắp tăng dần, cột dài nhất nằm trên cùng), line cho diễn tiến theo tháng, tiêu đề ghi rõ đây là dữ liệu mô phỏng. Vẽ thẳng giao dịch thô thì mỗi đơn thành một cột chồng lên nhau — trông như tổng nhưng không phải.

NGỘ NHẬN THƯỜNG GẶP: Năm cái bẫy của DA01

Đọc không khai dtype (mã "0101" thành 101, qty thành 2.0); chuẩn hoá thiếu một bước (strip mà quên lower thì "North" và "north" là hai vùng); drop_duplicates() không subset (chỉ bỏ dòng giống HỆT, đơn nhập lại với "Book"/"book" vẫn bị đếm hai lần); avg_price = trung bình unit_price (trung bình của tỷ số thay cho tỷ số của tổng); vẽ barh từ giao dịch thô thay vì từ bảng KPI. Thêm một lỗi pandas hay gặp: df = raw (không .copy()) rồi sửa df là sửa luôn raw.

CÂU DỰ ĐOÁN

sales.csv có hai dòng O05 giống hệt nhau, O03 thiếu qty, O04 có date "bad", O07 qty = -1, O08 thiếu giá. Theo chính sách của bài, còn bao nhiêu đơn và revenue bao nhiêu?

sai	7 đơn, revenue 185	Quên bỏ bản trùng O05 — đơn 30 bị cộng hai lần.
ĐÚNG	6 đơn, revenue 155	Loại O03, O04, O07, O08 và bản O05 thứ hai: 20 + 20 + 30 + 40 + 15 + 30 = 155.
sai	11 dòng, revenue 215	Không làm sạch, chỉ điền thiếu bằng 0: O04 vẫn cộng 40, O07 trừ 10, bản trùng cộng 30.

TỰ KIỂM TRA

1. Đơn P7 bị nhập hai lần: " west,Book" và "West,book". Hai đơn P8, P9 giống nhau mọi cột trừ order_id. Lệnh bỏ trùng nào đúng với grain "một dòng = một đơn"?

- drop_duplicates(subset=["date", "region", "qty", "unit_price"]) — Xoá nhầm P9: hai đơn thật khác order_id chỉ "trông giống nhau".

✔ duplicated(subset="order_id", keep="first") trên các dòng đã hợp lệ — Khoá của grain là order_id: P7 còn một dòng, P8 và P9 đều được giữ.

• drop_duplicates() không tham số — Chỉ bỏ dòng giống hệt ở mọi cột — "Book"/"book" khác nhau nên đơn P7 bị đếm hai lần.

Đáp án: Khoá của grain là order_id: P7 còn một dòng, P8 và P9 đều được giữ.

2. Vùng north có 3 đơn: 2 cuốn × 10, 3 cuốn × 10, 1 cuốn × 15. Giá bán bình quân mỗi cuốn là?

• 65 / 3 ≈ 21.67 — Đó là doanh thu bình quân mỗi ĐƠN, không phải mỗi cuốn.

• (10 + 10 + 15) / 3 ≈ 11.67 — Trung bình đơn giá coi đơn 1 cuốn nặng bằng đơn 3 cuốn.

✔ 65 / 6 ≈ 10.83 — tổng revenue chia tổng qty — Tỷ số của tổng: cuốn nào cũng nặng như nhau.

Đáp án: Tỷ số của tổng: cuốn nào cũng nặng như nhau.

3. Đọc file có mã đơn "0101", "0102"... bằng pd.read_csv(path) không khai dtype thì...

✔ order_id thành số 101, 102 — mất số 0 đầu, không còn khớp với hệ thống khác — pandas tự đoán kiểu: cột toàn chữ số thành int.

• Báo lỗi ngay — Không lỗi gì — đó mới là chỗ nguy hiểm.

• Vẫn giữ nguyên "0101" — Chỉ giữ khi khai dtype=str (hoặc dtype={"order_id": str}).

Đáp án: pandas tự đoán kiểu: cột toàn chữ số thành int.

TỰ GIẢI THÍCH

Vì sao rows_in phải bằng rows_out cộng tổng các lý do loại, và vì sao mỗi dòng chỉ được một lý do?

Mẫu trả lời: Mỗi dòng raw phải đi đúng một nơi: vào clean, hoặc vào rejected với một lý do. Nếu tổng không khớp là có dòng bị mất (lọc hai lần, groupby bỏ NaN) hoặc bị đếm hai lần. Một dòng vừa sai ngày vừa thiếu qty mà đếm cả hai lý do thì tổng các lý do lớn hơn số dòng bị loại và phép đối chiếu vô nghĩa; nên kiểm theo thứ tự cố định và dừng ở lỗi đầu tiên.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	load_sales: sales.csv → 11 dòng, mọi cột là chữ.
Bước 2	clean_sales: chuẩn hoá region, ép kiểu date / qty / unit_price → gắn lý do → bỏ trùng order_id → 6 đơn + 5 dòng rejected.
Bước 3	quality_report: 11 = 6 + (1 bad_date + 2 bad_qty + 1 bad_price + 1 duplicate).
Bước 4	kpi_table: north 65, south 90; 2026-01 110, 2026-02 45 — tổng vẫn 155.
Bước 5	revenue_chart: barh 2 cột (65, 90) + line 2 điểm (110, 45).

CÂU PHẢN VÍ DỤ

Thêm đơn hợp lệ "O11,2026-02-10, South,Pen,1,20" vào cuối sales.csv. Revenue của south thành bao nhiêu?

Lựa chọn: 20 | 90 | 110 · Đáp án: 110

LUYỆN CÓ HỖ TRỢ · Lập kế hoạch trước khi tự viết

Đi lại đường dữ liệu của bài trước khi tự viết code.

Yêu cầu thêm: Bộ chấm chạy lại cả pipeline trên một file khác bạn chưa thấy: mã đơn có số 0 đầu, ngày 30/02, ô ngày trống, qty viết bằng chữ, qty = 0, giá không phải số, một đơn nhập hai lần với cách viết khác, và hai đơn thật giống nhau. Viết cứng 6 / 155 sẽ trượt.

BÀI TẬP ĐỘC LẬP · Boss — tự viết, nộp lần đầu trước khi xem lời giải

Boss cuối khoá, phần 1. sales.csv (cấp sẵn) là 11 giao dịch mô phỏng có lỗi cố ý; grain: một dòng = một đơn, khoá order_id. Tự viết năm hàm thuần, không hàm nào được sửa dữ liệu đầu vào. (1) load_sales(path): đọc mọi cột dưới dạng chữ (dtype=str). (2) clean_sales(raw) → (clean, rejected): region bỏ khoảng trắng hai đầu và viết thường; date phải là ngày YYYY-MM-DD có thật; qty và unit_price phải là số > 0; mỗi dòng lỗi mang MỘT lý do theo thứ tự bad_date → bad_qty → bad_price; trong các dòng hợp lệ, order_id gặp lại thì giữ dòng đầu, dòng sau là duplicate. clean thêm month ("YYYY-MM") và revenue = qty × unit_price; rejected là các dòng raw bị loại kèm cột reason. (3) quality_report(raw, clean, rejected): đếm dòng vào/ra và từng lý do; tổng không khớp thì ValueError. (4) kpi_table(clean, by) theo "region" hoặc "month": orders, qty, revenue và avg_price = revenue / qty. (5) revenue_chart(clean) → Figure hai Axes: barh doanh thu theo vùng (tăng dần) và line doanh thu theo tháng, có tiêu đề ghi rõ dữ liệu mô phỏng và nhãn trục.

Thực hành: Viết và chạy từng hàm trên sales.csv, in kết quả ra đối chiếu (11 dòng → 6 đơn → 155) rồi mới sang hàm sau. Nộp lần đầu trước khi xem lời giải.

Gọi	Kết quả mong đợi
`sales.csv`	`11 dòng → 6 đơn, revenue 155; rejected: O03 bad_qty, O04 bad_date, O05 duplicate, O07 bad_qty, O08 bad_price`

`kpi_table(clean, "region")`	`north: 3 đơn, 65, avg_price 10.83 · south: 3 đơn, 90, avg_price 15.0`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`load_sales(path)`	Đọc file CSV bán hàng, giữ nguyên raw. · path: str — file CSV có dòng tiêu đề · DataFrame đủ các cột theo thứ tự của file; MỌI ô là chuỗi (dtype=str), ô trống là NaN · Không ép kiểu, không strip, không bỏ dòng nào.
`clean_sales(raw)`	Chuẩn hoá, kiểm từng dòng, gắn lý do loại, bỏ trùng theo order_id. · raw: DataFrame từ load_sales — KHÔNG được sửa · (clean, rejected). clean: dòng hợp lệ; region đã strip + lower; date kiểu datetime; qty, unit_price là số; thêm month = "YYYY-MM" và revenue = qty × unit_price. rejected: các dòng RAW bị loại (giữ text gốc) + cột reason · Lý do theo thứ tự kiểm, mỗi dòng một lý do: bad_date (không phải ngày YYYY-MM-DD có thật) → bad_qty (thiếu, không phải số, hoặc ≤ 0) → bad_price (thiếu, không phải số, hoặc ≤ 0) → duplicate (trong các dòng hợp lệ, order_id đã gặp — giữ dòng đầu).
`quality_report(raw, clean, rejected)`	Tóm tắt chất lượng dữ liệu và đối chiếu số dòng. · raw; clean, rejected do clean_sales trả về · dict đủ 6 khoá: rows_in, rows_out, bad_date, bad_qty, bad_price, duplicate (lý do không gặp thì 0) · len(raw) khác len(clean) + len(rejected) → raise ValueError.
`kpi_table(clean, by)`	Bảng KPI theo vùng hoặc theo tháng. · clean; by: "region" hoặc "month" · DataFrame, index là giá trị của cột by (tăng dần); cột orders (số đơn), qty (tổng), revenue (tổng), avg_price = revenue / qty của nhóm · Không sửa clean.
`revenue_chart(clean)`	Vẽ hai biểu đồ từ bảng KPI đã tổng hợp (không từ giao dịch thô). · clean · Figure đúng 2 Axes: axes[0] barh revenue theo region, sắp TĂNG dần; axes[1] line revenue theo month, có marker · Mỗi Axes có title, tiêu đề ghi rõ "dữ liệu mô phỏng"; axes[0] có xlabel; axes[1] có xlabel và ylabel. Không gọi plt.show() trong hàm.

BỘ CHẤM (19 ca)

Ca	Gọi	Mong đợi
load_sales: 11 dòng × 6 cột, đúng tên cột	`(lambda r: (len(r), list(r.columns)))(load_sales('sales.csv'))`	`(11, ['order_id', 'date', 'region', 'product', 'qty', 'unit_price'])`
load_sales đọc mọi cột là chữ: " North " còn dấu cách, qty là "2", ô trống là NaN	`(lambda r: (r.loc[0, 'region'], r.loc[0, 'qty'], int(r['qty'].isna().sum())))(load_sales('sales.csv'))`	`(' North ', '2', 1)`
File mới: mã đơn "0101" giữ số 0 đầu	`load_sales(_studio.write('sales_q2.csv', 'order_id,date,region,product,qty,unit_price\n0101,2026-03-01,East,Pen,2,20\n0102,2026-03-02, west,Book,1,10\n0103,2026-02-30,east,Pen,1,20\n0104,,West,Book,2,10\n0105,2026-03-04,WEST ,Book,two,10\n0102,2026-03-02,West,book,1,10\n0106,2026-04-01,east,Pen,3,abc\n0107,2026-04-02,east,Pen,4,15\n0108,2026-04-02,east,Pen,4,15\n0109,2026-04-05, West,Book,2,12.5\n0110,2026-04-06,west,Book,0,12\n0111,2026-03-15,EAST,Book,5,8\n0112,2026-04-20,west,Pen,1,20\n'))['order_id'].head(2).tolist()`	`['0101', '0102']`
sales.csv → đúng 6 đơn hợp lệ	`sorted(clean_sales(load_sales('sales.csv'))[0]['order_id'])`	`['O01', 'O02', 'O05', 'O06', 'O09', 'O10']`
sales.csv → tổng revenue = 155	`float(clean_sales(load_sales('sales.csv'))[0]['revenue'].sum())`	`155.0`
clean: region chuẩn hoá, month dạng "YYYY-MM", date là datetime, revenue là số	`(lambda c: (sorted(set(c['region'])), sorted(set(c['month'])), __import__('pandas').api.types.is_datetime64_any_dtype(c['date']), __import__('pandas').api.types.is_numeric_dtype(c['revenue'])))(clean_sales(load_sales('sales.csv'))[0])`	`(['north', 'south'], ['2026-01', '2026-02'], True, True)`
rejected: 5 dòng, mỗi dòng đúng một lý do	`sorted(map(tuple, clean_sales(load_sales('sales.csv'))[1][['order_id', 'reason']].values.tolist()))`	`[('O03', 'bad_qty'), ('O04', 'bad_date'), ('O05', 'duplicate'), ('O07', 'bad_qty'), ('O08', 'bad_price')]`

rejected giữ text raw (date vẫn là "bad"); raw không bị sửa	`(lambda raw: (lambda res: (res[1].loc[res[1]['order_id'] == 'O04', 'date'].tolist(), raw.loc[0, 'region'], len(raw)))(clean_sales(raw)))(load_sales('sales.csv'))`	`(['bad'], ' North ', 11)`
quality_report trên sales.csv: 11 = 6 + 1 + 2 + 1 + 1	`(lambda raw: quality_report(raw, *clean_sales(raw)))(load_sales('sales.csv'))`	`{'rows_in': 11, 'rows_out': 6, 'bad_date': 1, 'bad_qty': 2, 'bad_price': 1, 'duplicate': 1}`
quality_report: số dòng không khớp (mất một dòng clean) → ValueError	`(lambda raw: (lambda c, r: _studio.call(quality_report, raw, c.iloc[1:], r).error)(*clean_sales(raw)))(load_sales('sales.csv'))`	`'ValueError'`
File mới: report 13 → 7 (2 bad_date, 2 bad_qty, 1 bad_price, 1 duplicate)	`(lambda raw: quality_report(raw, *clean_sales(raw)))(load_sales(_studio.write('sales_q2.csv', 'order_id,date,region,product,qty,unit_price\n0101,2026-03-01,East,Pen,2,20\n0102,2026-03-02, west,Book,1,10\n0103,2026-02-30,east,Pen,1,20\n0104,,West,Book,2,10\n0105,2026-03-04,WEST ,Book,two,10\n0102,2026-03-02,West,book,1,10\n0106,2026-04-01,east,Pen,3,abc\n0107,2026-04-02,east,Pen,4,15\n0108,2026-04-02,east,Pen,4,15\n0109,2026-04-05, West,Book,2,12.5\n0110,2026-04-06,west,Book,0,12\n0111,2026-03-15,EAST,Book,5,8\n0112,2026-04-20,west,Pen,1,20\n')))`	`{'rows_in': 13, 'rows_out': 7, 'bad_date': 2, 'bad_qty': 2, 'bad_price': 1, 'duplicate': 1}`
File mới: 7 đơn, revenue 255 — bỏ trùng theo order_id, giữ hai đơn thật giống nhau	`(lambda c: (sorted(c['order_id']), float(c['revenue'].sum())))(clean_sales(load_sales(_studio.write('sales_q2.csv', 'order_id,date,region,product,qty,unit_price\n0101,2026-03-01,East,Pen,2,20\n0102,2026-03-02, west,Book,1,10\n0103,2026-02-30,east,Pen,1,20\n0104,,West,Book,2,10\n0105,2026-03-04,WEST ,Book,two,10\n0102,2026-03-02,West,book,1,10\n0106,2026-04-01,east,Pen,3,abc\n0107,2026-04-02,east,Pen,4,15\n0108,2026-04-02,east,Pen,4,15\n0109,2026-04-05, West,Book,2,12.5\n0110,2026-04-06,west,Book,0,12\n0111,2026-03-15,EAST,Book,5,8\n0112,2026-04-20,west,Pen,1,20\n')))[0])`	`(['0101', '0102', '0107', '0108', '0109', '0111', '0112'], 255.0)`
kpi_table theo region: orders, revenue và avg_price = revenue / qty	`(lambda t: (t['orders'].to_dict(), t['revenue'].to_dict(), t['avg_price'].round(2).to_dict()))(kpi_table(clean_sales(load_sales('sales.csv'))[0], 'region'))`	`({'north': 3, 'south': 3}, {'north': 65.0, 'south': 90.0}, {'north': 10.83, 'south': 15.0})`
kpi_table theo month; tổng các tháng = 155	`(lambda t: (t['revenue'].to_dict(), float(t['revenue'].sum())))(kpi_table(clean_sales(load_sales('sales.csv'))[0], 'month'))`	`({'2026-01': 110.0, '2026-02': 45.0}, 155.0)`
kpi_table trên 3 giao dịch của bài tập gốc: A 30, B 60, tổng 90	`(lambda t: (t['revenue'].to_dict(), t['avg_price'].to_dict(), float(t['revenue'].sum())))(kpi_table(__import__('pandas').DataFrame({'order_id': ['a1', 'a2', 'b1'], 'region': ['A', 'A', 'B'], 'qty': [1, 2, 3], 'unit_price': [10, 10, 20], 'revenue': [10, 20, 60]}), 'region'))`	`({'A': 30, 'B': 60}, {'A': 10.0, 'B': 20.0}, 90.0)`
File mới: KPI theo region và theo month	`(lambda c: (kpi_table(c, 'region')['revenue'].to_dict(), kpi_table(c, 'region')['avg_price'].round(2).to_dict(), kpi_table(c, 'month')['revenue'].to_dict()))(clean_sales(load_sales(_studio.write('sales_q2.csv', 'order_id,date,region,product,qty,unit_price\n0101,2026-03-01,East,Pen,2,20\n0102,2026-03-02, west,Book,1,10\n0103,2026-02-30,east,Pen,1,20\n0104,,West,Book,2,10\n0105,2026-03-04,WEST ,Book,two,10\n0102,2026-03-02,West,book,1,10\n0106,2026-04-01,east,Pen,3,abc\n0107,2026-04-02,east,Pen,4,15\n0108,2026-04-02,east,Pen,4,15\n0109,2026-04-05, West,Book,2,12.5\n0110,2026-04-06,west,Book,0,12\n0111,2026-03-15,EAST,Book,5,8\n0112,2026-04-20,west,Pen,1,20\n')))[0])`	`({'east': 200.0, 'west': 55.0}, {'east': 13.33, 'west': 13.75}, {'2026-03': 90.0, '2026-04': 165.0})`
revenue_chart: 2 Axes — barh 2 vùng (65, 90) và line 2 tháng (110, 45)	`(lambda f: (len(f.axes), [float(p.get_width()) for p in f.axes[0].patches], [float(y) for y in f.axes[1].lines[0].get_ydata()]))(revenue_chart(clean_sales(load_sales('sales.csv'))[0]))`	`(2, [65.0, 90.0], [110.0, 45.0])`

File mới: barh sắp tăng dần (west 55 rồi east 200), line 90 → 165	`(lambda f: ([float(p.get_width()) for p in f.axes[0].patches], [float(y) for y in f.axes[1].lines[0].get_ydata()]))(revenue_chart(clean_sales(load_sales(_studio.write('sales_q2.csv', 'order_id,date,region,product,qty,unit_price\n0101,2026-03-01,East,Pen,2,20\n0102,2026-03-02, west,Book,1,10\n0103,2026-02-30,east,Pen,1,20\n0104,,West,Book,2,10\n0105,2026-03-04,WEST,Book,two,10\n0102,2026-03-02,West,book,1,10\n0106,2026-04-01,east,Pen,3,abc\n0107,2026-04-02,east,Pen,4,15\n0108,2026-04-02,east,Pen,4,15\n0109,2026-04-05, West,Book,2,12.5\n0110,2026-04-06,west,Book,0,12\n0111,2026-03-15,EAST,Book,5,8\n0112,2026-04-20,west,Pen,1,20\n')))[0]))`	`([55.0, 200.0], [90.0, 165.0])`
revenue_chart: Axes nào cũng có tiêu đề, có nhãn trục, ghi rõ dữ liệu mô phỏng	`(lambda f: (lambda text: (all(a.get_title() for a in f.axes), bool(f.axes[0].get_xlabel()), bool(f.axes[1].get_xlabel()), bool(f.axes[1].get_ylabel()), 'mô phỏng' in text or 'synthetic' in text))(' '.join([a.get_title() for a in f.axes] + [f.get_suptitle()]).lower()))(revenue_chart(clean_sales(load_sales('sales.csv'))[0]))`	`(True, True, True, True, True)`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy, pandas. File kèm bài: sales.csv (data). Bài có cửa tự kiểm tra (gate): lời giải mẫu chỉ mở sau khi hoàn thành.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 5 bước)
Khung 1 — Bước 1. load_sales: sales.csv → 11 dòng, mọi cột là chữ.
Khung 2 — Bước 2. clean_sales: chuẩn hoá region, ép kiểu date / qty / unit_price → gắn lý do → bỏ trùng order_id → 6 đơn + 5 dòng rejected.
Khung 3 — Bước 3. quality_report: 11 = 6 + (1 bad_date + 2 bad_qty + 1 bad_price + 1 duplicate).
Khung 4 — Bước 4. kpi_table: north 65, south 90; 2026-01 110, 2026-02 45 — tổng vẫn 155.
Khung 5 — Bước 5. revenue_chart: barh 2 cột (65, 90) + line 2 điểm (110, 45).
Khung đối chiếu — ngộ nhận Năm cái bẫy của DA01. Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Đọc không khai dtype (mã "0101" thành 101, qty thành 2.0); chuẩn hoá thiếu một bước (strip mà quên lower thì "North" và "north" là hai vùng); drop_duplicates() không subset (chỉ bỏ dòng giống HỆT, đơn nhập lại với "Book"/"book" vẫn bị đếm hai lần); avg_price = trung bình unit_price (trung bình của tỷ số thay cho tỷ số của tổng); vẽ barh từ giao dịch thô thay vì từ bảng KPI. Thêm một lỗi pandas hay gặp: df = raw (không .copy()) rồi sửa df là sửa luôn raw.
Khung kết quả — 2 cặp vào/ra. sales.csv → 11 dòng → 6 đơn, revenue 155; rejected: O03 bad_qty, O04 bad_date, O05 duplicate, O07 bad_qty, O08 bad_price; kpi_table(clean, "region") → north: 3 đơn, 65, avg_price 10.83 · south: 3 đơn, 90, avg_price 15.0

Nguồn: pe-v4 · python-learning-data.json#DA01 · parser pe-v4/build · file build 047-DA01.json

DA02 · Mini-project: điểm số và GPA

Mã bài: DA02 · Nâng cao · M8 · Dự án DA · 50 phút · 40 XP · thứ tự 48

MỤC TIÊU

Làm sạch bảng điểm và tính GPA có trọng số tín chỉ.

CẦN NHỚ TRƯỚC

DA01

BỐI CẢNH

Boss cuối khoá, phần 2. grades.csv (cấp sẵn, thang 10) có lỗi cố ý; grain: một sinh viên, một môn, một điểm. Tự viết sáu hàm thuần. (1) load_grades(path): mọi cột là chữ. (2) clean_grades(raw) → (clean, report): student_id và course bỏ khoảng trắng hai đầu và viết HOA; score thiếu hoặc không phải số → missing_score (KHÔNG thay bằng 0); score ngoài [0, 10] → out_of_range; credits thiếu, không phải số hoặc ≤ 0 → bad_credits; trong các dòng hợp lệ, cặp (student_id, course) gặp lại → duplicate, giữ dòng đầu; report đếm rows_in, rows_out và từng lý do. (3) letter(gpa): A ≥ 8.5, B ≥ 7.0, C ≥ 5.5, D ≥ 4.0, còn lại F. (4) gpa_table(clean): mỗi sinh viên một dòng — courses, credits, gpa = Σ(điểm × tín chỉ) / Σ tín chỉ (làm tròn 2 chữ số), letter — sắp gpa giảm dần. (5) class_gpa(clean): GPA chung theo tín chỉ của cả lớp. (6) grade_chart(table) → Figure: hist GPA với bins [0, 4, 5.5, 7, 8.5, 10], tiêu đề ghi cỡ mẫu n, có nhãn hai trục.

VÍ DỤ MINH HOẠ

Sinh viên A có điểm [6, 9], tín chỉ [1, 2] → GPA = (6×1 + 9×2) / 3 = 8.0, không phải 7.5. grades.csv: 18 dòng → 13 dòng hợp lệ, 7 sinh viên, GPA chung 6.67.

ĐIỂM KIẾN THỨC

1. Schema trước khi tính: score là số trong [0, 10], credits là số dương, khoá của grain là cặp (student_id, course). Bỏ trùng chỉ theo student_id là xoá mất các môn khác của cùng một người.
2. Thiếu điểm KHÁC điểm 0: "abs", "N/A" hay ô trống nghĩa là chưa có điểm. Điền 0 kéo GPA xuống và có thể biến một sinh viên khá thành trượt. Loại dòng đó khỏi phép tính và báo số dòng thiếu trong report.
3. Điểm ngoài khoảng (85 do nhập nhầm thang 100, -1) là lỗi nhập liệu: loại và báo, đừng tự chia 10 khi chưa có quy tắc nghiệp vụ. "s02 " và "S02" là một người — chuẩn hoá mã (strip + upper) TRƯỚC khi bỏ trùng và groupby.
4. GPA = Σ(điểm × tín chỉ) / Σ(tín chỉ): môn 4 tín chỉ nặng gấp đôi môn 2 tín chỉ. GPA chung của lớp cũng là tỷ số của tổng trên mọi dòng điểm; trung bình các GPA (mean of means) chỉ trùng kết quả khi mọi người học cùng số tín chỉ.
5. Hạng chữ theo biên dưới: A ≥ 8.5, B ≥ 7.0, C ≥ 5.5, D ≥ 4.0, còn lại F — dùng >= ở mọi mốc, xếp hạng trên GPA đã làm tròn 2 chữ số. Histogram dùng chính các mốc đó làm biên bin (bin [a, b) gồm biên trái) và ghi cỡ mẫu n trong tiêu đề, để người đọc biết 3 sinh viên hạng B là trên tổng bao nhiêu.

NGỘ NHẬN THƯỜNG GẶP: "Điểm thiếu thì coi như 0 cho đủ dòng"

Thiếu điểm (vắng thi, chưa nhập, "N/A") không phải là được 0 điểm. fillna(0) làm GPA của S02 rơi từ 7.5 xuống 3.21 — từ hạng B thành F — chỉ vì một ô "abs". Đúng là loại dòng khỏi phép tính GPA và báo số dòng thiếu. Cùng họ với lỗi này: lấy trung bình điểm không trọng số, bỏ trùng theo sai khoá, quên strip mã sinh viên, và dùng > thay cho >= ở mốc hạng.

CÂU DỰ ĐOÁN

Sinh viên A có điểm [6, 9] với tín chỉ [1, 2]. GPA của A là?

ĐÚNG	8.0 = (6×1 + 9×2) / 3	Trung bình có trọng số tín chỉ: môn 2 tín chỉ nặng gấp đôi.
sai	7.5 = (6 + 9) / 2	Trung bình không trọng số — coi môn 1 tín chỉ nặng bằng môn 2 tín chỉ.
sai	24	Đó mới là tử số Σ(điểm × tín chỉ), chưa chia tổng tín chỉ.

TỰ KIỂM TRA

1. grades.csv có "S03,Chi,CS101,4,4.0" và "S03,Chi,cs101 ,4,4". Làm sạch thế nào cho đúng grain?

• drop_duplicates(subset="student_id") — Mỗi sinh viên chỉ còn đúng một môn — GPA sai hết.

✔ Chuẩn hoá course (strip + upper), ép score về số, rồi duplicated(subset=["student_id", "course"]) — giữ bản đầu — Sau chuẩn hoá hai dòng cùng khoá (S03, CS101): một môn chỉ được tính một lần.

• drop_duplicates() ngay trên raw — "CS101"/"cs101 " và "4.0"/"4" khác nhau ở dạng chữ nên không bị coi là trùng: môn bị tính hai lần.

Đáp án: Sau chuẩn hoá hai dòng cùng khoá (S03, CS101): một môn chỉ được tính một lần.

2. GPA của một sinh viên đúng bằng 7.0 thì hạng chữ là?

- C — vì chưa vượt quá 7.0 — Viết > thay vì >= là lệch một hạng ngay tại biên.
- ✔ B — mốc B là ≥ 7.0, biên dưới thuộc hạng trên — Dùng >= ở mọi mốc.
- Tuỳ cách làm tròn — GPA đã làm tròn 2 chữ số; 7.0 nằm đúng mốc B.

Đáp án: Dùng >= ở mọi mốc.

3. Lớp có X học 20 tín chỉ, GPA 8.0, và Y học 4 tín chỉ, GPA 5.0. GPA chung theo tín chỉ của lớp là?

- ✔ (8.0×20 + 5.0×4) / 24 = 7.5 — Tỷ số của tổng: mỗi tín chỉ nặng như nhau.
- 8.0 — Chỉ lấy người học nhiều tín chỉ hơn — bỏ mất Y.
- (8.0 + 5.0) / 2 = 6.5 — Trung bình các GPA (mean of means) coi 4 tín chỉ của Y nặng bằng 20 tín chỉ của X.

Đáp án: Tỷ số của tổng: mỗi tín chỉ nặng như nhau.

TỰ GIẢI THÍCH

Vì sao GPA là Σ(điểm × tín chỉ) / Σ(tín chỉ) chứ không phải trung bình điểm, và vì sao không được thay điểm thiếu bằng 0?

Mẫu trả lời: Môn nhiều tín chỉ chiếm nhiều thời lượng hơn nên phải nặng hơn: điểm [6, 9] với tín chỉ [1, 2] cho GPA 8.0, trung bình thường ra 7.5 là coi hai môn ngang nhau. Điểm thiếu là chưa có dữ liệu chứ không phải được 0: thay bằng 0 vừa kéo tử số xuống vừa giữ nguyên mẫu số, nên GPA tụt oan (7.5 thành 3.21). Đúng là loại dòng thiếu khỏi cả tử lẫn mẫu và ghi số dòng bị loại vào report.

BẢNG TRACE (trạng thái trước → sau)

Bước	Trạng thái
Bước 1	load_grades: grades.csv → 18 dòng chữ.
Bước 2	clean_grades: strip + upper mã → ép số → missing_score 2, out_of_range 1, bad_credits 1, duplicate 1 → 13 dòng.
Bước 3	gpa_table: S01 (8×3 + 6.5×2 + 9×4) / 9 = 8.11 → B; S06 (6×2 + 7.5×4) / 6 = 7.0 → B.
Bước 4	class_gpa: 287 / 43 = 6.67 (trung bình 7 GPA lại ra 6.5).
Bước 5	grade_chart: hist theo [0, 4, 5.5, 7, 8.5, 10] → cột F, D, C, B, A cao 1, 1, 1, 3, 1; tiêu đề n=7.

CÂU PHẢN VÍ DỤ

Sinh viên có MATH101 = 9.0 (3 tín chỉ) và CS101 = "abs" (4 tín chỉ). GPA theo chính sách của bài?

Lựa chọn: 4.5 | 9.0 | 3.86 · Đáp án: 9.0

LUYỆN CÓ HỖ TRỢ · Lập kế hoạch trước khi tự viết

Đi lại đường dữ liệu của bài trước khi tự viết code.

Yêu cầu thêm: Bộ chấm dùng thêm một bảng điểm khác chưa thấy: "N/A", điểm -1 và 10.5, tín chỉ "abc" hoặc trống, mã có dấu cách đầu, và GPA rơi đúng các mốc 8.5, 5.5, 4.0, 3.99 — lệch một dấu >= là trượt.

BÀI TẬP ĐỘC LẬP · Boss — tự viết, nộp lần đầu trước khi xem lời giải

Boss cuối khoá, phần 2. grades.csv (cấp sẵn, thang 10) có lỗi cố ý; grain: một sinh viên, một môn, một điểm. Tự viết sáu hàm thuần. (1) load_grades(path): mọi cột là chữ. (2) clean_grades(raw) → (clean, report): student_id và course bỏ khoảng trắng hai đầu và viết HOA; score thiếu hoặc không phải số → missing_score (KHÔNG thay bằng 0); score ngoài [0, 10] → out_of_range; credits thiếu, không phải số hoặc ≤ 0 → bad_credits; trong các dòng hợp lệ, cặp (student_id, course) gặp lại → duplicate, giữ dòng đầu; report đếm rows_in, rows_out và từng lý do. (3) letter(gpa): A ≥ 8.5, B ≥ 7.0, C ≥ 5.5, D ≥ 4.0, còn lại F. (4) gpa_table(clean): mỗi sinh viên một dòng — courses, credits, gpa = Σ(điểm × tín chỉ) / Σ tín chỉ (làm tròn 2 chữ số), letter — sắp gpa giảm dần. (5) class_gpa(clean): GPA chung theo tín chỉ của cả lớp. (6) grade_chart(table) → Figure: hist GPA với bins [0, 4, 5.5, 7, 8.5, 10], tiêu đề ghi cỡ mẫu n, có nhãn hai trục.

Thực hành: Làm từng hàm và in ra đối chiếu với grades.csv (18 → 13 dòng; S01 = 8.11) trước khi sang hàm sau. Nộp lần đầu trước khi xem lời giải.

Gọi	Kết quả mong đợi
`grades.csv`	`report: 18 → 13 (missing_score 2, out_of_range 1, bad_credits 1, duplicate 1)`
`gpa_table + class_gpa`	`S04 8.64 A · S01 8.11 B · S02 7.5 B · S06 7.0 B · S07 6.29 C · S03 4.43 D · S05 3.5 F; GPA chung 6.67`

HỢP ĐỒNG HÀM

Chữ ký	Nhận / trả / tác dụng
`load_grades(path)`	Đọc bảng điểm CSV, giữ nguyên raw. · path: str — file CSV có dòng tiêu đề · DataFrame đủ các cột của file; MỌI ô là chuỗi (dtype=str), ô trống / "N/A" là NaN · Không ép kiểu, không strip, không bỏ dòng.

`clean_grades(raw)`	Chuẩn hoá mã, ép kiểu, loại dòng lỗi có lý do và bỏ trùng theo (student_id, course). · raw: DataFrame từ load_grades — KHÔNG được sửa · (clean, report). clean: dòng hợp lệ; student_id và course đã strip + upper; score, credits là số. report: dict đủ 6 khoá rows_in, rows_out, missing_score, out_of_range, bad_credits, duplicate · Mỗi dòng một lý do, theo thứ tự: missing_score (score thiếu hoặc không phải số — KHÔNG thay bằng 0) → out_of_range (score < 0 hoặc > 10) → bad_credits (thiếu, không phải số, hoặc ≤ 0) → duplicate (trong các dòng hợp lệ, cặp student_id + course đã gặp — giữ dòng đầu).
`letter(gpa)`	Đổi GPA thang 10 sang hạng chữ. · gpa: số thực · "A" nếu ≥ 8.5; "B" nếu ≥ 7.0; "C" nếu ≥ 5.5; "D" nếu ≥ 4.0; còn lại "F" · Biên dưới thuộc hạng trên: dùng >= ở mọi mốc.
`gpa_table(clean)`	Bảng tổng hợp từng sinh viên. · clean (bảng đã làm sạch) · DataFrame index student_id, sắp gpa GIẢM dần; cột courses (số môn), credits (tổng tín chỉ), gpa = Σ(score × credits) / Σ credits làm tròn 2 chữ số, letter = letter(gpa) · Không sửa clean; có thể thêm cột phụ (vd. points = Σ score × credits).
`class_gpa(clean)`	GPA chung của cả lớp, mỗi TÍN CHỈ nặng như nhau. · clean · float làm tròn 2 chữ số = Σ(score × credits) / Σ credits trên mọi dòng hợp lệ · Không phải trung bình các GPA của từng sinh viên (mean of means).
`grade_chart(table)`	Histogram phân bố GPA theo đúng các mốc hạng. · table: kết quả gpa_table · Figure một Axes: hist cột gpa với bins = GRADE_BINS = [0, 4, 5.5, 7, 8.5, 10] · Tiêu đề ghi cỡ mẫu "n=<số sinh viên>"; có xlabel và ylabel. Không gọi plt.show() trong hàm.

BỘ CHẤM (14 ca)

Ca	Gọi	Mong đợi
load_grades: 18 dòng, đúng cột, mọi ô là chữ ("3", "s02 ")	`(lambda r: (len(r), list(r.columns), r.loc[0, 'credits'], r.loc[3, 'student_id']))(load_grades('grades.csv'))`	`(18, ['student_id', 'name', 'course', 'credits', 'score'], '3', 's02 ')`
clean_grades trên grades.csv: báo cáo 18 → 13	`clean_grades(load_grades('grades.csv'))[1]`	`{'rows_in': 18, 'rows_out': 13, 'missing_score': 2, 'out_of_range': 1, 'bad_credits': 1, 'duplicate': 1}`
File mới: báo cáo 15 → 9 (N/A, -1, 10.5, tín chỉ "abc"/trống, bản trùng " B03")	`clean_grades(load_grades(_studio.write('grades_k2.csv', 'student_id,name,course,credits,score\nB01,Lan,MATH101,3,8.5\nB01,Lan,CS101,3,8.5\nB02,Minh,MATH101,2,10\nB02,Minh,CS101,4,N/A\nB02,Minh,PHYS101,2,-1\nb03,Nga,MATH101,3,5.5\nB03,Nga,PHYS101,3,5.5\n B03,Nga,MATH101,3,5.5\nB04,Oanh,CS101,4,4\nB04,Oanh,MATH101,abc,9\nB05,Phú,CS101,5,3.99\nB05,Phú,MATH101,,7\nB06,Quang,CS101,2,10.5\nB06,Quang,MATH101,2,7\nB06,Quang,PHYS101,1,8.5\n')))[1]`	`{'rows_in': 15, 'rows_out': 9, 'missing_score': 1, 'out_of_range': 2, 'bad_credits': 2, 'duplicate': 1}`
clean: mã sinh viên / mã môn đã chuẩn hoá, điểm là số; raw không bị sửa	`(lambda raw: (lambda c: (sorted(set(c['student_id'])), sorted(set(c['course'])), __import__('pandas').api.types.is_numeric_dtype(c['score']), raw.loc[3, 'student_id']))(clean_grades(raw)[0]))(load_grades('grades.csv'))`	`(['S01', 'S02', 'S03', 'S04', 'S05', 'S06', 'S07'], ['CS101', 'MATH101', 'PHYS101'], True, 's02 ')`
letter: biên dưới thuộc hạng trên (8.5 → A, 7.0 → B, 5.5 → C, 4.0 → D)	`[letter(x) for x in [10, 8.5, 8.49, 7.0, 6.99, 5.5, 5.49, 4.0, 3.99, 0]]`	`['A', 'A', 'B', 'B', 'C', 'C', 'D', 'D', 'F', 'F']`
gpa_table: GPA = Σ(điểm × tín chỉ) / Σ tín chỉ, điểm thiếu không tính là 0	`{k: round(float(v), 2) for k, v in gpa_table(clean_grades(load_grades('grades.csv'))[0])['gpa'].items()}`	`{'S01': 8.11, 'S02': 7.5, 'S03': 4.43, 'S04': 8.64, 'S05': 3.5, 'S06': 7.0, 'S07': 6.29}`
gpa_table: tổng tín chỉ, số môn và hạng chữ	`(lambda t: (t['credits'].to_dict(), t['courses'].to_dict(), t['letter'].to_dict()))(gpa_table(clean_grades(load_grades('grades.csv'))[0]))`	`({'S01': 9, 'S02': 3, 'S03': 7, 'S04': 7, 'S05': 4, 'S06': 6, 'S07': 7}, {'S01': 3, 'S02': 1, 'S03': 2, 'S04': 2, 'S05': 1, 'S06': 2, 'S07': 2}, {'S01': 'B', 'S02': 'B', 'S03': 'D', 'S04': 'A', 'S05': 'F', 'S06': 'B', 'S07': 'C'})`
gpa_table sắp GPA giảm dần	`gpa_table(clean_grades(load_grades('grades.csv'))[0]).index.tolist()`	`['S04', 'S01', 'S02', 'S06', 'S07', 'S03', 'S05']`

Bài tập gốc: A có điểm [6, 9], tín chỉ [1, 2] → GPA 8.0, không phải 7.5	`{k: round(float(v), 2) for k, v in gpa_table(__import__('pandas').DataFrame({'student_id': ['A', 'A', 'B'], 'course': ['X', 'Y', 'X'], 'credits': [1, 2, 3], 'score': [6.0, 9.0, 8.0]}))['gpa'].items()}`	`{'A': 8.0, 'B': 8.0}`
File mới: GPA và hạng chữ ngay tại các mốc	`(lambda t: ({k: round(float(v), 2) for k, v in t['gpa'].items()}, t['letter'].to_dict()))(gpa_table(clean_grades(load_grades(_studio.write('grades_k2.csv', 'student_id,name,course,credits,score\nB01,Lan,MATH101,3,8.5\nB01,Lan,CS101,3,8.5\nB02,Minh,MATH101,2,10\nB02,Minh,CS101,4,N/A\nB02,Minh,PHYS101,2,-1\nb03,Nga,MATH101,3,5.5\nB03,Nga,PHYS101,3,5.5\n B03,Nga,MATH101,3,5.5\nB04,Oanh,CS101,4,4\nB04,Oanh,MATH101,abc,9\nB05,Phú,CS101,5,3.99\nB05,Phú,MATH101,,7\nB06,Quang,CS101,2,10.5\nB06,Quang,MATH101,2,7\nB06,Quang,PHYS101,1,8.5\n')))[0]))`	`({'B01': 8.5, 'B02': 10.0, 'B03': 5.5, 'B04': 4.0, 'B05': 3.99, 'B06': 7.5}, {'B01': 'A', 'B02': 'A', 'B03': 'C', 'B04': 'D', 'B05': 'F', 'B06': 'B'})`
class_gpa: tỷ số của tổng (6.67 và 6.25), không phải trung bình các GPA	`(round(class_gpa(clean_grades(load_grades('grades.csv'))[0]), 2), round(class_gpa(clean_grades(load_grades(_studio.write('grades_k2.csv', 'student_id,name,course,credits,score\nB01,Lan,MATH101,3,8.5\nB01,Lan,CS101,3,8.5\nB02,Minh,MATH101,2,10\nB02,Minh,CS101,4,N/A\nB02,Minh,PHYS101,2,-1\nb03,Nga,MATH101,3,5.5\nB03,Nga,PHYS101,3,5.5\n B03,Nga,MATH101,3,5.5\nB04,Oanh,CS101,4,4\nB04,Oanh,MATH101,abc,9\nB05,Phú,CS101,5,3.99\nB05,Phú,MATH101,,7\nB06,Quang,CS101,2,10.5\nB06,Quang,MATH101,2,7\nB06,Quang,PHYS101,1,8.5\n')))[0]), 2))`	`(6.67, 6.25)`
grade_chart: hist 5 bin theo mốc hạng, chiều cao [1, 1, 1, 3, 1]	`(lambda f: (len(f.axes), [float(p.get_height()) for p in f.axes[0].patches], [round(float(p.get_x()), 2) for p in f.axes[0].patches]))(grade_chart(gpa_table(clean_grades(load_grades('grades.csv'))[0])))`	`(1, [1.0, 1.0, 1.0, 3.0, 1.0], [0.0, 4.0, 5.5, 7.0, 8.5])`
File mới: hist [1, 1, 1, 1, 2], tiêu đề ghi n=6	`(lambda f: ([float(p.get_height()) for p in f.axes[0].patches], 'n=6' in f.axes[0].get_title().replace(' ', '')))(grade_chart(gpa_table(clean_grades(load_grades(_studio.write('grades_k2.csv', 'student_id,name,course,credits,score\nB01,Lan,MATH101,3,8.5\nB01,Lan,CS101,3,8.5\nB02,Minh,MATH101,2,10\nB02,Minh,CS101,4,N/A\nB02,Minh,PHYS101,2,-1\nb03,Nga,MATH101,3,5.5\nB03,Nga,PHYS101,3,5.5\n B03,Nga,MATH101,3,5.5\nB04,Oanh,CS101,4,4\nB04,Oanh,MATH101,abc,9\nB05,Phú,CS101,5,3.99\nB05,Phú,MATH101,,7\nB06,Quang,CS101,2,10.5\nB06,Quang,MATH101,2,7\nB06,Quang,PHYS101,1,8.5\n')))[0])))`	`([1.0, 1.0, 1.0, 1.0, 2.0], True)`
grade_chart: tiêu đề có cỡ mẫu n=7, có nhãn hai trục	`(lambda f: ('n=7' in f.axes[0].get_title().replace(' ', ''), bool(f.axes[0].get_xlabel()), bool(f.axes[0].get_ylabel())))(grade_chart(gpa_table(clean_grades(load_grades('grades.csv'))[0])))`	`(True, True, True)`

MÔI TRƯỜNG CHẠY

Bài này dùng numpy / pandas / matplotlib chạy ngay trong trình duyệt (Pyodide). Lần đầu mở bài, trình duyệt tải thêm thư viện (vài MB) nên chờ lâu hơn một chút. Biểu đồ vẽ bằng backend Agg: plt.show() không mở cửa sổ, bộ chấm đọc thẳng tiêu đề, nhãn trục và dữ liệu của Figure. Chạy trên máy để xem hình. Thư viện: matplotlib, numpy, pandas. File kèm bài: grades.csv (data). Bài có cửa tự kiểm tra (gate): lời giải mẫu chỉ mở sau khi hoàn thành.

KỊCH BẢN MINH HOẠ — ƯU TIÊN (trace 5 bước)
Khung 1 — Bước 1. load_grades: grades.csv → 18 dòng chữ.
Khung 2 — Bước 2. clean_grades: strip + upper mã → ép số → missing_score 2, out_of_range 1, bad_credits 1, duplicate 1 → 13 dòng.
Khung 3 — Bước 3. gpa_table: S01 (8×3 + 6.5×2 + 9×4) / 9 = 8.11 → B; S06 (6×2 + 7.5×4) / 6 = 7.0 → B.
Khung 4 — Bước 4. class_gpa: 287 / 43 = 6.67 (trung bình 7 GPA lại ra 6.5).
Khung 5 — Bước 5. grade_chart: hist theo [0, 4, 5.5, 7, 8.5, 10] → cột F, D, C, B, A cao 1, 1, 1, 3, 1; tiêu đề n=7.
Khung đối chiếu — ngộ nhận "Điểm thiếu thì coi như 0 cho đủ dòng". Chia đôi màn hình: bên sai làm theo ngộ nhận, bên đúng theo giải thích. Thiếu điểm (vắng thi, chưa nhập, "N/A") không phải là được 0 điểm. fillna(0) làm GPA của S02 rơi từ 7.5 xuống 3.21 — từ hạng B thành F — chỉ vì một ô "abs". Đúng là loại dòng khỏi phép tính GPA và báo số dòng thiếu. Cùng họ với lỗi này: lấy trung bình điểm không trọng số, bỏ trùng theo sai khoá, quên strip mã sinh viên, và dùng > thay cho >= ở mốc hạng.
Khung kết quả — 2 cặp vào/ra. grades.csv → report: 18 → 13 (missing_score 2, out_of_range 1, bad_credits 1, duplicate 1); gpa_table + class_gpa → S04 8.64 A · S01 8.11 B · S02 7.5 B · S06 7.0 B · S07 6.29 C · S03 4.43 D · S05 3.5 F; GPA chung 6.67

Nguồn: pe-v4 · python-learning-data.json#DA02 · parser pe-v4/build · file build 048-DA02.json

Phụ lục · Bảng tra ứng viên animation

Cột “Dữ liệu” ghi bài có sẵn những gì để dựng hình: trace (chuỗi trạng thái), scaffold (template lắp mảnh), io (cặp vào/ra). Bài không có mục nào thì ghi rõ là chưa rút được ý tưởng — không bịa.

★★ = ưu tiên cao (trace từ 5 bước và có scaffold): 20 bài. ★ = ưu tiên (trace từ 3 bước): 28 bài. Không dấu = chưa đủ dữ liệu: 0 bài.

Mã	Tên bài	Dữ liệu	Ý tưởng animation (rút từ JSON)
P01 ★	Program, process và tài nguyên	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: ancestors(TABLE, 410) → chain = [410] … parent_of[1] là 0, không có trong bảng → dừng, trả [410, 305, 120, 1]
P02 ★	Vòng đời, fork, exec và wait	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: Popen([sys.executable, "-c", code]) → con được tạo, cha chạy tiếp ngay … wait() trả về → exit code: 0, n của SystemExit(n), 1 nếu lỗi
P03 ★	Scheduling và context switch	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: ready.popleft() → lấy process đầu hàng ready … left > run → về cuối hàng với phần còn lại left - run
P04 ★	stdin, stdout, stderr và pipe	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: input=data → các dòng đi vào stdin (fd 0) của con … text=True → r.stdout và r.stderr là str, không phải bytes
P05 ★	subprocess.run và kết quả	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: run([sys.executable, "-c", code]) → cha chờ con kết thúc … stdout.strip() == expected → "PASS", khác thì "FAIL"
P06 ★	Popen, communicate và timeout	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: Popen(…) → con chạy, cha nhận proc ngay … kill() rồi communicate() → thu nốt output, returncode có giá trị
P07 ★	multiprocessing: Process và Queue	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: Queue() → kênh IPC dùng chung giữa cha và con … cha get() đủ len(numbers) lần → join hết → dict
T01 ★	Thread, tài nguyên và concurrency	trace 4 bước, scaffold 3 slot, io 4 cặp	Chuỗi 4 khung theo bảng trace: scope_of("heap") → "shared": mọi thread thấy cùng object … who_am_i() trong Thread(name="saver") → "saver"
T02 ★	Thread target, start và join	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: Thread(target=download, args=(url, sizes)) → mô tả việc, chưa chạy … return sizes → đọc kết quả sau khi join hết
T03 ★	Subclass Thread	trace 4 bước, scaffold 4 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: SumWorker([1, 2]) → __init__: lưu numbers, total = None … worker.join() → đọc total an toàn
T04 ★	GIL, CPU-bound và I/O-bound	trace 4 bước, scaffold 3 slot, io 4 cặp	Chuỗi 4 khung theo bảng trace: serial → cộng mọi task: 2 + 3 + 4 = 9 … best_strategy → nhanh nhất; bằng nhau chọn cách rẻ hơn
T05 ★	Lock, race condition và deadlock	trace 4 bước, scaffold 3 slot, io 4 cặp	Chuỗi 4 khung theo bảng trace: deposit(5) → with self.lock: đọc, cộng, ghi balance … safe_plan(["log", "db"], ["db", "cache", "log"]) → False
T06 ★	Queue và tác vụ lưu nền	trace 4 bước, scaffold 4 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: submit(record) → put bản sao vào self.jobs … queue.Empty → worker kết thúc, join xong
G01 ★	GUI, CLI và cửa sổ	trace 4 bước, scaffold 4 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: tk.Tk() → tạo cửa sổ chính root … root.mainloop() → chờ sự kiện tới khi cửa sổ đóng
G02 ★	Widget và callback	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: tk.Entry(frame, show="*") → ô mật khẩu che ký tự … login() → get() hai Entry rồi status.config(text=…)
G03 ★	Dữ liệu nhập và lựa chọn	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: name_var.get().strip() → họ tên đã bỏ khoảng trắng … marks_list.insert(tk.END, line) → thêm dòng cuối Listbox
G04 ★	Layout: pack, place, grid	trace 4 bước, scaffold 4 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: form.pack(fill="x") → khung form giãn ngang trong root … Nút Lưu grid(row=len(fields), columnspan=2) → hàng cuối

G05 ★	OOP và tách logic khỏi giao diện	trace 4 bước, scaffold 4 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: App(root) → __init__ tạo self.marks, self.mark_var … ValueError → showerror; hợp lệ → append rồi summary()
G06 ★	Event loop và worker nền	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: Bấm Tính → start_task() start worker, hẹn poll_result … poll_result() có kết quả → status.config trên thread GUI
R02 ★	Dự án PW5–PW9: lưu, shell, thread và GUI	trace 4 bước, io 3 cặp	Chuỗi 4 khung theo bảng trace: Ô nhập → validate() → self.marks[tên] = điểm … Bấm Báo cáo → subprocess report.py → stdout lên self.output
X01 ★	Nhập dữ liệu bằng NumPy và pandas	trace 4 bước, scaffold 4 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: read_csv(sep=";") → tách đúng 3 cột name, class, mark … schema(df) → rows 4, missing 1, mean_mark 8.17
X02 ★	Excel, Stata, SAS, HDF5 và MATLAB	trace 4 bước, scaffold 4 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: Path("Lab/Marks.XLSX").suffix.lower() → ".xlsx" … to_stata(write_index=False) + read_stata → đúng cột name, mark
X03 ★	SQL với sqlite3 và pandas	trace 4 bước, scaffold 4 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: make_db(ROWS) → bảng marks 4 dòng trong RAM … AVG(mark) … fetchone() → (8.25,), trả 8.25
X04 ★★	Matplotlib: dữ liệu và loại biểu đồ	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: fig, ax = plt.subplots() → một Figure chứa một Axes … return fig → nơi gọi tự show, lưu hoặc kiểm tra
X05 ★★	Matplotlib: tùy chỉnh và xuất	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: ax.bar(names, marks) → mỗi sinh viên một cột … plt.close(fig) → figure đóng, giải phóng bộ nhớ
N01 ★★	Array, dtype, shape và axis	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: np.arange(1, 13) → 12 số từ 1 đến 12 … sum() → 78
N02 ★★	Indexing, mask và vectorization	trace 5 bước, scaffold 3 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: a = [3, 5, 8, 12]; a >= 0 → [True, True, True, True] … np.clip(..., 0, 10) → chặn trên ở 10
N03 ★	Broadcasting và chuẩn hoá dữ liệu	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: mean(axis=0, keepdims=True) → [[3., 30.]], shape (1, 2) … np.where(std == 0, 1, std) → cột hằng chia cho 1, ra 0
N04 ★★	Thống kê, NaN và trọng số	trace 5 bước, scaffold 4 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: np.isfinite([4, nan, 8, 10]).sum() → 3 quan sát … np.average([7, 8], weights=[2, 3]) → 7.6
N05 ★★	View, copy và sai số số thực	trace 5 bước, scaffold 2 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: a[:2] → view, chung bộ nhớ với a … np.isclose(tổng, 0.3) → True, còn == cho False
N06 ★★	Đọc dữ liệu và mô phỏng tái lập	trace 5 bước, scaffold 3 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: genfromtxt(skip_header=1, usecols=(1, 2)) → array (4, 2) … default_rng(7) tạo lại → đúng 5 số đó
PD01 ★	Series, DataFrame và index	trace 4 bước, scaffold 3 slot, io 3 cặp	Chuỗi 4 khung theo bảng trace: df.loc["S02", "mark"] → tìm hàng có NHÃN S02 → 9. … df[["mark"]] → DataFrame một cột, shape (n, 1).
PD02 ★★	CSV, Excel và schema	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: sep=";" → tách đúng ba cột id, mark, note. … to_csv(path, index=False) → dòng đầu là id,mark.
PD03 ★★	Làm sạch thiếu, trùng và sai kiểu	trace 5 bước, scaffold 3 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: str.strip().str.lower() → " North " và "NORTH" thành "north". … reset_index(drop=True) → index 0, 1, 2… liền mạch.
PD04 ★★	Lọc, tính cột và cập nhật an toàn	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: out = df.copy() → bảng riêng, bảng gốc không đổi. … sort_values(..., ascending=False) → đơn lớn nhất lên đầu.
PD05 ★★	GroupBy và KPI có mẫu số rõ ràng	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: region.fillna("unknown") → không nhóm nào bị bỏ. … tổng các vùng = tổng nguồn → không mất dòng.
PD06 ★★	Merge, concat và kiểm tra cardinality	trace 5 bước, scaffold 3 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: how="left" → giữ mọi đơn, đơn thiếu giá có NaN. … concat(..., ignore_index=True) → index 0..n-1.

PD07 ★★	Long, wide, pivot và melt	trace 5 bước, scaffold 3 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: melt(id_vars="region") → mỗi (vùng, tháng) một dòng. … sum() trước = sum() sau → hình dạng đổi, tổng giữ nguyên.
PD08 ★★	Ngày giờ, resample và rolling	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: to_datetime(errors="coerce") → "bad", "2026-13-01" thành NaT. … sort_index().rolling(2) → cửa sổ hai ngày liền nhau.
PD09 ★★	EDA, chất lượng dữ liệu và export	trace 5 bước, scaffold 4 slot, io 3 cặp	Chuỗi 5 khung theo bảng trace: duplicated() → đánh dấu bản thứ hai trở đi của khoá. … to_csv(path, index=False) + nhật ký → file sạch, số dòng trước/sau.
M01 ★★	Figure, Axes và chọn biểu đồ	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: plt.subplots(1, 2, layout="constrained") → 1 Figure, 2 Axes … return fig → nơi gọi đọc fig.axes[0], fig.axes[1]
M02 ★★	Phân bố, scatter và outlier	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: all_scores = [… của mọi nhóm] → gộp, n = len(all_scores) … ax.scatter(hours, scores, alpha=0.5) → cặp quan sát, chỗ chồng đậm hơn
M03 ★★	Subplot và heatmap đọc được	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: max(... for table in tables.values()) → vmax chung … fig.colorbar(im, ax=list(axes[0]), label=unit) → một thước màu có đơn vị
M04 ★★	Màu, annotation và kể chuyện dữ liệu	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: revenue.to_numpy().argmax() → vị trí cột lớn nhất … title=f"{top} has the highest revenue" → tiêu đề nói kết quả
M05 ★★	Ticks, scale và ngày trên trục	trace 5 bước, scaffold 3 slot, io 2 cặp	Chuỗi 5 khung theo bảng trace: pd.to_datetime(df["date"]) → chuỗi thành Timestamp … any(size <= 0 …) → ValueError trước khi vẽ log
M06 ★	Layout, xuất hình và kiểm tra chất lượng	trace 4 bước, scaffold 3 slot, io 2 cặp	Chuỗi 4 khung theo bảng trace: plt.subplots(figsize=(6, 3), layout="constrained") → 6×3 inch … result[title] = buf.getvalue() → bytes để kiểm hoặc ghi file
DA01 ★	Mini-project: doanh thu từ raw tới chart	trace 5 bước, io 2 cặp	Chuỗi 5 khung theo bảng trace: load_sales: sales.csv → 11 dòng, mọi cột là chữ. … revenue_chart: barh 2 cột (65, 90) + line 2 điểm (110, 45).
DA02 ★	Mini-project: điểm số và GPA	trace 5 bước, io 2 cặp	Chuỗi 5 khung theo bảng trace: load_grades: grades.csv → 18 dòng chữ. … grade_chart: hist theo [0, 4, 5.5, 7, 8.5, 10] → cột F, D, C, B, A cao 1, 1, 1, 3, 1; tiêu đề n=7.

Ý tưởng ở cột cuối được rút máy móc từ bước đầu và bước cuối của bảng trace trong JSON, không phải đề xuất sáng tạo — xem khối “kịch bản minh hoạ” ở trang của bài để có đủ các khung.